

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Kiến trúc Phần mềm (CO3017)

BÁO CÁO BÀI TẬP LỚN

Intelligent Restaurant Management System (IRMS)

Giảng viên: Trần Trương Tuấn Phát

Lớp học phần: L01

Nhóm: 8

Khoa: Khoa học và Kỹ thuật Máy tính

MSSV	Họ và tên	Email
2311896	Đỗ Quang Long	long.doquang1896@hcmut.edu.vn
2311982	Lê Hoàng Luân	luan.le2311982@hcmut.edu.vn
2310990	Lê Thúy Hiền	hien.lethuy@hcmut.edu.vn
2213698	Nguyễn Hải Trung	trung.nguyen2004@hcmut.edu.vn
2211384	Tô Thế Hưng	hung.totothehung@hcmut.edu.vn
2310737	Trần Nhật Đăng	dang.trannh388@hcmut.edu.vn

[Source code](#)

Thành phố Hồ Chí Minh, tháng 4 năm 2026

Mục lục

1	Tổng quan hệ thống	2
1.1	Bối cảnh nghiệp vụ	2
1.2	Mục tiêu của hệ thống	2
1.3	Phạm vi hệ thống (IRMS)	3
1.4	Các bên liên quan	3
1.5	Yêu cầu chức năng	5
1.6	Yêu cầu phi chức năng ở góc nhìn toàn dự án	6
1.7	Lựa chọn kiến trúc chủ đạo	8
2	Mô hình hóa hệ thống	10
2.1	Danh mục use cases	10
2.2	Góc nhìn sử dụng	13
2.2.1	Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn	13
2.2.2	Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ	13
2.2.3	Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai	13
2.2.4	Kịch bản 4: Từ cảnh báo tồn kho thấp đến quyết định của quản lý	14
2.3	Đặc tả use cases	17
3	Kiến trúc phần mềm	47
3.1	Tổng quan kiến trúc	47
3.2	Tổng quan cho các góc nhìn chính	48
3.2.1	Tổng quan cho góc nhìn mô-đun	49
3.2.2	Tổng quan cho góc nhìn thành phần và kết nối	53
3.2.3	Tổng quan cho góc nhìn phân bố	54
3.3	Đặc trưng kiến trúc	55
3.4	Sơ đồ năng lực nghiệp vụ hỗ trợ	55
3.4.1	Sơ đồ phân rã năng lực logic	55
3.4.2	Sơ đồ phân rã năng lực logic hoàn chỉnh	56
3.4.3	Sơ đồ chuỗi phục vụ	57
3.5	Góc nhìn phân rã	58
3.5.1	Các ứng dụng phía người dùng	58
3.5.2	Các dịch vụ nghiệp vụ	58
3.5.3	Sơ đồ tổng quan	59
3.5.4	Chi tiết Module View cho tuyến order-to-cash	62
3.6	Góc nhìn thành phần và kết nối	63
3.6.1	Các thành phần chính	63
3.6.2	Các kiểu kết nối chính	64
3.6.3	Cơ sở lựa chọn mô hình kết nối hỗn hợp	65
3.6.4	Sơ đồ thành phần cho tuyến lệnh vận hành	68
3.6.5	Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc	69
3.7	Góc nhìn triển khai	70
3.7.1	Tổng quan phân bố hệ thống	70

3.7.2	Cơ sở thiết kế triển khai	72
3.7.3	Sơ đồ triển khai cho runtime Docker local và cô lập lỗi	77
3.7.4	Install View cho gói triển khai Docker local	78
3.7.5	Assignment view	81
3.8	Nguyên tắc thiết kế	81
3.9	Áp dụng nguyên lý SOLID vào thiết kế	81
3.10	Khả năng mở rộng trong tương lai	83
4	Thiết kế chi tiết	85
4.1	Góc nhìn lớp	85
4.2	Giải thích theo từng cụm lớp	88
4.3	Thiết kế hành vi	92
4.3.1	Nhóm đặt bàn và bàn ăn	93
4.3.2	Nhóm gọi món và điều phối bếp	98
4.3.3	Nhóm hóa đơn, thanh toán và biên lai	105
4.3.4	Nhóm tồn kho, cảnh báo và phân tích	111
4.3.5	Nhóm quản trị cấu hình và vận hành	116
5	Hiện thực	121
A	Phụ lục	126
A.1	Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc	126
A.1.1	So sánh các kiểu kiến trúc ứng viên	127
A.1.2	Hồ sơ quyết định kiến trúc	127
A.1.3	Tổng quan danh mục ADR	127
A.1.4	Nhóm quyết định về phong cách kiến trúc và phân rã	130
A.1.5	Nhóm quyết định về giao tiếp runtime	132
A.1.6	Nhóm quyết định về dữ liệu, nhất quán và báo cáo	135
A.1.7	Nhóm quyết định về bảo mật, mở rộng và triển khai	137
A.2	Liên kết giữa phụ lục và các phần chính	139
A.3	Lý do chọn các góc nhìn và loại sơ đồ	139
A.4	Lý do chọn các thành phần và loại hình kiến trúc nền	140
A.5	Bảng thuật ngữ và ký hiệu viết tắt	142
A.6	Hướng dẫn chạy project	142
A.7	Phân công công việc	144

1 Tổng quan hệ thống

1.1 Bối cảnh nghiệp vụ

Hệ thống **Intelligent Restaurant Management System (IRMS)** được thiết kế nhằm tối ưu hóa hoạt động vận hành của nhà hàng, bao gồm phục vụ tại bàn, khu vực bếp, quầy thanh toán và các hoạt động điều phối theo ca. Trong thực tế, nhiều sai sót thường xảy ra vào các giờ cao điểm, chẳng hạn như đơn gọi món bị ghi nhầm, ghi chú dị ứng không được lưu, trạng thái bàn không cập nhật kịp thời, hóa đơn bị lệch khi tách hóa đơn hoặc áp dụng khuyến mãi, cũng như tồn kho cuối ngày không phản ánh đúng mức tiêu hao thực tế. Những vấn đề này ảnh hưởng trực tiếp đến chất lượng phục vụ, trải nghiệm khách hàng và hiệu quả vận hành của nhà hàng.

Vì vậy, IRMS không chỉ đơn thuần là một ứng dụng ghi nhận đơn gọi món trên màn hình mà phải đóng vai trò như một **nền tảng số trung tâm**, liên kết chặt chẽ các khu vực phục vụ phía trước, khu bếp và bộ phận quản trị. Hệ thống cần đảm bảo các chức năng nghiệp vụ toàn diện, bao gồm quản lý đặt bàn và xếp bàn, tiếp nhận và điều phối đơn gọi món, quản lý tiến trình chế biến trong bếp, lập và thanh toán hóa đơn, theo dõi tồn kho, cung cấp báo cáo và phân tích hiệu suất vận hành, kiểm soát truy cập dựa trên vai trò, ghi nhận nhật ký kiểm toán, đồng thời có khả năng tích hợp với các hệ thống bên ngoài khi cần thiết.

IRMS được xây dựng như một hệ thống nghiệp vụ hoàn chỉnh với luồng xử lý lỗi rõ ràng, hỗ trợ các quy trình bất đồng bộ, đồng thời đảm bảo khả năng mở rộng linh hoạt để đáp ứng nhu cầu vận hành phức tạp và đa dạng trong môi trường nhà hàng hiện đại.

1.2 Mục tiêu của hệ thống

Hệ thống *Intelligent Restaurant Management System (IRMS)* được xây dựng nhằm hỗ trợ số hóa và tối ưu hóa các hoạt động vận hành trong nhà hàng. Thông qua việc tích hợp các chức năng quản lý đơn hàng, điều phối bếp, quản lý bàn, thanh toán và theo dõi tồn kho, hệ thống hướng tới việc nâng cao hiệu quả làm việc của nhân viên cũng như cải thiện trải nghiệm phục vụ khách hàng. Các mục tiêu chính của hệ thống bao gồm:

1. Giảm sai sót trong quá trình gọi món và xử lý đơn hàng

Hệ thống sử dụng thực đơn số hóa và giao diện nhập liệu có cấu trúc để giúp nhân viên phục vụ tạo đơn nhanh chóng và chính xác. Các tùy chọn như ghi chú món ăn, yêu cầu đặc biệt hoặc cảnh báo dị ứng giúp hạn chế nhầm lẫn trong quá trình phục vụ và chế biến.

2. Tăng hiệu quả phối hợp giữa bộ phận phục vụ và bếp

Khi đơn hàng được xác nhận, hệ thống sẽ tự động chuyển thông tin đến màn hình hiển thị bếp (*Kitchen Display System – KDS*).

3. Đảm bảo cập nhật và đồng bộ trạng thái hoạt động theo thời gian gần thực

Các thông tin quan trọng như trạng thái bàn, tiến độ chế biến món ăn, trạng thái đơn hàng và thanh toán cần được cập nhật liên tục trên các giao diện liên quan.

4. Hỗ trợ quản lý bàn và tối ưu hóa quy trình phục vụ khách hàng

Hệ thống giúp nhân viên theo dõi tình trạng bàn, quản lý đặt bàn và ước lượng thời gian chờ của khách.

5. Tăng hiệu quả quản lý tài chính và thanh toán

IRMS hỗ trợ tạo hóa đơn tự động, tính toán tổng chi phí bao gồm món ăn, dịch vụ và khuyến mãi. Hệ thống cho phép nhiều phương thức thanh toán khác nhau, đồng thời hỗ trợ tách hóa đơn khi cần thiết.

6. Hỗ trợ theo dõi tồn kho và cung cấp dữ liệu phân tích cho quản lý

Hệ thống ghi nhận việc sử dụng nguyên liệu và cung cấp cảnh báo khi tồn kho xuống thấp. Ngoài ra, các

báo cáo bán hàng và phân tích hoạt động giúp nhà quản lý đánh giá hiệu suất kinh doanh và đưa ra quyết định phù hợp.

1.3 Phạm vi hệ thống (IRMS)

Hệ thống được thiết kế nhằm số hóa và tối ưu quy trình vận hành nhà hàng, từ phục vụ khách hàng đến quản lý bếp, kho và kinh doanh. Xét theo kiến trúc hướng dịch vụ, nhóm quyết định phân rã thành các service theo năng lực nghiệp vụ như Reservation & Seating Service, Ordering & Menu Service, Kitchen Coordination Service, Billing & Settlement Service, Inventory Monitoring Service, Reporting Service, IAM & Audit Service, Notification Service và các adapter biên nội bộ. Cụ thể, hệ thống bao gồm các nhóm năng lực chính sau:

- **Phân hệ Phục vụ và Gọi món (Front-of-House)**

- **Quản lý gọi món:** Hỗ trợ nhân viên thực hiện gọi món nhanh chóng trên máy tính bảng/terminal. Xử lý các yêu cầu như tùy chỉnh món, ghi chú dị ứng, các gói combo và thay đổi nguyên liệu.
- **Cập nhật thực đơn thực tế:** Hiển thị danh mục món ăn và trạng thái còn/hết hàng theo thời gian thực.
- **Quản lý bàn và đặt chỗ:** Cung cấp sơ đồ bàn, quản lý trạng thái bàn (trống/đang dùng), danh sách chờ và lịch hẹn đặt trước cho bộ phận Host.

- **Phân hệ Điều hành Bếp (Kitchen Display System – KDS)**

- **Điều phối tự động:** Tự động chuyển đơn hàng từ bàn đến các khu vực bếp tương ứng.
- **Tối ưu hóa quy trình:** Sắp xếp và ưu tiên đơn hàng dựa trên vị trí khu vực bếp, loại món và thời gian chế biến để đảm bảo tốc độ phục vụ.
- **Tương tác thời gian thực:** Cho phép đầu bếp cập nhật tiến độ món ăn để nhân viên phục vụ có thể theo dõi và trả món kịp thời.

- **Phân hệ Thanh toán và Tài chính**

- **Xử lý hóa đơn linh hoạt:** Hỗ trợ tính phí dịch vụ, áp dụng chương trình khuyến mãi, tách hóa đơn và quản lý tiền tip.
- **Đa dạng phương thức thanh toán:** Hỗ trợ tiền mặt, thẻ và ví điện tử thông qua PaymentGateway, đồng thời xuất biên lai qua ReceiptDeliveryAdapter.

- **Phân hệ Quản trị và Kho vận**

- **Quản lý kho tự động:** Tự động khấu trừ nguyên liệu khi có đơn hàng và phát thông báo cảnh báo khi lượng tồn kho xuống thấp.
- **Báo cáo và Phân tích (Analytics):** Cung cấp hệ thống dashboard về doanh thu, các món bán chạy, hiệu suất nhân viên và nhận diện các điểm nghẽn trong vận hành.
- **Kiểm soát và Bảo mật:** Quản lý quyền truy cập dựa trên vai trò (RBAC) và lưu vết lịch sử (nhật ký kiểm toán) cho các tác động quan trọng vào hệ thống.

1.4 Các bên liên quan

Bảng 1: Các bên liên quan chính của IRMS

Bên liên quan	Vai trò	Mối quan tâm chính
Khách hàng	Người dùng/đơn vị vận hành	• Đặt bàn trước • Nhận thông báo xác nhận và nhắc nhở • Thanh toán đa phương thức, nhận hóa đơn • Gọi món, yêu cầu tùy chỉnh và ghi chú dị ứng khi gọi món
Lễ tân	Người dùng/đơn vị vận hành	• Quản lý sơ đồ bàn, theo dõi trạng thái trống/bận • Xử lý đặt bàn trước và danh sách chờ • Gửi thông báo xác nhận/nhắc nhở cho khách hàng • Ước tính thời gian chờ, phân bổ bàn hợp lý
Phục vụ	Người dùng/đơn vị vận hành	• Tạo đơn gọi món, tiếp nhận yêu cầu gọi món từ khách • Theo dõi đơn, thời gian phục vụ và tình trạng thanh toán theo từng bàn
Nhân viên bếp	Người dùng/đơn vị vận hành	• Cập nhật trạng thái món: Đã bắt đầu, Đang nấu, Sẵn sàng phục vụ • Phản hồi cảnh báo khi món sắp trễ • Cập nhật nguyên liệu đã sử dụng sau khi chế biến
Thu ngân	Người dùng/đơn vị vận hành	• Lập hóa đơn tổng hợp (đồ ăn, đồ uống, dịch vụ) • Xử lý thanh toán: tiền mặt, ví điện tử • Tách hóa đơn và quản lý tiền boa • Thực hiện hoàn tiền khi cần và ghi vào nhật ký kiểm toán
Quản lý	Người dùng/đơn vị vận hành	• Xem báo cáo: doanh thu, giờ cao điểm, món bán chạy • Giám sát tồn kho, nhận cảnh báo ngưỡng tối thiểu • Cấu hình thực đơn, giá và chương trình khuyến mãi • Xuất báo cáo cho kế toán, đánh giá hiệu suất • Theo dõi hiệu quả nhân viên và điểm nghẽn bếp

Tiếp tục ở trang sau

Bên liên quan	Vai trò	Mối quan tâm chính
Quản trị viên	Người dùng/đơn vị vận hành	- Phân quyền truy cập theo vai trò - Xem và quản lý nhật ký kiểm toán toàn hệ thống - Xử lý sự cố kỹ thuật, bảo đảm hệ thống luôn sẵn sàng hoạt động - Cấu hình và bảo trì hệ thống bán hàng tại quầy
PaymentGateway	Port/adaptor nội bộ	- Xử lý giao dịch thẻ hoặc ví điện tử ở mức hợp đồng thanh toán của hệ thống - Hỗ trợ nhánh tiền mặt và nhánh tham chiếu giao dịch trong phạm vi xử lý thanh toán của hệ thống - Trả về kết quả giao dịch cho hệ thống bán hàng tại quầy

1.5 Yêu cầu chức năng

IRMS phân rã thành các service theo năng lực nghiệp vụ. Mỗi service giữ một ranh giới trách nhiệm riêng, có dữ liệu và luật nghiệp vụ thuộc phạm vi của mình, đồng thời phối hợp với service khác bằng API đồng bộ hoặc event bất đồng bộ tùy theo tính chất của thao tác.

1. Reservation & Seating Service

Service này hỗ trợ tiếp đón khách và quản lý bàn trong nhà hàng, bao gồm đặt bàn, danh sách chờ, ghép/chuyển bàn và giải phóng bàn. Các thao tác cần phản hồi tức thời như nhận khách, gán bàn hay mở TableSession được xử lý đồng bộ qua API để đảm bảo lễ tân và nhân viên phục vụ cập nhật trạng thái nhanh chóng.

Quản lý trạng thái bàn cho phép nhân viên theo dõi sơ đồ nhà hàng theo thời gian thực, cập nhật trạng thái và điều phối khách phù hợp với sức chứa. Khi mở một phiên bàn, service phát event TableSessionOpened để các service khác như gọi món, báo cáo hay kiểm toán nhận biết ngữ cảnh phục vụ mới mà không cần gọi trực tiếp đến service đặt bàn.

2. Ordering & Menu Service

Service này quản lý thực đơn số, tùy chọn món, trạng thái còn/hết món và đơn gọi món. Để đảm bảo tính nhất quán, hệ thống lưu ảnh chụp giao dịch tại thời điểm xác nhận nhằm tránh ảnh hưởng khi thực đơn thay đổi. Các thao tác tạo và xác nhận đơn được xử lý qua API đồng bộ để phản hồi tức thời cho nhân viên phục vụ và bếp.

Duyệt thực đơn và gọi món cung cấp thông tin món ăn, giá, trạng thái khả dụng, ghi chú dị ứng và các tùy chọn đi kèm. Khi đơn được xác nhận, service phát event OrderConfirmed để các service như Kitchen Coordination Service, Inventory Monitoring Service, Reporting Service, IAM & Audit Service và Notification Service xử lý các tác vụ liên quan theo cơ chế bất đồng bộ, giúp giảm thời gian xác nhận đơn.

3. Kitchen Coordination Service

Service này chịu trách nhiệm chuyển đơn gọi món đã xác nhận thành các tác vụ chế biến tại bếp. Thành phần quản lý phiếu bếp, trạm bếp, hàng đợi, mức ưu tiên, tiến độ chế biến và bàn giao món cho phục vụ, giúp tách biệt

logic vận hành bếp khởi Order và OrderItem

Tiếp nhận và hiển thị phiếu bếp được kích hoạt từ các event như OrderConfirmed hoặc KitchenTicketCreated. Khi đầu bếp cập nhật tiến độ món, service phát các event như OrderItemPrepared để giao diện phục vụ, báo cáo và thông báo cùng sử dụng. Các cập nhật cần phản hồi nhanh được xử lý qua API đồng bộ, còn thông báo và tổng hợp dữ liệu vận hành được thực hiện bất đồng bộ qua event.

4. Billing & Settlement Service

Service này chịu trách nhiệm toàn bộ vùng thanh toán, bao gồm lập hóa đơn, tách hóa đơn, tính thuế, phí phục vụ, tiền boa, áp mã khuyến mãi, ghi nhận thanh toán, phát hành biên lai và xử lý hoàn tiền. Các thao tác như lập hóa đơn, ghi nhận thanh toán hoặc hoàn tiền cần phản hồi rõ ràng nên được xử lý qua API đồng bộ.

Lập hóa đơn và thanh toán sử dụng dữ liệu từ các món đã gọi và phục vụ để tạo hóa đơn. Sau khi phát hành hóa đơn hoặc ghi nhận thanh toán, service phát các event như BillIssued, PaymentCaptured và RefundRequested nhằm hỗ trợ in biên lai, ghi kiểm toán, cập nhật báo cáo và đồng bộ trạng thái bàn theo cơ chế bất đồng bộ.

5. Inventory Monitoring Service

Service này quản lý tồn kho, công thức nguyên liệu, giao dịch kho và các quy tắc cảnh báo mức tồn thấp. Thành phần chủ yếu tiêu thụ các event nghiệp vụ để cập nhật biến động kho theo cơ chế bất đồng bộ, thay vì tham gia trực tiếp vào tuyến xử lý gọi món hay thanh toán.

Cập nhật tồn kho và cảnh báo được kích hoạt từ các event như OrderConfirmed, OrderItemPrepared hoặc PaymentCaptured. Khi tồn kho giảm dưới ngưỡng an toàn, service phát event InventoryLow để các service như Notification Service, Reporting Service hoặc giao diện quản lý tiếp tục xử lý. Cách tiếp cận này giúp phản ánh tiêu hao thực tế mà không làm chậm quá trình gọi món.

6. Reporting Service, IAM & Audit Service và Notification Service

Reporting Service tổng hợp dữ liệu vận hành để xây dựng các báo cáo như doanh thu, món bán chạy, giờ cao điểm, hiệu suất bếp, tỷ lệ hoàn tiền và hiệu quả nhân viên. Dữ liệu được cập nhật qua event và bộ xử lý nền, giúp các truy vấn quản trị không ảnh hưởng trực tiếp đến dữ liệu giao dịch đang hoạt động.

IAM & Audit Service quản lý định danh người dùng, phân quyền theo vai trò, phiên làm việc và nhật ký kiểm toán. Mọi thao tác nhạy cảm như hoàn tiền, hủy món, thay đổi giá, mở lại hóa đơn, cập nhật quyền truy cập hoặc điều chỉnh tồn kho đều được ghi nhận đầy đủ thông tin người thực hiện, thời điểm, giá trị trước/sau và lý do thay đổi.

Notification Service xử lý các thông báo như xác nhận đặt bàn, cảnh báo tồn kho, món sẵn sàng và thanh toán. Service tiêu thụ event bất đồng bộ để tránh ảnh hưởng đến giao dịch chính khi kênh thông báo gặp sự cố.

1.6 Yêu cầu phi chức năng ở góc nhìn toàn dự án

Giả định thiết kế cho một nhà hàng tầm trung, được sử dụng xuyên suốt báo cáo để đánh giá hiệu năng, độ tin cậy và khả năng mở rộng của hệ thống.

Bảng 2: Hai điều kiện vận hành dùng làm cơ sở thiết kế của IRMS

Điều kiện	Tải giả định	Ý nghĩa với kiến trúc
Điều kiện 1 — Ca thông thường	20–25 bàn hoạt động, 8–12 nhân viên đăng nhập đồng thời, tối đa 35 đơn gọi món mỗi giờ	Hệ thống phải phản hồi mượt cho các tác vụ tạo đơn gọi món, cập nhật màn hình bếp, đổi trạng thái bàn và chốt hóa đơn mà chưa cần mở rộng hay hy sinh trải nghiệm thao tác.
Điều kiện 2 — Giờ cao điểm	35–40 bàn hoạt động, 15–18 nhân viên đăng nhập đồng thời, tối đa 60 đơn gọi món mỗi giờ	Kiến trúc phải giữ ổn định tuyến giao dịch nóng; các luồng nền như báo cáo, cảnh báo mức tồn thấp và đồng bộ thông báo không được làm nghẽn gọi món, điều phối bếp hay thanh toán.

Trong hai điều kiện trên, hệ thống chịu các ràng buộc sau:

- **Ràng buộc phạm vi:** Hệ thống hiện phục vụ một nhà hàng đơn lẻ nhưng có khả năng mở rộng trong tương lai.
- **Ràng buộc thiết bị:** Nhân viên thao tác qua tablet, màn hình bếp và web nội bộ với giao diện ưu tiên nhanh, đơn giản.
- **Ràng buộc mạng:** Hệ thống chịu được gián đoạn mạng ngắn hạn mà không mất hoặc trùng đơn.
- **Ràng buộc thanh toán:** Thanh toán điện tử qua cổng bên ngoài, không lưu dữ liệu thẻ thô.
- **Ràng buộc kiểm soát:** Dữ liệu được lưu tập trung và hỗ trợ nhật ký kiểm toán cho các thao tác nhạy cảm.

Bảng 3: Yêu cầu phi chức năng

Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Hiệu năng & độ phản hồi	Tra cứu thực đơn và trạng thái bàn có thời gian phản hồi dưới 1 giây; thao tác xác nhận đơn gọi món và gửi bếp có thời gian phản hồi dưới 2 giây ở Điều kiện 1 và dưới 3 giây ở Điều kiện 2; phiếu phải hiển thị trên màn hình bếp không quá 2 giây sau khi đơn được xác nhận.	Áp dụng cho toàn bộ giờ mở cửa, đặc biệt tại màn hình gọi món, màn hình bếp và quầy thu ngân là ba điểm không được gây nghẽn thao tác người dùng.	Tách riêng các miền Ordering, Kitchen Coordination và Billing để giảm cạnh tranh tài nguyên; dùng mô hình đọc hoặc bộ nhớ đệm cho thực đơn và trạng thái thường đọc; giữ các tác vụ trên tuyến giao dịch nóng ở dạng gọi đồng bộ ngắn, còn báo cáo và cảnh báo chuyển sang xử lý nền.
Độ tin cậy & toàn vẹn giao dịch	Không làm mất đơn gọi món đã xác nhận; tỷ lệ tạo phiếu trùng hoặc ghi nhận thanh toán trùng phải thấp hơn 0.1%; khôi phục sau lỗi một nút ứng dụng trong vòng tối đa 10 phút.	Áp dụng ở cả Điều kiện 1 và Điều kiện 2; vẫn phải giữ đúng khi mạng nội bộ gián đoạn ngắn hạn dưới 60 giây hoặc khi đối tác thanh toán phản hồi chậm.	Dùng giao dịch cục bộ theo dịch vụ, khóa chống xử lý lặp cho gọi món, thanh toán và hoàn tiền, khóa lạc quan khi cập nhật trạng thái, cùng hộp thư ra và bộ truyền sự kiện cho sự kiện miền để tránh vừa mất dữ liệu vừa xử lý lặp.

Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Khả năng mở rộng	Hệ thống phải chịu được tối thiểu 1.5 lần tải của Điều kiện 2 trong các đợt cao điểm ngắn 10–15 phút mà không cần thiết kế lại; riêng các miền Ordering, Kitchen Coordination và Billing phải có thể tăng số phiên bản chạy độc lập khi tải tăng, báo cáo và phân tích không được kéo chậm tuyến giao dịch nóng.	Áp dụng cho bữa trưa, bữa tối, ngày cuối tuần hoặc các đơn đặt nhóm lớn	Chọn kiến trúc tách miền nghiệp vụ rõ ràng để phân bổ tải theo nhu cầu thực tế; mở rộng theo chiều ngang các miền có nhu cầu cao; cô lập báo cáo bằng mô hình đọc và ảnh chụp dữ liệu để truy vấn phân tích không tranh chấp với cơ sở dữ liệu giao dịch.
Bảo mật & khả năng kiểm toán	100% thao tác hoàn tiền, hủy món đã gửi bếp, ghi đề giá, mở lại hóa đơn và thay đổi quyền phải có nhật ký người thực hiện, thời điểm, giá trị trước, giá trị sau và lý do; phiên thu ngân hoặc quản lý tự khóa sau 15 phút không hoạt động; hệ thống không lưu dữ liệu thẻ gốc.	Áp dụng liên tục trong toàn bộ vòng đời vận hành và đặc biệt quan trọng ở các ca đổi nhân sự, bàn giao ca hoặc xử lý khiếu nại sau thanh toán.	Dùng IAM tập trung, phân quyền theo vai trò, chính sách kiểm tra quyền trong từng dịch vụ, nhật ký kiểm toán bất biến cho hành động nhạy cảm, và tích hợp thanh toán qua đối tác ngoài để giới hạn phạm vi dữ liệu nhạy cảm nằm trong IRMS.
Khả năng quan sát & vận hành	100% yêu cầu ghi dữ liệu phải mang mã liên kết xuyên suốt; cảnh báo mức tồn thấp phải xuất hiện trong vòng 60 giây sau khi vượt ngưỡng; bảng điều khiển vận hành có độ trễ dữ liệu không quá 5 phút; lỗi ở cổng vào hoặc PaymentGateway phải sinh cảnh báo vận hành trong vòng 1 phút.	Áp dụng cho cả thời gian phục vụ và giai đoạn chốt ca, khi quản lý cần đối chiếu chênh lệch đơn gọi món, hóa đơn, tồn kho và thao tác hoàn tiền.	Ghi nhật ký tập trung theo dịch vụ, truy vết xuyên suốt bằng mã liên kết, thu thập số đo cho độ trễ phiếu và thanh toán, tổng hợp bảng điều khiển từ mô hình đọc, cùng với dấu thời gian sự kiện để có thể truy vết một đơn từ lúc tạo đến lúc đóng hóa đơn.
Khả năng bảo trì & mở rộng	Khi thêm một phương thức thanh toán hoặc một kênh thông báo mới, thay đổi chủ yếu chỉ nằm ở bộ kết nối, cấu hình và kiểm thử tích hợp; thay đổi thực đơn, giá, khuyến mãi hoặc ngưỡng mức tồn thấp không được buộc sửa chéo bộ điều khiển, thanh toán và tồn kho cùng lúc.	Áp dụng trong suốt vòng đời dự án, đặc biệt khi nhà hàng đổi chính sách giá, khuyến mãi theo mùa hoặc cần thí điểm thêm kênh thanh toán mới.	Phân rã theo miền nghiệp vụ, áp dụng nguyên lý SOLID và đảo ngược phụ thuộc cho cổng ngoài, tách lớp điều phối ứng dụng khỏi quy tắc miền, chuẩn hóa hợp đồng giữa các dịch vụ để mỗi thay đổi được khu trú trong biên chức năng tương ứng.

1.7 Lựa chọn kiến trúc chủ đạo

Từ bối cảnh vận hành ở các giờ cao điểm, yêu cầu chức năng và các yêu cầu phi chức năng, kiến trúc của IRMS được xác định là **service-based architecture theo miền nghiệp vụ**, kết hợp **event-driven processing** cho các hệ quả nền. Trong đó, service-based là kiến trúc chính: hệ thống được chia theo các servicenhu Reservation & Seating, Ordering & Menu, Kitchen Coordination, Billing & Settlement, Inventory Monitoring, Reporting, IAM & Audit và Notification. Event-driven đóng vai trò hỗ trợ để tách các tác vụ không cần phản hồi tức thời khỏi tuyến giao dịch nóng.

Bảng 4: Cơ sở lựa chọn kiến trúc chủ đạo của IRMS

Yêu cầu	Ý nghĩa	Lựa chọn
Luồng gọi món, bếp và thanh toán cần phản hồi nhanh, đặc biệt trong giờ cao điểm.	Tuyến giao dịch chính phải ngắn, rõ trạng thái thành công/thất bại và không bị kéo dài bởi các tác vụ phụ.	Dùng API đồng bộ cho các use case tuyến đầu; giữ logic nghiệp vụ trong service theo miền.
Các miền đặt bàn, gọi món, bếp, thanh toán, tồn kho, báo cáo và kiểm toán có trách nhiệm khác nhau.	Cần ranh giới trách nhiệm rõ để tránh một service hoặc một lớp kỹ thuật ôm quá nhiều nghiệp vụ.	Chọn service-based architecture theo miền nghiệp vụ làm kiểu kiến trúc chính.
Thông báo, báo cáo, cảnh báo tồn kho và kiểm toán không luôn cần phản hồi ngay cho thao tác chính.	Các hệ quả phụ nên được tách khỏi tuyến giao dịch nóng để giảm độ trễ và giảm phụ thuộc chéo giữa các miền.	Dùng event-driven processing, RabbitMQ, outbox và bộ xử lý sự kiện nền cho các luồng bất đồng bộ.
Hệ thống cần toàn vẹn dữ liệu cho đơn gọi món, hóa đơn, thanh toán, hoàn tiền và kiểm toán.	Không nên dùng event-driven thuần cho mọi thao tác vì nhiều bước cần xác nhận đồng bộ và kiểm soát lỗi trực tiếp.	Duy trì giao dịch cục bộ trong từng service; phát event sau khi trạng thái chính đã được ghi nhận.
Báo cáo và dashboard có thể chấp nhận độ trễ có kiểm soát.	Truy vấn phân tích không nên gây áp lực trực tiếp lên dữ liệu giao dịch nóng.	Dùng mô hình đọc được cập nhật qua sự kiện hoặc tiến trình nền.
Thanh toán, thông báo và biên lai là các điểm dễ thay đổi theo chính sách vận hành.	Lỗi nghiệp vụ không nên phụ thuộc trực tiếp vào nhà cung cấp cụ thể hoặc thiết bị vật lý.	Đặt PaymentGateway, NotificationChannelAdapter và ReceiptDeliveryAdapter sau các adapter nội bộ.

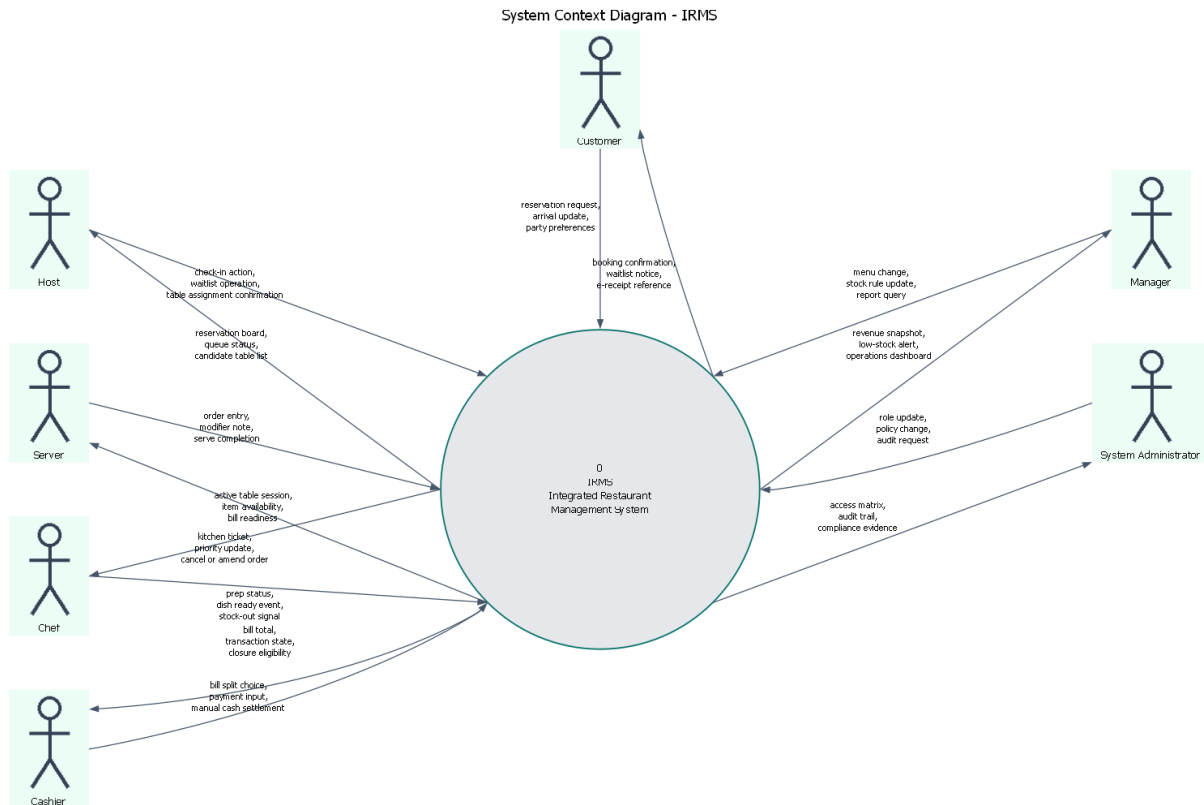
So sánh với các kiến trúc khác

- **Kiến trúc phân lớp tập trung:** dễ khởi tạo nhưng dễ làm mờ ranh giới giữa các nghiệp vụ như gọi món, bếp, thanh toán, tồn kho và báo cáo.
- **Microservices đầy đủ:** có khả năng cô lập triển khai tốt nhưng phát sinh chi phí vận hành, giám sát và quản lý giao dịch phân tán lớn hơn nhu cầu hiện tại.
- **Event-driven thuần:** không phù hợp với các thao tác cần phản hồi tức thời, trạng thái lỗi rõ ràng và kiểm soát trùng lặp chặt chẽ như gọi món hay thanh toán.

Vì vậy, kiến trúc được chọn là một cấu trúc lai: **service-based là nền tảng phân rã và tổ chức hệ thống**, còn **event-driven là cơ chế phối hợp cho các hệ quả bất đồng bộ**. Quyết định chuyển từ định hướng modular ban đầu sang service-based architecture được ghi chú trong [ADR-0001](#) và [ADR-0002](#). Cơ chế REST đồng bộ, event-driven, outbox/RabbitMQ, PostgreSQL vật lý duy nhất và các adapter biên lần lượt được ghi chú ở [ADR-0004](#), [ADR-0005](#), [ADR-0006](#), [ADR-0007](#), [ADR-0008](#) và [ADR-0011](#).

2 Mô hình hóa hệ thống

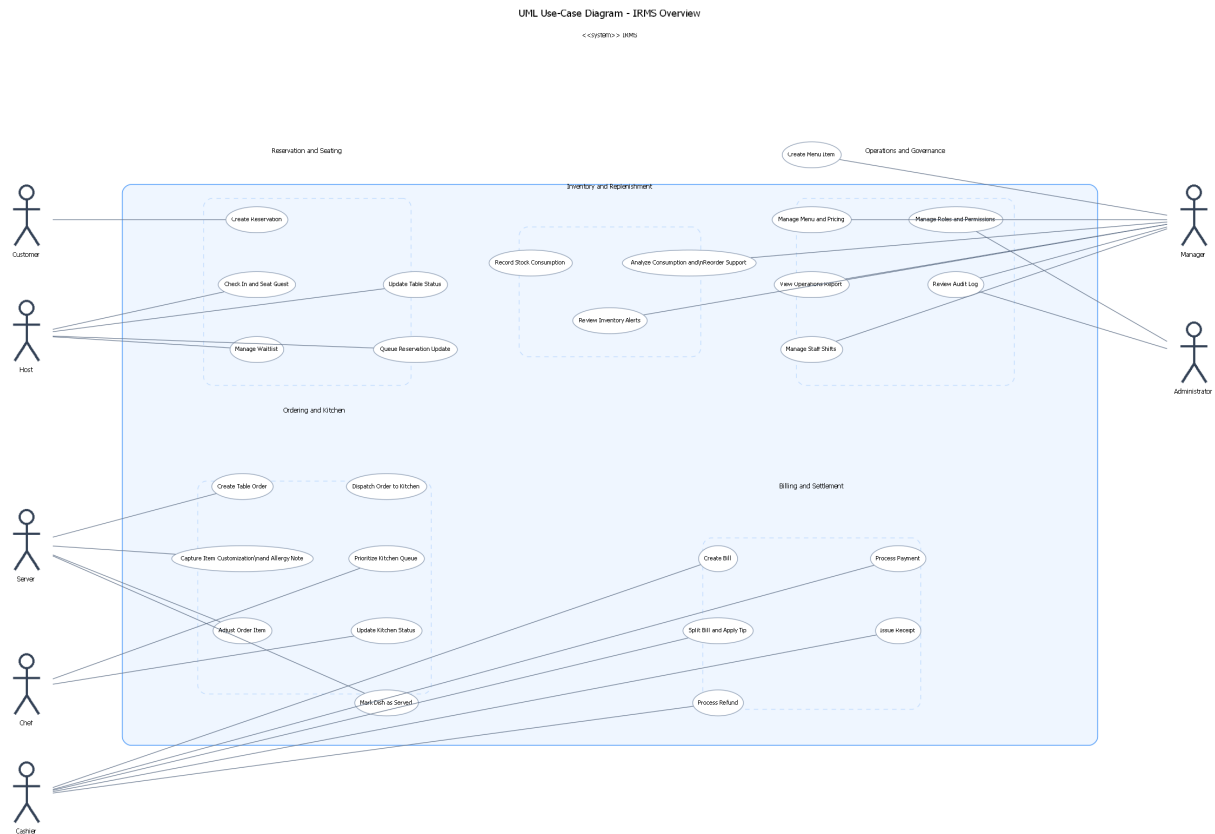
2.1 Danh mục use cases



Hình 1: Sơ đồ ngữ cảnh của IRMS

Hình 1 thể hiện *context/process diagram*: IRMS ở trung tâm điều phối vận hành, các vai trò con người nằm ngoài ranh giới này.

- **Các vai trò vận hành:** Gồm Customer, Host / Reception, Server / Floor Staff, Kitchen Operations, Cashier, Manager và System Administrator. Đây là những điểm tương tác và quyết định yêu cầu phản hồi của hệ thống.
- **Các vai trò không phải hệ thống ngoài:** các kênh thanh toán, thông báo và biên lai được xem là adapter biên nội bộ.
- **Các luồng nghiệp vụ:** như “reservation request”, “kitchen ticket”, “transaction state” hay “dấu vết kiểm toán”. Trách nhiệm của IRMS ở mức trao đổi dữ liệu, không chỉ ở mức vai trò sử dụng.
- **Ranh giới trách nhiệm:** IRMS chịu trách nhiệm điều phối đặt bàn, gọi món, bếp, hóa đơn, tồn kho và quản trị; các chi tiết như thanh toán, thông báo và biên lai được xử lý qua service hoặc adapter biên nội bộ.

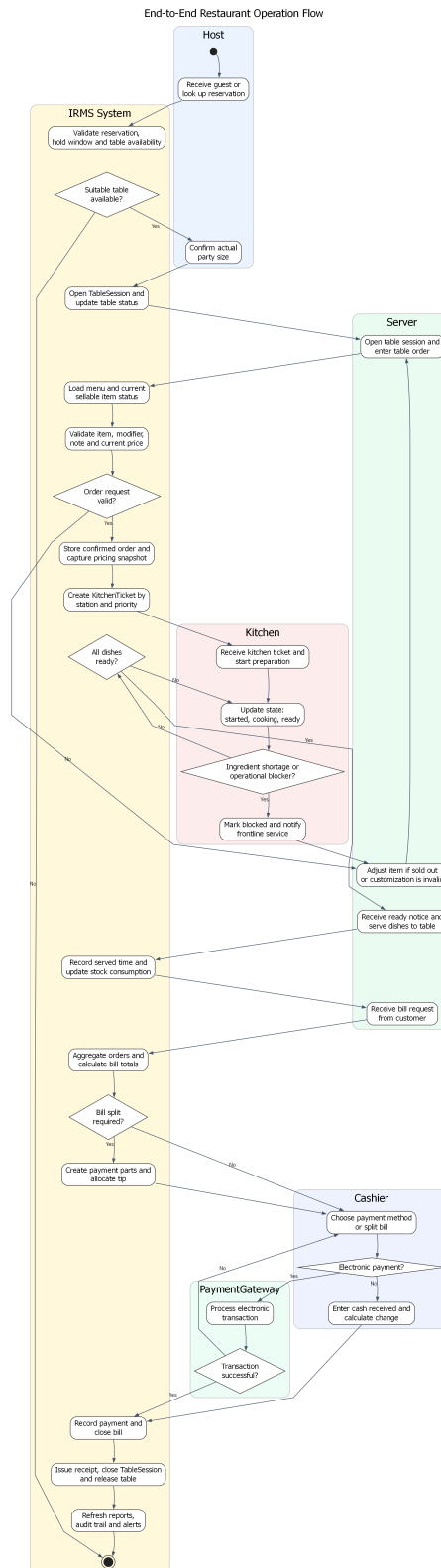


Hình 2: Sơ đồ tổng quan use cases của IRMS

Hình 2 thể hiện sơ đồ use cases tổng quát. Các use cases được gom theo các cụm lớn bám theo nghiệp vụ. Gồm các khối Reservation and Seating, Ordering and Kitchen, Billing and Settlement, Inventory and Replenishment Support và Operations and Governance.

Các cụm chức năng.

- **Reservation and Seating:** Gồm đặt bàn, tiếp nhận khách, danh sách chờ, giữ chỗ và mở phiên phục vụ tại bàn. Thuộc service Reservation & Seating Service.
- **Ordering and Kitchen:** Gồm tạo đơn, điều phối phiếu bếp, cập nhật tiến độ chế biến, xử lý ưu tiên và xác nhận món đã phục vụ. Thuộc service Ordering & Menu Service và Kitchen Coordination Service.
- **Billing and Settlement:** Gồm tạo hóa đơn, tách hóa đơn, xử lý thanh toán, hoàn tiền và phát hành biên lai; Thuộc service Billing & Settlement Service.
- **Inventory and Replenishment Support:** Gồm ghi nhận tiêu hao, cảnh báo mức tồn thấp và hỗ trợ quyết định nhập thêm nguyên liệu từ dữ liệu lịch sử. Thuộc service Inventory Monitoring Service.
- **Governance and Analytics:** Gồm quản trị thực đơn, phân quyền, kiểm toán, báo cáo và điều hành vận hành; đây là phạm vi của Reporting Service và IAM & Audit Service.



Hình 3: Sơ đồ hoạt động của luồng vận hành đầu-cuối nhà hàng

Hình 3 mô tả một vòng vận hành điển hình của nhà hàng, từ tiếp nhận khách đến giải phóng bàn sau thanh toán. Một bàn chỉ được có một TableSession đang mở, một đơn đã xác nhận phải sinh phiếu cho bếp, và một hóa đơn chỉ được đóng sau khi thanh toán được ghi nhận hợp lệ.

Ở góc nhìn kiến trúc, sơ đồ tách rõ **đường giao dịch nóng** khỏi **đường hỗ trợ**. Các thao tác như xác nhận đơn,

cập nhật trạng thái món sẵn sàng, ghi nhận thanh toán và đóng phiên bàn là các điểm phải phản hồi dứt khoát qua gọi đồng bộ giữa các biên phù hợp. Ngược lại, các tác vụ như làm mới báo cáo, phát cảnh báo mức tồn thấp, đồng bộ trạng thái phụ trợ hay gửi thông báo có thể đi trên tuyến bất đồng bộ theo hướng event-driven nếu điều đó không làm sai trạng thái phục vụ trước mặt khách hàng.

Bảng 5: Danh mục 26 use cases chính được đặc tả chi tiết

Nhóm	Mã use case
Nhóm đặt bàn và bàn ăn	UC-RES-01, UC-RES-02, UC-RES-03, UC-RES-04, UC-RES-05
Nhóm gọi món và điều phối bếp	UC-ORD-01, UC-ORD-02, UC-ORD-03, UC-KIT-01, UC-KIT-02, UC-KIT-03, UC-ORD-04
Nhóm thanh toán	UC-BIL-01, UC-BIL-02, UC-PAY-01, UC-PAY-02, UC-PAY-03
Nhóm tồn kho, cảnh báo và hỗ trợ nhập hàng	UC-INV-01, UC-INV-02, UC-INV-03
Nhóm quản trị, kiểm toán và phân tích	UC-REP-01, UC-ADM-01, UC-ADM-02, UC-ADM-03, UC-ADM-04, UC-OPS-01

2.2 Góc nhìn sử dụng

2.2.1 Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn

1. Khách hàng hoặc lễ tân tạo đặt bàn.
2. Reservation & Seating Service kiểm tra khả dụng, gán bàn hoặc giữ chỗ và ghi nhận ReservationCreated nếu cần phát thông báo.
3. Khi khách đến, Host UI xác nhận tiếp nhận và mở TableSession.
4. Trạng thái bàn được cập nhật qua TableStatusChanged để tuyến phục vụ nhìn thấy bàn sẵn sàng nhận order.

2.2.2 Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ

1. Phục vụ tạo đơn gọi món và cấu hình tùy chọn món hoặc gợi ý dự ứng trên Server POS UI.
2. Ordering & Menu Service xác nhận đơn gọi món và phát OrderPlaced.
3. Kitchen Coordination Service nhận order, tách phiếu theo trạm bếp và hiển thị lên Kitchen Display UI.
4. Bếp cập nhật trạng thái từng dòng phiếu qua các mốc như OrderItemStarted và OrderReady; màn hình gọi món nhận lại trạng thái sẵn sàng.
5. Phục vụ giao món và đánh dấu đã phục vụ; Inventory Monitoring Service nhận sự kiện liên quan để ghi nhận tiêu hao và cập nhật kho.

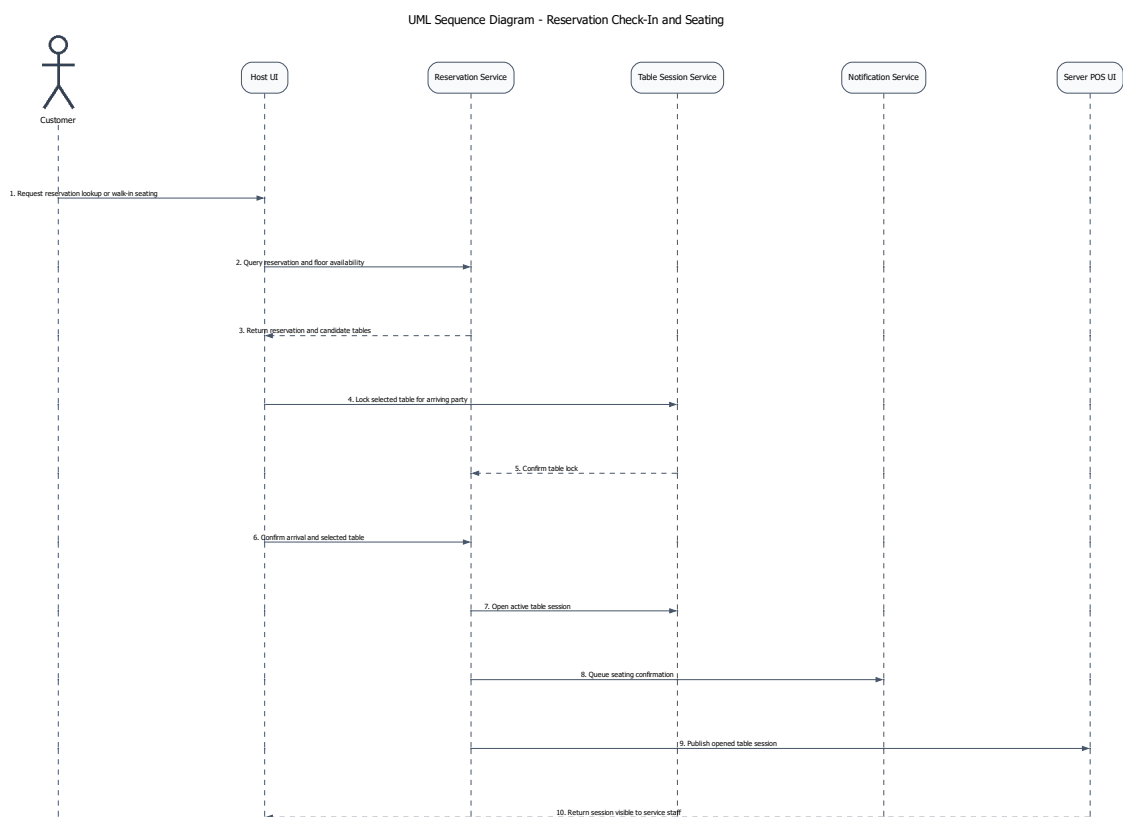
2.2.3 Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai

1. Thu ngân hoặc phục vụ gom các đơn gọi món của bàn thành hóa đơn.
2. Billing & Settlement Service áp dụng khuyến mãi, tiền boa và tách hóa đơn nếu cần.
3. Thanh toán được thực hiện qua tiền mặt hoặc qua PaymentGateway.

4. Khi thanh toán đủ, hệ thống ghi nhận PaymentCompleted, phát hành biên lai và chuyển TableSession sang chuẩn bị đóng.

2.2.4 Kịch bản 4: Từ cảnh báo tồn kho thấp đến quyết định của quản lý

1. Inventory Monitoring Service cập nhật lượng tồn kho thực tế sau mỗi món đã phục vụ hoặc sau mỗi lần điều chỉnh thủ công.
2. Nếu xuống dưới ngưỡng, hệ thống phát StockLow.
3. Manager Console nhận cảnh báo, hiển thị nguyên liệu bị ảnh hưởng và món có nguy cơ hết hàng.
4. Quản lý quyết định nhập thêm hàng hoặc tạm ẩn món trong menu.



Hình 4: Sơ đồ trình tự cho kịch bản từ đặt bàn đến tiếp nhận và xếp bàn

Hình 4 thể hiện luồng hoạt động từ đặt bàn đến tiếp nhận và xếp bàn: Reservation đại diện cho đặt chỗ trước khi khách ngồi vào bàn; TableSession đại diện cho phiên phục vụ đã thực sự được mở. Việc tách rõ ràng giúp hệ thống xử lý đúng các tình huống thường gặp như no-show, walk-in, đổi bàn do thay đổi sức chứa hoặc check-in muộn so với khung giữ chỗ.

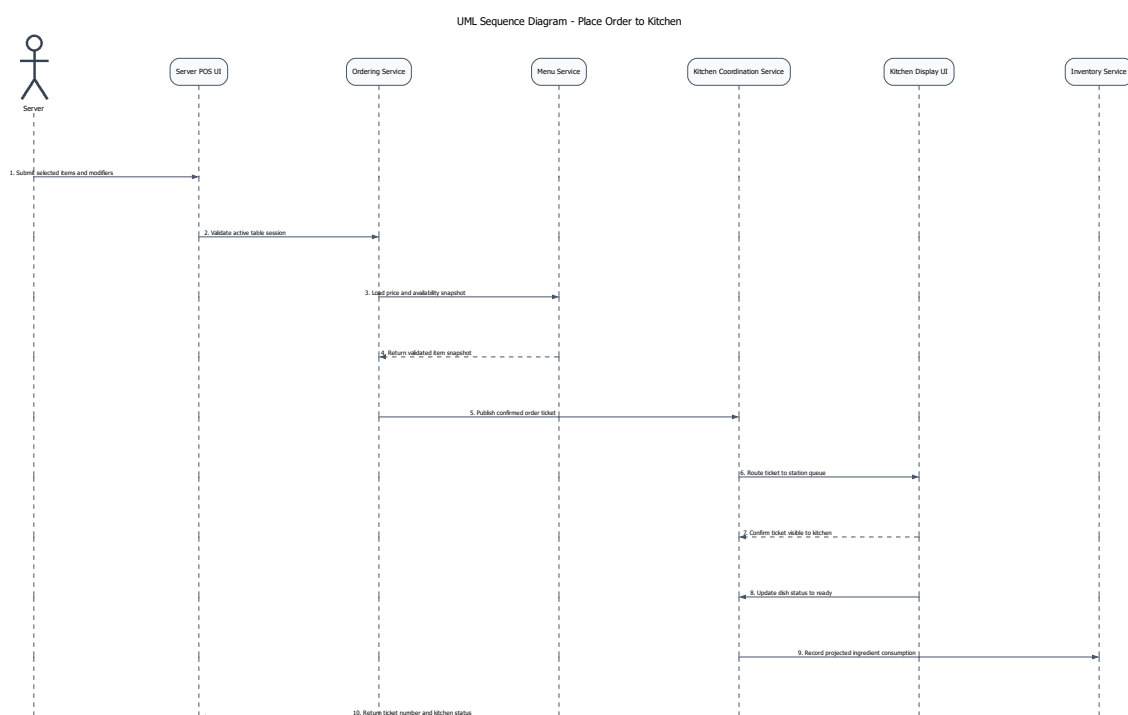
Các điểm trong sơ đồ

- **Host UI và Reservation & Seating Service** chịu trách nhiệm kiểm tra khả dụng, giữ chỗ và xác nhận tiếp nhận khách.

- **Reservation & Seating Service** là ranh giới nơi bàn được chuyển từ trạng thái chờ sang trạng thái đang phục vụ thông qua TableSession.
- **Notification Service** và **Server POS UI** chỉ được kích hoạt sau khi phiên bàn đã được mở thành công.

Ý nghĩa

- Phải có cơ chế chống mở trùng TableSession cho cùng một bàn hoặc cùng một lượt tiếp nhận.
- Trạng thái bàn và trạng thái đặt bàn không nên bị ép gộp thành một thực thể duy nhất, nếu không các chuyển tiếp quan trọng của vận hành sẽ trở nên khó khăn.



Hình 5: Sơ đồ trình tự cho kịch bản từ gọi món đến điều phối bếp và phục vụ

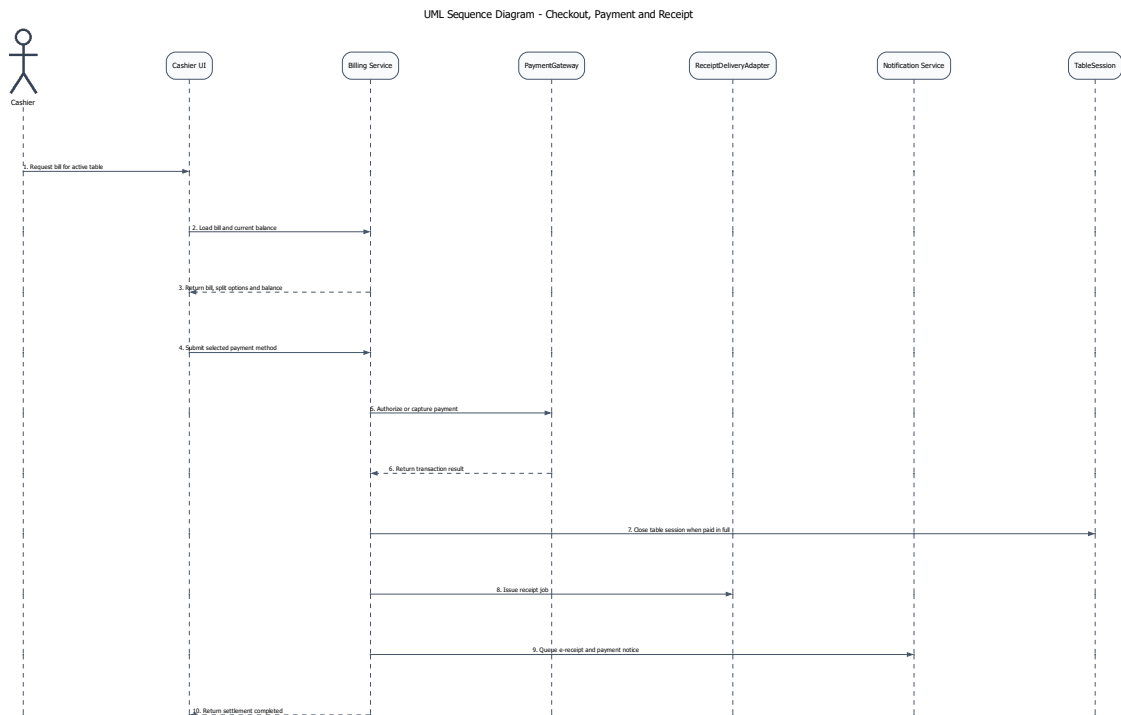
Hình 5 là sơ đồ kiểm tra **khả năng đáp ứng** của IRMS. Nguyên tắc chính là *xác nhận đơn thành công trước, rồi mới lan truyền sang bếp và các luồng hỗ trợ*. Giúp rút ngắn thời gian phản hồi tại màn hình gọi món và bảo vệ đơn đã chốt khi một thành phần phía sau như KDS hoặc đồng bộ trạng thái gặp trục trặc tạm thời.

Các điểm trong sơ đồ.

- **Ordering & Menu Service** là ranh giới chốt giao dịch gọi món;
- **Kitchen Coordination Service** và **Kitchen Display UI** xử lý phần chuyển vận hành sau giao dịch, bao gồm định tuyến phiếu và phản hồi trạng thái chế biến.
- **Menu Service** lưu giữ trạng thái khả dụng và giá tại thời điểm xác nhận đơn, giúp tránh nhập nhằng khi menu thay đổi sau đó.
- **Inventory Monitoring Service** chỉ tham gia ở nhánh hỗ trợ, không chặn bước xác nhận đơn ở tuyến đầu.

Ý nghĩa

- Đơn gọi món, phiếu bếp và cập nhật trạng thái không nên bị ràng buộc trong cùng một bước đồng bộ dài.
- Cơ chế chống lặp, correlation ID và khả năng thử lại là bắt buộc nếu kiến trúc muốn vừa nhanh vừa không làm mất dữ liệu đơn.



Hình 6: Sơ đồ trình tự cho kịch bản từ hóa đơn đến thanh toán và biên lai

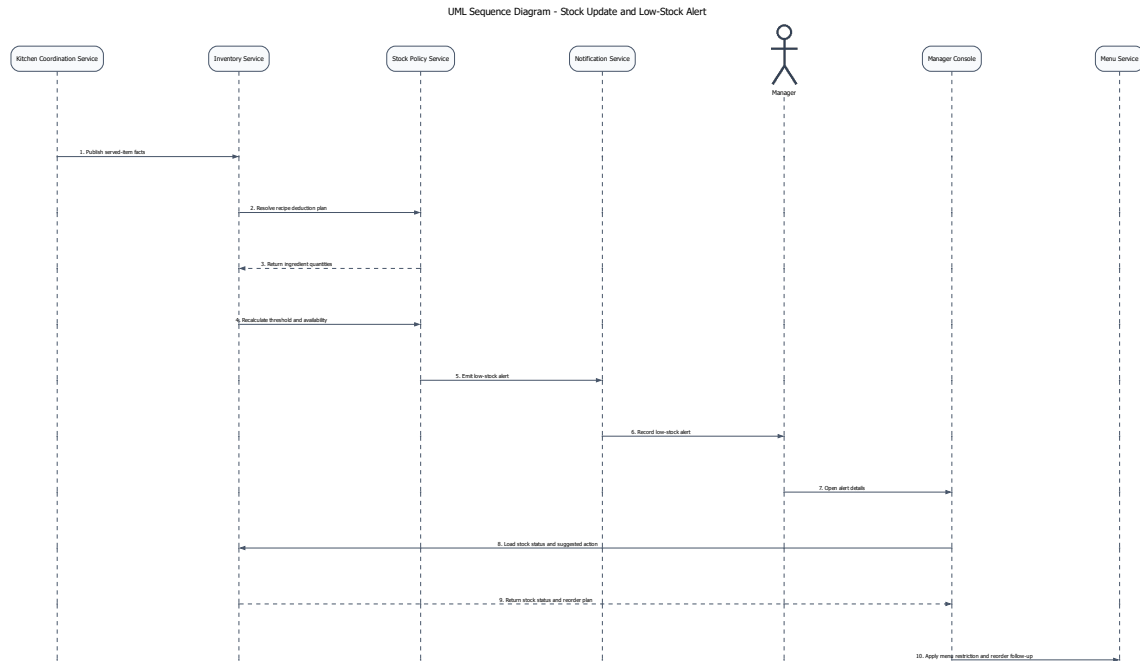
Hình 6 mô tả luồng thanh toán cần ưu tiên độ tin cậy và tính toàn vẹn giao dịch. Billing & Settlement Service xử lý tính tiền, khuyến mãi, thanh toán, phát hành biên lai và đóng phiên bản, nên cần cơ chế đồng bộ ngăn gọn để xác định rõ trạng thái giao dịch. Vì luồng này nhạy cảm với lỗi và độ trễ từ hệ thống ngoài, kiến trúc yêu cầu timeout rõ ràng, chống xử lý lặp, ghi kiểm toán và tách biệt adapter tích hợp để giảm rủi ro.

Các điểm trong sơ đồ.

- **Billing & Settlement Service** giữ vai trò bộ điều phối của tuyến thanh toán và đảm bảo idempotency cho các thao tác nhạy cảm.
- **PaymentGateway** đại diện cho port thanh toán, được tách khỏi logic thanh toán lõi.
- **ReceiptDeliveryAdapter**, **Notification Service** và **TableSession** chỉ được kích hoạt sau khi kết quả thanh toán đã rõ ràng.

Ý nghĩa

- PaymentGateway không nên được phép chi phối trực tiếp trạng thái hóa đơn mà không đi qua lớp điều phối của hệ thống.
- Việc đóng TableSession, phát hành biên lai và gửi e-receipt phải phụ thuộc vào cùng một kết quả thanh toán đã được xác nhận.



Hình 7: Sơ đồ trình tự cho kịch bản từ cảnh báo mức tồn thấp đến quyết định của quản lý

Hình 7 mô tả cập nhật tồn kho và phát cảnh báo được xử lý trên tuyến hỗ trợ bất đồng bộ để tránh làm chậm luồng gọi món và thanh toán. Kitchen Coordination Service, Inventory Monitoring Service, Stock Policy Service và Notification Service phối hợp qua event nhằm cập nhật tiêu hao và cảnh báo tồn kho. Đánh đổi là trạng thái tồn kho có thể cập nhật chậm hơn giao dịch thực tế, nhưng điều này chấp nhận được vì không ảnh hưởng đến các thao tác cần phản hồi tức thời.

Các điểm kiến trúc đọc được từ sơ đồ.

- **Kitchen Coordination Service** phát sinh dữ kiện phục vụ để tính tiêu hao nguyên liệu.
- **Inventory Monitoring Service** và **Stock Policy Service** cập nhật tồn kho và đánh giá ngưỡng cảnh báo.
- **Notification Service, Manager Console** và **Menu Service** xử lý cảnh báo và phản ứng quản trị sau đó.

Ý nghĩa

- Cập nhật tồn kho và cảnh báo nên được xử lý tách khỏi tuyến phản hồi thời gian thực của gọi món và thanh toán.
- Việc tách riêng tầng chính sách và tầng thông báo giúp hệ thống dễ thay đổi và mở rộng hơn.

2.3 Đặc tả use cases

Use Case ID	UC-RES-01
Use Case Name	Tạo đặt bàn
Primary Actors	Khách hàng, Lễ tân

Description	Tạo một đặt bàn hợp lệ cho ngày, giờ và số lượng khách cụ thể.
Trigger	Khách hàng gửi yêu cầu đặt bàn qua quầy lễ tân, điện thoại hoặc web/app.
Preconditions	- Nhà hàng có mở nhận đặt bàn trong khung giờ yêu cầu. - Có tối thiểu tên và số điện thoại khách hàng. - Sơ đồ bàn và năng lực phục vụ đã được cấu hình.
Postconditions	- Tạo đặt bàn với trạng thái Pending hoặc Confirmed. - Hệ thống giữ chỗ hoặc gợi ý bàn phù hợp. - Khách nhận mã đặt bàn và thông báo xác nhận.
Normal Flow	- Lễ tân nhập thời gian, số khách, tên và số điện thoại. - Hệ thống kiểm tra và validate dữ liệu đầu vào. - Hệ thống kiểm tra sức chứa, bàn trống và xung đột đặt bàn. - Hệ thống gợi ý bàn hoặc nhóm bàn phù hợp. - Lễ tân chọn bàn và nhập ghi chú nếu có. - Hệ thống tạo đặt bàn và sinh mã đặt bàn. - Hệ thống gửi thông báo xác nhận cho khách.
Alternative Flows	A1. Không đủ sức chứa (từ bước 3) 3.1 Hệ thống đề xuất khung giờ khác hoặc đưa khách vào danh sách chờ. A2. Yêu cầu đặc biệt (từ bước 4) 4.1 Lễ tân lọc danh sách bàn theo khu vực hoặc thuộc tính yêu cầu. A3. Không gán được bàn (từ bước 5) 5.1 Hệ thống tạo đặt bàn trạng thái Pending và giữ chỗ theo số lượng khách.
Exception Flows	E1. Dữ liệu không hợp lệ (từ bước 2) 2.1 Hệ thống yêu cầu nhập lại thông tin khách. E2. Lỗi đồng bộ sơ đồ bàn (từ bước 3) 3.2 Hệ thống khóa thao tác và yêu cầu tải lại dữ liệu.
Business Rules	- Không vượt quá sức chứa tối đa của bàn hoặc nhóm bàn. - Nhóm khách lớn phải được xác nhận bởi lễ tân hoặc quản lý.

Use Case ID	UC-RES-02
Use Case Name	Nhận khách và sắp bàn
Primary Actors	Lễ tân, Phục vụ

Description	Tiếp nhận khách đến nhà hàng và gán khách vào bàn hoặc nhóm bàn phù hợp.
Trigger	Khách đến quầy lễ tân hoặc được gọi từ danh sách chờ.
Preconditions	- Khách có đặt bàn hoặc có thể được xếp chỗ tại chỗ. - Sơ đồ bàn và trạng thái bàn được đồng bộ theo thời gian thực.
Postconditions	- Một phiên phục vụ bàn (TableSession) được mở. - Bàn được chuyển sang trạng thái Occupied hoặc Reserved-Arrived. - Phục vụ nhận được thông báo tiếp nhận bàn mới.
Normal Flow	- Lễ tân tìm đặt bàn theo mã, số điện thoại hoặc tên khách. - Hệ thống hiển thị thông tin đặt bàn, trạng thái, thời điểm đến và bàn gợi ý. - Lễ tân xác nhận khách đến, số khách thực tế và thời gian check-in. - Hệ thống kiểm tra hiệu lực đặt bàn và trạng thái bàn. - Lễ tân điều chỉnh hoặc ghép bàn nếu số lượng khách thay đổi. - Hệ thống mở phiên phục vụ (TableSession). - Hệ thống cập nhật trạng thái bàn thành Occupied hoặc Reserved-Arrived. - Hệ thống phân công bàn cho phục vụ theo khu vực. - Phục vụ nhận thông báo và chuẩn bị phục vụ.
Alternative Flows	A1. Khách vắng lai (từ bước 1) - Lễ tân chọn chức năng xếp bàn trực tiếp. - Hệ thống gợi ý bàn trống phù hợp. - Lễ tân chọn bàn và tiếp tục từ bước 5. A2. Thay đổi số lượng khách (từ bước 3) - Lễ tân cập nhật số khách thực tế. - Hệ thống gợi ý lại bàn phù hợp hoặc yêu cầu ghép bàn.
Exception Flows	E1. Bàn chưa sẵn sàng (từ bước 4) - Hệ thống đề xuất bàn thay thế hoặc đưa khách vào danh sách chờ. E2. Đặt bàn hết hiệu lực (từ bước 4) - Hệ thống yêu cầu xác nhận phục hồi từ lễ tân hoặc quản lý. E3. Lỗi đồng bộ sơ đồ bàn (từ bước 4) - Hệ thống tạm dừng thao tác và yêu cầu tải lại dữ liệu.
Business Rules	- Mỗi bàn chỉ có một TableSession hoạt động tại một thời điểm. - Không được gán bàn nếu bàn đang được sử dụng. - Thời gian giữ chỗ tối đa được cấu hình theo ca phục vụ.

Use Case ID	UC-RES-03
Use Case Name	Quản lý danh sách chờ
Primary Actors	Lễ tân
Description	Ghi nhận khách chờ bàn và điều phối khách khi có bàn phù hợp.
Trigger	Nhà hàng hết bàn phù hợp hoặc khách bị trì hoãn.
Preconditions	- Lễ tân có quyền truy cập danh sách chờ. - Hệ thống có khả năng ước tính thời gian chờ.
Postconditions	- Khách được thêm vào danh sách chờ với trạng thái Waiting. - Khách được gọi theo thứ tự ưu tiên khi có bàn phù hợp.
Normal Flow	- Lễ tân nhập thông tin khách, số lượng khách và kênh liên hệ. - Hệ thống ước tính thời gian chờ dựa trên tình trạng bàn. - Lễ tân xác nhận thêm khách vào danh sách chờ. - Khi có bàn trống, hệ thống tìm khách phù hợp theo sức chứa và ưu tiên. - Lễ tân chọn khách để gọi. - Hệ thống gửi thông báo và cập nhật trạng thái Called. - Lễ tân xác nhận khách đến và chuyển sang use case Nhận khách và sắp bàn.
Alternative Flows	A1. Ưu tiên khách (từ bước 4) 4.1 Lễ tân chọn khách có ưu tiên cao hơn (VIP, đặt trước) thay vì theo FIFO. A2. Cập nhật thông tin (từ bước 3) 3.1 Lễ tân cập nhật số điện thoại, số lượng khách hoặc hủy yêu cầu.
Exception Flows	E1. Khách không phản hồi (từ bước 6) 6.1 Nếu khách không phản hồi trong thời gian quy định, hệ thống chuyển trạng thái sang Skipped. E2. Thời gian chờ thay đổi (từ bước 2) 2.1 Nếu thời gian chờ tăng đột biến, hệ thống yêu cầu lễ tân xác nhận lại trước khi hiển thị.
Business Rules	- Thứ tự mặc định là FIFO trong cùng nhóm sức chứa. - Có thể override thứ tự theo ưu tiên. - Chỉ gọi khách khi có bàn phù hợp với số lượng và yêu cầu. - Thời gian giữ lượt gọi được cấu hình (timeout).

Use Case ID	UC-RES-04
--------------------	-----------

Use Case Name	Cập nhật trạng thái bàn và giải phóng bàn
Primary Actors	Lễ tân, Phục vụ, Thu ngân
Description	Cập nhật trạng thái bàn, đóng phiên phục vụ và giải phóng bàn cho lượt khách tiếp theo.
Trigger	Khách rời bàn, hoàn tất phục vụ hoặc cần thay đổi trạng thái bàn.
Preconditions	- Bàn tồn tại trong hệ thống. - Người thao tác có quyền cập nhật trạng thái. - Nếu có TableSession, hệ thống truy xuất được trạng thái hóa đơn.
Postconditions	- Trạng thái bàn được cập nhật nhất quán. - TableSession được đóng nếu đủ điều kiện. - Lịch sử thay đổi được ghi nhận.
Normal Flow	- Người dùng chọn bàn trên sơ đồ bàn. - Hệ thống hiển thị trạng thái bàn, phiên phục vụ và hóa đơn liên quan. - Người dùng chọn trạng thái đích (Cleaning, Available, OutOfService). - Nếu chuyển sang trạng thái giải phóng, hệ thống kiểm tra hóa đơn hoặc split còn mở. - Nếu tất cả hóa đơn đã hoàn tất, hệ thống đóng TableSession và chuyển bàn sang Cleaning. - Phục vụ xác nhận đã dọn bàn xong. - Hệ thống chuyển bàn sang trạng thái Available. - Hệ thống cập nhật danh sách chờ và gợi ý khách tiếp theo nếu có.
Alternative Flows	A1. Chuyển sang OutOfService (từ bước 3) 3.1 Người dùng chuyển bàn sang trạng thái OutOfService để bảo trì hoặc khóa khu vực. A2. Chuyển bàn trong khi phục vụ (từ bước 3) 3.2 Nếu khách đổi bàn, hệ thống cập nhật bàn mới nhưng giữ nguyên TableSession.
Exception Flows	E1. Chưa thanh toán (từ bước 4) 4.1 Hệ thống từ chối giải phóng bàn và yêu cầu hoàn tất thanh toán. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện cập nhật đồng thời và yêu cầu tải lại dữ liệu. E3. Không đủ quyền (từ bước 3) 3.3 Hệ thống chặn thao tác nếu người dùng không có quyền.

Business Rules	- Chỉ chuyển sang Available khi không còn TableSession và hóa đơn mở. - Mọi thay đổi trạng thái phải được ghi log (thời gian, người thực hiện). - Không được giải phóng bàn khi còn công nợ.
-----------------------	--

Use Case ID	UC-RES-05
Use Case Name	Gửi thông báo cho khách hàng
Primary Actors	Hệ thống, Lễ tân
Description	Gửi thông báo cho khách hàng khi có sự kiện như đặt bàn, thay đổi hoặc đến lượt chờ.
Trigger	Sự kiện nghiệp vụ phát sinh hoặc lễ tân kích hoạt gửi thủ công.
Preconditions	- Có thông tin liên hệ hợp lệ. - Mẫu thông báo và kênh gửi đã được cấu hình. - Hệ thống có kênh thông báo nội bộ qua NotificationChannelAdapter.
Postconditions	- Tạo bản ghi NotificationMessage. - Thông báo được gửi hoặc đưa vào hàng chờ. - Lịch sử gửi được lưu lại.
Normal Flow	- Hệ thống nhận sự kiện cần gửi thông báo. - Hệ thống xác định loại thông báo và chọn mẫu nội dung phù hợp. - Hệ thống kiểm tra thông tin liên hệ và kênh gửi ưu tiên. - Hệ thống cá nhân hóa nội dung thông báo. - Hệ thống tạo NotificationMessage và gửi qua kênh đã chọn. - Hệ thống ghi nhận kết quả và cập nhật trạng thái gửi.
Alternative Flows	A1. Gửi thủ công (từ bước 1) 1.1 Lễ tân kích hoạt gửi lại thông báo cho khách. A2. Chuyển kênh gửi (từ bước 3) 3.1 Nếu kênh chính không khả dụng, hệ thống chuyển sang kênh dự phòng.
Exception Flows	E1. Thiếu thông tin liên hệ (từ bước 3) 3.2 Hệ thống đánh dấu cần xử lý thủ công. E2. Lỗi dịch vụ gửi (từ bước 5) 5.1 Hệ thống đưa thông báo vào hàng chờ retry. E3. Bị chặn spam (từ bước 5) 5.2 Hệ thống bỏ qua gửi trùng và ghi nhận lý do.

Business Rules	- Mỗi thông báo phải liên kết với Reservation hoặc WaitlistEntry. - Thông báo có thời hạn phải kèm thời gian phản hồi rõ ràng. - Không gửi trùng thông báo trong khoảng thời gian cấu hình.
-----------------------	---

Use Case ID	UC-ORD-01
Use Case Name	Tạo đơn gọi món tại bàn
Primary Actors	Phục vụ
Description	Tạo đơn gọi món cho một bàn đang phục vụ, bao gồm món, số lượng và ghi chú.
Trigger	Phục vụ chọn chức năng tạo đơn gọi món trên bàn đang hoạt động.
Preconditions	- TableSession đang hoạt động. - Phục vụ có quyền thao tác trên khu vực.
Postconditions	- Đơn gọi món được lưu với trạng thái Draft hoặc Confirmed. - Các dòng món được liên kết với phiên phục vụ.
Normal Flow	- Phục vụ mở bàn và chọn chức năng tạo đơn gọi món. - Hệ thống kiểm tra trạng thái TableSession. - Hệ thống hiển thị thực đơn và danh sách món. - Phục vụ chọn món, số lượng và ghi chú cho từng dòng món. - Hệ thống tính tạm tính theo giá hiện hành. - Phục vụ lưu đơn ở trạng thái Draft hoặc xác nhận gửi bếp. - Hệ thống lưu đơn gọi món và cập nhật trạng thái.
Alternative Flows	A1. Tạo nhiều đơn (từ bước 1) 1.1 Phục vụ tạo nhiều đơn gọi món liên tiếp trong cùng một phiên phục vụ. A2. Dùng mẫu đơn (từ bước 3) 3.1 Phục vụ chọn mẫu đơn có sẵn cho bàn đoàn hoặc combo.
Exception Flows	E1. Món hết hàng (từ bước 4) 4.1 Hệ thống từ chối thêm món và yêu cầu chọn món thay thế. E2. Phiên bàn không hợp lệ (từ bước 2) 2.1 Hệ thống từ chối tạo đơn nếu TableSession đã đóng.
Business Rules	- Mỗi đơn phải thuộc một TableSession. - Giá món được cố định tại thời điểm xác nhận đơn. - Một bàn có thể có nhiều đơn trong cùng phiên phục vụ.

Use Case ID	UC-ORD-02
Use Case Name	Tùy biến món và ghi chú dị ứng
Primary Actors	Phục vụ, Bếp
Description	Ghi nhận tùy chọn món, mức chế biến và ghi chú dị ứng cho từng dòng món.
Trigger	Phục vụ chọn chỉnh sửa chi tiết một dòng món trong đơn gọi món.
Preconditions	- Món hỗ trợ tùy chọn hoặc ghi chú. - Đơn gọi món ở trạng thái Draft hoặc chưa gửi bếp. - Cấu hình modifier đã được thiết lập.
Postconditions	- Tùy chọn được lưu dưới dạng OrderItemModifier. - Ghi chú dị ứng được gắn vào dòng món.
Normal Flow	- Phục vụ mở chi tiết dòng món. - Hệ thống hiển thị các nhóm tùy chọn và giới hạn chọn. - Phục vụ chọn các tùy biến cho món. - Phục vụ nhập ghi chú dị ứng hoặc yêu cầu đặc biệt. - Hệ thống kiểm tra quy tắc chọn (min/max). - Hệ thống lưu cấu hình tùy chọn vào dòng món. - Khi đơn được gửi bếp, thông tin tùy biến và ghi chú được hiển thị.
Alternative Flows	A1. Dừng cấu hình mẫu (từ bước 2) 2.1 Phục vụ chọn nhanh cấu hình có sẵn cho món. A2. Gắn cờ dị ứng (từ bước 4) 4.1 Hệ thống tự động đánh dấu nổi bật các ghi chú dị ứng quan trọng.
Exception Flows	E1. Vi phạm quy tắc chọn (từ bước 5) 5.1 Hệ thống yêu cầu điều chỉnh số lượng lựa chọn. E2. Ghi chú quá dài (từ bước 4) 4.2 Hệ thống yêu cầu rút gọn hoặc chuyển sang ghi chú nội bộ. E3. Đơn đã gửi bếp (từ bước 1) 1.1 Hệ thống từ chối chỉnh sửa nếu đơn đã được xác nhận.
Business Rules	- Ghi chú dị ứng phải hiển thị rõ ràng trên phiếu bếp. - Modifier có thể thay đổi giá món. - Không cho phép chỉnh sửa sau khi gửi bếp.

Use Case ID	UC-ORD-03
--------------------	-----------

Use Case Name	Gửi đơn gọi món sang bếp
Primary Actors	Phục vụ, Bếp
Description	Chuyển đơn gọi món đã xác nhận đến các trạm bếp để chế biến.
Trigger	Phục vụ nhấn gửi bếp hoặc hệ thống tự động gửi.
Preconditions	- Đơn có ít nhất một dòng món hợp lệ. - Đơn ở trạng thái Draft. - Kết nối tới hệ thống bếp khả dụng.
Postconditions	- Tạo một hoặc nhiều KitchenTicket. - Đơn chuyển sang trạng thái Confirmed/Queued.
Normal Flow	- Phục vụ xác nhận các dòng món cần gửi. - Hệ thống kiểm tra trạng thái món, modifier và tồn kho logic. - Hệ thống xác định trạm bếp cho từng dòng món. - Hệ thống tách đơn thành các KitchenTicket theo trạm bếp. - Hệ thống xếp hàng phiếu bếp theo ưu tiên và thời gian. - Hệ thống cập nhật trạng thái đơn thành Confirmed/Queued. - Màn hình bếp hiển thị phiếu và phát tín hiệu thông báo.
Alternative Flows	A1. Gửi món sau (từ bước 1) 1.1 Phục vụ đánh dấu một số món để gửi sau. A2. Gộp phiếu bếp (từ bước 4) 4.1 Hệ thống gộp các phiếu bếp gần nhau theo cấu hình thời gian.
Exception Flows	E1. Không có trạm bếp (từ bước 3) 3.1 Hệ thống chặn gửi và cảnh báo lỗi cấu hình menu. E2. Mất kết nối bếp (từ bước 5) 5.1 Hệ thống đưa phiếu vào hàng đợi và retry sau.
Business Rules	- Mỗi dòng món phải được định tuyến đến ít nhất một trạm bếp. - Chỉ dòng món hợp lệ mới được gửi. - Không cho phép gửi lại các dòng đã được gửi trước đó.

Use Case ID	UC-KIT-01
Use Case Name	Cập nhật tiến độ chế biến
Primary Actors	Bếp
Description	Theo dõi và cập nhật trạng thái món từ khi nhận phiếu bếp đến khi sẵn sàng phục vụ.

Trigger	Nhân viên bếp chọn một phiếu bếp trên màn hình KDS.
Preconditions	- KitchenTicket đã được tạo và gán trạm bếp. - Nhân viên đã đăng nhập vào hệ thống bếp.
Postconditions	- Trạng thái món và phiếu bếp được cập nhật. - Màn hình phục vụ nhận được tiến độ mới nhất.
Normal Flow	- Nhân viên bếp mở phiếu bếp ở trạng thái Queued. - Nhân viên cập nhật trạng thái từng món (Started, Cooking, Ready). - Hệ thống ghi nhận thời gian cho từng trạng thái. - Hệ thống so sánh thời gian chế biến với SLA. - Khi tất cả món hoàn tất, hệ thống chuyển phiếu sang Ready. - Hệ thống gửi thông báo đến màn hình phục vụ để ra món.
Alternative Flows	A1. Ưu tiên phiếu (từ bước 1) 1.1 Nhân viên bếp ưu tiên xử lý phiếu VIP hoặc món cần ra trước. A2. Giữ món (từ bước 2) 2.1 Nhân viên đánh dấu Hold for service để chờ đồng bộ với món khác.
Exception Flows	E1. Thiếu nguyên liệu (từ bước 2) 2.2 Nhân viên đánh dấu Blocked và gửi yêu cầu thay món. E2. Mất kết nối KDS (từ bước 3) 3.1 Hệ thống chuyển sang hàng đợi cục bộ và đồng bộ lại sau.
Business Rules	- Chỉ trạm bếp được phân công mới được cập nhật trạng thái. - Mọi thay đổi trạng thái phải được ghi log. - Trạng thái phiếu phụ thuộc vào trạng thái tất cả món.

Use Case ID	UC-KIT-02
Use Case Name	Ra món cho bàn
Primary Actors	Phục vụ
Description	Nhận món từ bếp và phục vụ đúng bàn, đúng thứ tự.
Trigger	Hệ thống thông báo món hoặc phiếu bếp ở trạng thái Ready.
Preconditions	- Món ở trạng thái Ready. - TableSession còn hoạt động.

Postconditions	- Dòng món được cập nhật trạng thái Served. - Thời gian phục vụ được ghi nhận.
Normal Flow	- Phục vụ mở danh sách món sẵn sàng. - Phục vụ xác nhận nhận món từ bếp. - Hệ thống hiển thị thông tin bàn, vị trí ngồi và ghi chú. - Phục vụ giao món cho khách. - Phục vụ xác nhận “Đã phục vụ”. - Hệ thống cập nhật trạng thái từng dòng món thành Served. - Nếu tất cả món đã phục vụ, hệ thống cập nhật trạng thái phục vụ của bàn.
Alternative Flows	A1. Xác nhận theo phiếu (từ bước 1) 1.1 Phục vụ xác nhận toàn bộ phiếu thay vì từng món. A2. Giữ món theo course (từ bước 3) 3.1 Phục vụ trì hoãn ra món để đồng bộ theo course.
Exception Flows	E1. Sai món hoặc sai tùy biến (từ bước 3) 3.2 Phục vụ trả món về bếp và ghi lý do. E2. Bàn tạm vắng (từ bước 4) 4.1 Phục vụ đánh dấu trì hoãn phục vụ. E3. Trạng thái không hợp lệ (từ bước 5) 5.1 Hệ thống từ chối nếu món chưa ở trạng thái Ready.
Business Rules	- Chỉ món ở trạng thái Ready mới được phục vụ. - Mọi thao tác trả món phải có lý do. - Trạng thái phục vụ phải được ghi log để phân tích SLA.

Use Case ID	UC-KIT-03
Use Case Name	Ưu tiên hàng chờ bếp và xử lý món gấp
Primary Actors	Bếp, Quản lý
Description	Điều chỉnh mức ưu tiên của ticket hoặc món để xử lý các trường hợp gấp và giảm trễ hạn.
Trigger	Ticket gần vượt SLA, khách VIP hoặc yêu cầu can thiệp từ quản lý.
Preconditions	- KitchenTicket hoặc item tồn tại trong hàng chờ. - Có dữ liệu SLA và thứ tự hàng chờ. - Người thao tác có quyền thay đổi ưu tiên.

Postconditions	- Mức ưu tiên được cập nhật. - Hàng chờ được sắp xếp lại. - Lý do thay đổi được ghi log.
Normal Flow	- Nhân viên mở hàng chờ của trạm bếp. - Hệ thống hiển thị độ trễ và các ticket có nguy cơ trễ. - Người thao tác chọn ticket hoặc món cần ưu tiên. - Hệ thống áp dụng quy tắc ưu tiên (SLA, VIP, course). - Người thao tác xác nhận hoặc nhập lý do ưu tiên. - Hệ thống cập nhật mức ưu tiên. - Hệ thống sắp xếp lại hàng chờ. - Hệ thống làm nổi bật ticket ưu tiên trên KDS.
Alternative Flows	A1. Tự động ưu tiên (từ bước 2) 2.1 Hệ thống tự động tăng ưu tiên nếu ticket gần vượt SLA. A2. Ưu tiên theo món (từ bước 3) 3.1 Người thao tác chỉ ưu tiên một phần của ticket (ví dụ món khai vị).
Exception Flows	E1. Không đủ quyền (từ bước 3) 3.2 Hệ thống yêu cầu xác nhận từ quản lý. E2. Quá tải trạm bếp (từ bước 2) 2.2 Hệ thống cảnh báo và gợi ý điều phối sang trạm khác. E3. Thiếu dữ liệu SLA (từ bước 4) 4.1 Hệ thống chỉ cho phép ưu tiên thủ công.
Business Rules	- Mọi hành động ưu tiên phải có lý do. - Không được gây trì hoãn vô hạn cho ticket khác (anti-starvation). - Ưu tiên có thể áp dụng ở mức ticket hoặc item.

Use Case ID	UC-ORD-04
Use Case Name	Sửa hoặc hủy dòng món
Primary Actors	Phục vụ, Quản lý
Description	Điều chỉnh hoặc hủy dòng món trong đơn gọi món theo trạng thái và quyền hạn.
Trigger	Phục vụ chọn dòng món để sửa hoặc hủy.
Preconditions	- Đơn chưa hoàn tất thanh toán. - Người dùng có quyền chỉnh sửa theo trạng thái món.

Postconditions	- Dòng món được cập nhật hoặc chuyển trạng thái Voided. - Lịch sử thay đổi được ghi nhận. - Nếu đã gửi bếp, hệ thống phát thông báo đến KDS.
Normal Flow	- Phục vụ chọn dòng món cần chỉnh sửa. - Hệ thống hiển thị trạng thái món và quyền chỉnh sửa. - Phục vụ chọn sửa thông tin hoặc hủy dòng món. - Nếu món đã gửi bếp, hệ thống yêu cầu nhập lý do. - Nếu cần, hệ thống yêu cầu phê duyệt từ quản lý. - Hệ thống cập nhật dữ liệu dòng món. - Hệ thống phát sự kiện đến KDS và cập nhật tạm tính.
Alternative Flows	A1. Chỉnh sửa trước khi gửi bếp (từ bước 3) 3.1 Phục vụ chỉnh sửa tự do khi món ở trạng thái Draft. A2. Hủy sau khi chế biến (từ bước 4) 4.1 Hệ thống yêu cầu phê duyệt bắt buộc khi món đã ở trạng thái Cooking hoặc Ready.
Exception Flows	E1. Không đủ quyền (từ bước 5) 5.1 Hệ thống từ chối thao tác nếu không có quyền. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện chỉnh sửa đồng thời và yêu cầu tải lại.
Business Rules	- Hủy món sau khi gửi bếp phải có lý do. - Có thể yêu cầu phê duyệt nếu vượt ngưỡng cấu hình. - Không được mất lịch sử trạng thái và giá ban đầu.

Use Case ID	UC-BIL-01
Use Case Name	Lập hóa đơn
Primary Actor	Thu ngân
Description	Use case này cho phép thu ngân tạo hóa đơn thanh toán cho bàn.
Preconditions	- Bàn đã có đơn gọi món. - Các món đã được ghi nhận trong hệ thống. - Thu ngân đã đăng nhập vào hệ thống.

Postconditions	- Hóa đơn được tạo và lưu trong hệ thống. - Tổng số tiền cần thanh toán được tính toán và hiển thị. - Hóa đơn ở trạng thái chờ thanh toán.
Main Flow	- Thu ngân chọn bàn cần thanh toán trong hệ thống. - Hệ thống hiển thị danh sách các món ăn, đồ uống và dịch vụ đã được gọi cho bàn đó. - Thu ngân kiểm tra thông tin hóa đơn và nhấn tiếp theo. - Hệ thống tính tổng tiền bao gồm các khoản phí và thuế (nếu có). - Thu ngân xác nhận lập hóa đơn. - Hệ thống tạo hóa đơn và hiển thị thông tin chi tiết. - Thu ngân tiến hành bước thanh toán.
Alternative Flows	A1. Điều chỉnh món trong hóa đơn: Tại bước 3, nếu thu ngân phát hiện sai sót trong danh sách món. 3a. Thu ngân yêu cầu điều chỉnh món (thay đổi số lượng hoặc hủy món). 3b. Hệ thống cập nhật lại danh sách món. 3c. Hệ thống quay lại bước 3 của luồng chính. A2. Hủy thao tác lập hóa đơn: Tại bước 5, nếu thu ngân quyết định không tiếp tục lập hóa đơn. 5a. Thu ngân chọn hủy thao tác. 5b. Hệ thống hủy quá trình lập hóa đơn và quay lại màn hình quản lý bàn.
Exception Flows	E1. Lỗi hệ thống khi tạo hóa đơn: 5a. Hệ thống không thể tạo hóa đơn do lỗi kỹ thuật. 5b. Hệ thống hiển thị thông báo: “Không thể tạo hóa đơn. Vui lòng thử lại sau.” 5c. Use case kết thúc. E2. Không tìm thấy dữ liệu đơn món: 2a. Hệ thống không tìm thấy dữ liệu đơn món của bàn đã chọn. 2b. Hệ thống hiển thị thông báo: “Không có dữ liệu để lập hóa đơn.” 2c. Hệ thống quay lại màn hình quản lý bàn.

Use Case ID	UC-BIL-02
Use Case Name	Tách hóa đơn và thêm tiền boa
Primary Actor	Thu ngân, Phục vụ
Description	Use case này cho phép nhân viên tách hóa đơn thành nhiều phần thanh toán và ghi nhận tiền boa theo yêu cầu của khách.
Trigger	Khách yêu cầu thanh toán riêng theo đầu người, theo món, hoặc theo số tiền.
Preconditions	- Hóa đơn đã được tạo trong hệ thống. - Hóa đơn chưa hoàn tất thanh toán.
Postconditions	- Các phần thanh toán (Bill Split) được tạo trong hệ thống. - Mỗi phần hóa đơn sẵn sàng để thực hiện thanh toán. - Tiền boa được ghi nhận theo cấu hình của nhân viên.
Main Flow	- Thu ngân mở hóa đơn cần tách. - Hệ thống hiển thị danh sách món, số ghế và tổng tiền của hóa đơn. - Thu ngân chọn phương thức chia hóa đơn (chia đều, theo món, theo ghế, theo số tiền hoặc kết hợp). - Thu ngân chọn mức tiền boa gợi ý. - Hệ thống tính lại số tiền của từng phần thanh toán. - Hệ thống hiển thị tổng tiền của từng phần và chênh lệch (nếu có). - Thu ngân xác nhận cấu hình tách hóa đơn. - Hệ thống tạo các phần hóa đơn tương ứng.
Alternative Flows	A1. Tách một phần món ra thanh toán trước: Tại bước 3. 3a. Thu ngân chọn các món cần tách ra thanh toán. 3b. Hệ thống tạo một phần hóa đơn mới cho các món đã chọn. 3c. Phần còn lại vẫn giữ trong hóa đơn gốc. A2. Áp dụng tiền boa cho từng phần hóa đơn: Tại bước 4. 4a. Thu ngân nhập tiền boa cho từng phần thanh toán. 4b. Hệ thống cập nhật tổng tiền của từng phần hóa đơn.

Exception Flows	E1. Tổng tiền các phần không khớp • 7a. Hệ thống phát hiện tổng các phần thanh toán không khớp với tổng hóa đơn. • 7b. Hệ thống hiển thị thông báo yêu cầu điều chỉnh lại cấu hình tách hóa đơn. • 7c. Hệ thống quay lại bước 3. E2. Phần hóa đơn đã thanh toán • 3a. Hệ thống phát hiện một phần hóa đơn đã được thanh toán. • 3b. Hệ thống không cho phép thay đổi phạm vi món trong phần đó. • 3c. Hệ thống yêu cầu hoàn tiền hoặc có phê duyệt trước khi chỉnh sửa.
------------------------	---

Use Case ID	UC-PAY-01
Use Case Name	Xử lý thanh toán
Primary Actor	Thu ngân
Description	Use case này cho phép thu ngân thực hiện thanh toán cho hóa đơn hoặc từng phần hóa đơn bằng các phương thức như tiền mặt, thẻ hoặc ví điện tử.
Trigger	Thu ngân chọn chức năng thanh toán trên một hóa đơn hoặc một phần hóa đơn trong hệ thống.
Preconditions	• Hóa đơn hoặc phần hóa đơn đã được tạo trong hệ thống. • Hóa đơn chưa hoàn tất thanh toán. • Thu ngân đã đăng nhập vào hệ thống.
Postconditions	• Một bản ghi thanh toán được lưu trong hệ thống. • Số tiền đã thanh toán của hóa đơn được cập nhật. • Nếu tổng tiền đã thanh toán đủ, hóa đơn được đánh dấu hoàn tất.

Main Flow	1. Thu ngân chọn hóa đơn hoặc phân hóa đơn cần thanh toán. 2. Hệ thống hiển thị tổng số tiền cần thanh toán. 3. Thu ngân chọn phương thức thanh toán. 4. Nếu thanh toán điện tử, hệ thống gửi yêu cầu xử lý giao dịch. 5. Nếu thanh toán tiền mặt, thu ngân nhập số tiền khách đưa. 6. Hệ thống tính tiền thối (nếu có). 7. Hệ thống ghi nhận giao dịch thanh toán và cập nhật số tiền còn lại của hóa đơn. 8. Nếu hóa đơn chưa thanh toán đủ, hệ thống cho phép tiếp tục thực hiện thanh toán. 9. Nếu hóa đơn đã thanh toán đủ, hệ thống đóng hóa đơn và hiển thị biên lai.
Alternative Flows	A1. Sử dụng tiền đặt cọc Tại bước 2 của Main Flow. • 2a. Hệ thống phát hiện hóa đơn có tiền đặt cọc từ trước. • 2b. Hệ thống hiển thị số tiền đặt cọc và số tiền còn lại cần thanh toán. • 2c. Hệ thống tự động trừ tiền đặt cọc vào tổng hóa đơn. • 2d. Thu ngân tiếp tục thanh toán phần tiền còn lại.
Exception Flows	E1. Thanh toán điện tử thất bại • 4a. Hệ thống nhận phản hồi giao dịch thất bại. • 4b. Hệ thống hiển thị thông báo lỗi thanh toán. • 4c. Thu ngân chọn phương thức thanh toán khác hoặc thử lại. E2. Thiết bị thanh toán không phản hồi • 4a. Hệ thống không nhận được phản hồi từ thiết bị thanh toán. • 4b. Hệ thống hiển thị thông báo lỗi. • 4c. Thu ngân chuyển sang phương thức thanh toán khác.

Use Case ID	UC-PAY-02
Use Case Name	Phát hành biên lai
Primary Actor	Thu ngân, Khách hàng

Description	Use case này cho phép hệ thống tạo và phát hành biên lai cho khách hàng sau khi thanh toán được ghi nhận thành công.
Trigger	Thanh toán của hóa đơn hoặc split bill được xác nhận thành công.
Preconditions	• Thanh toán đã được ghi nhận hợp lệ trong hệ thống. • Hệ thống đã cấu hình kênh phát hành biên lai qua ReceiptDeliveryAdapter (print, email hoặc sms ở mức adapter nội bộ).
Postconditions	• Biên lai (Receipt) được tạo và lưu trong hệ thống. • Biên lai được liên kết với hóa đơn và giao dịch thanh toán. • Yêu cầu phát hành biên lai được ghi nhận theo kênh đã chọn.
Main Flow	1. Hệ thống tạo nội dung biên lai gồm danh sách món, giảm giá, phí, phương thức thanh toán và thời gian. 2. Thu ngân chọn phương thức phát hành biên lai qua print adapter hoặc kênh điện tử. 3. Hệ thống ghi nhận yêu cầu phát hành biên lai theo lựa chọn. 4. Hệ thống lưu biên lai và đánh dấu trạng thái yêu cầu phát hành. 5. Nếu có thông tin khách hàng, hệ thống tạo yêu cầu phát hành bản sao biên lai qua email hoặc sms ở mức adapter nội bộ.
Alternative Flows	A1. Xuất biên lai rút gọn hoặc chi tiết • 2a. Thu ngân chọn loại biên lai (rút gọn hoặc chi tiết). • 2b. Hệ thống tạo biên lai theo định dạng tương ứng. A2. Biên lai theo split bill • 2a. Hệ thống phát hiện giao dịch thuộc split bill. • 2b. Mỗi split được tạo một biên lai riêng tương ứng.

Exception Flows	E1. Kênh phát hành biên lai không hợp lệ 3a. Hệ thống không tìm thấy ReceiptDeliveryAdapter phù hợp hoặc thiếu địa chỉ nhận đối với kênh điện tử. 3b. Hệ thống hiển thị thông báo lỗi và cho phép thu ngân chọn lại kênh phát hành. E2. Không tạo được yêu cầu gửi biên lai điện tử 5a. Hệ thống không tạo được yêu cầu gửi biên lai điện tử qua NotificationChannelAdapter. 5b. Hệ thống vẫn lưu biên lai và đánh dấu “chưa gửi”. 5c. Thu ngân có thể gửi lại thủ công sau.
------------------------	---

Use Case ID	UC-PAY-03
Use Case Name	Hoàn tiền
Primary Actor	Thu ngân, Quản lý
Description	Use case này cho phép hệ thống thực hiện hoàn tiền một phần hoặc toàn phần cho giao dịch thanh toán đã được ghi nhận trước đó.
Trigger	Khách yêu cầu hoàn tiền do hủy món, khiếu nại hoặc điều chỉnh hóa đơn sau thanh toán.
Preconditions	- Tồn tại giao dịch thanh toán thành công cần hoàn tiền. - Người thực hiện có quyền hoàn tiền hoặc được quản lý phê duyệt.
Postconditions	- Một bản ghi hoàn tiền (Refund) được tạo và liên kết với giao dịch gốc. - Hệ thống cập nhật lại số dư hóa đơn và trạng thái thanh toán. - Nhật ký kiểm toán ghi nhận người thực hiện, lý do và số tiền hoàn.
Main Flow	- Thu ngân chọn giao dịch thanh toán cần hoàn tiền. - Hệ thống hiển thị số tiền đã thanh toán và số tiền còn có thể hoàn. - Thu ngân nhập số tiền hoàn và lý do hoàn tiền. - Nếu vượt ngưỡng cho phép, hệ thống yêu cầu quản lý xác nhận. - Hệ thống xử lý hoàn tiền qua PaymentGateway hoặc ghi nhận hoàn tiền mặt. - Hệ thống cập nhật trạng thái hoàn tiền và số dư hóa đơn.

Alternative Flows	A1. Hoàn tiền một phần theo món • 3a. Thu ngân chọn các món cần hoàn tiền. • 3b. Hệ thống tính lại số tiền tương ứng. • 3c. Tiếp tục luồng hoàn tiền như bình thường. A2. Chuyển hoàn tiền thành tín dụng khách hàng • 4a. Quản lý chọn phương án không hoàn tiền trực tiếp. • 4b. Hệ thống ghi nhận số tiền vào ví nội bộ của khách hàng.
Exception Flows	E1. PaymentGateway từ chối hoàn tiền • 5a. Hệ thống gửi yêu cầu hoàn tiền thất bại. • 5b. Hệ thống ghi nhận trạng thái “Pending Review”. • 5c. Thông báo cho kế toán hoặc quản lý xử lý. E2. Số tiền hoàn vượt giới hạn • 3a. Hệ thống phát hiện số tiền hoàn lớn hơn mức còn lại. • 3b. Hệ thống chặn thao tác và yêu cầu nhập lại. E3. Giao dịch không còn đủ điều kiện hoàn tiền • 2a. Hệ thống kiểm tra giao dịch đã vượt quá thời hạn hoàn tiền được cấu hình (refund_window_hours, mặc định 24 giờ). • 2b. Hệ thống hiển thị thông báo và yêu cầu xử lý ngoại lệ.

Use Case ID	UC-INV-01
Use Case Name	Cập nhật tiêu hao tồn kho sau phục vụ
Primary Actor	Hệ thống, Quản lý kho
Description	Use case này mô tả việc hệ thống tự động khấu trừ tồn kho dựa trên món đã phục vụ, đồng thời cho phép quản lý kho thực hiện điều chỉnh thủ công khi cần thiết.
Trigger	Một món ăn được xác nhận đã phục vụ hoặc quản lý kho tạo giao dịch điều chỉnh tồn kho.
Preconditions	• Món ăn đã được ánh xạ công thức nguyên liệu (RecipeLine). • Hệ thống tồn kho đang hoạt động bình thường.
Postconditions	• Giao dịch kho (StockTransaction) được ghi nhận. • Số lượng tồn kho của nguyên liệu được cập nhật chính xác.

Main Flow	1. Hệ thống ghi nhận món ăn được phục vụ thành công. 2. Hệ thống tra cứu công thức nguyên liệu (RecipeLine) của món. 3. Hệ thống tính toán lượng nguyên liệu cần trừ theo số lượng món. 4. Hệ thống tạo giao dịch kho để khấu trừ tồn kho. 5. Quản lý kho có thể tạo giao dịch điều chỉnh thủ công nếu cần. 6. Hệ thống cập nhật số lượng tồn kho và lưu lịch sử giao dịch.
Alternative Flows	A1. Khấu trừ tồn kho khi gửi bếp • 2a. Hệ thống được cấu hình khấu trừ tại thời điểm gửi bếp thay vì phục vụ. • 2b. Tồn kho được trừ ngay khi món được chuyển sang bếp chế biến. A2. Ghi nhận hao hụt nguyên liệu đầu ca • 5a. Quản lý kho tạo giao dịch prep/waste đầu ca. • 5b. Hệ thống cập nhật tồn kho tương ứng.
Exception Flows	E1. Thiếu công thức nguyên liệu • 2a. Hệ thống không tìm thấy RecipeLine của món. • 2b. Hệ thống không thực hiện khấu trừ và tạo cảnh báo cho quản lý. E2. Xung đột cập nhật tồn kho • 4a. Hệ thống phát hiện đồng thời nhiều giao dịch gây sai lệch tồn kho. • 4b. Hệ thống tạm khóa cập nhật hoặc đưa vào hàng chờ xử lý.

Use Case ID	UC-INV-02
Use Case Name	Nhận cảnh báo sắp hết hàng
Primary Actor	Hệ thống, Quản lý, Bếp
Description	Use case này mô tả việc hệ thống tự động phát hiện nguyên liệu sắp hết khi tồn kho giảm xuống dưới ngưỡng cấu hình và gửi cảnh báo đến các bộ phận liên quan.
Trigger	Số lượng tồn kho của nguyên liệu giảm xuống thấp hơn ngưỡng cảnh báo (LowStockRule).
Preconditions	• Ngưỡng cảnh báo tồn kho đã được cấu hình cho từng nguyên liệu. • Hệ thống thông báo nội bộ đang hoạt động.

Postconditions	• Cảnh báo sắp hết hàng được tạo và gửi đến các vai trò liên quan. • Danh sách nguyên liệu cần bổ sung được cập nhật trong hệ thống.
Main Flow	1. Hệ thống kiểm tra tồn kho sau mỗi lần cập nhật. 2. Nếu tồn kho của nguyên liệu thấp hơn ngưỡng cấu hình, hệ thống tạo cảnh báo. 3. Hệ thống gửi thông báo đến Quản lý và Bếp. 4. Quản lý xem danh sách nguyên liệu sắp hết và các món liên quan. 5. Quản lý quyết định đặt hàng bổ sung hoặc điều chỉnh kế hoạch sử dụng.
Alternative Flows	A1. Gọi ý đặt hàng tự động • 3a. Hệ thống phân tích lịch sử tiêu thụ nguyên liệu. • 3b. Hệ thống đề xuất số lượng đặt hàng phù hợp. A2. Gom cảnh báo theo ca • 2a. Hệ thống gom nhiều cảnh báo trong một khoảng thời gian. • 2b. Hệ thống gửi một thông báo tổng hợp thay vì gửi riêng lẻ.
Exception Flows	E1. Cấu hình ngưỡng không hợp lệ • 1a. Hệ thống phát hiện ngưỡng cảnh báo thiếu hoặc sai định dạng. • 1b. Hệ thống bỏ qua việc tạo cảnh báo và ghi nhận lỗi cấu hình. • 1c. Quản lý được thông báo để cập nhật lại cấu hình. E2. Lỗi kênh thông báo • 3a. Hệ thống không gửi được thông báo đến người dùng. • 3b. Hệ thống lưu cảnh báo vào hàng đợi để gửi lại sau.

Use Case ID	UC-INV-03
Use Case Name	Phân tích tiêu hao và đề xuất nhập hàng
Primary Actor	Quản lý, Quản lý kho
Description	Use case này mô tả việc hệ thống phân tích dữ liệu tiêu hao, tồn kho và cảnh báo để đề xuất số lượng nhập hàng phù hợp cho kỳ vận hành tiếp theo.
Trigger	Quản lý mở chức năng phân tích nhập hàng theo ngày, tuần hoặc kỳ vận hành.

Preconditions	- Hệ thống đã có dữ liệu tồn kho và lịch sử tiêu hao (StockTransaction). - Có cấu hình mức tồn an toàn hoặc min-max. - Người dùng có quyền xem dữ liệu kho và báo cáo.
Postconditions	- Hệ thống tạo danh sách đề xuất nhập hàng cho từng nguyên liệu. - Quản lý có thể lưu hoặc xuất báo cáo đề xuất. - Các nguyên liệu có rủi ro được đánh dấu cảnh báo.
Main Flow	- Quản lý chọn kỳ phân tích và nhóm nguyên liệu. - Hệ thống tải dữ liệu tiêu hao và tồn kho liên quan. - Hệ thống tính mức tiêu hao trung bình theo kỳ. - Hệ thống xác định mức tồn mục tiêu và điểm đặt hàng lại. - Hệ thống tạo đề xuất số lượng nhập cho từng nguyên liệu. - Quản lý xem và điều chỉnh đề xuất nếu cần. - Quản lý lưu hoặc xuất báo cáo.
Alternative Flows	A1. Thiếu dữ liệu lịch sử 3a. Hệ thống phát hiện dữ liệu không đủ để phân tích. 3b. Hệ thống chuyển sang phương pháp min-max hoặc safety stock. 3c. Đề xuất vẫn được tạo nhưng gắn nhãn độ tin cậy thấp. A2. So sánh nhiều kỳ 1a. Quản lý chọn nhiều kỳ (cuối tuần, lễ, cao điểm). 1b. Hệ thống tính và so sánh mức tiêu hao giữa các kỳ. 1c. Hiện thị đề xuất theo từng kịch bản.
Exception Flows	E1. Dữ liệu không đồng nhất 2a. Hệ thống phát hiện sai đơn vị hoặc thiếu ánh xạ nguyên liệu. 2b. Hệ thống không tạo đề xuất cho nguyên liệu đó. 2c. Ghi nhận cần xử lý thủ công. E2. Lỗi hệ thống 2a. Hệ thống không truy xuất được dữ liệu kho. 2b. Hiện thị thông báo lỗi và yêu cầu thử lại.

Use Case ID	UC-REP-01
--------------------	-----------

Use Case Name	Xem báo cáo doanh thu và vận hành
Primary Actor	Quản lý
Description	Use case này mô tả việc hệ thống cung cấp báo cáo tổng hợp về doanh thu, hiệu suất phục vụ, hoạt động bếp và các chỉ số vận hành của nhà hàng.
Trigger	Quản lý mở dashboard báo cáo hoặc yêu cầu xuất báo cáo theo khoảng thời gian.
Preconditions	- Dữ liệu giao dịch, gọi món, bếp và tồn kho đã được tổng hợp vào hệ thống báo cáo. - Người dùng có quyền truy cập chức năng báo cáo.
Postconditions	- Hệ thống hiển thị dashboard báo cáo hoặc file xuất (PDF/CSV). - Các chỉ số vận hành được tổng hợp theo bộ lọc đã chọn.
Main Flow	- Quản lý chọn khoảng thời gian và bộ lọc báo cáo. - Hệ thống truy xuất dữ liệu báo cáo từ mô hình đọc. - Hệ thống hiển thị doanh thu, món bán chạy, giờ cao điểm và hiệu suất phục vụ. - Quản lý xem chi tiết theo ca, khu vực hoặc bếp. - Quản lý chọn xuất báo cáo nếu cần.
Alternative Flows	A1. Lọc và cập nhật báo cáo 1a. Quản lý thay đổi bộ lọc (thời gian, khu vực, ca làm). 1b. Hệ thống truy vấn lại dữ liệu và cập nhật dashboard. A2. Xuất báo cáo 5a. Quản lý chọn định dạng xuất (PDF/CSV). 5b. Hệ thống tạo file và cung cấp cho người dùng tải về. A3. Hiển thị cảnh báo bất thường 3a. Hệ thống phát hiện chỉ số bất thường (ví dụ: hoàn tiền tăng đột biến). 3b. Hệ thống hiển thị cảnh báo trên dashboard.

Exception Flows	E1. Dữ liệu báo cáo chưa cập nhật • 2a. Hệ thống hiển thị thời điểm đồng bộ gần nhất. • 2b. Báo cáo có thể bị giới hạn độ mới của dữ liệu. E2. Lỗi truy xuất dữ liệu • 2a. Hệ thống không truy xuất được dữ liệu báo cáo. • 2b. Hiển thị thông báo lỗi và yêu cầu thử lại.
------------------------	--

Use Case ID	UC-ADM-01
Use Case Name	Quản lý menu và giá bán
Primary Actor	Quản lý
Description	Use case này mô tả việc quản lý cập nhật món ăn, điều chỉnh giá bán, khuyến mãi và trạng thái hiển thị của thực đơn trong hệ thống.
Trigger	Quản lý mở màn hình cấu hình thực đơn trong hệ thống.
Preconditions	• Người dùng có quyền quản trị thực đơn. • Hệ thống đã có danh mục món cơ sở.
Postconditions	• Thực đơn hoặc thay đổi giá được lưu trong hệ thống. • Các thay đổi được áp dụng theo thời điểm công bố. • Các đơn hàng đang mở giữ nguyên giá đã ghi nhận trước đó.
Main Flow	1. Quản lý chọn món hoặc danh mục món cần chỉnh sửa. 2. Hệ thống hiển thị thông tin chi tiết (giá, trạng thái, khuyến mãi). 3. Quản lý chỉnh sửa thông tin món (tên, giá, trạng thái, khuyến mãi). 4. Quản lý lưu thay đổi và chọn công bố ngay hoặc lên lịch. 5. Hệ thống kiểm tra dữ liệu và cập nhật phiên bản thực đơn mới.
Alternative Flows	A1. Công bố theo lịch • 4a. Quản lý chọn thời gian áp dụng (ca sáng, ca tối, ngày lễ). • 4b. Hệ thống lưu phiên bản và kích hoạt theo thời gian đã đặt. A2. Áp dụng khuyến mãi • 3a. Quản lý áp dụng khuyến mãi theo món, danh mục hoặc hóa đơn. • 3b. Hệ thống cập nhật chính sách giá tương ứng.

Exception Flows	E1. Thiếu dữ liệu cấu hình • 5a. Hệ thống phát hiện thiếu thông tin bắt buộc (ví dụ: trạm bếp hoặc thuế). • 5b. Hệ thống không cho phép công bố thực đơn. • 5c. Yêu cầu người dùng bổ sung thông tin. E2. Xung đột lịch hiệu lực • 4a. Hệ thống phát hiện trùng lịch áp dụng giá hoặc khuyến mãi. • 4b. Hệ thống yêu cầu người dùng điều chỉnh trước khi lưu.
------------------------	--

Use Case ID	UC-ADM-02
Use Case Name	Quản lý vai trò và phân quyền
Primary Actor	Quản trị viên, Quản lý
Description	Use case này mô tả việc gán vai trò và phân quyền cho người dùng trong hệ thống nhằm kiểm soát các thao tác theo cơ chế RBAC.
Trigger	Quản trị viên mở màn hình quản lý người dùng hoặc phân quyền hệ thống.
Preconditions	• Tài khoản người dùng đã tồn tại hoặc đang được tạo mới. • Chính sách RBAC và danh mục quyền đã được định nghĩa.
Postconditions	• Người dùng được gán vai trò và quyền truy cập phù hợp. • Hệ thống cập nhật AuthorizationPolicy cho các thao tác liên quan. • Thay đổi quyền được ghi nhận trong nhật ký kiểm toán.
Main Flow	1. Quản trị viên tìm kiếm người dùng cần cập nhật. 2. Hệ thống hiển thị vai trò và quyền hiện tại. 3. Quản trị viên gán hoặc thay đổi vai trò/quyền theo chính sách. 4. Hệ thống lưu thay đổi phân quyền. 5. Hệ thống yêu cầu người dùng đăng nhập lại. 6. Hệ thống ghi nhận thay đổi vào nhật ký kiểm toán.

Alternative Flows	A1. Gán vai trò tạm thời • 3a. Quản trị viên chọn vai trò có thời hạn. • 3b. Hệ thống gán vai trò theo khoảng thời gian hiệu lực. A2. Giới hạn theo khu vực • 3a. Quản trị viên giới hạn quyền theo khu vực (bếp, sảnh, quầy). • 3b. Hệ thống áp dụng phạm vi quyền tương ứng.
Exception Flows	E1. Gỡ quyền quản trị cuối cùng • 3a. Hệ thống phát hiện đây là quyền quản trị cuối cùng. • 3b. Hệ thống chặn thao tác để tránh mất quyền hệ thống. E2. Xung đột chính sách quyền • 3a. Hệ thống phát hiện vai trò xung đột chính sách RBAC. • 3b. Yêu cầu quản trị viên điều chỉnh trước khi lưu.

Use Case ID	UC-ADM-03
Use Case Name	Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm
Primary Actor	Quản lý, Quản trị viên
Description	Use case này mô tả việc tra cứu và truy vết các hành động nhạy cảm như hoàn tiền, hủy món, ghi đề giá, thay đổi quyền hoặc mở lại hóa đơn.
Trigger	Người dùng có thẩm quyền mở màn hình kiểm toán để điều tra, rà soát hoặc kiểm tra nội bộ.
Preconditions	• Hệ thống đã ghi nhận AuditLog cho các thao tác nhạy cảm. • Người dùng có quyền truy cập nhật ký kiểm toán. • Dữ liệu giao dịch và correlation id có thể truy xuất.
Postconditions	• Hệ thống hiển thị chuỗi hành động liên quan đến truy vấn. • Có thể xuất báo cáo hoặc đánh dấu bản ghi cần theo dõi. • Hành vi truy cập nhật ký kiểm toán được ghi lại.

Main Flow	1. Người dùng mở màn hình nhật ký kiểm toán. 2. Hệ thống kiểm tra quyền truy cập. 3. Người dùng nhập bộ lọc tìm kiếm (thời gian, người thao tác, loại hành động, mã liên kết). 4. Hệ thống truy vấn và hiển thị danh sách nhật ký kiểm toán. 5. Người dùng chọn một bản ghi để xem chi tiết (giá trị trước, sau, lý do, ngữ cảnh). 6. Người dùng xuất báo cáo hoặc gắn cờ theo dõi nếu cần.
Alternative Flows	A1. Truy vết từ giao dịch • 3a. Người dùng truy xuất nhật ký kiểm toán từ hóa đơn/payment/order. • 3b. Hệ thống hiển thị toàn bộ chuỗi liên quan. A2. Che dữ liệu theo quyền • 4a. Hệ thống ẩn thông tin nhạy cảm nếu không đủ quyền. • 4b. Chỉ hiển thị dữ liệu rút gọn.
Exception Flows	E1. Không đủ quyền truy cập • 2a. Hệ thống từ chối truy cập hoặc hiển thị bản rút gọn. E2. Không có dữ liệu phù hợp • 4a. Hệ thống thông báo không tìm thấy kết quả. • 4b. Người dùng điều chỉnh bộ lọc tìm kiếm.

Use Case ID	UC-ADM-04
Use Case Name	Tạo món mới
Primary Actor	Quản lý
Description	Use case này mô tả việc quản lý tạo mới một món ăn trong thực đơn và khai báo thông tin nguyên liệu để phục vụ bán hàng và quản lý tồn kho.
Trigger	Quản lý chọn chức năng tạo món mới trong màn hình quản lý menu.
Preconditions	• Người dùng có quyền quản trị menu. • Hệ thống đã có danh mục nguyên liệu trong kho.

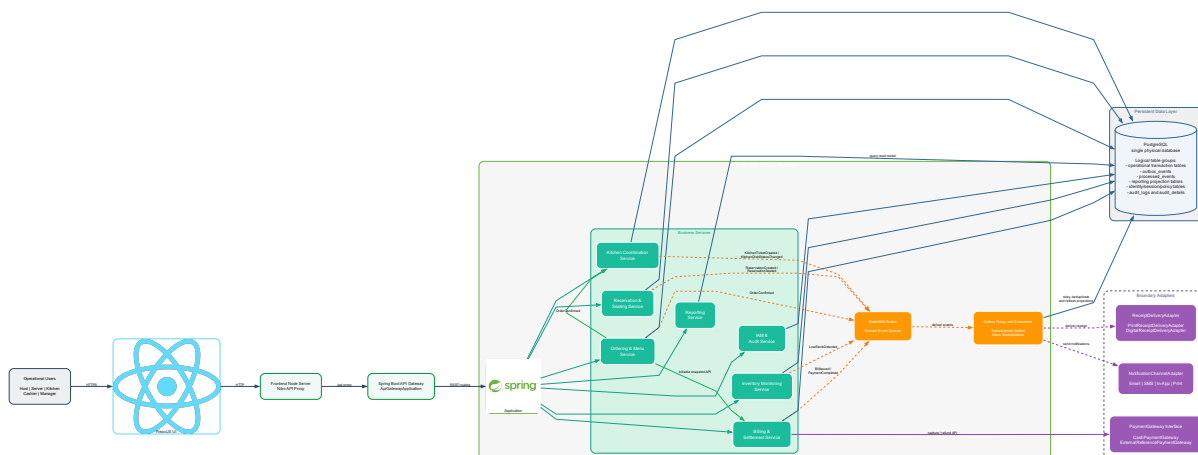
Postconditions	- Món mới được tạo và lưu trong hệ thống menu. - Công thức nguyên liệu (recipe) được liên kết với món. - Món có thể được sử dụng trong gọi món; tồn kho được trừ theo công thức khi món bắt đầu được chế biến.
Main Flow	- Quản lý chọn chức năng tạo món mới. - Hệ thống hiển thị form nhập thông tin món. - Quản lý nhập thông tin món (tên, giá, mô tả, danh mục, trạng thái bán). - Quản lý khai báo danh sách nguyên liệu và định lượng. - Quản lý lưu thông tin món. - Hệ thống kiểm tra dữ liệu hợp lệ. - Hệ thống lưu món và công thức nguyên liệu vào hệ thống.
Alternative Flows	A1. Thiếu thông tin bắt buộc 3a. Hệ thống phát hiện thiếu tên món hoặc giá. 3b. Hệ thống hiển thị thông báo lỗi. 3c. Quay lại bước 3. A2. Thiếu nguyên liệu 4a. Hệ thống phát hiện chưa khai báo nguyên liệu. 4b. Hệ thống yêu cầu bổ sung nguyên liệu trước khi lưu.
Exception Flows	E1. Lỗi hệ thống khi lưu 5a. Hệ thống không thể lưu món mới. 5b. Hệ thống hiển thị thông báo lỗi và yêu cầu thử lại.

Use Case ID	UC-OPS-01
Use Case Name	Quản lý ca làm nhân viên
Primary Actor	Quản lý
Description	Use case này mô tả việc phân công nhân sự cho từng khu vực và ca làm việc trong nhà hàng.
Trigger	Quản lý mở lịch làm việc và tạo hoặc cập nhật ca làm.
Preconditions	- Tài khoản nhân viên đã tồn tại. - Mẫu ca làm và cấu hình khu vực đã được thiết lập.

Postconditions	• Ca làm (ShiftAssignment) được tạo hoặc cập nhật. • Nhân viên nhận được lịch làm việc mới.
Main Flow	1. Quản lý chọn ngày, khu vực và loại ca. 2. Hệ thống hiển thị danh sách nhân viên và các ca đã có. 3. Quản lý gán nhân viên vào ca và khu vực. 4. Hệ thống kiểm tra thời điểm kết thúc và trùng ca làm việc. 5. Quản lý lưu lịch làm việc. 6. Hệ thống ghi nhận thông báo nội bộ cho lịch ca vừa thay đổi.
Alternative Flows	A1. Cập nhật ca đã có • 1a. Quản lý chọn một ca làm việc đã tồn tại. • 1b. Hệ thống hiển thị thông tin ca để quản lý chỉnh sửa. A2. Ca bị trùng thời gian • 4a. Hệ thống phát hiện nhân viên đã có ca trùng khoảng thời gian. • 4b. Hệ thống chặn lưu và yêu cầu điều chỉnh ca làm.
Exception Flows	E1. Trùng ca hoặc thời gian không hợp lệ • 4a. Hệ thống phát hiện xung đột lịch làm. • 4b. Chặn lưu và yêu cầu điều chỉnh. E2. Không tìm thấy nhân viên hoặc ca cần cập nhật • 3a. Hệ thống không tìm thấy nhân viên hoặc ca làm việc cần cập nhật. • 3b. Hệ thống thông báo lỗi và không lưu thay đổi.

3 Kiến trúc phần mềm

3.1 Tổng quan kiến trúc



Hình 8: Sơ đồ tổng quan kiến trúc và các trách nhiệm lưu trữ logic của IRMS

IRMS được thiết kế theo kiến trúc service-based kết hợp event-driven processing, gồm ba lớp chính: giao diện người dùng, dịch vụ nghiệp vụ và dữ liệu cùng các xử lý nền. Kiến trúc này giúp phân tách rõ các chức năng nghiệp vụ và xử lý các tác vụ phụ qua cơ chế sự kiện mà không ảnh hưởng đến luồng giao dịch chính.

Ở lớp giao diện, ReactJS cung cấp nền tảng thống nhất cho các vai trò như lễ tân, phục vụ, bếp, thu ngân và quản lý. Cách tiếp cận này giúp các bộ phận dùng chung cơ chế xác thực, điều hướng và đồng bộ trạng thái, đảm bảo các thông tin như bàn, đơn gọi món, phiếu bếp và hóa đơn luôn được cập nhật nhất quán trong quá trình vận hành.

Phía sau giao diện là Frontend Node Server của TanStack Start/Nitro, chịu trách nhiệm phục vụ React và chuyển tiếp các yêu cầu /api đến ApiGatewayApplication. Backend sử dụng GatewayProxyController làm cổng vào để định tuyến yêu cầu đến các service Spring Boot theo cấu hình docker-compose.yml. Cách tổ chức này giúp frontend không phụ thuộc vào cấu trúc nội bộ của backend, tăng khả năng thay đổi và mở rộng hệ thống.

Lớp backend được xây dựng bằng Spring Boot và tổ chức thành các service theo miền nghiệp vụ. Mỗi service phụ trách một chức năng riêng như Reservation & Seating Service quản lý đặt bàn và trạng thái bàn, Ordering Service xử lý đơn gọi món, Kitchen Coordination Service điều phối chế biến món ăn, Billing & Settlement Service quản lý hóa đơn và thanh toán, Inventory Monitoring Service theo dõi tồn kho, Reporting Service phục vụ báo cáo quản trị và IAM Audit Service đảm nhiệm kiểm soát truy cập cùng ghi nhận kiểm toán. Cách phân tách này giúp hệ thống rõ ràng về trách nhiệm, dễ bảo trì và thuận lợi cho mở rộng sau này.

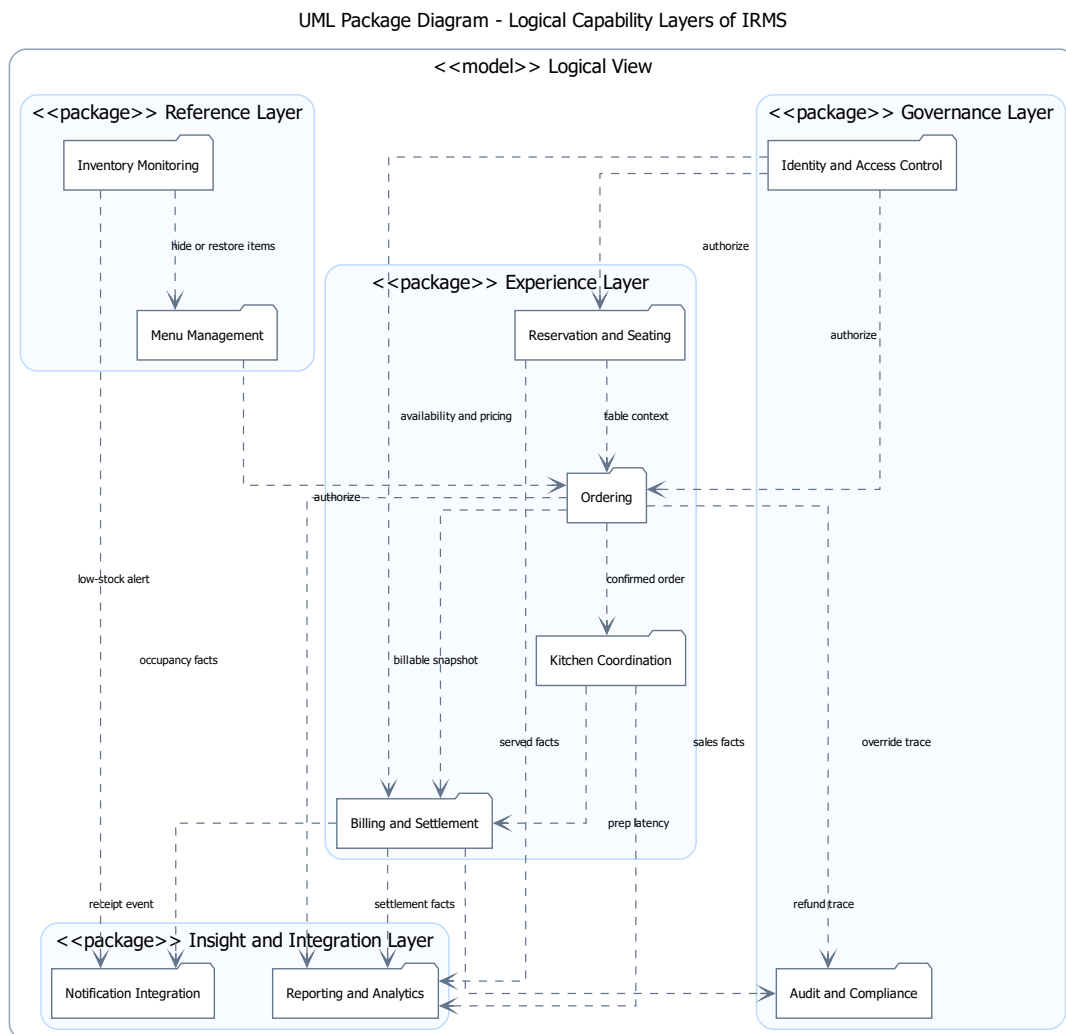
Việc phân chia service theo miền nghiệp vụ giúp kiến trúc phản ánh đúng quy trình vận hành thực tế của nhà hàng. Nhờ ranh giới trách nhiệm rõ ràng, thay đổi ở một nghiệp vụ sẽ ít ảnh hưởng đến các phần khác của hệ thống, từ đó tăng khả năng bảo trì và mở rộng.

Lớp dữ liệu của IRMS sử dụng một PostgreSQL vật lý duy nhất nhưng được phân tách thành các nhóm dữ liệu logic như dữ liệu giao dịch, outbox/inbox, định danh kiểm toán và read model cho báo cáo. Mô hình đọc phục vụ báo cáo được cập nhật qua các xử lý nền thay vì truy vấn trực tiếp lên dữ liệu giao dịch nóng, giúp giảm tải và tách biệt trách nhiệm lưu trữ trong hệ thống.

Ngoài tuyến xử lý chính, kiến trúc còn sử dụng `outbox_events`, `processed_events`, RabbitMQ và các bộ xử lý nền để thực hiện các tác vụ phụ như gửi thông báo, phát hành biên lai và cập nhật mô hình đọc. Các sự kiện được xử lý bất đồng bộ nhằm giảm tải cho các thao tác trực tiếp như gọi món và thanh toán, đồng thời hỗ trợ cơ chế chống xử lý lặp cho consumer.

Cuối cùng, các điểm tích hợp được đặt ở rìa kiến trúc thông qua các adapter như `PaymentGateway`, `NotificationChannelAdapter` và `ReceiptDeliveryAdapter`. Cách tổ chức này giúp tách biệt lỗi nghiệp vụ khỏi thay đổi của hệ thống thanh toán, thông báo và phát hành biên lai, từ đó tăng khả năng bảo trì và mở rộng.

3.2 Tổng quan cho các góc nhìn chính



Hình 9: Sơ đồ năng lực nghiệp vụ hỗ trợ phân rã module của IRMS

Hình 9 trình bày sơ đồ năng lực nghiệp vụ của hệ thống IRMS dưới dạng UML Package, trong đó hệ thống được tổ chức dựa trên các năng lực nghiệp vụ (capabilities) thay vì theo giao diện người dùng hoặc cấu trúc kỹ thuật.

Mục tiêu của sơ đồ hỗ trợ này là xác định các miền nghiệp vụ độc lập, đồng thời thể hiện mối quan hệ tương tác giữa chúng trước khi được ánh xạ sang các module và service trong các góc nhìn kiến trúc chính.

Trong kiến trúc IRMS, sơ đồ năng lực này được xây dựng theo hướng service-based architecture kết hợp với cơ chế event-driven interaction ở mức khái niệm. Mỗi package tương ứng với một miền năng lực có ranh giới rõ ràng. Sự tương tác giữa các miền được thể hiện thông qua hai cơ chế chính, bao gồm giao tiếp đồng bộ theo mô hình request/response trong luồng nghiệp vụ chính và giao tiếp bất đồng bộ thông qua domain events nhằm truyền tải thông tin vận hành mà không làm tăng mức độ phụ thuộc giữa các miền.

Từ cách tổ chức này, các năng lực nghiệp vụ trong hệ thống được phân nhóm theo vai trò trong kiến trúc tổng thể.

Thứ nhất, nhóm năng lực tham chiếu và điều kiện vận hành đóng vai trò cung cấp dữ liệu nền và đảm bảo điều kiện cho các luồng nghiệp vụ chính. *Menu Management* chịu trách nhiệm quản lý thông tin món ăn, giá bán và trạng thái hiển thị, trong khi *Inventory Monitoring* theo dõi tình trạng tồn kho và phát hiện các điều kiện thiếu hụt nguyên liệu. Nhóm này không trực tiếp tham gia xử lý giao dịch nhưng ảnh hưởng đến tính hợp lệ của dữ liệu trong toàn bộ hệ thống.

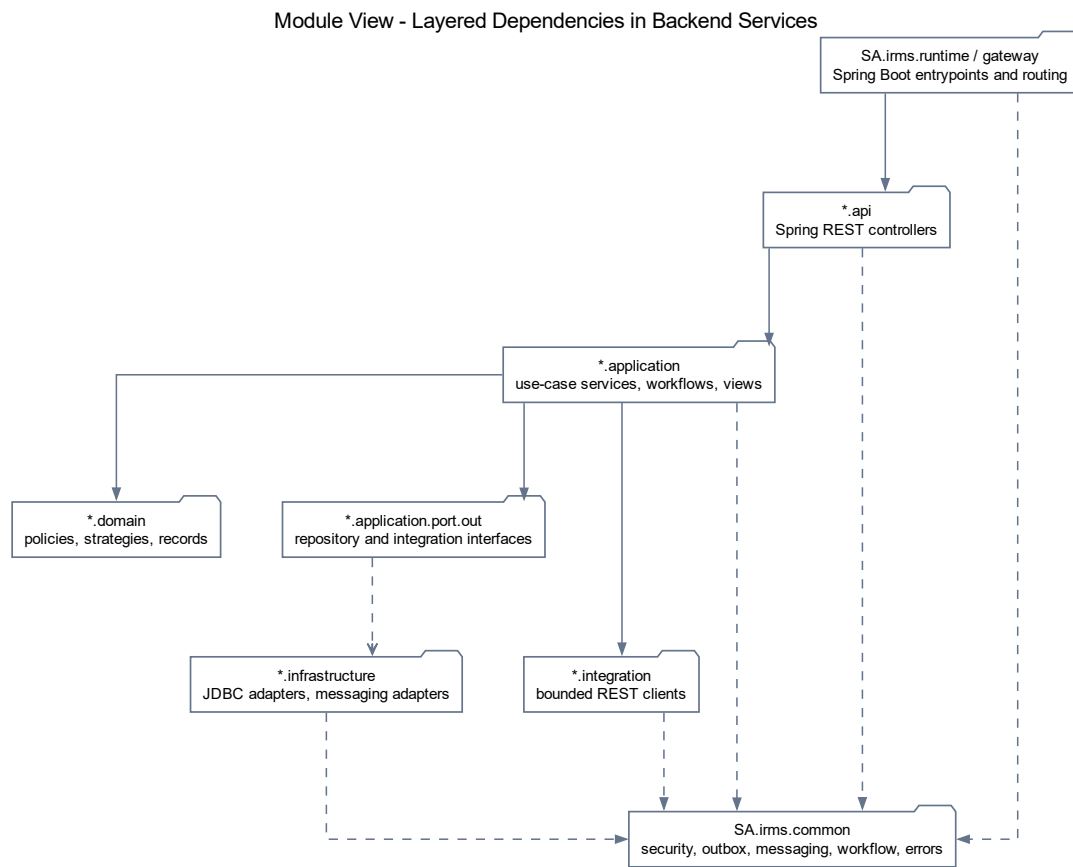
Thứ hai, chuỗi năng lực cốt lõi phản ánh toàn bộ vòng đời phục vụ trong nhà hàng. *Reservation & Seating* đảm nhiệm việc quản lý đặt bàn và phân bổ chỗ ngồi, tạo ngữ cảnh cho *Ordering* trong quá trình tiếp nhận đơn hàng. Sau đó, *Kitchen Coordination* tiếp nhận thông tin từ đơn hàng để điều phối quá trình chế biến món ăn, trong khi *Billing & Settlement* thực hiện thanh toán và kết thúc phiên phục vụ. Các năng lực trong chuỗi này có quan hệ phụ thuộc logic rõ ràng nhằm đảm bảo luồng nghiệp vụ được xử lý nhất quán.

Thứ ba, nhóm năng lực hỗ trợ và phân tích chịu trách nhiệm xử lý thông tin vận hành theo hướng bất đồng bộ. *Notification Integration* tiếp nhận các sự kiện phát sinh để thực hiện thông báo, trong khi *Reporting & Analytics* tổng hợp dữ liệu từ nhiều miền nghiệp vụ nhằm phục vụ thống kê và phân tích. Việc tách nhóm này giúp giảm tải cho luồng xử lý chính và tăng khả năng mở rộng hệ thống.

Thứ tư, nhóm năng lực kiểm soát hệ thống đảm bảo các yêu cầu về bảo mật, phân quyền và truy vết. *Identity & Access Control* quản lý xác thực và phân quyền người dùng, trong khi *Audit & Compliance* ghi nhận lịch sử hoạt động để phục vụ kiểm toán và tuân thủ. Đây là các năng lực xuyên suốt toàn hệ thống nhưng không tham gia trực tiếp vào luồng nghiệp vụ chính.

3.2.1 Tổng quan cho góc nhìn mô-đun

Góc nhìn mô-đun mô tả các đơn vị hiện thực chính của backend IRMS, cách chúng được tổ chức thành package, trách nhiệm của từng module và quan hệ phụ thuộc tĩnh giữa các module. Góc nhìn này cũng ghi nhận các cấu trúc dữ liệu dùng chung như `outbox_events`, `processed_events`, nhật ký kiểm toán và mô hình đọc báo cáo khi chúng ảnh hưởng đến nhiều module. Các cấu trúc dữ liệu này là nhóm bảng hoặc trách nhiệm lưu trữ logic trong cùng PostgreSQL, không phải các database vật lý riêng. Module View không mô tả tiến trình runtime hay luồng message khi hệ thống chạy; các nội dung đó thuộc Component-and-Connector View và Deployment View.



Hình 10: Module View tổng quan theo lớp phụ thuộc của IRMS

Hình 10 thể hiện quy tắc phụ thuộc theo lớp bằng Module View, bám vào package và lớp hiện thực trong backend.

Mô tả

- GatewayProxyController đi qua GatewayRouteLocator và RestClientConfiguration để định tuyến REST, không gọi trực tiếp vào dữ liệu nghiệp vụ.
- Các controller như OrdersController, KitchenController và BillsController phụ thuộc vào service ứng dụng tương ứng, đúng với cấu trúc package trong backend.
- Các thành phần hạ tầng dùng chung như RabbitMqEventPublisher, RabbitMqOutboxRelay và ManualAckConsumerSupport được đặt riêng để xử lý sự kiện và bộ xử lý sự kiện nền.

Catalog mô-đun và ánh xạ hiện thực. Bảng 32 thể hiện giới mô-đun theo package trong backend. Các hình Module View ở phần này mô tả tổ chức tính và hướng phụ thuộc giữa package, còn các sơ đồ Component-and-Connector phía sau mới mô tả cộng tác runtime.

Bảng 32: Catalog mô-đun backend IRMS

Module	Trách nhiệm
Gateway	Điểm vào REST, chuẩn hóa đường /api/** và định tuyến đến service nghiệp vụ theo cấu hình.
Runtime / Bootstrap	Chứa các endpoint Spring Boot và profile khởi động từng runtime trong Docker.
Reservation	Đặt bàn, danh sách chờ, sơ đồ bàn, check-in và phiên phục vụ.
Ordering	Thực đơn, tạo/xác nhận order, tùy biến món, combo, snapshot giá và gửi bếp.
Kitchen	Điều phối phiếu bếp, trạng thái món, deadline, ưu tiên và workflow chế biến.
Billing	Lập hóa đơn, tách hóa đơn, thanh toán, hoàn tiền, biên nhận và settlement.
Inventory	Theo dõi tồn kho, tiêu hao nguyên liệu, cảnh báo mức tồn thấp và đề xuất nhập hàng.
Notification	Ghi nhận yêu cầu thông báo và chọn adapter kênh gửi nội bộ.
Reporting	Dashboard, báo cáo vận hành và các bảng mô hình đọc/projection.
Identity/Audit	Xác thực, vai trò, quyền, session, policy và nhật ký kiểm toán.
Common Platform	Hạ tầng dùng chung: security/context, lỗi chuẩn, workflow, outbox, RabbitMQ và consumer support.
PDF / Receipt Adapter	Render/phát hành biên nhận qua adapter nội bộ, không mô tả máy in vật lý hay nhà cung cấp ngoài.

Bảng 33: Interface và phụ thuộc chính của các mô-đun

Module	Class/interface tiêu biểu và contract nhìn thấy	Phụ thuộc và dữ liệu dùng chung
Gateway	GatewayProxyController; GatewayRouteLocator; contract HTTP proxy cho route /api/**.	Dùng cấu hình URL service và common error/context; không gọi trực tiếp dữ liệu nghiệp vụ.
Runtime / Bootstrap	ApiGatewayApplication, OrderingServiceApplication, KitchenServiceApplication, BillingServiceApplication, ReservationServiceApplication, InventoryServiceApplication, NotificationServiceApplication, ReportingServiceApplication, IdentityAuditServiceApplication.	Chọn service boundary bằng profile, port và biến môi trường trong docker-compose.yml.
Reservation	ReservationService, ReservationCheckInService, ReservationTableAssignmentService.	Dùng identity/common để kiểm soát thao tác; ghi bảng reservation/table session và event qua outbox nếu phát sinh.
Ordering	OrdersService, OrderMenuSelectionService, ComboPricingPolicy, OrderStatusPolicy.	Dùng reservation/table-session context, kitchen integration client và common outbox/messaging.
Kitchen	KitchenService, KitchenOrderRoutingService, KitchenWorkflowMediator, KitchenTicketTransitionPolicy, KitchenTicketItemRepositoryPort.	Nhận order/kitchen events; dùng ManualAckConsumerSupport và processed_events cho bộ xử lý sự kiện nền.
Billing	BillingService, BillingPaymentService, BillingReceiptService, PaymentGateway, ReceiptDeliveryAdapter, BillSplitStrategy.	Dùng billable snapshot từ ordering, adapter thành toán/biên nhận và DomainEventPublisher.
Inventory	InventoryService, InventoryStockChangedConsumer, LowStockSeverityPolicy, ReorderRecommendationStrategy.	Tiêu thụ event qua RabbitMQ/manual ack; ghi inventory tables và có thể phát cảnh báo tồn thấp.
Notification	NotificationService, NotificationRequestedConsumer, NotificationChannelAdapter và các adapter email/SMS/in-app/print.	Nhận yêu cầu thông báo từ sự kiện; kênh gửi được che sau adapter biên nội bộ.
Reporting	ReportingQueryService, ReportingProjectionConsumer, các projection/materializer.	Dùng domain events, processed_events và các bảng reporting_*_projection.
Identity/Audit	AuthService, AuditService, IdentityRepository, IdentityPolicyRepository, AuditRecordingRequestedConsumer.	Duy trì user/session/role/policy và audit_logs; các module bảo vệ thao tác dùng chính sách identity.
Common Platform	TransactionalOutboxPublisher, RabbitMqOutboxRelay, RabbitMqEventPublisher, ManualAckConsumerSupport, ApiEnvelope.	Crosscutting package cho outbox_events, processed_events, envelope/correlation và lỗi chuẩn.

Bảng 34: Các loại quan hệ dùng trong Module View

Relation type	Ý nghĩa trong Module View	Ví dụ trong IRMS
is-part-of	Module/package con thuộc một package lớn hơn.	ordering.api, ordering.application, ordering.domain, ordering.infrastructure thuộc SA.irms.ordering.
uses / depends-on	Phụ thuộc source-level qua import, constructor, method call, config hoặc bean wiring.	OrdersController phụ thuộc OrdersService; BillingPaymentService phụ thuộc PaymentGateway.
allowed-to-use	Quy tắc tăng được phép dùng tầng thấp hơn hoặc package dùng chung.	*.api dùng *.application; *.application dùng *.domain và port/interface.
data dependency	Module đọc/ghi hoặc bị ảnh hưởng bởi cấu trúc dữ liệu dùng chung.	RabbitMqOutboxRelay dùng outbox_events; bộ xử lý sự kiện nền dùng processed_events.
crosscuts	Package dùng chung hỗ trợ nhiều module nghiệp vụ.	SA.irms.common cung cấp security/context, outbox, messaging, workflow và lỗi chuẩn.
is-a	Quan hệ kế thừa hoặc hiện thực interface ở mức class/interface.	CashPaymentGateway hiện thực PaymentGateway; notification adapters hiện thực NotificationChannelAdapter.

Bảng 35: Cấu trúc dữ liệu dùng chung trong Module View

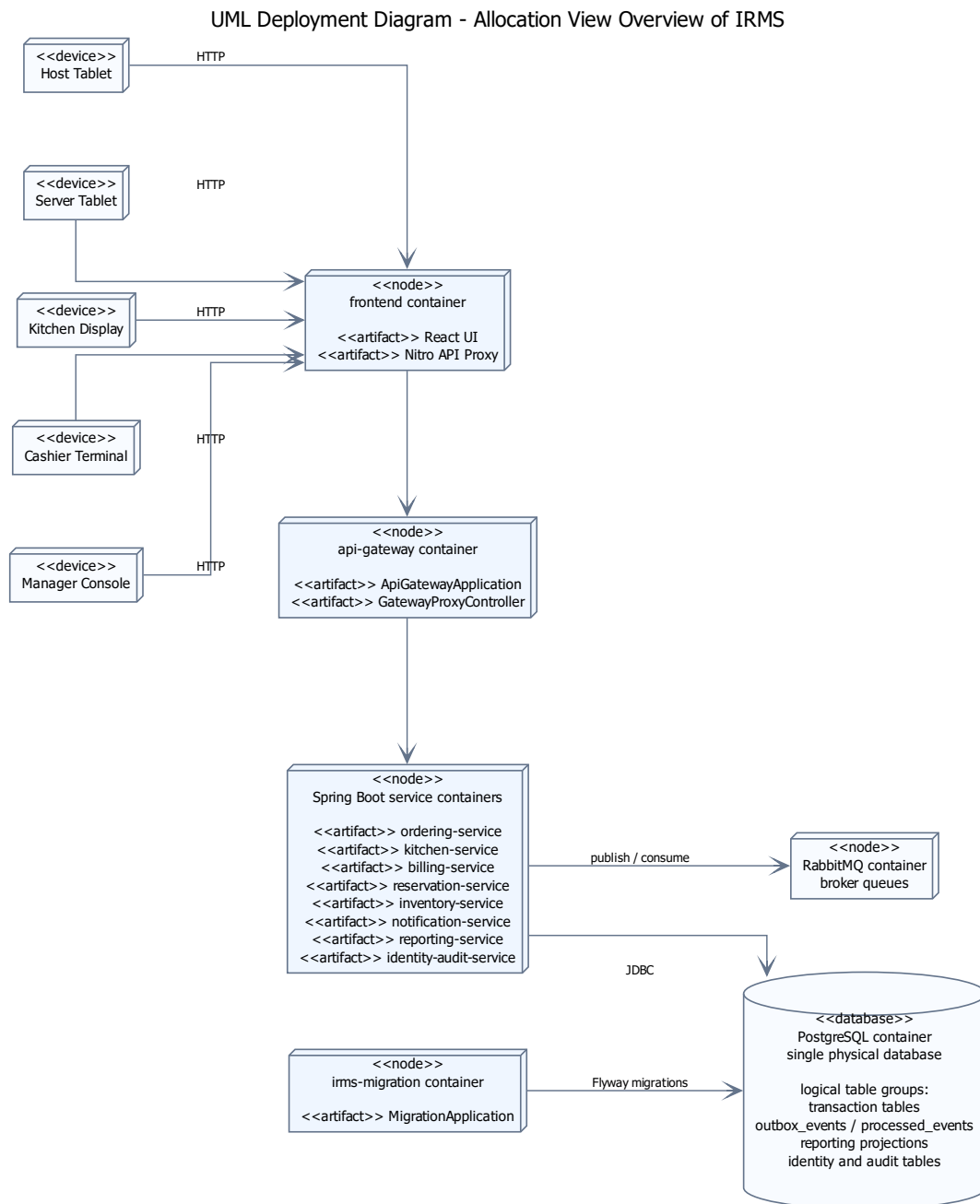
Cấu trúc	Module duy trì	Vai trò trong góc nhìn mô-đun	Thành phần liên quan
outbox_events	SA.irms.common.outbox và các module phát event.	Bảng dùng chung để ghi sự kiện cần phát bất đồng bộ trước khi chuyển sang RabbitMQ.	TransactionalOutbox Publisher; RabbitMqOutboxRelay.
processed_events	SA.irms.common.messaging.	Bảng ghi nhận sự kiện đã xử lý, giúp tránh xử lý lặp trong các luồng nền.	ManualAckConsumer Support.
reporting_*_ projection	SA.irms.reporting.	Mô hình đọc cho dashboard và báo cáo, được cập nhật qua các thành phần tổng hợp dữ liệu.	ReportingProjection Consumer; ReportingQueryService.
audit_logs, audit_details	SA.irms.identity.	Dữ liệu kiểm toán cho thao tác nhạy cảm và truy vết vận hành.	AuditService; AuditRecording RequestedConsumer.
identity/session/ policy data	SA.irms.identity.	Dữ liệu xác thực, phân quyền, session và chính sách được nhiều module tham chiếu.	AuthService; IdentityPolicy Repository.

3.2.2 Tổng quan cho góc nhìn thành phần và kết nối

Góc nhìn thành phần và kết nối mô tả các thành phần có mặt ở runtime và cách chúng giao tiếp qua REST, JDBC, AMQP, outbox và các adapter biên. Khác với Module View, góc nhìn này không chỉ mô tả package tĩnh mà làm rõ component nào cung cấp interface, component nào yêu cầu interface, port nào nằm ở biên subsystem và connector nào lắp ghép các interface đó. Trong IRMS, trọng tâm của C&C View là tuyến đi từ frontend/API Gateway vào controller/service nghiệp vụ, tuyến ghi dữ liệu vào PostgreSQL vật lý duy nhất, và tuyến event-driven đi qua outbox, RabbitMQ, bộ xử lý sự kiện nền, read model, notification và audit.

Phần chi tiết của Component-and-Connector View được trình bày ở Mục 3.6.1 và các hình thành phần bên dưới. Các hình này cần được đọc như sơ đồ runtime collaboration có component, port/interface và connector, không phải package diagram hay class diagram.

3.2.3 Tổng quan cho góc nhìn phân bố



Hình 11: Sơ đồ tổng quan cho góc nhìn phân bố của IRMS

Mô tả:

- **Lớp biên và thiết bị:** là nơi phát sinh thao tác nghiệp vụ thực tế như máy của lễ tân, máy của phục vụ, màn hình bếp, máy thu ngân và màn hình quản lý.

- **Lớp thực thi:** gồm frontend container, api-gateway container, nhóm container service Spring Boot và irms-migration container; đây là nơi các năng lực được hiện thực thành tiến trình hoặc vùng thực thi cụ thể.
- **Lớp dữ liệu và sự kiện:** gồm PostgreSQL container, RabbitMQ container, outbox_events, processed_events và các bảng hoặc view báo cáo; chúng tách dữ liệu giao dịch, dữ liệu đọc tổng hợp và hàng đợi sự kiện.
- **Lớp hệ thống ngoài:** hình này không vẽ hệ thống ngoài độc lập cho thanh toán, thông báo hoặc máy in vật lý; các năng lực đó được đặt sau adapter biên nội bộ trong phạm vi hiện tại.

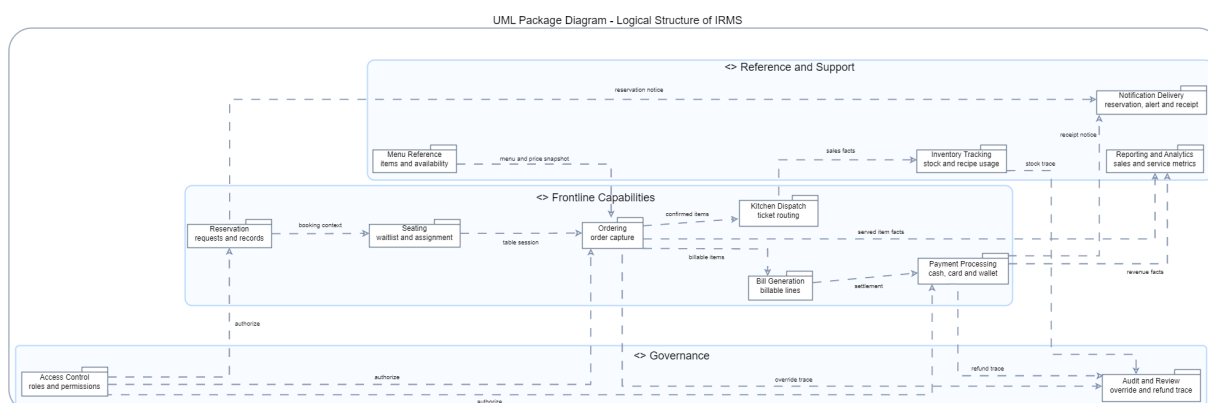
3.3 Đặc trưng kiến trúc

Những *đặc trưng kiến trúc* quan trọng là:

1. **Khả năng đáp ứng:** màn hình gọi món, màn hình bếp và màn hình thu ngân phải phản hồi nhanh để không làm chậm tuyến phục vụ trực tiếp với khách.
2. **Độ tin cậy:** đơn gọi món, thanh toán, hoàn tiền và cập nhật trạng thái bàn là các luồng không được mất dữ liệu hoặc xử lý lặp sai lệch.
3. **Khả năng mở rộng:** điểm nóng thường dồn vào giờ cao điểm và tập trung ở các miền Ordering, Kitchen Coordination và Billing thay vì toàn hệ thống.
4. **Bảo mật và khả năng kiểm toán:** mọi thao tác nhạy cảm phải được kiểm soát bằng quyền và truy vết lại được về sau.
5. **Khả năng quan sát:** hệ thống phải đủ nhật ký, vết thực thi và số đo để thấy độ trễ của phiếu bếp, thanh toán và các điểm nghẽn vận hành.
6. **Khả năng sửa đổi:** thực đơn, giá, khuyến mãi, ngưỡng cảnh báo tồn kho, kênh gửi thông báo và bộ kết nối thanh toán thay đổi thường xuyên nên cần biên thay đổi hợp.

3.4 Sơ đồ năng lực nghiệp vụ hỗ trợ

3.4.1 Sơ đồ phân rã năng lực logic



Hình 12: Sơ đồ phân rã năng lực logic

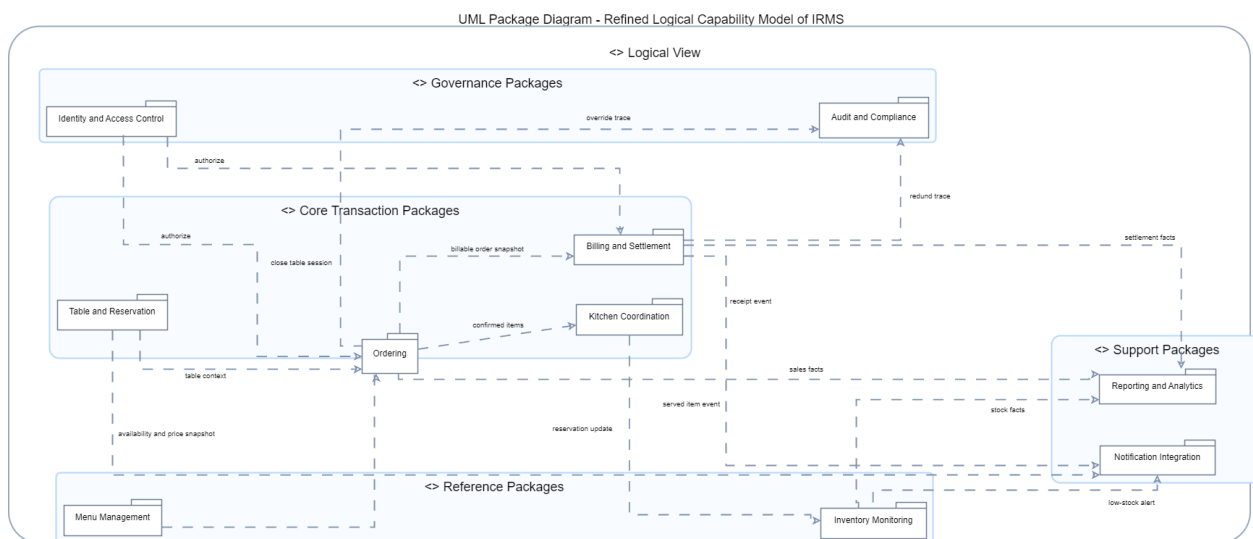
Hình 12 thể hiện bước phân rã ban đầu các năng lực nghiệp vụ trong hệ thống IRMS.

Trong sơ đồ này, các năng lực được xác định theo hướng chức năng bề mặt như quản lý bàn, tiếp nhận đơn gọi món, điều phối bếp, thanh toán, quản lý tồn kho, báo cáo, kiểm soát truy cập và giám sát hệ thống.

Mục tiêu là nhận diện đầy đủ phạm vi năng lực của hệ thống IRMS. Các năng lực được mô tả theo cách gần với quy trình vận hành thực tế để dễ kiểm tra và tránh bỏ sót chức năng quan trọng.

Sơ đồ này được xem là bước khởi tạo trong quá trình phân tích logic, đóng vai trò nền tảng để tiếp tục tái cấu trúc và gom nhóm các năng lực thành các miền nghiệp vụ ổn định hơn ở bước tiếp theo.

3.4.2 Sơ đồ phân rã năng lực logic hoàn chỉnh



Hình 13: Sơ đồ phân rã năng lực logic của IRMS

Trong cấu trúc này, các chức năng được tổ chức theo các miền nghiệp vụ có tính kết dính cao và phản ánh đầy đủ quy trình vận hành của nhà hàng, bao gồm:

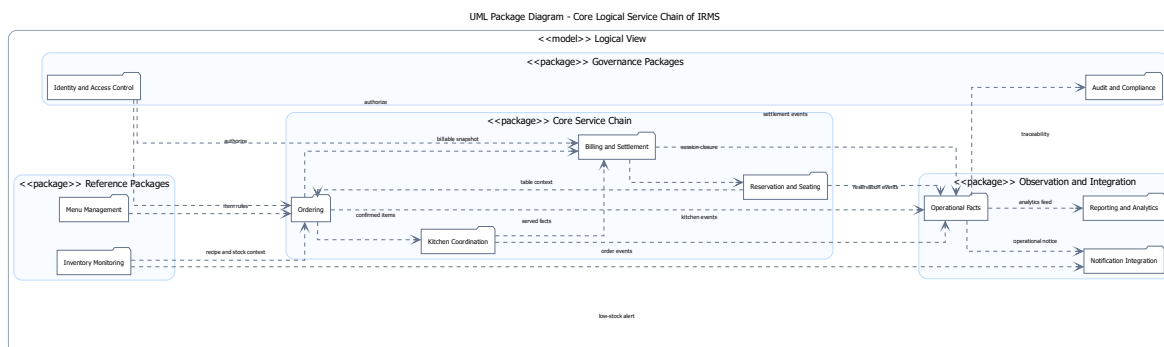
- Table & Reservation
- Menu Management
- Ordering
- Kitchen Coordination
- Billing & Settlement
- Inventory Monitoring
- Reporting & Analytics
- Identity & Access Control
- Audit & Compliance
- Notification Integration

Việc tái cấu trúc này giúp chuyển từ cách nhìn theo chức năng thao tác người dùng sang cách nhìn theo miền trách nhiệm nghiệp vụ. Thay vì xem hệ thống như tập hợp các chức năng rời rạc, mỗi năng lực được định nghĩa như một miền logic độc lập, có ranh giới rõ ràng và chịu trách nhiệm cho một phần xác định trong vòng đời vận hành của nhà hàng.

Từ góc độ kiến trúc, đây là bước chuyển từ functional decomposition sang domain-oriented decomposition. Các năng lực lúc này không chỉ mang ý nghĩa chức năng, mà trở thành các đơn vị nghiệp vụ có tính độc lập tương đối, có thể được ánh xạ trực tiếp hoặc gián tiếp sang các service trong kiến trúc service-based.

Bên cạnh đó, cấu trúc này cũng tạo nền tảng cho mô hình event-driven processing trong các góc nhìn kiến trúc tiếp theo. Các miền năng lực có thể trao đổi thông tin thông qua sự kiện nghiệp vụ thay vì phụ thuộc trực tiếp lẫn nhau, từ đó giảm coupling giữa các thành phần và tăng khả năng mở rộng của hệ thống trong dài hạn.

3.4.3 Sơ đồ chuỗi phục vụ



Hình 14: Sơ đồ năng lực cho chuỗi phục vụ lõi của IRMS

Hình 14 mô tả cách các năng lực logic lõi tương tác với nhau trong chuỗi phục vụ chính của hệ thống.

Chuỗi này bao gồm bốn miền năng lực chính:

- Reservation & Seating: thiết lập và duy trì ngữ cảnh phục vụ
- Ordering: ghi nhận và quản lý đơn gọi món
- Kitchen Coordination: điều phối quá trình chế biến món ăn
- Billing & Settlement: xử lý thanh toán và kết thúc phiên phục vụ

Sơ đồ thể hiện rằng hệ thống không vận hành như tập hợp các chức năng độc lập, mà được tổ chức theo một chuỗi năng lực có thứ tự rõ ràng, phản ánh trực tiếp vòng đời phục vụ của nhà hàng. Mỗi năng lực tương ứng với một giai đoạn trong quy trình nghiệp vụ tổng thể và chỉ chịu trách nhiệm trong phạm vi của giai đoạn đó.

Trong chuỗi này, các miền nghiệp vụ chỉ trao đổi lượng thông tin tối thiểu cần thiết để đảm bảo tính nhất quán của luồng xử lý. Reservation & Seating cung cấp ngữ cảnh phiên bàn hợp lệ; Ordering tạo và xác nhận thông tin đơn gọi món; Kitchen Coordination tiếp nhận và xử lý trạng thái chế biến; trong khi Billing & Settlement sử dụng dữ liệu đã được chốt để thực hiện quyết toán và kết thúc phiên phục vụ.

Cách tổ chức này giúp đảm bảo luồng nghiệp vụ chính có tính tuyến tính và dễ kiểm soát, đồng thời giảm thiểu sự phụ thuộc trực tiếp giữa các miền nghiệp vụ. Về mặt kiến trúc, đây là cơ sở để hiện thực hóa các tương tác giữa các năng lực thông qua cơ chế gọi dịch vụ đồng bộ trong luồng chính và cơ chế sự kiện nghiệp vụ (domain events) cho các tương tác mở rộng trong các góc nhìn triển khai phía sau.

3.5 Góc nhìn phân rã

Ở góc nhìn phân rã, hệ thống được chia thành hai lớp lớn: **các ứng dụng phía người dùng** và **các service nghiệp vụ**.

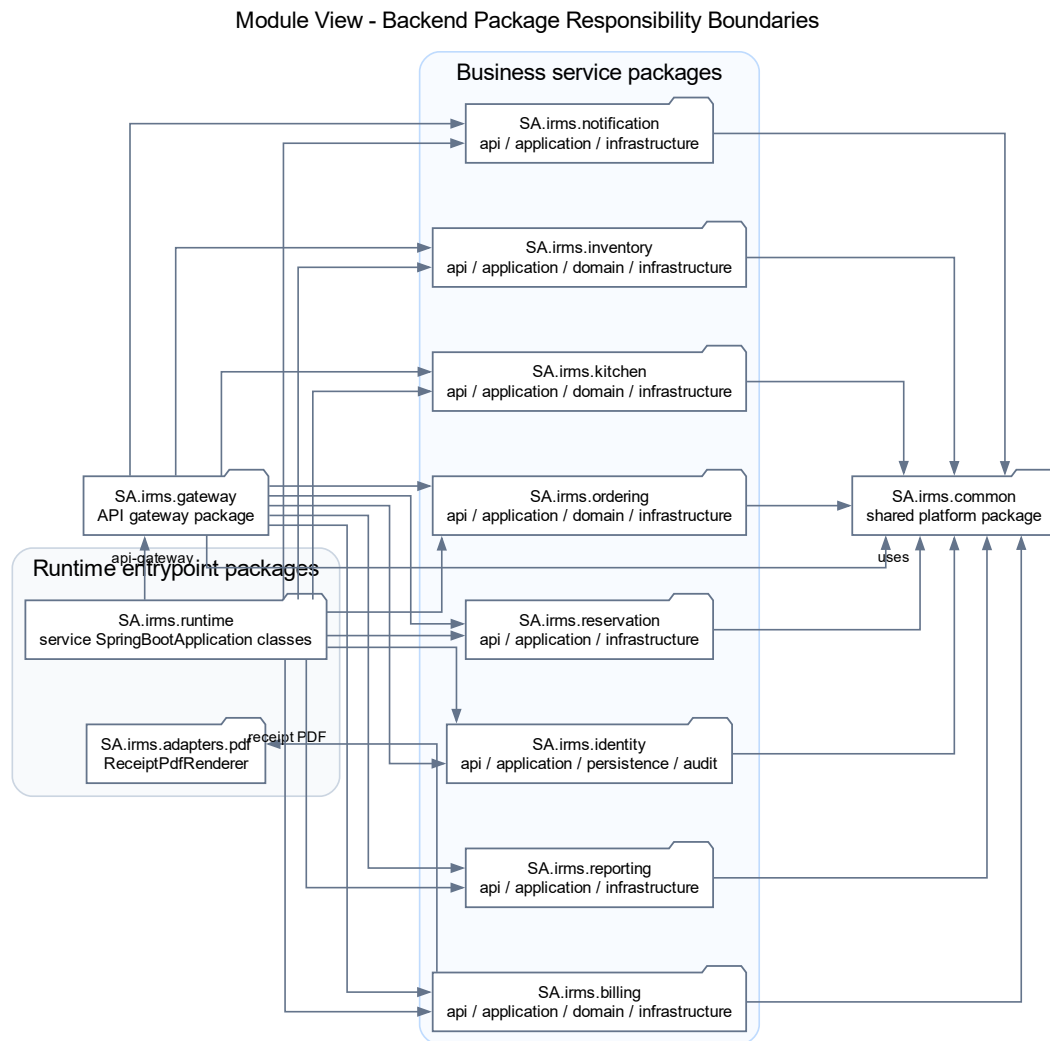
3.5.1 Các ứng dụng phía người dùng

- **Host/Reservation UI:** phục vụ đặt bàn, danh sách chờ, sơ đồ bàn và check-in.
- **Server POS UI:** phục vụ gọi món tại bàn, thay đổi món, theo dõi phiếu bếp và thực hiện tính tiền.
- **Kitchen Display UI:** hiển thị phiếu bếp theo trạm, mức ưu tiên, thời gian dự kiến hoàn thành và tiến độ chế biến.
- **Cashier UI:** tạo hóa đơn, tách hóa đơn, thanh toán, biên lai và hoàn tiền.
- **Manager Portal:** quản lý thực đơn, chương trình khuyến mãi, cảnh báo tồn kho, lịch làm việc và báo cáo.

3.5.2 Các dịch vụ nghiệp vụ

- **Reservation & Seating Service:** quản lý đặt bàn, danh sách chờ, bàn ăn và phiên phục vụ tại bàn.
- **Ordering Service:** quản lý thực đơn, tùy chọn món, đơn gọi món, dòng món và trạng thái của đơn từ đầu đến cuối.
- **Kitchen Coordination Service:** điều phối phiếu bếp, trạm bếp và cam kết thời gian ra món.
- **Billing Service:** quản lý hóa đơn, tách hóa đơn, thanh toán, biên lai, hoàn tiền và áp dụng khuyến mãi.
- **Inventory Service:** quản lý công thức, mặt hàng tồn kho, giao dịch kho và quy tắc cảnh báo mức tồn thấp.
- **Reporting Service:** quản lý mô hình đọc, ảnh chụp dữ liệu báo cáo và xuất báo cáo.
- **IAM/Access Service:** định danh, vai trò, quyền và thực thi chính sách truy cập.

3.5.3 Sơ đồ tổng quan



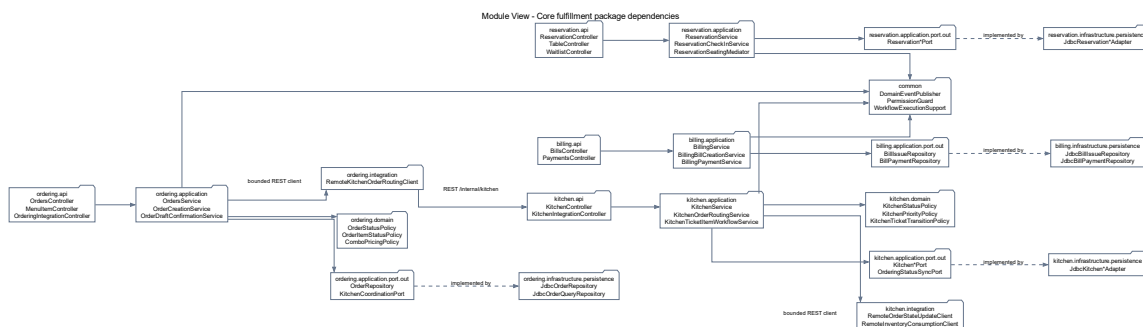
Hình 15: Module View tổng quan theo package backend của IRMS

Sơ đồ thể hiện cách các **package** và **class** trung tâm được chia theo nghĩa nghiệp vụ. ReservationService đại diện cho vùng đặt bàn; OrdersService xử lý tạo và xác nhận đơn; KitchenService xử lý trạng thái phiếu bếp; BillingService xử lý lập hóa đơn; còn InventoryService, ReportingQueryService, NotificationService, AuthService và AuditService giữ các vùng hỗ trợ và kiểm soát.

Một điểm quan trọng khác của sơ đồ là các **phụ thuộc giữa thành phần**. apiFetch gọi các endpoint đã khai báo trong ENDPOINTS, sau đó GatewayProxyController định tuyến đến service phù hợp. Các service nghiệp vụ cùng phát sự kiện qua RabbitMqEventPublisher, còn RabbitMqOutboxRelay xử lý phần chuyển tiếp nền.

Mô tả

- **Các thành phần frontend và gateway:** apiFetch, ENDPOINTS, permissions.ts, GatewayProxyController và GatewayRouteLocator.
- **Các service ứng dụng:** ReservationService, OrdersService, KitchenService, BillingService, InventoryService, ReportingQueryService, NotificationService, AuthService và AuditService.
- **Các thành phần sự kiện dùng chung:** RabbitMqEventPublisher và RabbitMqOutboxRelay.



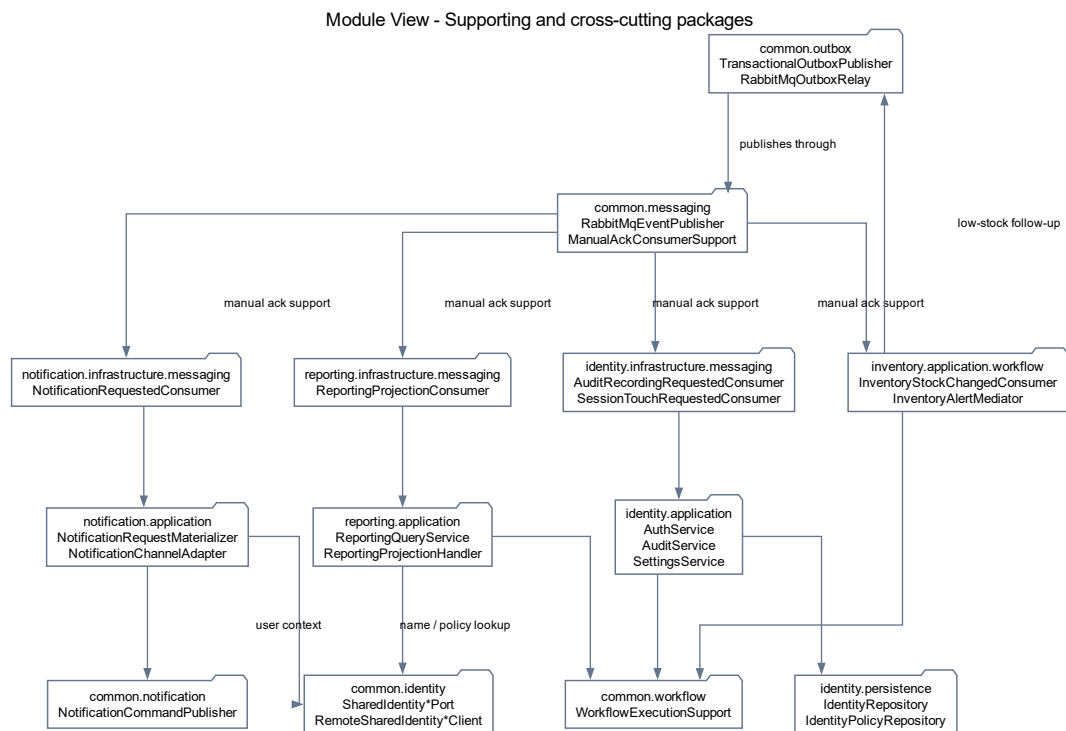
Hình 16: Module View chi tiết cho tuyến xử lý nghiệp vụ lõi

Trong cấu trúc này, OrdersService được đặt ở trung tâm để phản ánh vai trò cốt lõi của nghiệp vụ gọi món, nơi hệ thống phải kiểm tra trạng thái bàn, tính hợp lệ của món và tùy chọn, đồng thời chuyển thông tin sang bếp thông qua RemoteKitchenOrderRoutingClient. Service này tập trung vào xử lý dữ liệu gọi món và điều phối nghiệp vụ trực tiếp, tránh trở thành nơi chứa toàn bộ logic như khuyến mãi, tồn kho, thông báo hay phân tích.

Mối quan hệ giữa OrdersService, KitchenOrderRoutingService và BillingBillCreationService cũng thể hiện luồng nghiệp vụ rõ ràng: từ xác nhận đơn gọi món, sang điều phối chế biến tại bếp, rồi đến lập hóa đơn và thanh toán. Các service phía sau sử dụng dữ liệu đáng tin cậy từ đơn gọi món nhưng không can thiệp ngược vào cấu trúc nghiệp vụ gốc.

Mô tả

- **ReservationCheckInService và ReservationTableAssignmentService:** cung cấp ngữ cảnh bàn đang phục vụ.
- **OrdersService:** giữ vai trò trung tâm của tuyến nghiệp vụ nóng.
- **KitchenOrderRoutingService và BillingBillCreationService:** là hai hướng lan truyền quan trọng sau khi đơn được xác nhận.



Hình 17: Module View cho các package hỗ trợ và xuyên suốt

Hình trên mô tả các package hỗ trợ và các thành phần xử lý nền của hệ thống. Đây là những thành phần không nằm trực tiếp trong các giao dịch chính như gọi món hay thanh toán, nhưng vẫn đóng vai trò quan trọng đối với vận hành và quản lý hệ thống. Góc nhìn này tập trung vào các module liên quan đến messaging, event processing và runtime support thay vì các lớp nghiệp vụ lõi.

Trong kiến trúc IRMS, các consumer như InventoryStockChangedConsumer, ReportingProjectionConsumer, NotificationRequestedConsumer và AuditRecordingRequestedConsumer đảm nhận các chức năng khác nhau. InventoryStockChangedConsumer cập nhật biến động tồn kho dựa trên dữ liệu vận hành; ReportingProjectionConsumer xây dựng dữ liệu phục vụ báo cáo; NotificationRequestedConsumer xử lý việc gửi thông báo; còn AuditRecordingRequestedConsumer ghi nhận dữ liệu kiểm toán và theo dõi thao tác trong hệ thống.

Các thành phần này nhận dữ liệu sự kiện từ các service nghiệp vụ như ReservationService, OrdersService, KitchenService, BillingService và InventoryService thông qua RabbitMqEventPublisher. Nhờ đó, các chức năng hỗ trợ có thể được xử lý bất đồng bộ mà không ảnh hưởng trực tiếp đến các giao dịch chính.

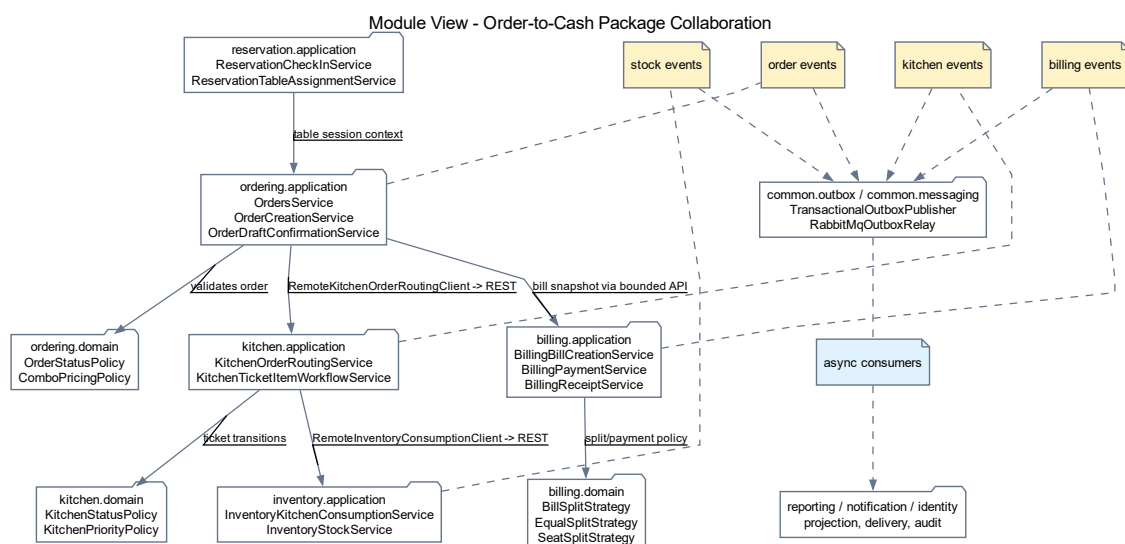
Việc tách riêng các service hỗ trợ giúp hệ thống dễ mở rộng và thuận tiện hơn cho quản trị vận hành. Các chức năng như báo cáo, cảnh báo tồn kho, phân quyền hay kiểm toán có thể hoạt động độc lập với tuyến xử lý chính của hệ thống.

Tuy nhiên, cách tổ chức này cũng dẫn đến độ trễ nhất định trong các chức năng hỗ trợ. Ví dụ, dữ liệu báo cáo hoặc cảnh báo tồn kho có thể cập nhật chậm hơn vài giây so với giao dịch thực tế. Đánh đổi này được chấp nhận vì các thao tác cần phản hồi ngay trong nhà hàng vẫn là đặt bàn, gọi món, chế biến và thanh toán.

Các thành phần chính trong hình.

- **ReservationService, OrdersService, KitchenService, BillingService và InventoryService:** là các thành phần nguồn phát sinh dữ kiện vận hành để nhóm hỗ trợ tiêu thụ.
- **RabbitMqEventPublisher và ManualAckConsumerSupport:** là lớp trung gian chuẩn hóa và xử lý sự kiện trước khi sang nhóm hỗ trợ.
- **InventoryStockChangedConsumer, ReportingProjectionConsumer và NotificationRequestedConsumer:** đại diện cho nhóm suy diễn, tổng hợp và tích hợp có thể chạy lệch nhịp với đường nóng.
- **AuditRecordingRequestedConsumer và AuditService:** đại diện cho nhóm kiểm soát xuyên suốt.
- **Các adapter kênh thông báo:** gồm EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter và PrintNotificationChannelAdapter.

3.5.4 Chi tiết Module View cho tuyến order-to-cash



Hình 18: Module View chi tiết cho tuyến order-to-cash của IRMS

Hình 18 đi sâu vào luồng nghiệp vụ Order-to-Cash: từ lúc khách nhận bàn, gọi món, báo xuống bếp, thanh toán cho đến khi hệ thống phát sự kiện thông báo và cập nhật báo cáo. Dưới góc nhìn Module View cho riêng lát cắt nghiệp vụ này, sơ đồ tập trung làm rõ các package và class chính tham gia xử lý như ReservationCheckInService, OrdersService, KitchenOrderRoutingService, BillingPaymentService và RabbitMqEventPublisher.

Trong thiết kế này, **mỗi service sẽ tự quản lý lớp điều phối và các port hạ tầng của riêng nó**. Cụ thể, ReservationSeatingMediator lo việc nhận và xếp bàn; OrdersService xử lý tạo đơn; RemoteKitchenOrderRoutingClient sẽ giao tiếp với KitchenIntegrationController; tiến độ làm món được KitchenTicketItemWorkflowService theo dõi; còn BillingPaymentService thì kết nối qua interface PaymentGateway. Các service này hoạt động độc lập, chỉ phụ thuộc nhau qua API, event hoặc interface chứ không truy cập chéo vào cấu trúc nội bộ của nhau.

Sơ đồ cũng thể hiện rõ việc phân tách giữa **luồng xử lý đồng bộ (synchronous)** và **luồng xử lý bất đồng bộ**

qua event (asynchronous). Các thao tác cần phản hồi cho user ngay lập tức (nhận bàn, chốt order, lập hóa đơn, thanh toán) sẽ được chạy đồng bộ. Ngược lại, các tác vụ "hậu cần" sẽ được đẩy vào RabbitMqEventPublisher để các consumer (ReportingProjectionConsumer, NotificationRequestedConsumer, AuditRecordingRequestedConsumer) tự động bắt lấy và xử lý. Việc tách bạch này giúp luồng chính chạy nhanh hơn và không bị block bởi các tác vụ phụ như gửi thông báo, lưu log audit hay tổng hợp báo cáo.

Cuối cùng, các controller và gateway đóng vai trò là các điểm chạm (entry point) đứng trước các service nghiệp vụ. Đây là nơi tiếp nhận request, validate dữ liệu và chuẩn hóa lời gọi. Nhờ có tầng biên này, các service nghiệp vụ bên trong không phải gánh thêm phần logic kỹ thuật, đồng thời đảm bảo ranh giới giao dịch (transaction boundary) của từng service được giữ nguyên vẹn.

Mô tả

- **Lát cắt service** được thể hiện bằng package và class thật: controller, service ứng dụng, interface và client tích hợp.
- **Tuyến phụ thuộc tĩnh** chỉ đi qua giao diện công khai và client REST cần thiết, nhờ đó tránh phụ thuộc chéo vào chi tiết nội bộ.
- **Tuyến hệ quả sự kiện** được tách sang RabbitMqEventPublisher và các bộ xử lý sự kiện nền, bảo vệ đường nóng khỏi tác vụ phụ.
- **PaymentGateway** là interface với hai hiện thực CashPaymentGateway và ExternalReferencePaymentGateway.
- **Quy tắc phụ thuộc runtime** ở hình này là cơ sở để kiểm chứng phân hiện thực hiện thực: nếu service truy cập xuyên qua chi tiết nội bộ của nhau, kiến trúc sẽ bị phá vỡ ngay ở mức thiết kế.

3.6 Góc nhìn thành phần và kết nối

Góc nhìn này cụ thể hóa các quyết định về gateway boundary, REST đồng bộ, event-driven processing, outbox, RabbitMQ và idempotency trong [ADR-0003](#), [ADR-0004](#), [ADR-0005](#), [ADR-0006](#), [ADR-0007](#) và [ADR-0010](#). Ở góc nhìn thành phần và kết nối, hệ thống được mô tả bằng các thành phần thực thi chính và các kiểu liên kết giữa chúng. Góc nhìn này đặc biệt quan trọng vì nó cho biết một yêu cầu đi qua những khối nào, khối nào giữ quyết định nghiệp vụ và khối nào chỉ đóng vai trò tích hợp hoặc hỗ trợ kỹ thuật.

3.6.1 Các thành phần chính

Góc nhìn này bóc tách hệ thống thành các khối thực thi cụ thể, đảm bảo sự phân tách rạch ròi giữa trải nghiệm giao diện, điều phối luồng, luật nghiệp vụ và tích hợp hạ tầng phần cứng lẫn phần mềm:

- **Các ứng dụng giao diện (Client Applications):** Gồm giao diện cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Lớp này tiếp nhận thao tác người dùng, gọi các endpoint trong ENDPOINTS của frontend và hiển thị lại dữ liệu do backend trả về:
 - Dùng apiFetch và ENDPOINTS để gọi REST API qua Nitro API Proxy.
 - Dùng permissions.ts và dữ liệu vai trò từ /api/auth/me để kiểm soát điều hướng theo quyền trên giao diện.
- **Cổng vào hệ thống (API Gateway & Frontend Proxy):** Frontend Node Server phục vụ React và chuyển tiếp /api; GatewayProxyController trong api-gateway định tuyến yêu cầu REST đến các service phía sau.

Thiết kế hiện tại chưa cho thấy WebSocket/SSE, rate limiting hay cấu hình Nginx riêng.

- **Các dịch vụ ứng dụng (Application Services):** Lớp này chịu trách nhiệm điều phối các use case. Thành phần này không chứa luật nghiệp vụ mà chỉ quản lý ranh giới giao dịch, gọi các đối tượng miền phù hợp và ủy quyền việc phát sự kiện.
- **Các service miền nghiệp vụ (Domain Services):** Đây là lớp lõi của kiến trúc, chứa các Aggregate Roots và các bất biến nghiệp vụ. Mọi quyết định nghiệp vụ như tính tiền hoặc xác định trạng thái bản chỉ được xử lý tại lớp này.
- **Lớp lưu trữ dữ liệu (Data Persistence):** Thực hiện lưu trữ thông qua mẫu Repository nhằm cô lập nghiệp vụ khỏi nền tảng SQL. Lớp này phân tách rõ giữa **Mô hình ghi (Transactional Schema)** phục vụ cho tốc độ giao dịch nóng và **Mô hình đọc (Reporting Projections)** phục vụ truy vấn phân tích để không gây nghẽn cổ chai cơ sở dữ liệu.
- **Bộ truyền sự kiện và xử lý nền (Event Bus & Background Workers):** Đảm bảo tính nhất quán cuối cùng (Eventual Consistency) của hệ thống. Hệ thống áp dụng **Outbox Pattern** với **Outbox Dispatcher** để quét và đẩy sự kiện một cách đáng tin cậy. Các hệ quả bất đồng bộ (như cập nhật kho, gửi thông báo) được đưa vào hàng đợi nền để không làm chậm luồng thao tác trực tiếp của nhân viên.
- **Các bộ kết nối thích nghi (Adapters):** Được thiết kế theo nguyên lý Đảo ngược phụ thuộc (Dependency Inversion), chia làm hai nhóm đã thấy trong thiết kế nhằm cách ly lỗi nghiệp vụ:
 - *Bộ kết nối thanh toán:* CashPaymentGateway và ExternalReferencePaymentGateway hiện thực interface PaymentGateway.
 - *Bộ kết nối thông báo và biên lai:* EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter, PrintNotificationChannelAdapter, PrintReceiptDeliveryAdapter và DigitalReceiptDeliveryAdapter.

3.6.2 Các kiểu kết nối chính

Để đáp ứng yêu cầu về hiệu năng cao và tính nhất quán của dữ liệu trong môi trường vận hành nhà hàng, IRMS kết hợp các kiểu kết nối đã có trong thiết kế và cấu hình. Các tương tác giữa các thành phần được định nghĩa qua bảng sau:

Bảng 36: Các kiểu kết nối chính trong IRMS

Kiểu kết nối	Vai trò và Đặc tính kỹ thuật	Ví dụ thực tế trong IRMS
Giao thức HTTP (REST API)	Kết nối đồng bộ (Synchronous), ưu tiên độ trễ thấp và phản hồi trạng thái giao dịch tức thì.	POS App gửi yêu cầu xác nhận đơn gọi món hoặc lập hóa đơn.
REST qua apiFetch	Frontend gọi định kỳ hoặc gọi lại sau thao tác người dùng để tải trạng thái mới.	Màn hình bếp gọi /api/kitchen/overview; màn hình báo cáo gọi các endpoint trong /api/reports.
Thông điệp Sự kiện (Pub/Sub)	Kết nối bất đồng bộ (Asynchronous) qua Message Broker, giúp giảm sự phụ thuộc trực tiếp (Decoupling) giữa các module.	Lan truyền sự kiện đơn hàng hoàn tất để trừ tồn kho và làm giàu dữ liệu báo cáo.
Adapter nội bộ	Interface và adapter trong thiết kế để cô lập biến thể nghiệp vụ hoặc kênh phát hành.	PaymentGateway, NotificationChannelAdapter và ReceiptDeliveryAdapter.
Kết nối Database (JDBC / Driver)	Truy xuất dữ liệu có cấu trúc và quản lý ranh giới giao dịch (Atomic Transactions).	Business Services lưu trữ trạng thái bàn và phiên phục vụ vào PostgreSQL.

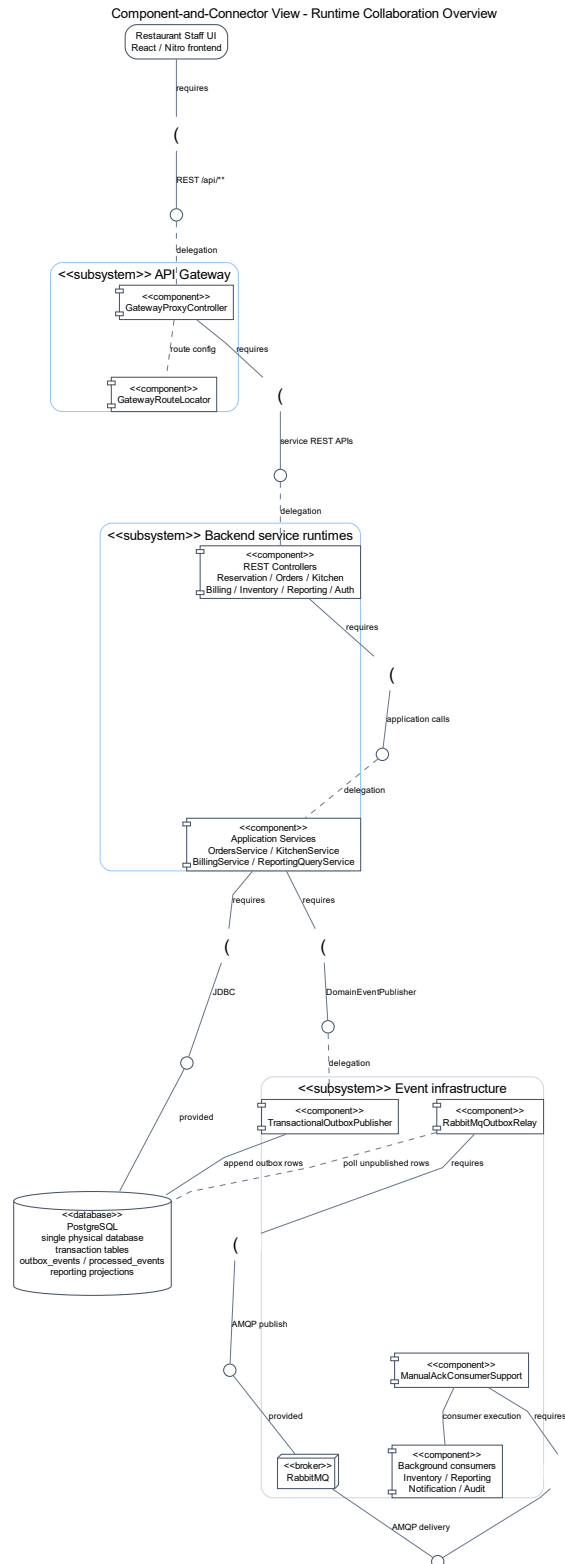
3.6.3 Cơ sở lựa chọn mô hình kết nối hỗn hợp

Kiến trúc IRMS không sử dụng duy nhất một kiểu kết nối mà kết hợp linh hoạt dựa trên hai nguyên tắc cốt lõi:

- Tuyến đầu ưu tiên đồng bộ (Sync for UX):** Các thao tác tương tác trực tiếp với khách hàng (đặt bàn, gọi món, thanh toán) được thực hiện qua API đồng bộ. Điều này giúp nhân viên nhận được phản hồi "Thành công" hoặc "Thất bại" ngay lập tức, đảm bảo luồng phục vụ không bị gián đoạn.
- Hậu phương ưu tiên bất đồng bộ (Async for Resilience):** Các hệ quả phát sinh sau giao dịch như gửi thông báo, cập nhật kho, audit và tổng hợp báo cáo được đẩy sang tuyến sự kiện qua RabbitMQ, outbox_events và các bộ xử lý nền. Việc này giúp bảo vệ hệ thống: nếu một tác vụ phụ bị chậm hoặc lỗi, nó cũng không làm nghẽn luồng nghiệp vụ chính.

Góc nhìn chuyên sâu về thành phần và kết nối:

Điểm then chốt trong kiến trúc IRMS là sự phân định rạch ròi giữa *kết nối phục vụ trải nghiệm người dùng* và *kết nối phục vụ phối hợp nội bộ*. Việc chọn đúng chiến lược giao tiếp cho từng ngữ cảnh không chỉ quyết định độ trễ phản hồi mà còn ảnh hưởng trực tiếp đến khả năng chịu lỗi (fault tolerance) và tính mở rộng của toàn hệ thống.

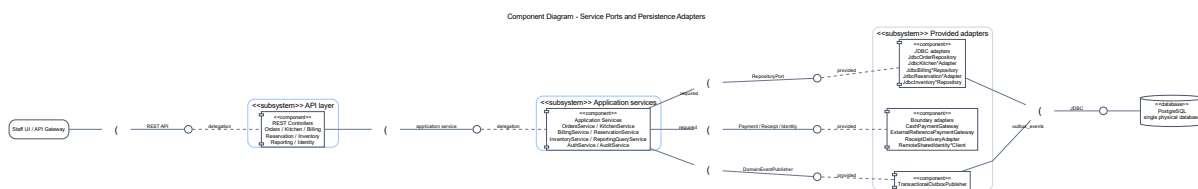


Hình 19: Sơ đồ thành phần và kết nối tổng quan của IRMS

Hình 19 thể hiện cấu trúc thực thi tổng quan của hệ thống theo cách nhìn thành phần. Thiết bị người dùng chỉ giao tiếp với lớp cổng vào; từ đó yêu cầu đi qua lớp điều phối ca sử dụng, xuống miền nghiệp vụ, lớp lưu trữ và các adapter biên. Ý nghĩa học thuật của hình này là làm rõ hệ thống không được tổ chức như một khối gọi chéo tùy ý, mà có hướng phụ thuộc rõ ràng từ ngoài vào trong.

Các thành phần chính

- **Các ứng dụng giao diện:** là nơi phát sinh yêu cầu từ lễ tân, phục vụ, bếp, thu ngân và quản lý.
- **Cổng vào giao diện lập trình ứng dụng:** tiếp nhận yêu cầu, xác thực, kiểm tra đầu vào và chuyển tiếp tới đúng ca sử dụng.
- **Các dịch vụ ứng dụng:** điều phối từng ca sử dụng cụ thể như tạo đặt bàn, xác nhận đơn gọi món, cập nhật phiếu bếp hoặc chốt hóa đơn.
- **Các service miền nghiệp vụ:** giữ luật nghiệp vụ cốt lõi, bất biến dữ liệu và quyết định nghiệp vụ của nhà hàng.
- **Lớp lưu trữ dữ liệu:** chịu trách nhiệm lưu trữ trạng thái giao dịch và bảo vệ ranh giới giao dịch.
- **Bộ phát sự kiện và bộ xử lý nền:** xử lý sự kiện và tác vụ nền không nên nằm trên tuyến giao dịch nóng.
- **Các port/adaptor biên:** bao bọc cách xử lý thanh toán, gửi thông báo và phát hành biên lai.



Hình 20: Sơ đồ thành phần chi tiết của lớp service nghiệp vụ trong IRMS

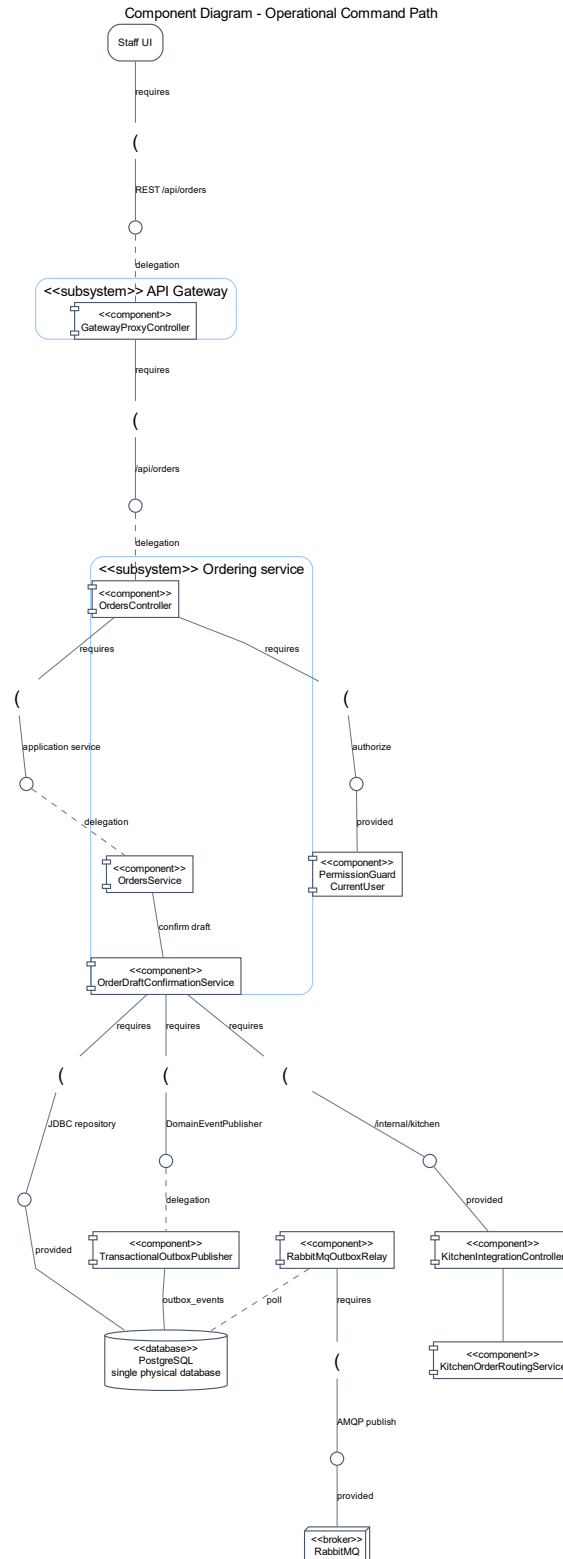
Hình 20 mở chi tiết lớp backend ở mức component, nhưng không trộn sang Class Diagram. Sơ đồ gom các controller REST, application service, required interface và provided adapter theo đúng vai trò trong thiết kế. Nhờ đó người đọc thấy rõ yêu cầu đi từ biên API vào lớp điều phối, sau đó phụ thuộc vào các port như repository, PaymentGateway, ReceiptDeliveryAdapter, SharedIdentity*Port và DomainEventPublisher.

Điểm chính của hình là hướng phụ thuộc được kiểm soát qua interface. Các application service không phụ thuộc trực tiếp vào JDBC adapter, gateway thanh toán hay cơ chế outbox cụ thể; những thành phần đó cung cấp lại năng lực qua adapter ở rìa. Cách biểu diễn này khớp với thiết kế hiện tại, nơi các lớp như JdbcOrderRepository, CashPaymentGateway, ExternalReferencePaymentGateway, RemoteSharedIdentity*Client và TransactionalOutboxPublisher nằm ở phía hiện thực kỹ thuật.

Các thành phần chính

- **API layer:** gom các controller REST của những module nghiệp vụ chính.
- **Application services:** gom các service điều phối use case như OrderService, KitchenService, BillingService, ReservationService, InventoryService, ReportingQueryService, AuthService và AuditService.
- **Required interfaces:** mô tả các port mà lớp ứng dụng cần, gồm port truy cập dữ liệu, integration port và event publisher.
- **Provided adapters:** mô tả các adapter cung cấp các port trên, gồm JDBC adapter, payment/receipt adapter, shared identity client và outbox publisher.

3.6.4 Sơ đồ thành phần cho tuyến lệnh vận hành



Hình 21: Sơ đồ thành phần cho tuyến lệnh vận hành của IRMS

Hình 21 mô tả luồng thực thi của một lệnh vận hành (operational command), từ giao diện người dùng đến khi dữ liệu được lưu trữ và các tác vụ phụ (side-effects) được kích hoạt. Trình tự này được thiết kế chặt chẽ để đảm bảo

hệ thống hoạt động ổn định và có khả năng truy vết (audit).

Đầu tiên, các yêu cầu đi qua bộ lọc xác thực và phân quyền ở tầng biên (edge) để kiểm tra tính hợp lệ trước khi chạm vào hệ thống lõi. Sau đó, request được chuyển đến các Application Service (Reservation, Ordering, Kitchen, Billing). Tại đây, Application Service chỉ đóng vai trò điều phối (orchestration): gọi các đối tượng Domain tương ứng, quản lý vòng đời giao dịch (transaction), và ghi sự kiện vào Outbox (Outbox Pattern) để chuẩn bị cho các tác vụ tiếp theo.

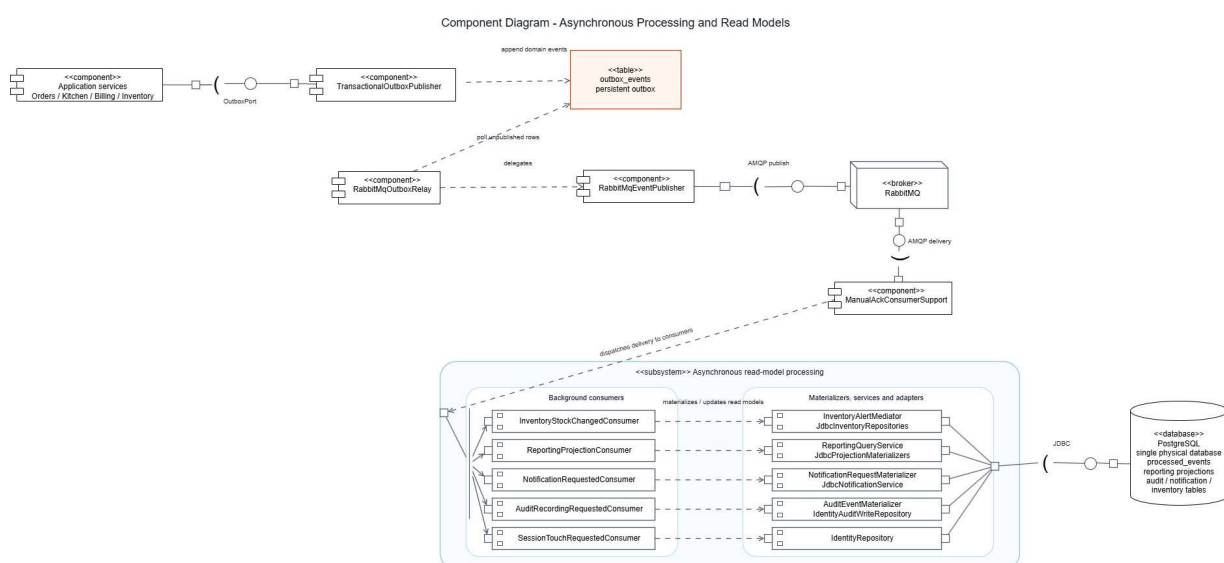
Thứ hai, thiết kế tách biệt rõ ràng giữa **giao dịch lõi** (core transaction) và **tác vụ phụ** (side-effects). Các giao dịch chính (như chốt đơn, ghi nhận thanh toán) bắt buộc phải được commit thành công vào cơ sở dữ liệu. Chỉ khi trạng thái lõi đã nhất quán, hệ thống mới phát sự kiện (event) để xử lý các tác vụ bất đồng bộ như báo xuống bếp, gửi thông báo hoặc cập nhật Read Model. Cơ chế này thực thi nguyên tắc “lõi đồng bộ, hỗ trợ bất đồng bộ”, giúp tối ưu thời gian phản hồi cho client nhưng vẫn đảm bảo toàn vẹn dữ liệu.

Cuối cùng, việc đặt PaymentGateway, ReceiptDeliveryAdapter và NotificationChannelAdapter ở rìa kiến trúc thể hiện rõ việc áp dụng mô hình Ports & Adapters. Tầng Domain chỉ định nghĩa các interface (Port), trong khi chi tiết tích hợp cụ thể với nhà cung cấp thanh toán hay máy in biên lai được giao cho các Adapter bên ngoài. Cách tổ chức này giúp lõi nghiệp vụ độc lập hoàn toàn với chi tiết hạ tầng.

Các thành phần chính

- **Cổng vào giao diện lập trình ứng dụng và lớp bộ điều khiển** cùng **bộ lọc xác thực / kiểm tra chính sách**: tiếp nhận yêu cầu, chuẩn hóa dữ liệu vào, xác thực và chặn hành vi vượt quyền trước khi đi vào lõi.
- **Các dịch vụ ứng dụng** của từng vùng Reservation, Ordering, Kitchen và Billing: điều phối use case, gọi đúng thành phần miền nghiệp vụ và kiểm soát biên giao dịch.
- **Các thành phần miền nghiệp vụ và kho lưu trữ**: hiện thực quy tắc miền và nơi lưu trạng thái bền vững.
- **Hộp thư ra / bộ phát sự kiện** cùng các port/adaptor như PaymentGateway, ReceiptDeliveryAdapter và NotificationChannelAdapter: tách bước chốt giao dịch khỏi bước xử lý biên hoặc xử lý nền.

3.6.5 Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc



Hình 22: Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc

Hình 22 mô tả kiến trúc event-driven sau khi các giao dịch lỗi đã được xác lập. Đây là Component Diagram của C&C View, không phải Class Diagram: các phần tử trong hình là service, outbox, RabbitMQ và bộ xử lý sự kiện nền đang tham gia vào tương tác runtime. Tại đây, IRMS tập trung vào việc hiện thực hóa nguyên tắc **Nhất quán cuối cùng (Eventual Consistency)** và **Tách biệt trách nhiệm truy vấn (CQRS - Command Query Responsibility Segregation)** ở mức độ thực dụng.

Cấu trúc phối hợp giữa các thành phần hỗ trợ được tổ chức như sau:

1. **Cơ chế phát và chuyển tiếp sự kiện:** Các service giao dịch ghi sự kiện vào outbox_events. Sau đó RabbitMqOutboxRelay đọc bản ghi chưa phát, dùng RabbitMqEventPublisher đẩy sự kiện sang RabbitMQ. Cấu trúc này khớp với TransactionalOutboxPublisher, RabbitMqOutboxRelay và cấu hình spring.rabbitmq.* trong backend.
2. **Cơ chế diễn dịch sự kiện (Event Projection):** Các sự kiện sau khi đi qua RabbitMQ được tiêu thụ bởi các bộ xử lý sự kiện như ReportingProjectionConsumer và InventoryStockChangedConsumer. Các thành phần này chuyển dữ liệu từ dạng giao dịch sang mô hình đọc hoặc projection phục vụ báo cáo và tồn kho, giúp yêu cầu từ màn hình quản lý không tranh chấp trực tiếp với luồng gọi món.
3. **Quản trị và Tuân thủ (Governance & Compliance):**
 - AuditRecordingRequestedConsumer: Độc lập với logic nghiệp vụ, chỉ tiêu thụ event để ghi audit log (lưu vết: ai, làm gì, khi nào, ngữ)
 - IAM / Authorization Service: Cung cấp các chính sách bảo vệ tài nguyên (Policy-based Access Control), đảm bảo các thao tác quản trị nhạy cảm như hoàn tiền hoặc điều chỉnh tồn kho luôn được kiểm soát chặt chẽ.
4. **Tích hợp thông báo và liên lạc (Notification Hub):** NotificationRequestedConsumer tiếp nhận sự kiện từ RabbitMQ; sau đó NotificationService ghi nhận thông báo qua các adapter kênh trong thiết kế:
 - EmailNotificationChannelAdapter, SmsNotificationChannelAdapter;
 - InAppNotificationChannelAdapter, PrintNotificationChannelAdapter.

Thiết kế hiện tại chưa thể hiện nhà cung cấp SMS/Email bên ngoài hay máy in vật lý, nên phần này chỉ được hiểu là biên adapter nội bộ.

Cơ sở thiết kế xử lý nền:

- **Tối ưu hóa tài nguyên ghi (Write Optimization):** Đẩy các tác vụ phụ (tổng hợp báo cáo, gửi thông báo) sang xử lý nền giúp giảm tải DB giao dịch chính, tối ưu tốc độ phản hồi cho các nghiệp vụ lõi (chốt đơn, thanh toán).
- **Khả năng quan sát (Observability):** Tách biệt các event consumer giúp dễ dàng monitor trạng thái và độ trễ của từng tác vụ (cập nhật projection, audit log, notification) thông qua log và các bảng outbox_events, processed_events.

3.7 Góc nhìn triển khai

3.7.1 Tổng quan phân bổ hệ thống

Ở góc nhìn triển khai, hệ thống được ánh xạ lên hạ tầng vật lý và hạ tầng thực thi nhằm làm rõ cách các thành phần phần mềm được phân bổ trên các nút triển khai (deployment nodes), cách các nút này giao tiếp với nhau và

cách hệ thống đáp ứng các yêu cầu phi chức năng như độ trễ, khả năng mở rộng và độ tin cậy. Mô hình này nhất quán với [ADR-0008](#) về một PostgreSQL vật lý duy nhất và [ADR-0014](#) về Docker Compose cho runtime cục bộ.

Hệ thống được tổ chức thành bốn tầng chính như sau:

1. Tầng thiết bị đầu cuối (Client Layer)

- Máy tính bảng dành cho nhân viên phục vụ, hỗ trợ thao tác đặt bàn và gọi món theo thời gian thực.
- Thiết bị lễ tân hoặc ki-ốt tiếp nhận khách, phục vụ việc đăng ký và phân bổ chỗ ngồi.
- Màn hình bếp tại từng trạm chế biến, hiển thị danh sách món cần chuẩn bị với yêu cầu cập nhật gần thời gian thực.
- Máy thu ngân, thực hiện các giao dịch thanh toán với yêu cầu độ trễ thấp và tính nhất quán cao.
- Trình duyệt của quản lý, phục vụ truy vấn báo cáo và giám sát hoạt động hệ thống.

Các thiết bị này hoạt động như các client gửi yêu cầu đồng bộ (synchronous request) tới hệ thống thông qua giao thức HTTP/HTTPS. Đặc điểm chung của tầng này là phụ thuộc vào mạng và yêu cầu độ trễ thấp đối với các thao tác nghiệp vụ chính.

2. Tầng cổng vào và phân phối (Gateway & Delivery Layer)

- Bộ đảo ngược lưu lượng hoặc **API Gateway**, đóng vai trò điểm truy cập duy nhất của hệ thống.
- Máy chủ phân phối giao diện tĩnh hoặc giao diện phía máy chủ, chịu trách nhiệm phục vụ nội dung frontend.

Tầng này chịu trách nhiệm:

- Điều phối yêu cầu từ client tới các service tương ứng.
- Thực hiện các kiểm tra kỹ thuật chung như xác thực, phân quyền và ghi nhận nhật ký truy cập.
- Giảm tải cho các service phía sau thông qua việc gom và chuẩn hóa các điểm truy cập.

Việc sử dụng API Gateway giúp cô lập tầng client với tầng ứng dụng, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và thay đổi cấu trúc backend trong tương lai.

3. Tầng ứng dụng (Application Layer)

- Khối xử lý phía sau bao gồm các service nghiệp vụ chính:
 - Service đặt bàn và quản lý chỗ ngồi.
 - Service gọi món và thực đơn.
 - Service điều phối bếp.
 - Service thanh toán.
 - Service quản lý tồn kho.
 - Service báo cáo và quản trị.
 - Service quản lý truy cập và kiểm toán.
- Bộ xử lý nền (Background Worker) thực hiện các tác vụ bất đồng bộ:
 - Xử lý sự kiện phát sinh từ hệ thống (event-driven processing).
 - Thực hiện các tác vụ retry khi xảy ra lỗi tạm thời.
 - Làm mới các ảnh chụp dữ liệu phục vụ báo cáo (reporting projections).

Các service trong tầng này có thể được triển khai trên cùng một cụm máy chủ hoặc container, nhưng được tách biệt về mặt logic theo miền nghiệp vụ. Các giao tiếp giữa service có thể bao gồm:

- Giao tiếp đồng bộ (REST API) cho các thao tác nghiệp vụ chính.
- Giao tiếp bất đồng bộ (message queue) cho các tác vụ nền.

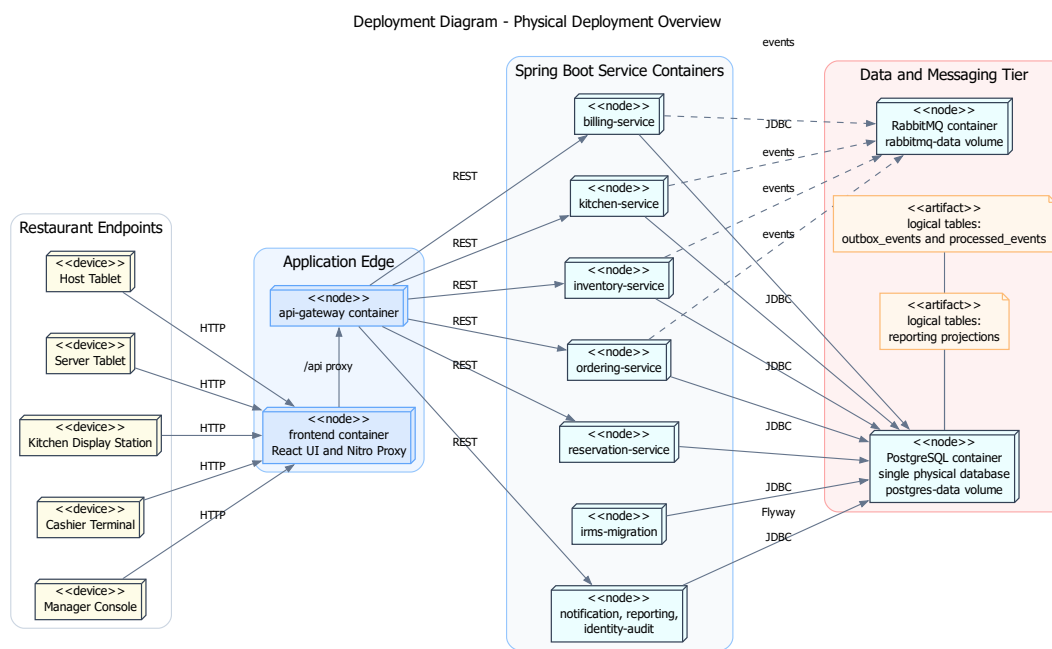
4. Tầng dữ liệu (Data Layer)

- Cơ sở dữ liệu giao dịch, lưu trữ dữ liệu vận hành chính như đơn hàng, thanh toán, tồn kho và người dùng.
- Mô hình đọc và khung nhìn báo cáo, được tối ưu cho truy vấn và phân tích dữ liệu.
- Các bảng outbox_events, processed_events và mô hình đọc báo cáo phục vụ xử lý nền và truy vấn quản trị.
- Hạ tầng RabbitMQ phục vụ hàng đợi sự kiện giữa các service.

Tầng dữ liệu được thiết kế tách biệt với tầng ứng dụng nhằm đảm bảo tính toàn vẹn dữ liệu, đồng thời hỗ trợ mở rộng theo nhu cầu đọc/ghi. Các thành phần như mô hình đọc và hệ thống logging thường được cập nhật thông qua luồng xử lý bất đồng bộ để giảm tải cho hệ thống giao dịch chính.

3.7.2 Cơ sở thiết kế triển khai

- **Tối ưu mạng nội bộ:** Thiết bị gọi món và bếp ưu tiên kết nối mạng LAN để giảm latency và đảm bảo vận hành liên tục khi kết nối WAN (Internet) không ổn định.
- **Độ sẵn sàng cao (High Availability):** Các module Billing và Ordering yêu cầu HA cao nhất. Cần ưu tiên giám sát, backup và khả năng scale vì ảnh hưởng trực tiếp đến doanh thu.
- **Tách biệt OLTP và OLAP:** Tách khối báo cáo khỏi tuyến giao dịch để các truy vấn phân tích (nặng) không gây nghẽn luồng xử lý gọi món và thanh toán (thời gian thực).
- **Bảo mật dữ liệu:** Áp dụng chính sách phân quyền và sao lưu riêng biệt cho dữ liệu thanh toán và audit log; đây là nhóm dữ liệu nhạy cảm, yêu cầu bảo vệ cao nhất.
- **Cô lập tài nguyên (Resource Isolation):** Tách bộ xử lý nền (Background Worker) giúp các tác vụ phụ như in biên lai điện tử hay gửi thông báo không tranh chấp tài nguyên với các thao tác tại quầy.



Hình 23: Sơ đồ triển khai tổng quan về bố trí vật lý của IRMS

Hình 23 mô tả cách hệ thống được triển khai trong môi trường thực tế của nhà hàng.

Điểm quan trọng nhất cần rút ra là IRMS được tổ chức theo mô hình **nhiều điểm chạm vật lý nhưng một lõi điều phối tập trung** . Các thiết bị tại hiện trường — như máy tính bảng phục vụ, thiết bị lễ tân, màn hình bếp và quầy thu ngân — được đặt gần người dùng để đảm bảo thao tác nhanh và thuận tiện. Tuy nhiên, toàn bộ trạng thái nghiệp vụ cốt lõi vẫn được duy trì tại khối xử lý trung tâm và hạ tầng dữ liệu, thay vì phân tán xuống các thiết bị.

Cách bố trí này giúp tránh tình trạng mất nhất quán khi mạng dao động hoặc khi nhiều thao tác xảy ra đồng thời tại các điểm khác nhau. Nó cũng phản ánh một nguyên tắc quan trọng: các thiết bị đầu cuối chỉ nên đóng vai trò tương tác, còn quyết định nghiệp vụ bền vững phải được neo tại backend.

Một đặc điểm thiết kế quan trọng khác là sự tách biệt giữa hai tuyến xử lý:

- **Tuyến xử lý trực tuyến (synchronous path)** phục vụ các thao tác yêu cầu phản hồi ngay như gọi món, cập nhật trạng thái bếp và thanh toán.
- **Tuyến xử lý bất đồng bộ (asynchronous path)** xử lý các hệ quả phụ thông qua **Outbox / hàng đợi** và bộ xử lý nền, bao gồm gửi thông báo, phát hành biên lai và cập nhật dữ liệu báo cáo.

Việc phân tách này cho phép hệ thống giữ được thời gian phản hồi ổn định trong giờ cao điểm, khi các thao tác trực tuyến cần được ưu tiên tuyệt đối.

Ngoài ra, sơ đồ cũng làm rõ vai trò của PostgreSQL, RabbitMQ, outbox_events và các bảng hoặc view báo cáo. Các tác vụ không yêu cầu phản hồi ngay được xử lý tách biệt, trong khi dữ liệu giao dịch chính vẫn được neo trong PostgreSQL để giữ tính nhất quán.

Từ góc độ vận hành, cách bố trí này còn mang lại khả năng **cô lập lỗi theo thành phần**. Khi bộ xử lý sự kiện nền, hàng đợi hoặc mô hình đọc báo cáo gặp sự cố, tuyến xử lý trực tuyến vẫn có thể tiếp tục hoạt động với mức suy giảm có kiểm soát.

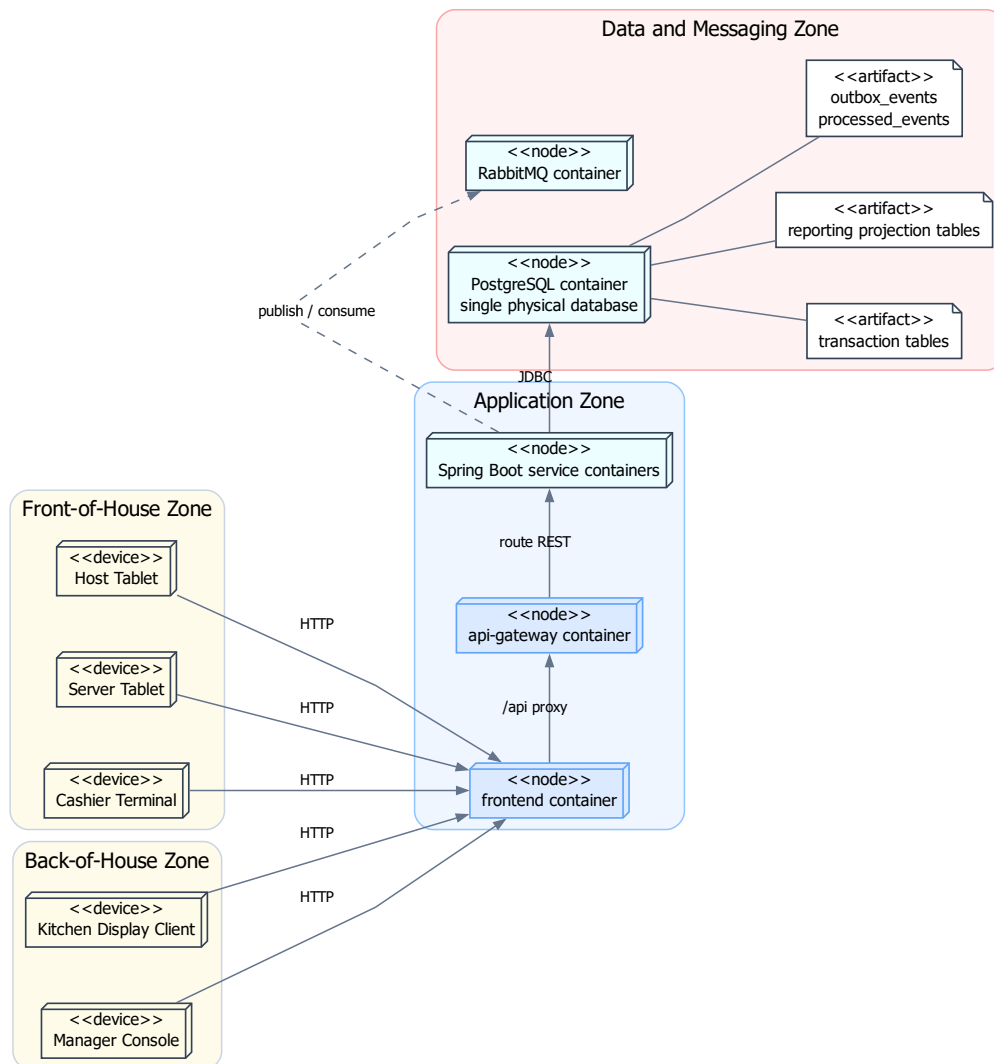
Các thành phần chính

- **Thiết bị đầu cuối:** máy tính bảng phục vụ, thiết bị lễ tân, màn hình bếp, máy thu ngân và trình duyệt quản lý.
- **Khối cổng vào:** frontend container và api-gateway container tiếp nhận, proxy và định tuyến yêu cầu từ thiết bị.
- **Khối xử lý trung tâm:** các container service Spring Boot thực hiện logic nghiệp vụ chính.
- **Hạ tầng sự kiện:** RabbitMQ, outbox_events và processed_events xử lý các tác vụ bất đồng bộ.
- **Hạ tầng dữ liệu:** PostgreSQL lưu dữ liệu giao dịch và mô hình đọc báo cáo.

Lý do thiết kế.

- Giữ một lõi xử lý trung tâm giúp đảm bảo tính nhất quán và đơn giản hóa triển khai.
- Tách tuyến bất đồng bộ giúp giảm tải cho tuyến trực tuyến.
- Phân lớp lưu trữ giúp hệ thống linh hoạt và dễ mở rộng hơn.

Deployment Diagram - Network Zones and Trust Boundaries



Hình 24: Sơ đồ triển khai theo vùng mạng và ranh giới tin cậy

Hình 24 mở rộng góc nhìn triển khai bằng cách làm rõ **ranh giới tin cậy** và cách hệ thống được phân chia thành các vùng mạng khác nhau.

Kiến trúc được chia thành các vùng: vùng thiết bị hiện trường, vùng ứng dụng, vùng dữ liệu và vùng hàng đợi sự kiện. Mỗi vùng đại diện cho một mức độ tin cậy và quyền truy cập khác nhau.

Nguyên tắc quan trọng nhất là **không có thành phần nào được phép vượt qua ranh giới mà không qua kiểm soát**. Các thiết bị đầu cuối chỉ giao tiếp với hệ thống thông qua frontend container và api-gateway. Các service phía sau mới được truy cập PostgreSQL và RabbitMQ.

Việc tách riêng vùng dữ liệu giúp bảo vệ dữ liệu nhạy cảm và giảm nguy cơ truy cập trái phép. Đồng thời, việc đặt RabbitMQ ở vùng riêng giúp cô lập tuyến sự kiện khỏi tuyến yêu cầu trực tuyến.

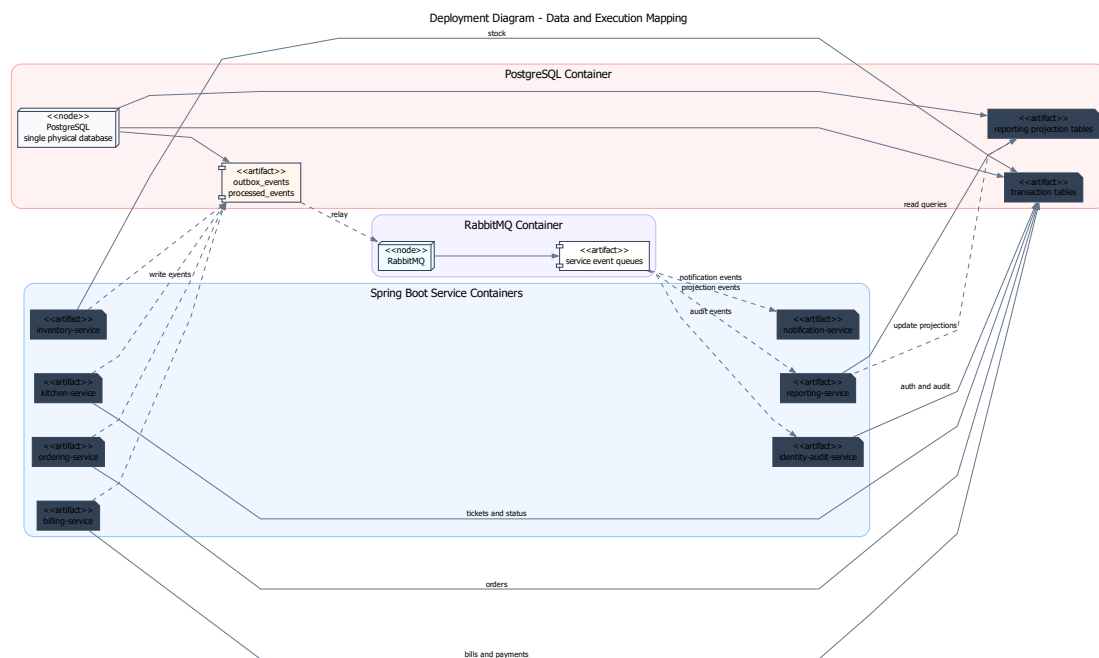
Sơ đồ này cũng hỗ trợ vận hành thực tế: khi xảy ra sự cố, việc phân vùng giúp nhanh chóng xác định lỗi thuộc lớp nào.

Các vùng chính.

- Vùng thiết bị hiện trường
- Vùng ứng dụng
- Vùng dữ liệu
- Vùng dữ liệu và hàng đợi sự kiện

Lý do thiết kế.

- Tăng cường bảo mật thông qua phân tách vùng
- Kiểm soát luồng truy cập giữa các thành phần
- Hỗ trợ vận hành và xử lý sự cố



Hình 25: Sơ đồ triển khai về ánh xạ dữ liệu và thực thi

Hình 25 làm rõ cách các service, dữ liệu và luồng thực thi được tổ chức trong hệ thống.

Sơ đồ này thể hiện rõ rằng IRMS theo **kiến trúc hướng dịch vụ kết hợp điều phối bằng sự kiện**. Các service nghiệp vụ xử lý giao dịch chính thông qua API đồng bộ, trong khi các hệ quả phụ được xử lý thông qua cơ chế bất đồng bộ dựa trên **Outbox và hàng đợi sự kiện**.

Luồng xử lý có thể chia thành hai phần:

- **Luồng giao dịch chính:** các service ghi dữ liệu vào cơ sở dữ liệu giao dịch.
- **Luồng bất đồng bộ:** các sự kiện được ghi vào Outbox, sau đó được bộ xử lý nền tiêu thụ để cập nhật mô hình đọc, gửi thông báo hoặc xử lý các tác vụ phụ.

Cách tổ chức này giúp đảm bảo rằng các giao dịch cốt lõi vẫn rõ ràng và nhất quán, trong khi hệ thống vẫn linh hoạt trong việc xử lý các nhu cầu mở rộng như báo cáo và tích hợp.

Ngoài ra, việc tách mô hình đọc khỏi dữ liệu giao dịch cho phép tối ưu hóa truy vấn mà không ảnh hưởng đến hiệu năng của hệ thống chính.

Các thành phần chính.

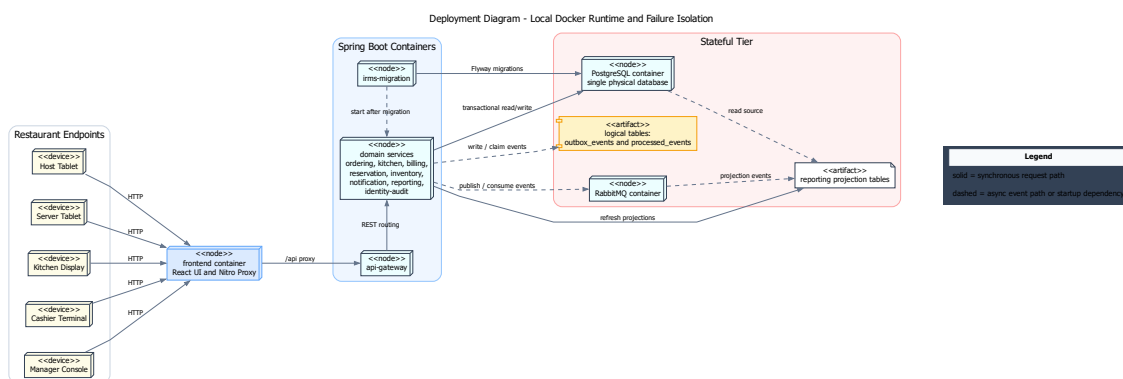
- Service nghiệp vụ
- Cơ sở dữ liệu giao dịch
- Outbox / hàng đợi sự kiện
- Bộ xử lý nền
- Mô hình đọc
- RabbitMQ và các consumer sự kiện

Lý do thiết kế.

- Giữ ranh giới service rõ ràng
- Tách xử lý bất đồng bộ để giảm phụ thuộc
- Tối ưu hóa truy vấn thông qua mô hình đọc

Ưu điểm của kiến trúc này là vừa đảm bảo tính rõ ràng của giao dịch, vừa hỗ trợ mở rộng linh hoạt. Đối lại, hệ thống cần được thiết kế cẩn thận về hợp đồng API, sự kiện và quyền sở hữu dữ liệu để tránh phụ thuộc chéo giữa các service.

3.7.3 Sơ đồ triển khai cho runtime Docker local và cô lập lỗi



Hình 26: Sơ đồ triển khai cho runtime Docker local và cô lập lỗi

Hình 26 mô tả môi trường runtime thực tế qua Docker Compose, tập trung vào cấu trúc container, luồng sự kiện và cơ chế cô lập lỗi (fault isolation).

- **Luồng Request:** Request từ frontend container (React/Node.js) đi qua api-gateway (sử dụng GatewayProxyController) để định tuyến REST đến các service Spring Boot. Phạm vi triển khai hiện tại tập trung vào cụm container đơn, chưa bao gồm Load Balancing hay Failover.

- **Quản lý dữ liệu:** Dữ liệu lưu tại PostgreSQL container với volume postgres-data. irms-migration sử dụng Flyway để thực hiện migration trước khi khởi động các service nghiệp vụ. Hiện tại hệ thống dùng một instance vật lý duy nhất, chưa triển khai Replication hay Hot-standby.
- **Cô lập lỗi (Fault Isolation):** Tách biệt giao dịch lỗi và các tác vụ phụ (side-effects). Giao dịch chính được xử lý đồng bộ; các tác vụ như cập nhật Read Model, gửi thông báo và ghi audit log được đẩy qua Outbox Pattern (outbox_events, processed_events) và RabbitMQ. Cơ chế này đảm bảo sự cố tại các module hỗ trợ không làm gián đoạn luồng nghiệp vụ chính tại nhà hàng.
- **Xử lý kịch bản lỗi:**
 - Gateway/Service lỗi: Endpoint tương ứng sẽ ngừng hoạt động dựa trên healthcheck của Docker.
 - Bộ xử lý nền lỗi: Sự kiện được lưu giữ trong Outbox table hoặc hàng đợi RabbitMQ để retry theo cấu hình.
 - Side-effect chậm: Không block luồng giao dịch chính; ưu tiên hoàn tất thanh toán và chốt đơn trước.
- **Suy giảm dịch vụ (Graceful Degradation):** Kiến trúc cho phép hệ thống duy trì hoạt động phần lõi khi các thành phần phụ gặp sự cố. Việc chấp nhận suy giảm có kiểm soát (chậm báo cáo/thông báo) giúp hệ thống tránh khỏi lỗi dây chuyền (cascading failure).

Mô tả

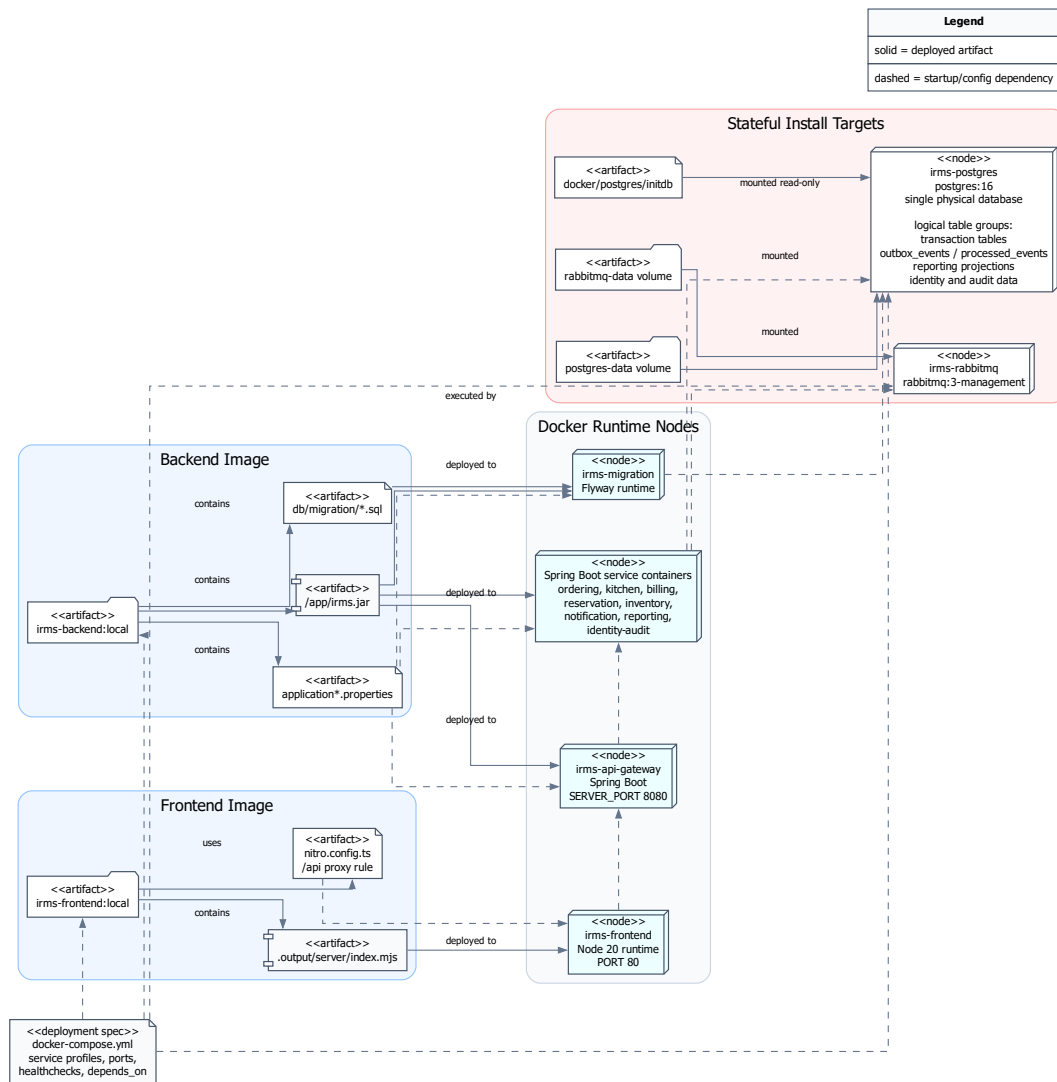
- **frontend container** tách giao diện React và proxy /api khỏi các service backend.
- **api-gateway** định tuyến REST đến các container service Spring Boot theo cấu hình thật.
- **PostgreSQL container** lưu dữ liệu giao dịch và các nhóm bảng hỗ trợ báo cáo trong một cơ sở dữ liệu vật lý duy nhất.
- **RabbitMQ container** cùng outbox_events và processed_events ngăn các tác vụ nền làm nghẽn trực tiếp tuyến giao dịch.
- **Reporting tables and views** cho phép tách nhu cầu đọc báo cáo khỏi phần xử lý lệnh nghiệp vụ.

Ngoài khả năng cô lập lỗi, sơ đồ cũng thể hiện rõ hướng mở rộng của hệ thống. Việc bổ sung thêm nút ứng dụng hoặc bộ xử lý nền có thể được thực hiện về sau, nhưng trong tài liệu hiện tại cần phân biệt rõ giữa cấu hình đã có trong docker-compose.yml và phương án mở rộng tương lai.

3.7.4 Install View cho gói triển khai Docker local

Deployment View ở trên trả lời các container và node runtime được đặt ở đâu. Install View dưới đây bổ sung góc nhìn còn lại: artifact, cấu hình và migration nào được đóng gói vào từng container khi chạy theo Docker Compose. Phần này chỉ mô tả runtime hiện tại, không bổ sung các hạ tầng như Kubernetes, dịch vụ đám mây, cân bằng tải, bản sao đọc dữ liệu hoặc nhà cung cấp ngoài khi chưa thuộc phạm vi triển khai.

UML Deployment Diagram - Install View for Local Docker Runtime



Hình 27: UML Deployment Diagram cho Install View của gói Docker local

Hình 27 dùng UML Deployment Diagram để thể hiện các node runtime và artifact cài đặt của gói Docker local. irms-backend:local chứa /app/irms.jar, các file application*.properties và migration SQL; cùng artifact JAR này được triển khai vào api-gateway, các service Spring Boot và irms-migration, nhưng vai trò runtime khác nhau nhờ profile và biến môi trường trong docker-compose.yml. irms-frontend:local chứa bundle Nitro ở .output/server/index.mjs; nitro.config.ts điều khiển proxy /api/** sang api-gateway. Các node trạng thái chỉ gồm PostgreSQL với postgres-data và RabbitMQ với rabbitmq-data, đúng với cấu hình hiện tại.

Bảng 37: Install View cho artifact backend và frontend

Artifact/config item	Nơi sử dụng	Vai trò runtime
Backend Spring Boot artifact irms.jar	backend/Dockerfile; backend/pom.xml; docker-compose.yml.	Image irms-backend:local được dùng lại cho api-gateway, các service nghiệp vụ và irms-migration. Vai trò service được chọn bằng SPRING_PROFILES_ACTIVE, IRMS_RUNTIME_SERVICE và SERVER_PORT.
Migration runtime và SQL migrations	Giai đoạn khởi tạo schema dữ liệu.	Container irms-migration chạy Flyway trước service nghiệp vụ, với điều kiện service_completed_successfully.
Frontend bundle / Nitro server	frontend/Dockerfile; frontend/package.json; frontend/nitro.config.ts.	Image irms-frontend:local chạy node .output/server/index.mjs, phục vụ UI React/TanStack và proxy /api/** tới api-gateway.
Runtime application properties	application.properties; application-docker.properties; application-*--service.properties.	Cấu hình datasource PostgreSQL, RabbitMQ, Flyway, CORS, security token, timeout phiên, outbox và URL service nội bộ.

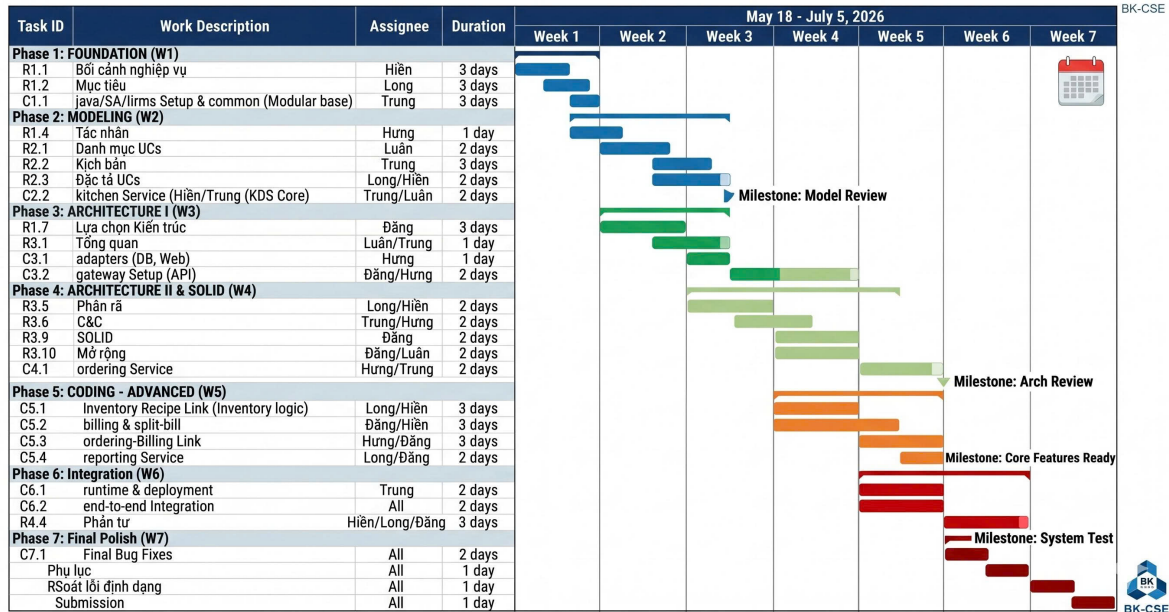
Bảng 38: Install View cho thành phần trạng thái và cấu hình Docker

Thành phần	Nơi sử dụng	Vai trò runtime
irms-postgres và postgres-data	docker-compose.yml; docker/postgres/initdb.	Một PostgreSQL container vật lý duy nhất lưu dữ liệu giao dịch, outbox_events, processed_events, các bảng projection báo cáo và dữ liệu identity/audit trong cùng PostgreSQL; các nhóm này không phải database riêng.
irms-rabbitmq và rabbitmq-data	docker-compose.yml; application.properties; application-docker.properties.	Broker cho tuyến event-driven; các service dùng spring.rabbitmq.*, manual ack và publisher confirm khi IRMS_RABBITMQ_ENABLED=true.
Thứ tự khởi động Docker	depends_on và healthcheck trong docker-compose.yml.	postgres và rabbitmq healthy trước, irms-migration hoàn tất, sau đó Spring Boot services chạy và cuối cùng frontend phụ thuộc api-gateway.

Bảng thể hiện thứ tự cài đặt/khởi động của phạm vi triển khai hiện tại: postgres và rabbitmq phải healthy trước, irms-migration chạy thành công để tạo/cập nhật schema, sau đó các service Spring Boot và cuối cùng là frontend phụ thuộc vào api-gateway. Đây là Install View của runtime Docker local hiện có, không phải mô tả triển khai cloud hay cụm sẵn sàng cao.

3.7.5 Assignment view

IRMS 7-WEEK INTEGRATED DEVELOPMENT SCHEDULE (4 WEEKS REPORT, 3 WEEKS CODE)



Hình 28: Góc nhìn lịch trình và phân công công việc

3.8 Nguyên tắc thiết kế

Kiến trúc của IRMS bám theo các nguyên tắc thiết kế sau:

- **High Cohesion, Low Coupling:** Mỗi service chỉ expose API, event và data contract cần thiết; ẩn hoàn toàn chi tiết triển khai (encapsulation).
- **Separation of Concerns (SoC):** Tách biệt rõ rệt vai trò giữa Controller, Application Service, Domain và Infrastructure.
- **Dependency Inversion:** Domain core chỉ phụ thuộc vào abstraction (interface); các adapter hạ tầng phụ thuộc ngược lại vào core, tránh gắn chặt vào library hoặc provider cụ thể.
- **Synchronous API (Core Transactions):** Xử lý đồng bộ các nghiệp vụ cần phản hồi tức thời như đặt bàn, gọi món và thanh toán để đảm bảo tính nhất quán trạng thái.
- **Asynchronous Events (Side-effects):** Các tác vụ phụ như cảnh báo kho, cập nhật báo cáo, thông báo và audit log được xử lý bất đồng bộ qua Outbox Pattern và Event Bus.
- **Domain-centric Modeling:** Tập trung mô hình hóa hành vi nghiệp vụ thực tế (Domain Behavior) thay vì thiết kế dựa trên quan hệ dữ liệu (Database-centric) thuần túy.

3.9 Áp dụng nguyên lý SOLID vào thiết kế

Phần này đối chiếu trực tiếp các nguyên tắc SOLID với thiết kế thực tế (implementation) của IRMS qua các class và interface tiêu biểu, thay vì liệt kê lý thuyết đơn thuần. Các interface như PaymentGateway, NotificationChannelAdapter và ReceiptDeliveryAdapter cụ thể hóa quyết định dùng boundary adapters trong [ADR-0011](#); cách tách lớp api, application, domain và infrastructure liên kết với [ADR-0013](#).

Bảng 39: Đối chiếu nguyên tắc SOLID với thiết kế IRMS

Nguyên tắc	Ví dụ thiết kế tiêu biểu	Ý nghĩa thiết kế
SRP	BillingPaymentService; BillingReceiptService; KitchenTicketTransitionPolicy; ManualAckConsumerSupport; RabbitMqOutboxRelay.	Trách nhiệm được tách theo service, policy, adapter và consumer; khi thay đổi thanh toán, trạng thái bếp hoặc retry RabbitMQ, phạm vi sửa được khoanh vào nhóm class đúng vai trò.
OCP	PaymentGateway với CashPaymentGateway, ExternalReferencePaymentGateway; BillSplitStrategy với các strategy chia hóa đơn; ReorderRecommendationStrategy.	Các điểm dễ thay đổi được mở qua interface/strategy/policy, nên có thể thêm biến thể mới bằng class hiện thực mới và cấu hình bean.
LSP	BillingPaymentService nhận List<PaymentGateway> và gọi supports(...); BillingSplitService nhận List<BillSplitStrategy>.	Service sử dụng contract ổn định; implementation có thể thay thế trong phạm vi hợp đồng đã định nghĩa.
ISP	PaymentGateway; ReceiptDeliveryAdapter; NotificationChannelAdapter; KitchenTicketItemRepositoryPort; DomainEventPublisher.	Interface được giữ nhỏ và gắn với một vai trò cụ thể, giúp client chỉ phụ thuộc vào hành vi nó thật sự cần.
DIP	BillingPaymentService phụ thuộc BillPaymentRepository, PaymentGateway, DomainEventPublisher; BillingReceiptService phụ thuộc ReceiptDeliveryAdapter; JdbcKitchenTicketItemRepository implements KitchenTicketItemRepositoryPort.	Service cấp cao phụ thuộc abstraction/port, còn hạ tầng hiện thực chi tiết JDBC, RabbitMQ hoặc adapter kênh.

Thanh toán và biên nhận. BillingPaymentService không biết chi tiết xử lý từng phương thức thanh toán, mà làm việc qua PaymentGateway. Các hiện thực như CashPaymentGateway và ExternalReferencePaymentGateway được xem là adapter thanh toán ở biên. Tương tự, BillingReceiptService phát hành biên nhận qua ReceiptDeliveryAdapter với DigitalReceiptDeliveryAdapter và PrintReceiptDeliveryAdapter. Đây là ví dụ mạnh cho OCP, ISP và DIP: service lõi giữ luồng thanh toán, còn biến thể phương thức/kênh nằm ở lớp hiện thực.

Thông báo. Nhóm NotificationChannelAdapter được tách thành các kênh nhỏ, gồm:

- EmailNotificationChannelAdapter, SmsNotificationChannelAdapter;
- InAppNotificationChannelAdapter, PrintNotificationChannelAdapter;
- PhoneNotificationChannelAdapter.

Những class này giúp service thông báo kiểm tra và ghi nhận kênh gửi mà không biến phần notification thành một interface quá lớn. Các adapter này được mô tả như điểm mở rộng nội bộ cho các kênh gửi, không làm lỗi thông báo phụ thuộc trực tiếp vào một kênh cụ thể.

Policy và strategy nghiệp vụ. DomainPolicyConfiguration đăng ký các nhóm policy/strategy như:

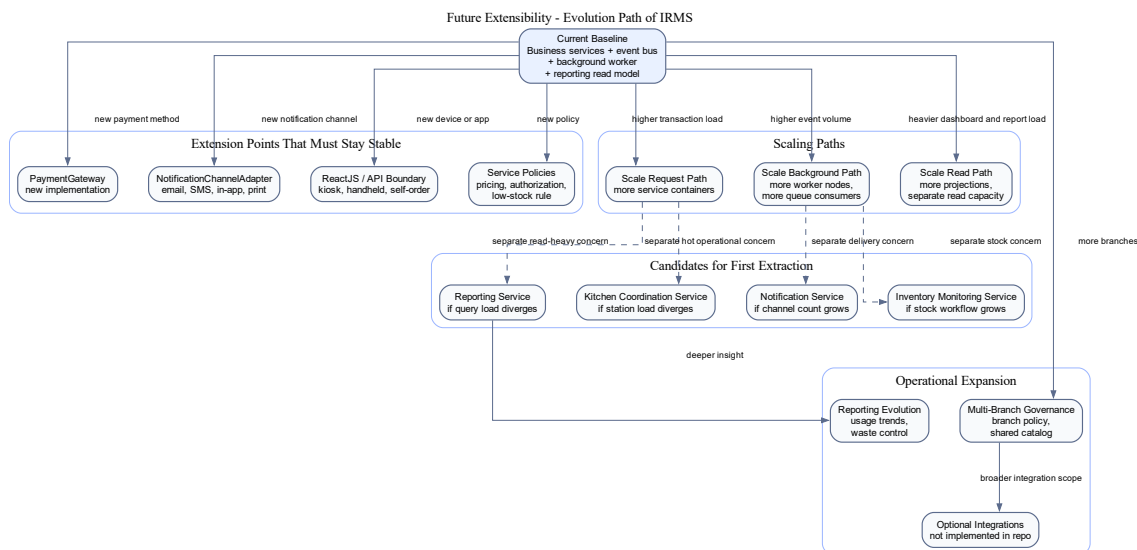
- OrderStatusPolicy, ComboPricingPolicy;
- KitchenTicketTransitionPolicy, KitchenDeadlinePolicy;
- LowStockSeverityPolicy, ReorderRecommendationStrategy;
- các hiện thực BillSplitStrategy.

Cách tách này làm rõ SRP: quy tắc chuyển trạng thái, tính deadline, đề xuất nhập hàng hoặc tách hóa đơn không bị trộn vào controller. Đồng thời, các strategy như BillSplitStrategy và ReorderRecommendationStrategy là điểm mở rộng OCP được thể hiện rõ trong thiết kế.

Outbox, RabbitMQ và bộ xử lý sự kiện nền. Ở tuyến event-driven, TransactionalOutboxPublisher hiện thực DomainEventPublisher, RabbitMqOutboxRelay phụ trách chuyển outbox sang RabbitMQ, còn ManualAckConsumerSupport gom logic ack, retry, deduplication và dead-letter cho consumer. Phần này là ví dụ SRP và DIP ở tầng hạ tầng: service nghiệp vụ chỉ publish event qua abstraction, còn cách phát, retry hoặc dead-letter nằm trong component kỹ thuật riêng.

Giới hạn cần tránh diễn giải quá mức. Các ví dụ trên cho thấy IRMS có áp dụng SOLID tại những điểm thiết kế chính, đặc biệt ở service, adapter, policy, strategy, port truy cập dữ liệu và event infrastructure. Điều đó không có nghĩa mọi class đều hoàn hảo theo SOLID. Ngoài ra, các interface như PaymentGateway, ReceiptDeliveryAdapter và NotificationChannelAdapter xác định ranh giới trừu tượng, không làm cho lỗi nghiệp vụ phụ thuộc vào một kênh thanh toán, thông báo hoặc in ấn cụ thể.

3.10 Khả năng mở rộng trong tương lai



Hình 29: Lộ trình mở rộng kiến trúc của IRMS trong tương lai

Hình 28 mô tả lộ trình tiến hóa (evolutionary path) của hệ thống khi mở rộng quy mô (tăng thiết bị, chi nhánh, tích hợp) mà không cần tái cấu trúc tầng lõi (Core Domain).

Trạng thái kiến trúc hiện tại: IRMS áp dụng mô hình SOA kết hợp Event-driven. Các nghiệp vụ chính (Core Transactions) sử dụng Sync API để đảm bảo tính nhất quán; các tác vụ phụ (Side-effects) được xử lý bất đồng bộ qua Event. Thiết kế này cho phép các service hỗ trợ như Reporting, Notification, Inventory có thể scale độc lập tùy theo tải trọng thực tế.

Nguyên tắc mở rộng không phá vỡ kiến trúc: Khả năng mở rộng (Extensibility) của IRMS được đo lường bằng việc duy trì sự ổn định của Core Domain (Order, Billing) khi thay đổi các thành phần biên:

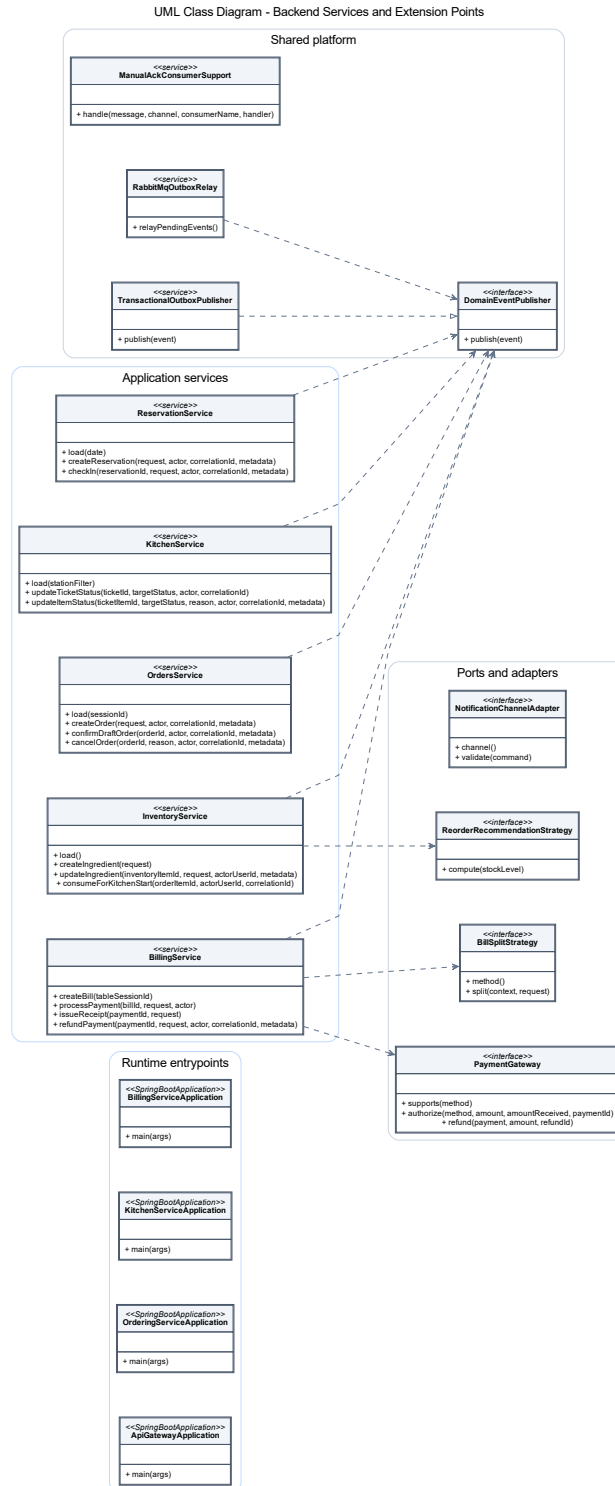
- **Tích hợp thanh toán/thông báo:** Chỉ tác động đến PaymentGateway và NotificationChannelAdapter.
- **Mở rộng quy mô:** Thêm chi nhánh hoặc thiết bị (Kiosk, Mobile) chỉ yêu cầu điều chỉnh tại tầng Deployment và API layer.
- **Điểm mở rộng chiến lược:** Các thành phần như Gateway, Adapter, Policy (quy tắc định giá/tồn kho) và Read Model cần được giữ ổn định để đảm bảo hệ thống tiến hóa có kiểm soát.

Các hướng mở rộng chính:

- **Mở rộng tích hợp (Integration):** Thêm Provider (thanh toán, thông báo) hoặc thiết bị ngoại vi qua mô hình Ports & Adapters.
- **Mở rộng theo tải (Scaling):** Tăng số lượng instance ứng dụng, Background Worker hoặc tách biệt hoàn toàn Read Model để tối ưu truy vấn.
- **Mở rộng phạm vi vận hành:** Hỗ trợ đa chi nhánh và quản trị tập trung (Centralized Management) trên cùng một mô hình nghiệp vụ lõi.
- **Mở rộng theo dịch vụ:** Chi tách thành các Microservices độc lập đối với những vùng có nhịp thay đổi (rate of change) hoặc tải trọng khác biệt hoàn toàn.

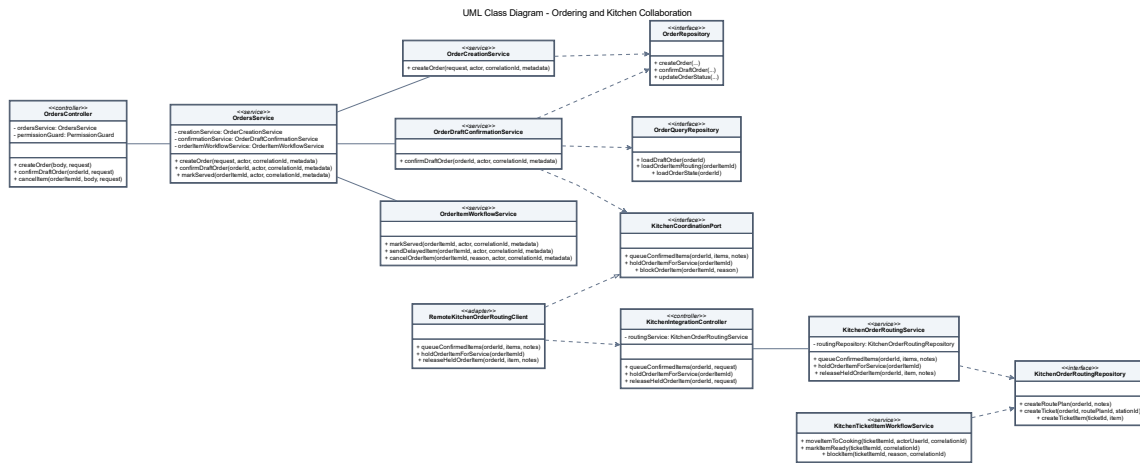
4 Thiết kế chi tiết

4.1 Góc nhìn lớp



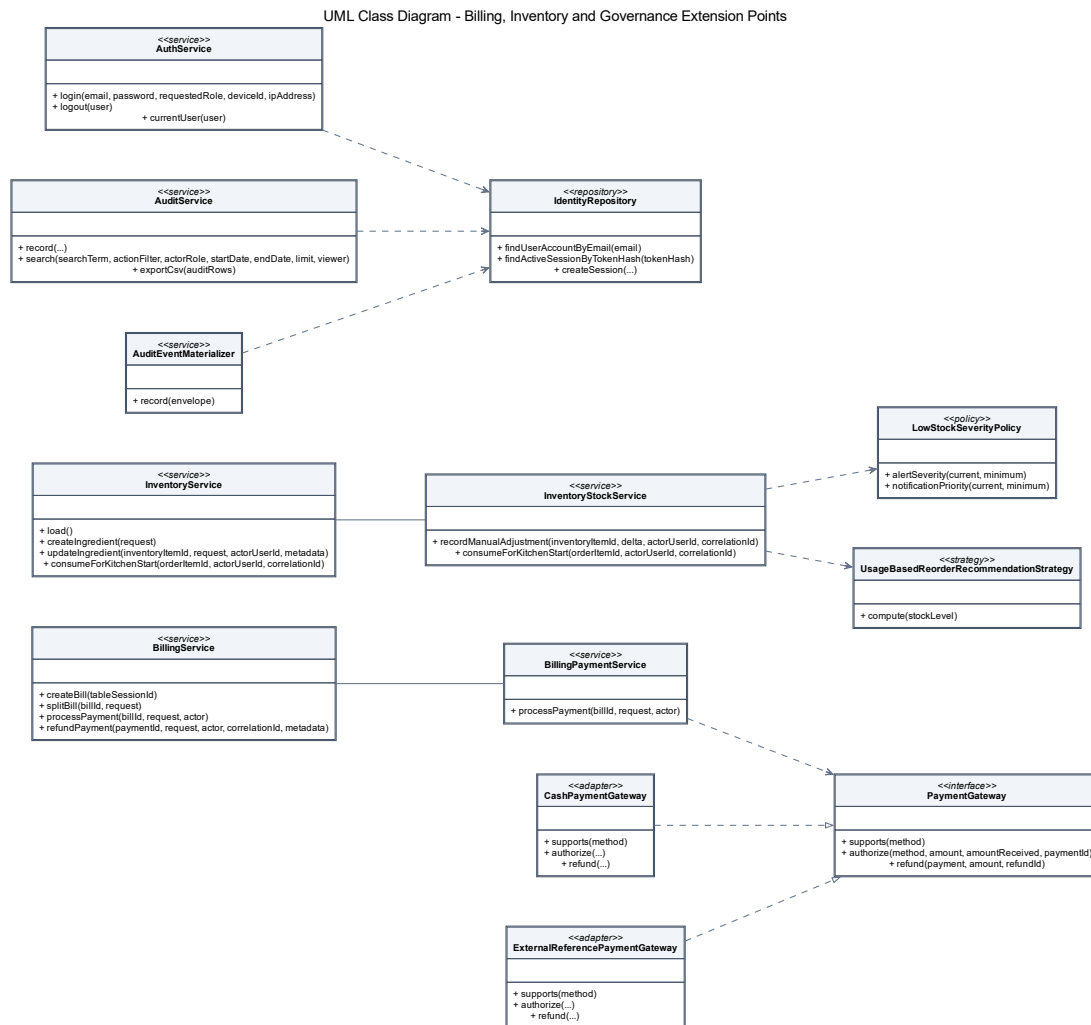
Hình 30: Sơ đồ lớp tổng quan mô tả các service ứng dụng và các điểm mở rộng chính của backend IRMS

Hình 29 đặt các class entypoint và service chính cạnh các interface dùng chung, gồm DomainEventPublisher, PaymentGateway, BillSplitStrategy, ReorderRecommendationStrategy và NotificationChannelAdapter.



Hình 31: Sơ đồ lớp mô tả quan hệ giữa cụm ordering và kitchen

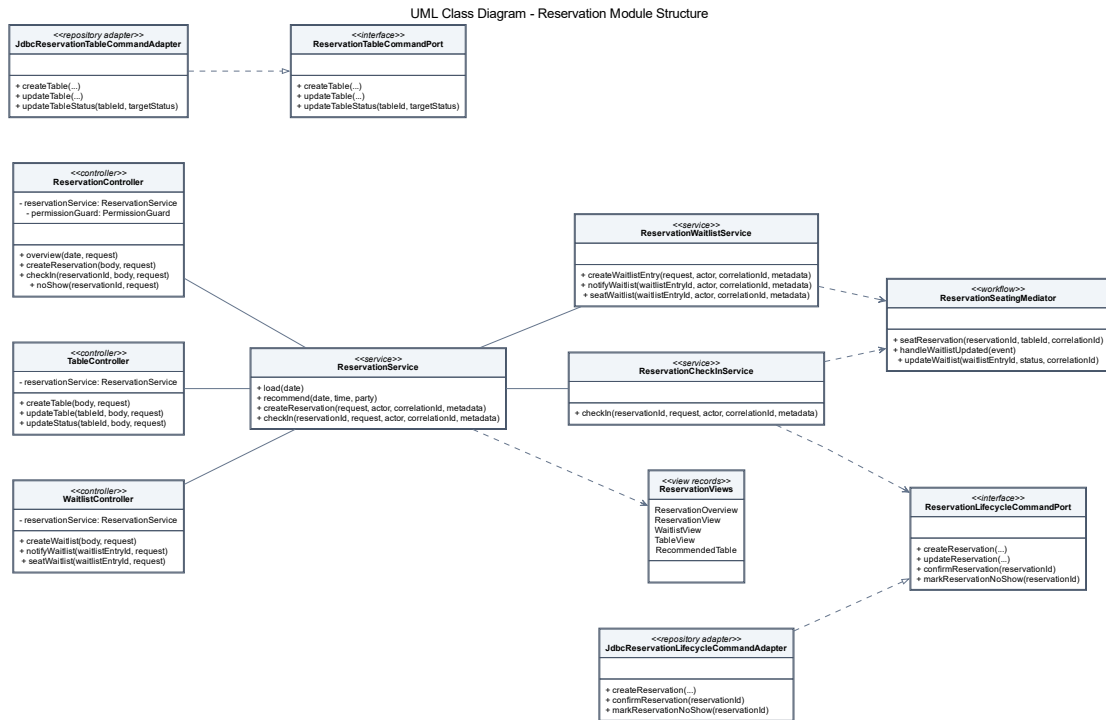
Hình 30 mô tả quan hệ giữa OrdersController, OrdersService, các service con của module ordering, cổng KitchenCoordinationPort, adapter RemoteKitchenOrderRoutingClient và phía nhận ở module kitchen. Quan hệ này khớp với C&C View: module gọi món không truy cập thẳng persistence của bếp mà đi qua client tích hợp và controller nội bộ.



Hình 32: Sơ đồ lớp mô tả các điểm mở rộng của cụm billing, inventory và governance

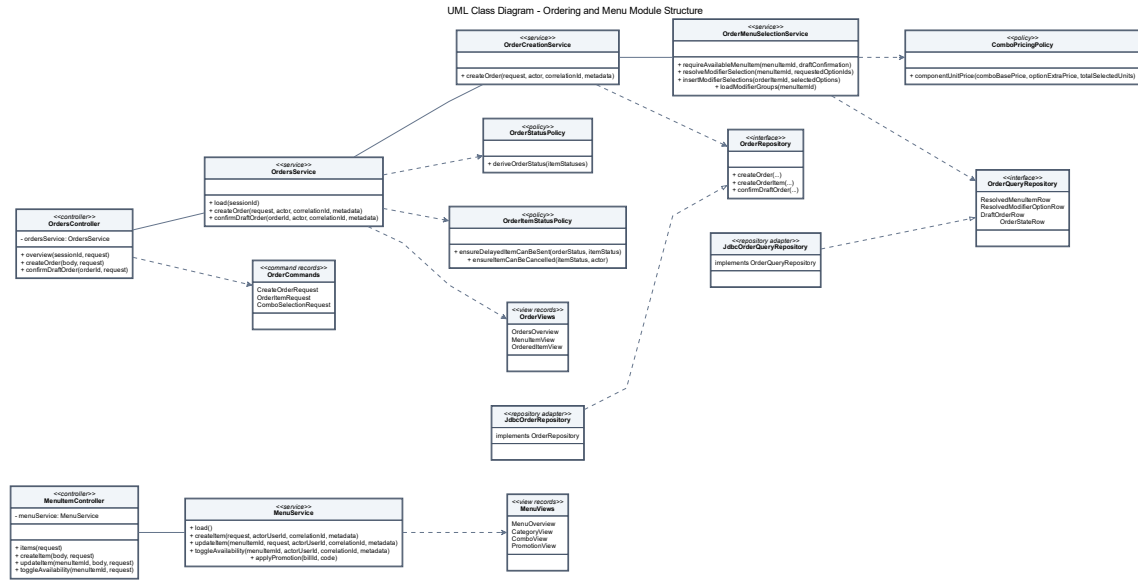
Hình 31 gom các điểm mở rộng nhạy cảm của hệ thống. Phần billing có PaymentGateway và các adapter thanh toán. Phần inventory có LowStockSeverityPolicy và UsageBasedReorderRecommendationStrategy. Vùng kiểm soát có AuthService, AuditService, AuditEventManager và IdentityRepository.

4.2 Giải thích theo từng cụm lớp



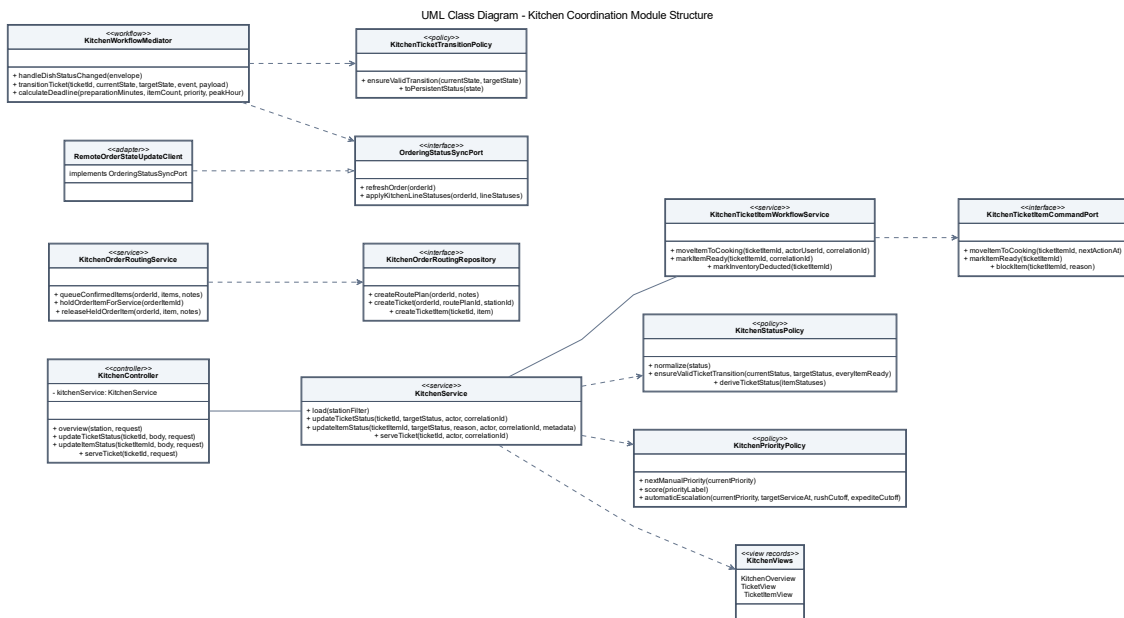
Hình 33: Sơ đồ lớp chi tiết cho module reservation

Hình 32 thể hiện các controller ReservationController, TableController, WaitlistController; service ReservationService, ReservationCheckInService, ReservationWaitlistService; workflow ReservationSeatingMediator; và các port/adaptor JDBC cho vòng đời đặt bàn và bàn ăn. Các method trong hình được lấy từ class thật, ví dụ load, recommend, createReservation và checkIn.



Hình 34: Sơ đồ lớp chi tiết cho module ordering và phần menu

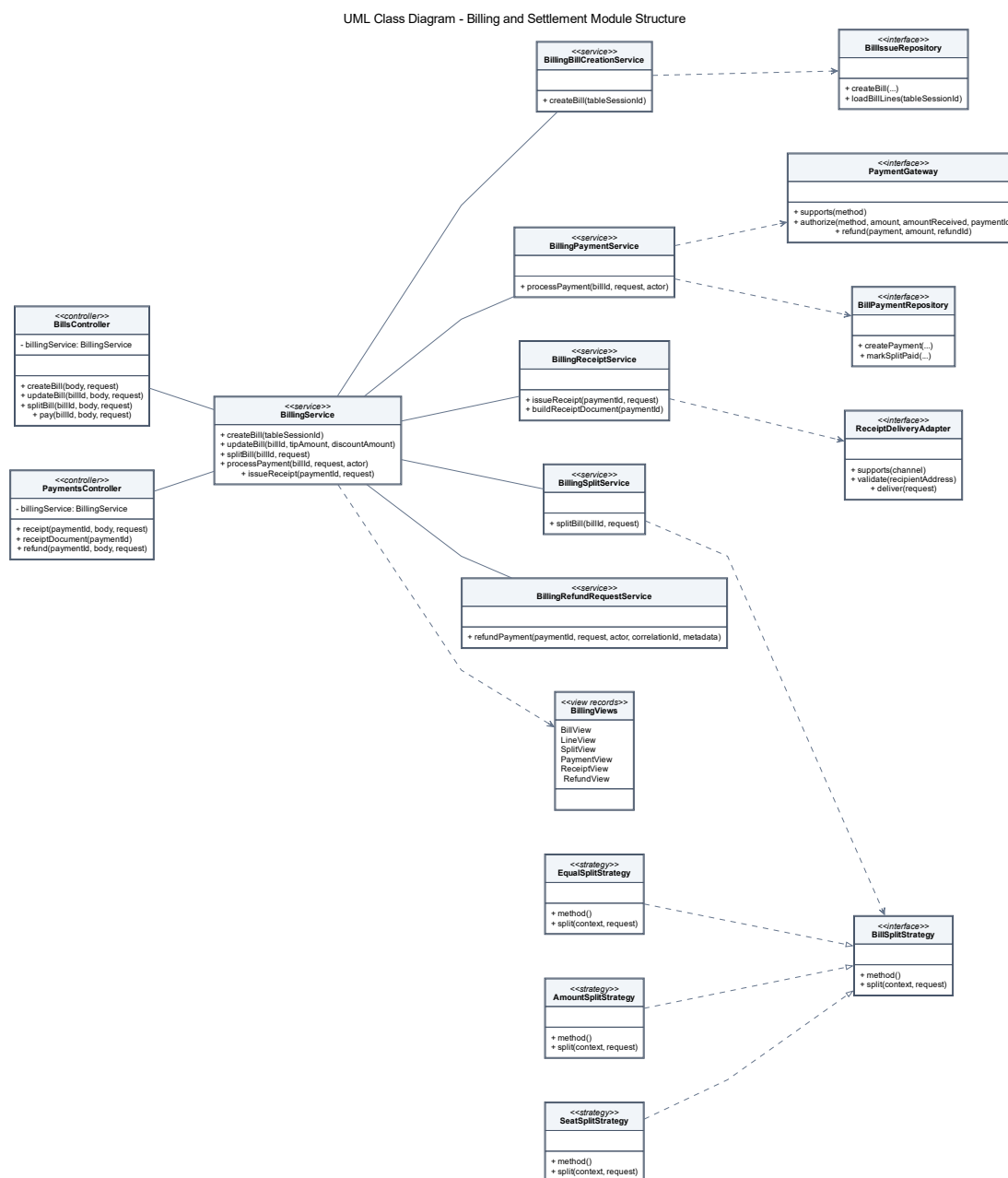
Hình 33 làm rõ vai trò của MenuItemController, OrdersController, MenuService, OrdersService, OrderCreationService, OrderMenuSelectionService, các policy đơn hàng và hai phạm vi triển khai repository OrderRepository, OrderQueryRepository. Các adapter JdbcOrderRepository và JdbcOrderQueryRepository được đặt ở phía hiện thực port, đúng với nguyên tắc tách application service khỏi truy cập JDBC.



Hình 35: Sơ đồ lớp chi tiết cho module kitchen

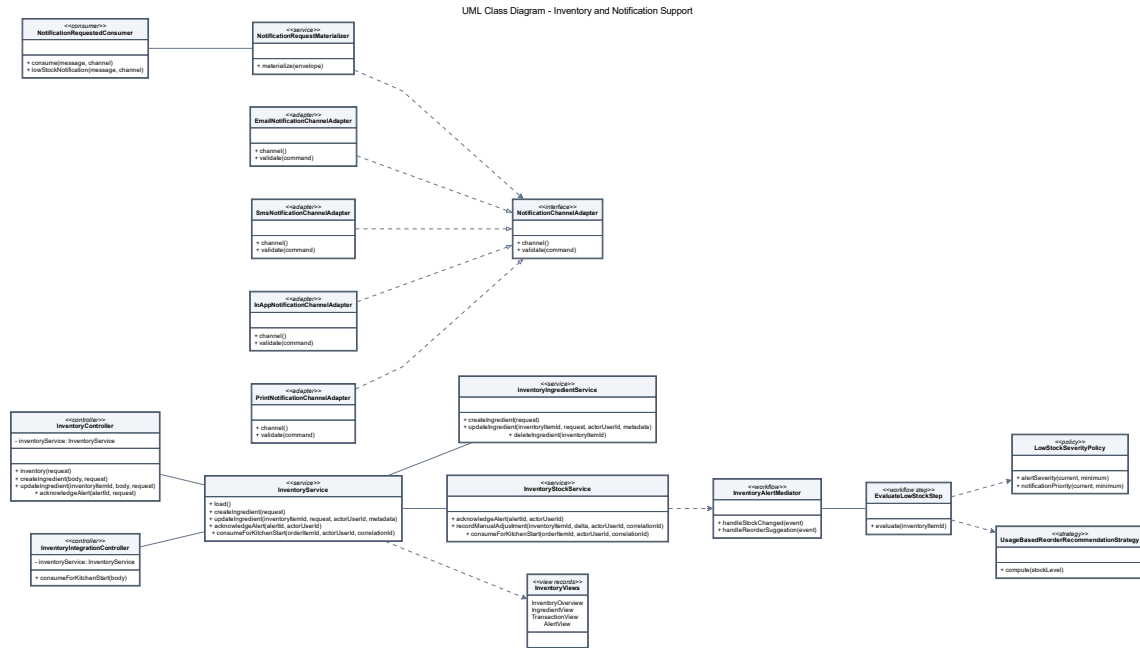
Hình 34 mô tả các lớp điều phối bếp thật như KitchenService, KitchenOrderRoutingService, KitchenTicketItemWorkflowService, KitchenWorkflowMediator, các policy trạng thái/ưu tiên và các port KitchenOrderRoutingRepository, KitchenTicketItemCommandPort, OrderingStatusSyncPort. Adapter

RemoteOrderStateUpdateClient thể hiện phụ thuộc quay về trạng thái gọi món thông qua interface thay vì truy cập trực tiếp.



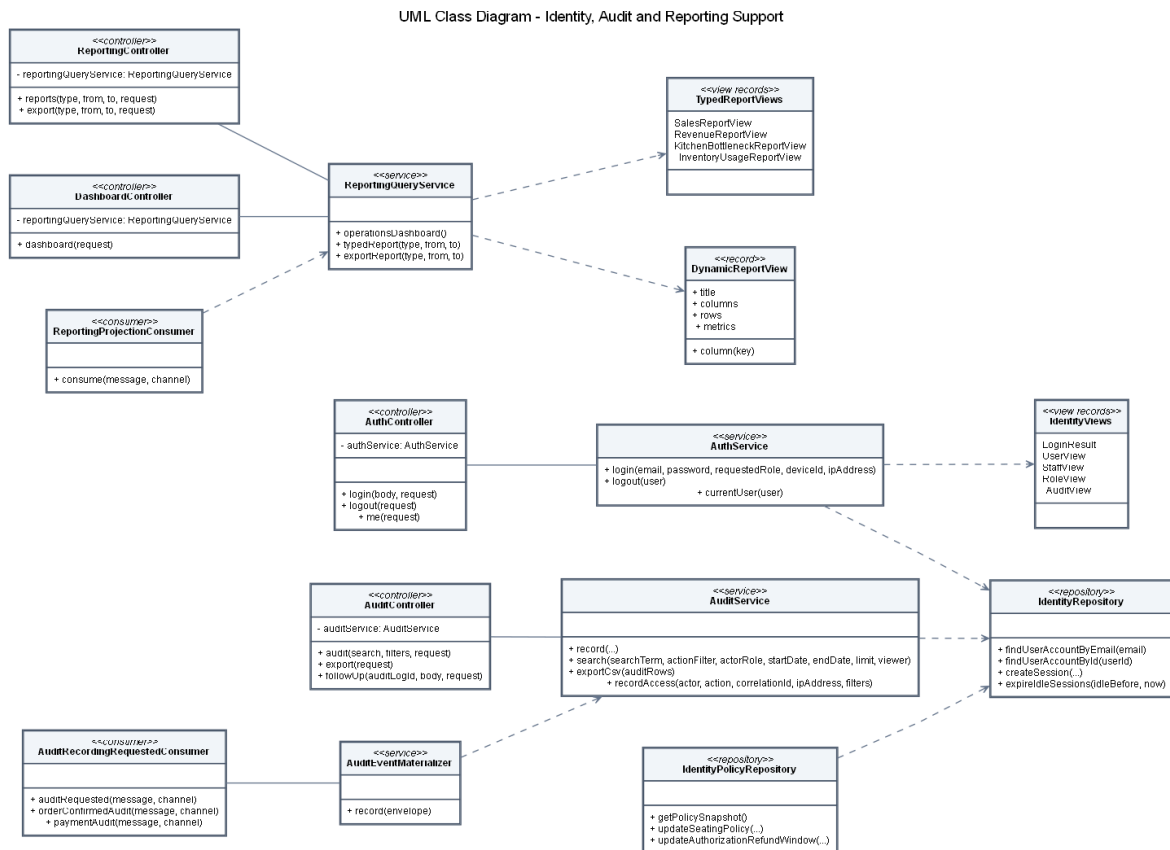
Hình 36: Sơ đồ lớp chi tiết cho module billing

Hình 35 thể hiện BillingService như facade ứng dụng. Các service con phụ trách lập hóa đơn, tách hóa đơn, thanh toán, biên nhận và hoàn tiền. Các điểm mở rộng chính là BillSplitStrategy, PaymentGateway và ReceiptDeliveryAdapter. Các strategy EqualSplitStrategy, AmountSplitStrategy và SeatSplitStrategy được biểu diễn bằng quan hệ hiện thực interface được dùng trong thiết kế.



Hình 37: Sơ đồ lớp chi tiết cho module inventory và notification

Hình 36 nối InventoryService, InventoryIngredientService, InventoryStockService với workflow cảnh báo tồn kho và nhóm thông báo. NotificationRequestedConsumer, NotificationRequestMaterializer và các adapter kênh gửi được giữ vì chúng là class chính trong thiết kế và cũng xuất hiện ở C&C View xử lý nền.



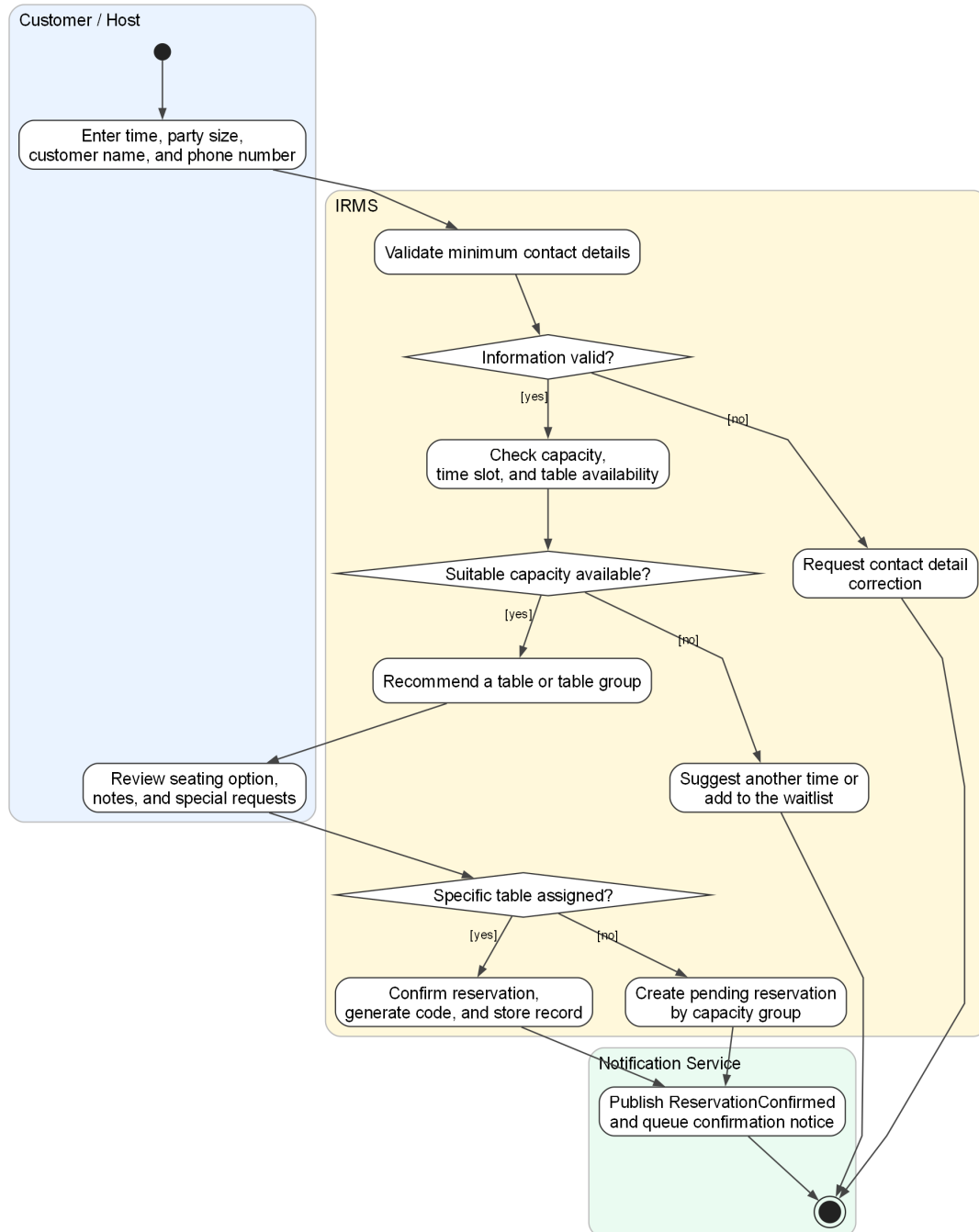
Hình 38: Sơ đồ lớp chi tiết cho module identity, kiểm toán và reporting

Hình 37 mô tả vùng kiểm soát bằng các controller, service, phạm vi triển khai và bộ xử lý sự kiện: AuthController, AuditController, ReportingController, DashboardController, AuthService, AuditService, AuditEventMaterializer, AuditRecordingRequestedConsumer, ReportingQueryService và ReportingProjectionConsumer. Quan hệ giữa kiểm toán/báo cáo với bộ xử lý sự kiện nhất quán với Component-and-Connector View: dữ liệu quản trị được cập nhật qua xử lý nền và phạm vi triển khai/query service riêng.

4.3 Thiết kế hành vi

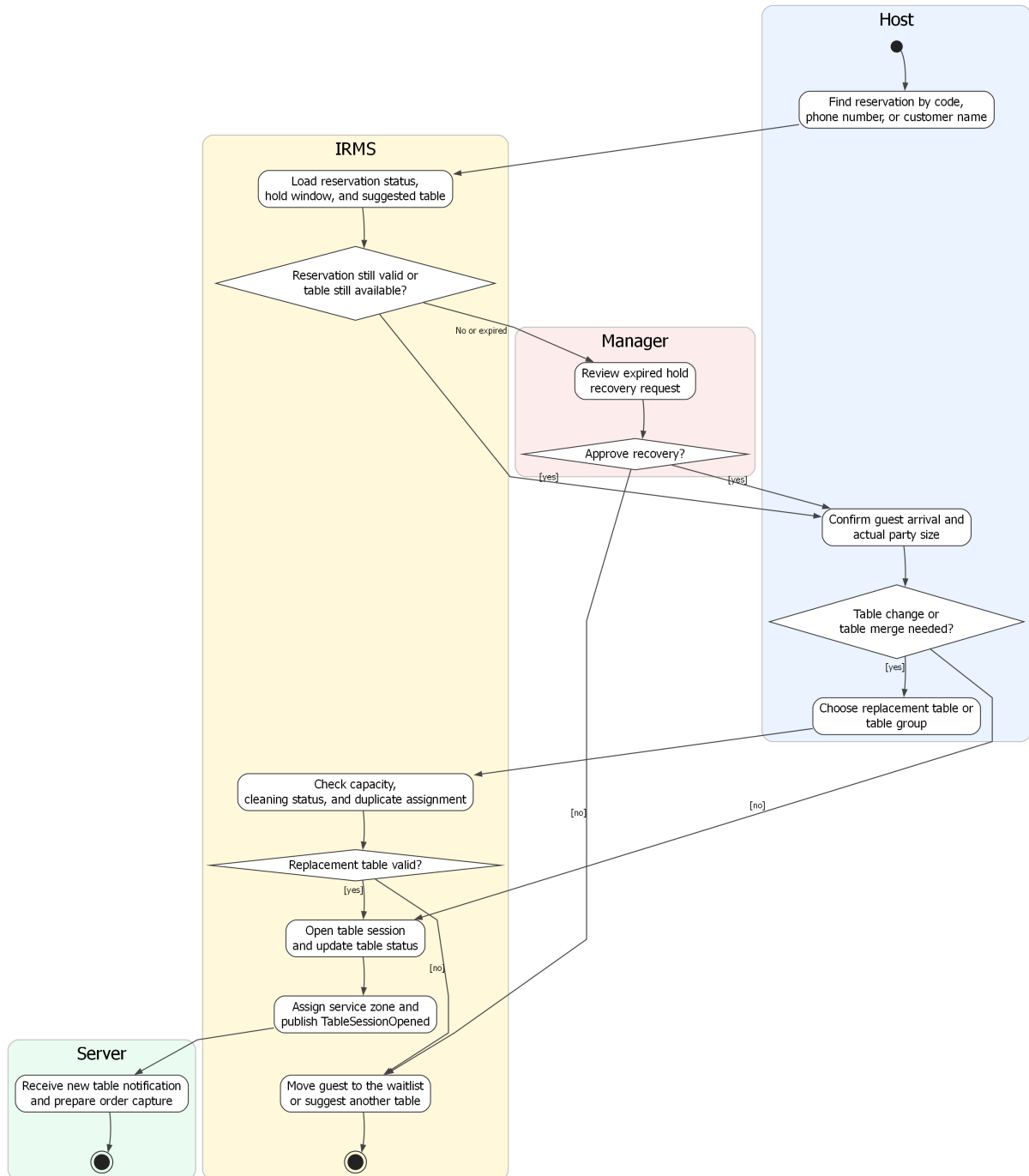
4.3.1 Nhóm đặt bàn và bàn ăn

UC-RES-01 -- Create Reservation



Hình 39: Sơ đồ hoạt động cho UC-RES-01 – Tạo đặt bàn

UC-RES-02 -- Check In Guest and Seat Table

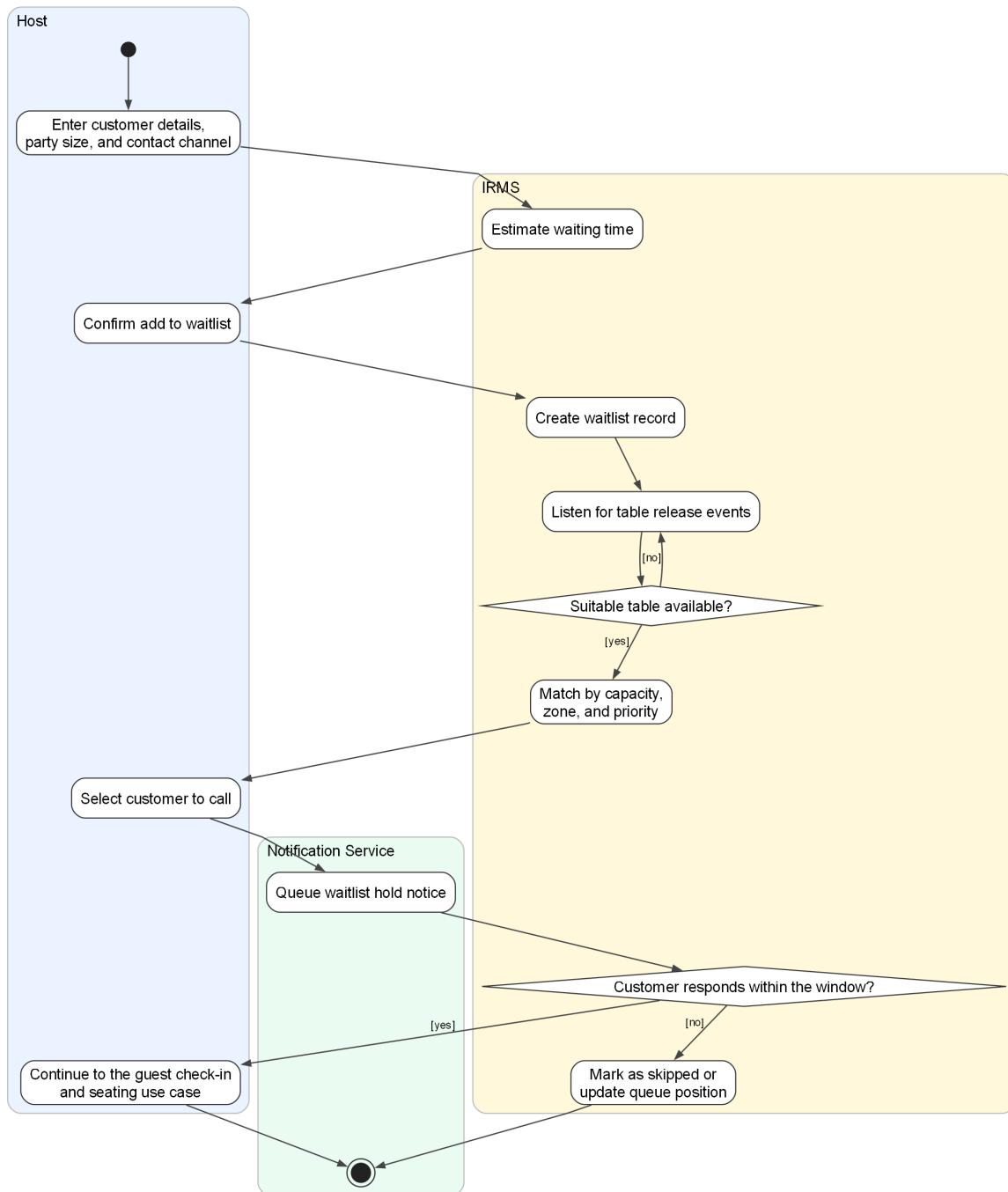


Hình 40: Sơ đồ hoạt động cho UC-RES-02 – Nhận khách và sắp bàn

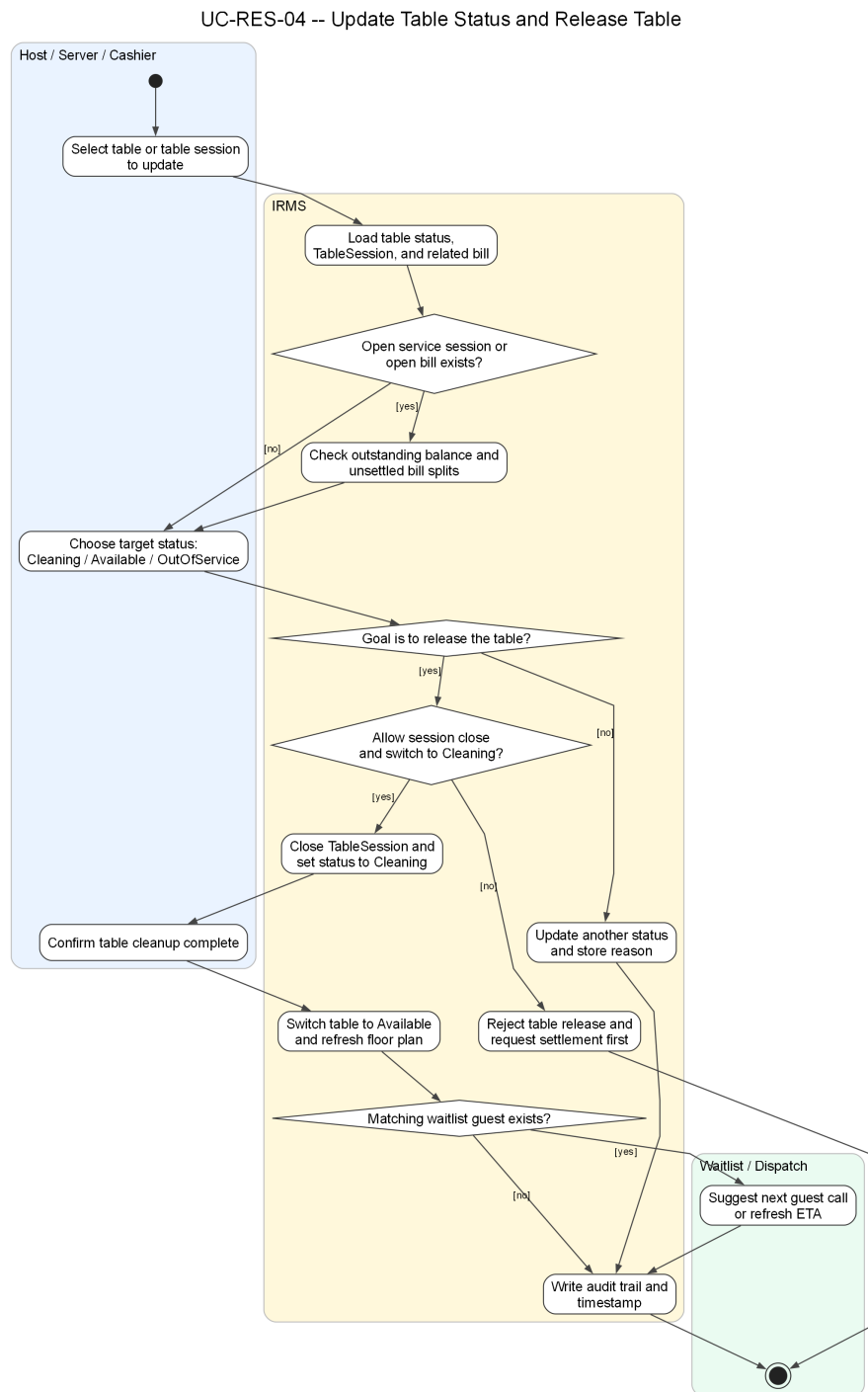
Phân tích sơ đồ UC-RES-02. Luồng hoạt động trong trường hợp này bám sát thực tế tại quầy lễ tân: tìm đặt bàn, kiểm tra hiệu lực, đối chiếu số khách thực tế, xác định có cần đổi bàn hoặc ghép bàn hay không, sau đó mới mở TableSession. Điểm mấu chốt là việc **mở phiên phục vụ bàn** luôn diễn ra sau bước kiểm tra bàn khả dụng, nhờ đó báo cáo giữ được bất biến quan trọng rằng một bàn không có hai phiên phục vụ hoạt động cùng lúc.

Ngoài ra, sơ đồ cũng làm rõ nhánh ngoại lệ khi đặt bàn đã quá giờ giữ chỗ. Trong tình huống này, hệ thống không tự động bỏ qua mà chuyển sang bước phê duyệt của quản lý.

UC-RES-03 -- Manage Waitlist



Hình 41: Sơ đồ hoạt động cho UC-RES-03 – Quản lý danh sách chờ

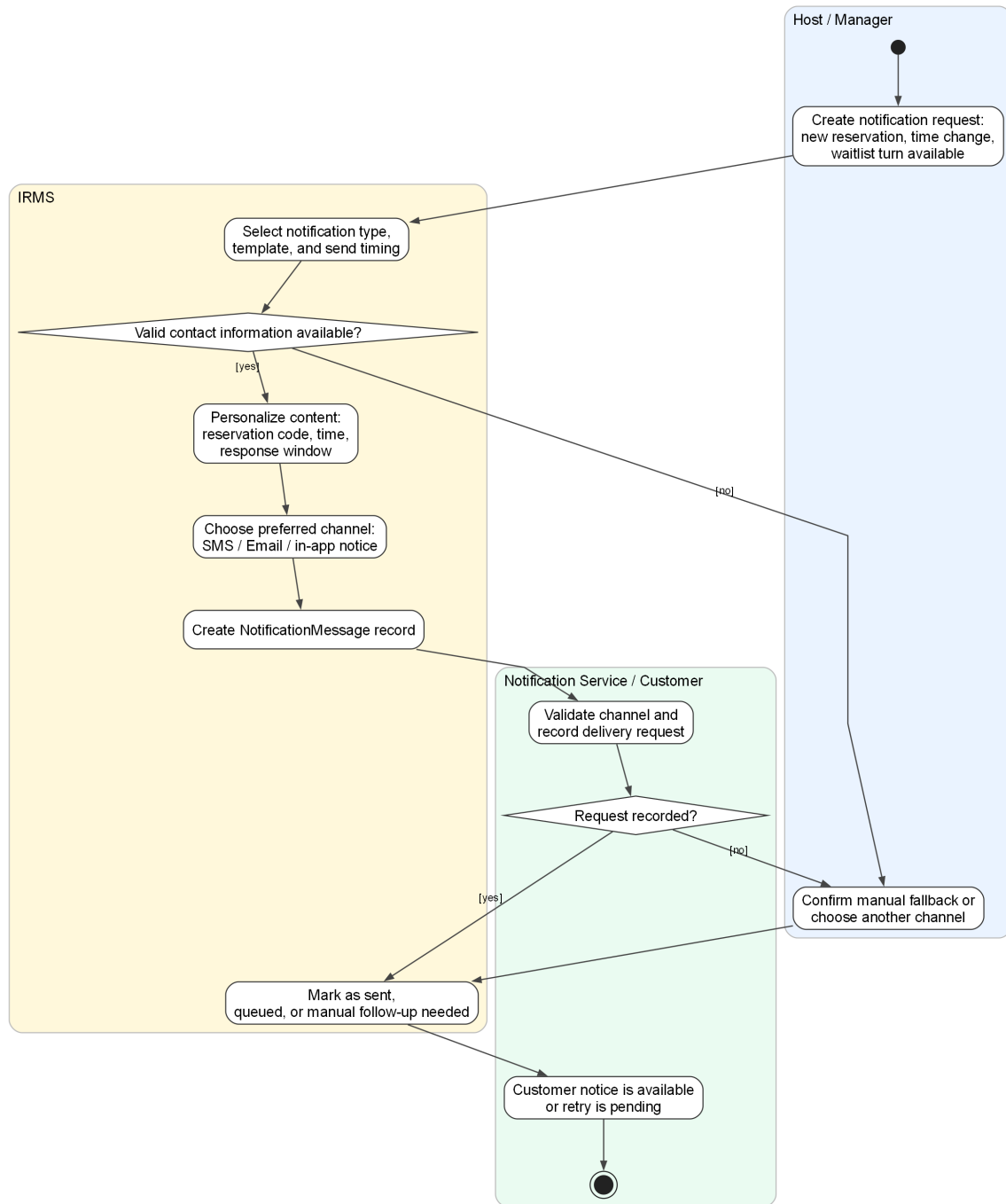


Hình 42: Sơ đồ hoạt động cho UC-RES-04 – Cập nhật trạng thái bàn và giải phóng bàn

Phân tích sơ đồ UC-RES-04.

- **Ràng buộc trạng thái Available:** Trạng thái bàn không cập nhật tự động sau thanh toán mà yêu cầu validation: đóng toàn bộ phiên phục vụ (session), tắt toán công nợ (billing) và xác nhận hoàn tất vệ sinh (cleaning).
- **Phối hợp liên module:** Đồng bộ hóa dữ liệu giữa ba miền nghiệp vụ: TableSession, Billing và Waitlist.
- **Hiệu quả vận hành:** Đảm bảo tính nhất quán giữa hệ thống và thực tế, tránh lỗi gán trùng bàn (double-booking) và cung cấp dữ liệu chính xác để tính toán hiệu suất vòng quay bàn.

UC-RES-05 -- Notify Customer

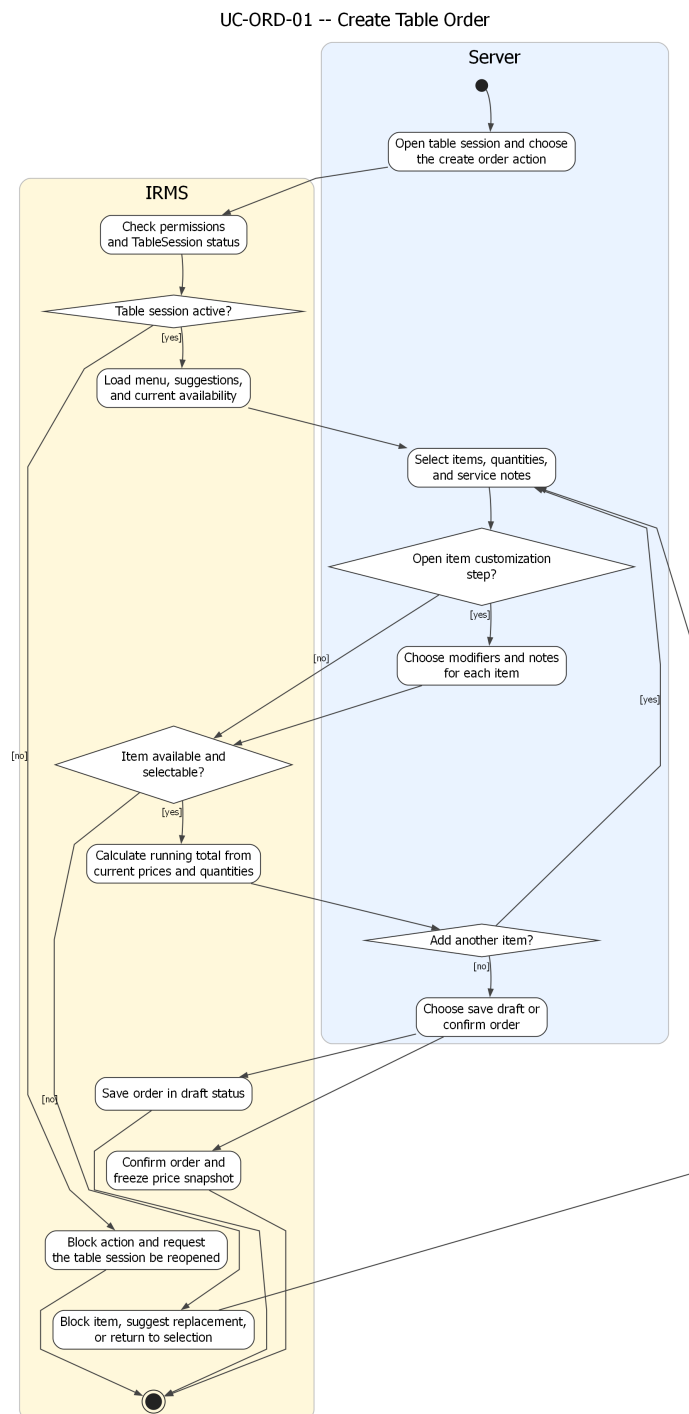


Hình 43: Sơ đồ hoạt động cho UC-RES-05 – Gửi thông báo cho khách hàng

Phân tích sơ đồ UC-RES-05.

- **Controlled Side-effect:** Thông báo là hệ quả có kiểm soát phát sinh từ các sự kiện nghiệp vụ (xác nhận đặt bàn, nhắc giờ, gọi khách từ waitlist).
- **Quản lý luồng:** Mỗi thông báo có đầu vào, điều kiện kích hoạt và trạng thái xử lý riêng biệt.
- **Phân tách trách nhiệm:** Cô lập logic nghiệp vụ (Core Domain) khỏi các chi tiết hạ tầng (Notification Adapter) và các cơ chế vận hành kỹ thuật (Retry, Fallback).

4.3.2 Nhóm gọi món và điều phối bếp



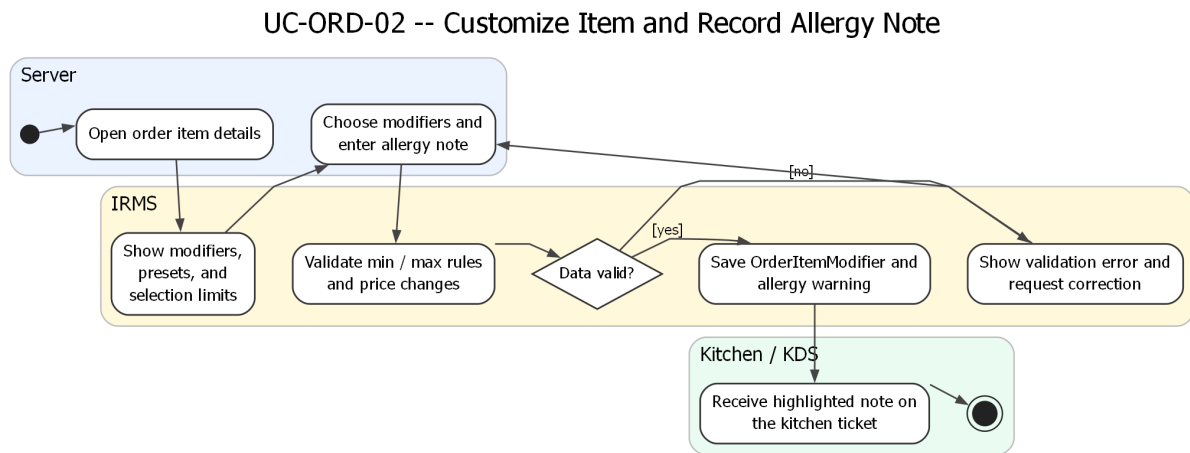
Hình 44: Sơ đồ hoạt động cho UC-ORD-01 – Tạo đơn gọi món tại bàn

Phân tích sơ đồ UC-ORD-01. Luồng hoạt động gọi món bao gồm các bước: tải thực đơn, kiểm tra quyền thao tác trên phiên bản, chọn món, kiểm tra trạng thái còn bán (quay lại chọn lại nếu hết hàng), tính tạm tính và rẽ nhánh “lưu nháp” hoặc “xác nhận đơn”. Thao tác này đóng vai trò chốt trạng thái (snapshot) về giá và tính hợp lệ của món ngay tại thời điểm xử lý.

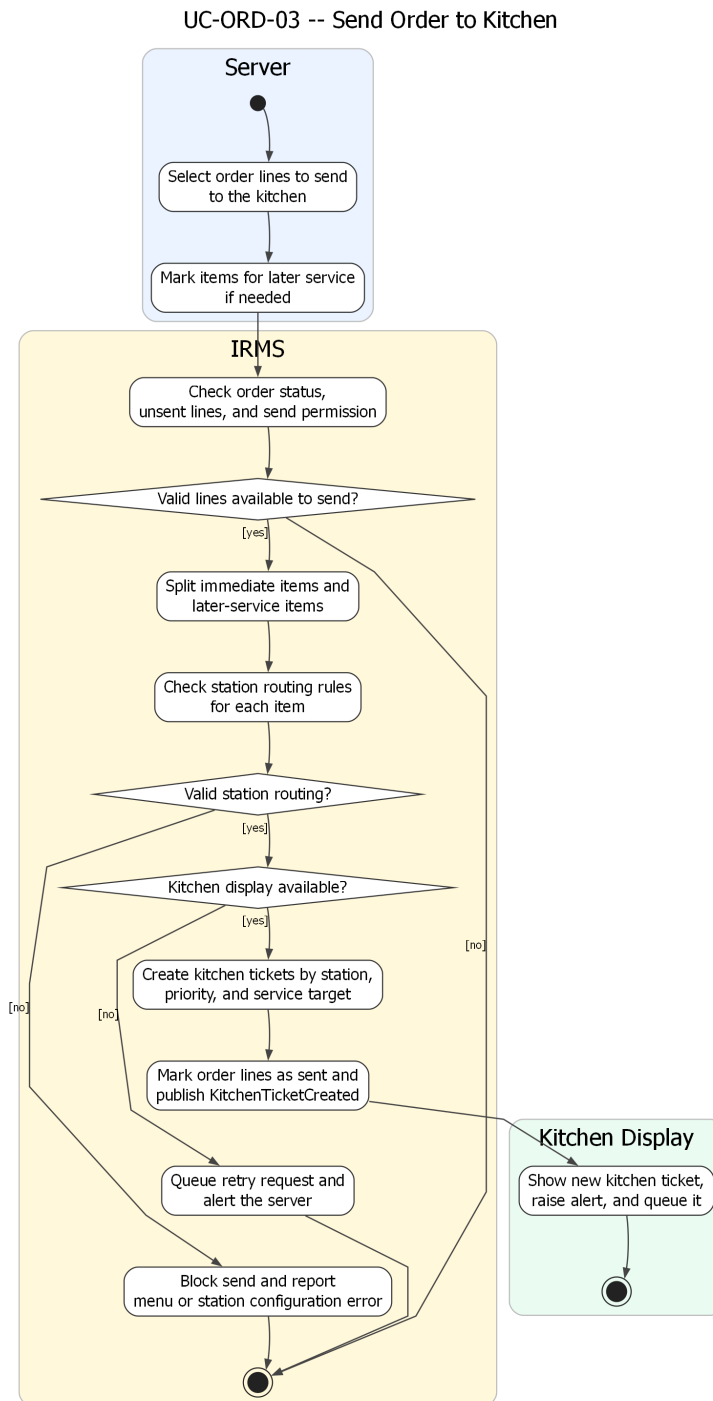
Sơ đồ cũng vạch rõ ranh giới trách nhiệm giữa hai module:

- **Ordering:** Giới hạn ở việc tạo và xác nhận đơn gọi món.

- **Kitchen Coordination:** Tách biệt hoàn toàn việc tạo phiếu bếp và điều phối sang trạm bếp (thuộc một sơ đồ riêng).



Hình 45: Sơ đồ hoạt động cho UC-ORD-02 – Tùy biến món và ghi chú dị ứng



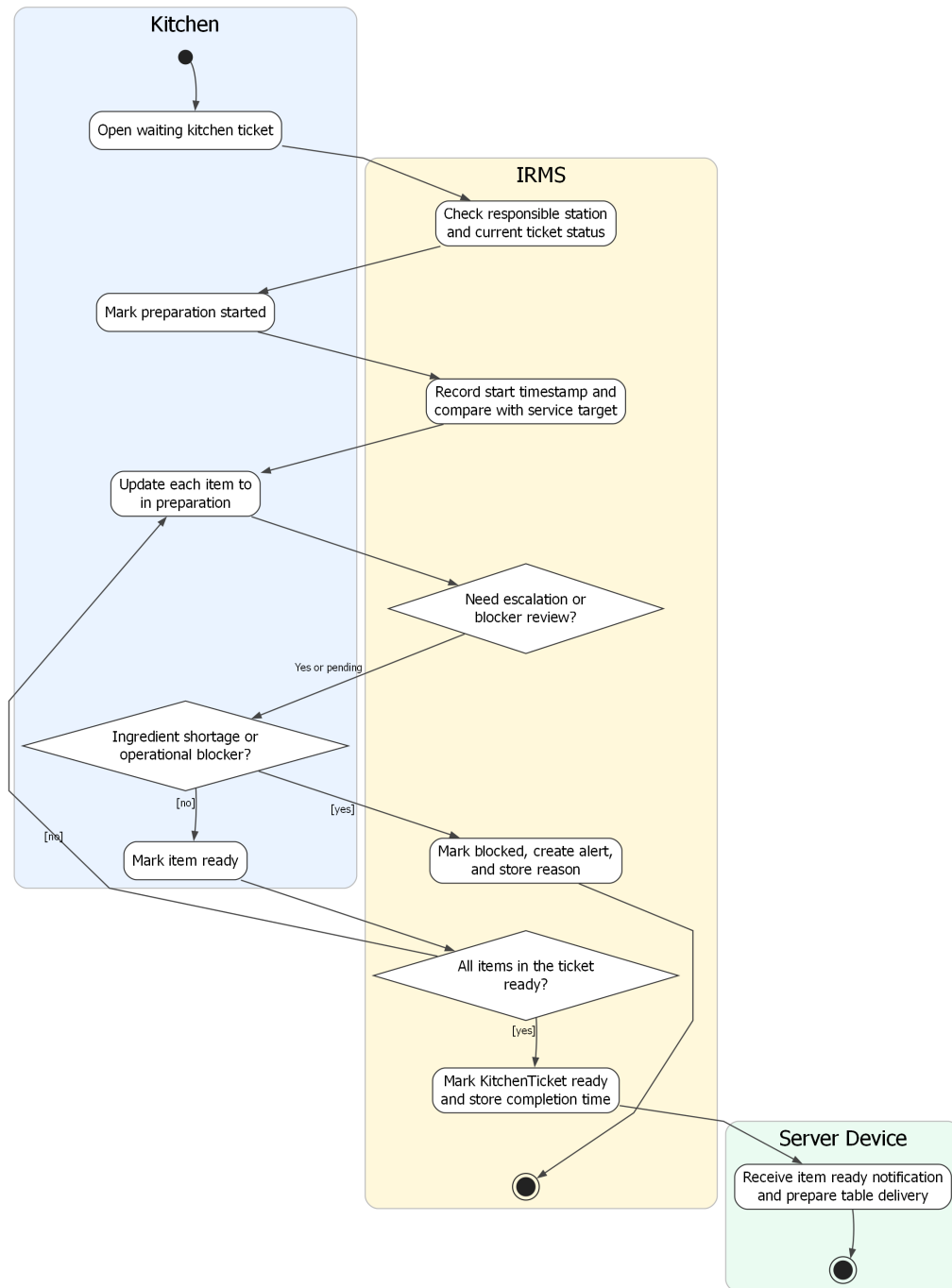
Hình 46: Sơ đồ hoạt động cho UC-ORD-03 – Gửi đơn gọi món sang bếp

Phân tích sơ đồ UC-ORD-03. Sơ đồ mô tả ranh giới giao tiếp giữa hai module Ordering (gọi món) và Kitchen Coordination (điều phối bếp), bao gồm:

- **Luồng xử lý chính:** Kiểm tra điều kiện → Phân loại món (làm ngay/làm sau) → Định tuyến theo trạm bếp (Station Routing) → Khởi tạo KitchenTicket → Đưa vào hàng đợi ưu tiên (Priority Queuing) → Cập nhật trạng thái đơn (Order).
- **Xử lý ngoại lệ (Exception Handling):** Nếu lỗi định tuyến trạm hoặc hệ thống màn hình bếp (KDS) offline, hệ thống đẩy request vào hàng đợi gửi lại (retry queue) và phát cảnh báo (alert) cho nhân viên.

- **Đáp ứng NFR:** Cơ chế xử lý lỗi này đảm bảo yêu cầu phi chức năng về độ tin cậy (Reliability) và toàn vẹn dữ liệu (Data Integrity), tuyệt đối không làm mất các KitchenTicket đã được xác nhận.

UC-KIT-01 -- Update Cooking Progress

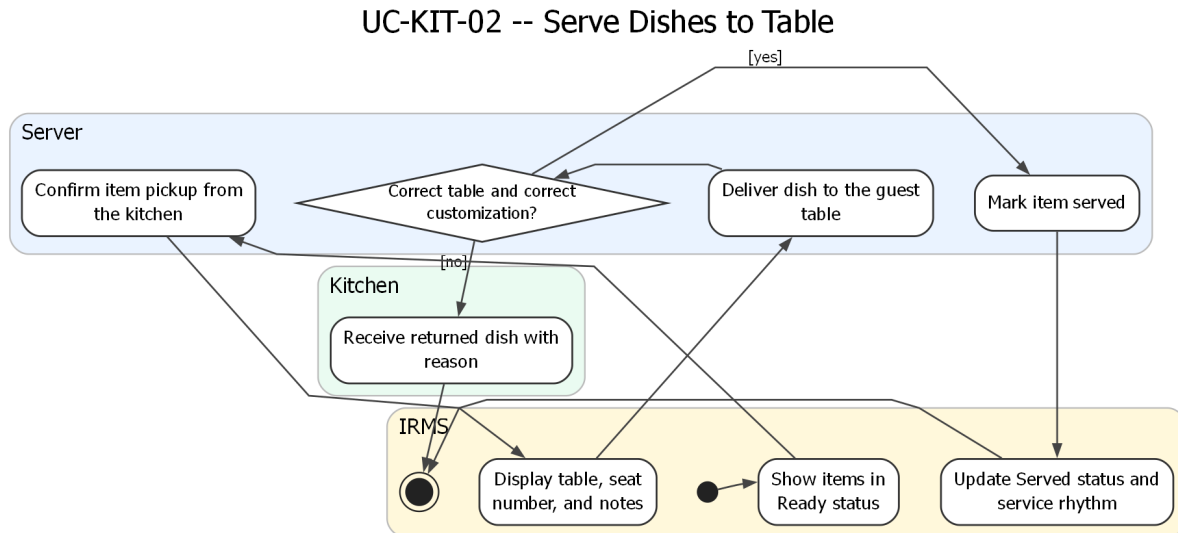


Hình 47: Sơ đồ hoạt động cho UC-KIT-01 – Cập nhật tiến độ chế biến

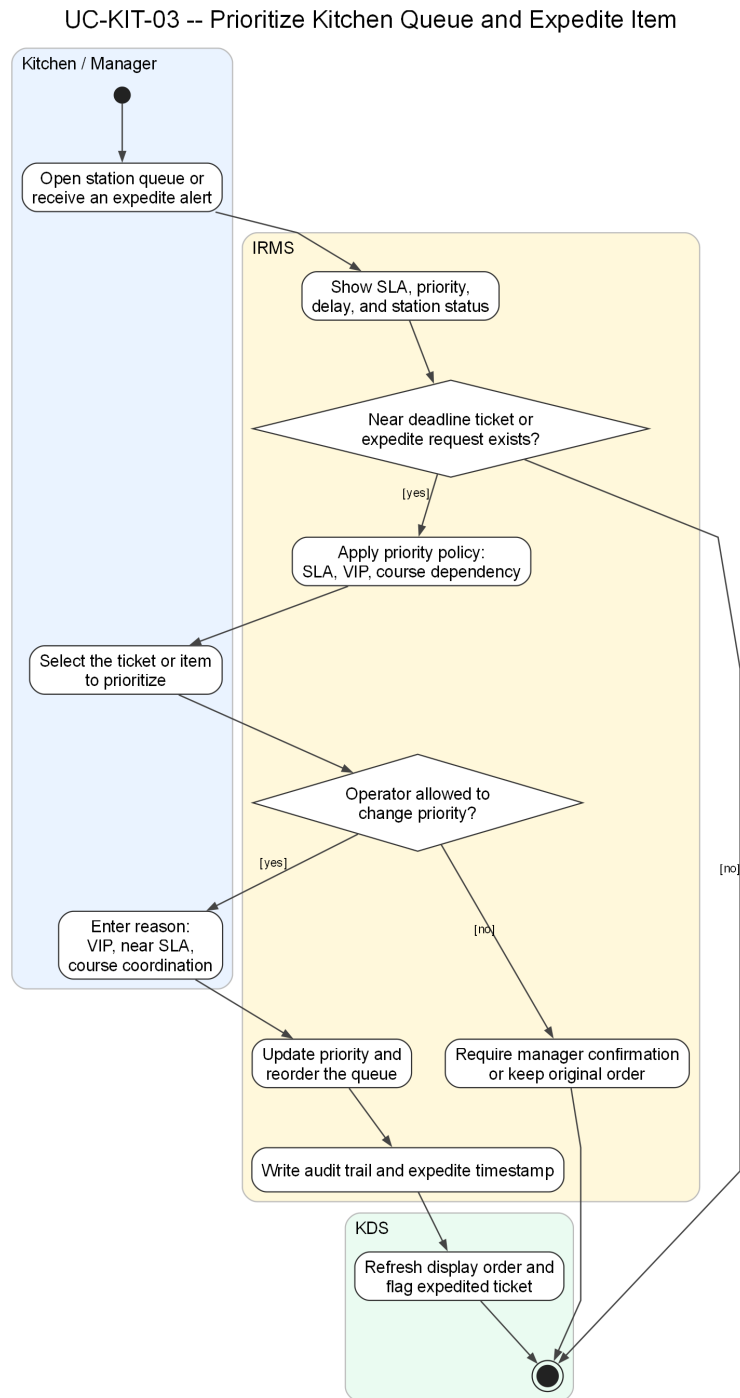
Phân tích sơ đồ UC-KIT-01. Luồng hoạt động bám sát quy trình xử lý thực tế tại trạm bếp:

- **Luồng xử lý chính:** Nhận phiếu (KitchenTicket) → Kiểm tra trạm đích → Bắt đầu chế biến → Ghi nhận thời gian (timestamp) qua mỗi lần chuyển trạng thái → Đánh giá nguy cơ trễ hạn (delay risk) → Xác nhận hoàn tất (Ready). Việc lưu vết timestamp chi tiết là cơ sở dữ liệu bắt buộc để phục vụ module phân tích hiệu suất vận hành (operational analytics) sau này.

- **Xử lý ngoại lệ (Thiếu nguyên liệu/Sự cố):** Hệ thống không được phép kết thúc ngầm (fail silently). Request bắt buộc chuyển sang trạng thái chặn (Blocked), đồng thời phát cảnh báo (alert) và phản hồi trạng thái (feedback) về khu vực phục vụ. Cơ chế này đảm bảo luồng giao tiếp hai chiều giữa bếp và phục vụ luôn đồng bộ và nhất quán với kiến trúc tổng thể.



Hình 48: Sơ đồ hoạt động cho UC-KIT-02 – Ra món cho bàn

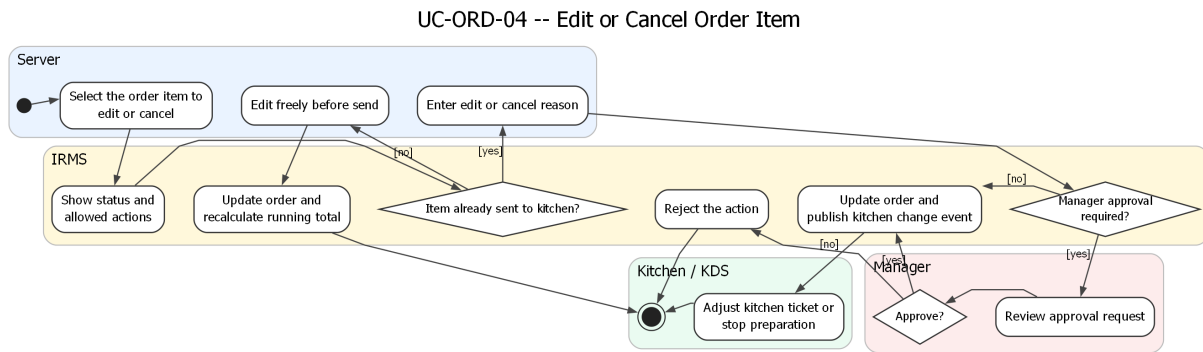


Hình 49: Sơ đồ hoạt động cho UC-KIT-03 – Ưu tiên hàng chờ bếp và xử lý món gấp

Phân tích sơ đồ UC-KIT-03. Luồng hoạt động định hình lại vai trò của hệ thống KDS: không chỉ hiển thị thụ động mà phải trực tiếp điều phối thứ tự thực thi (queue prioritization).

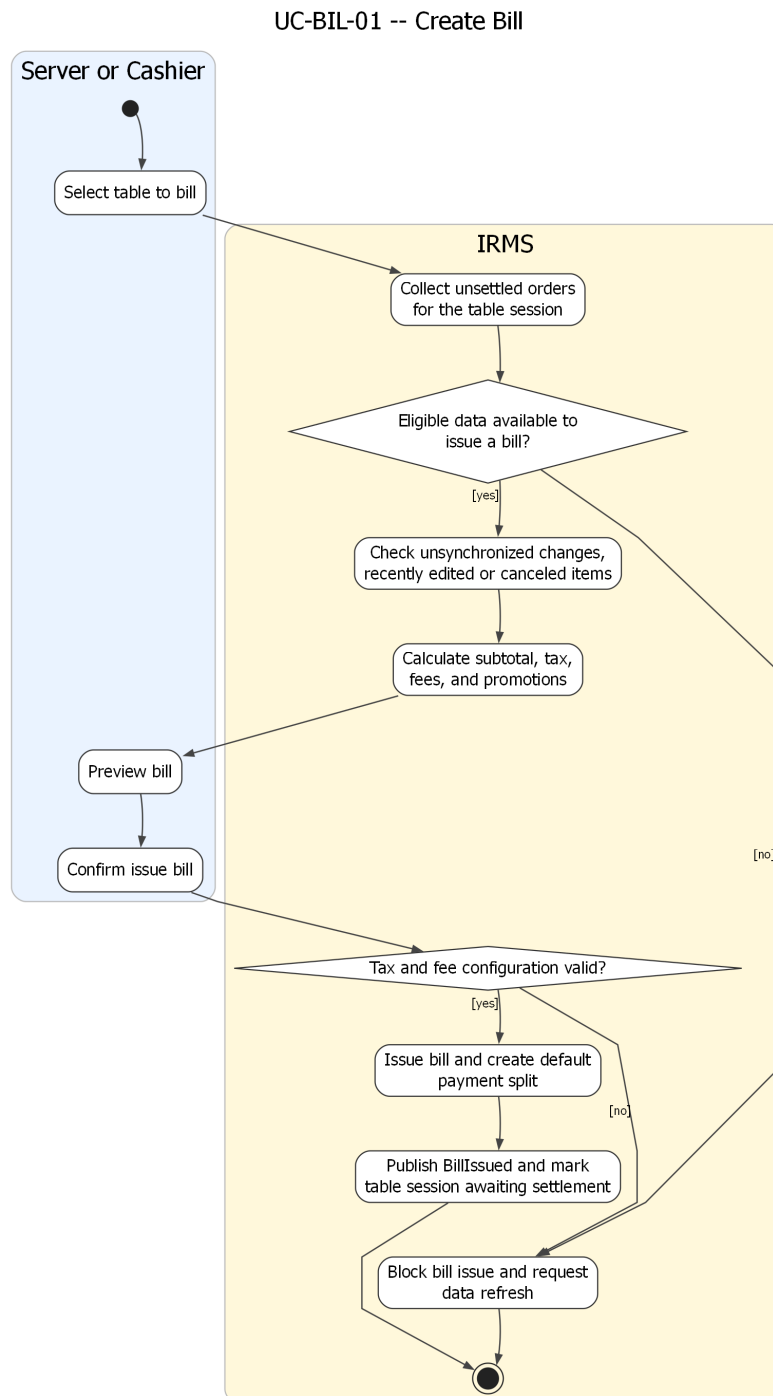
- **Tiêu chí phân luồng:** Trong giờ cao điểm, các KitchenTicket được sắp xếp tự động dựa trên: cam kết thời gian (SLA), cấu trúc món (course), phân hạng khách (VIP) và nguy cơ trễ hạn. Ca sử dụng này được tách độc lập để đáp ứng đúng yêu cầu “prioritize queues”.
- **Kiểm soát chính sách (Policy-driven):** Các can thiệp thay đổi thứ tự chế biến bắt buộc phải tuân thủ chính sách: yêu cầu phân quyền hợp lệ và ghi nhận lý do thay đổi (reason log), không cho phép tùy chỉnh tự do.

- **Khả năng truy vết (Traceability):** Cơ chế kiểm soát chặt chẽ giúp vận hành linh hoạt nhưng vẫn đảm bảo lưu vết (audit) đầy đủ để đối soát khi xảy ra khiếu nại về việc lên món sai thứ tự hoặc trễ SLA.



Hình 50: Sơ đồ hoạt động cho UC-ORD-04 – Sửa hoặc hủy dòng món

4.3.3 Nhóm hóa đơn, thanh toán và biên lai

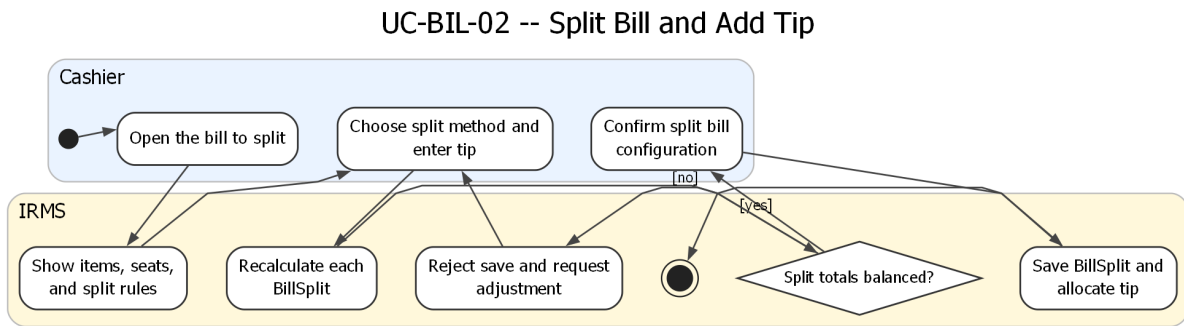


Hình 51: Sơ đồ hoạt động cho UC-BIL-01 – Tạo hóa đơn cho bàn

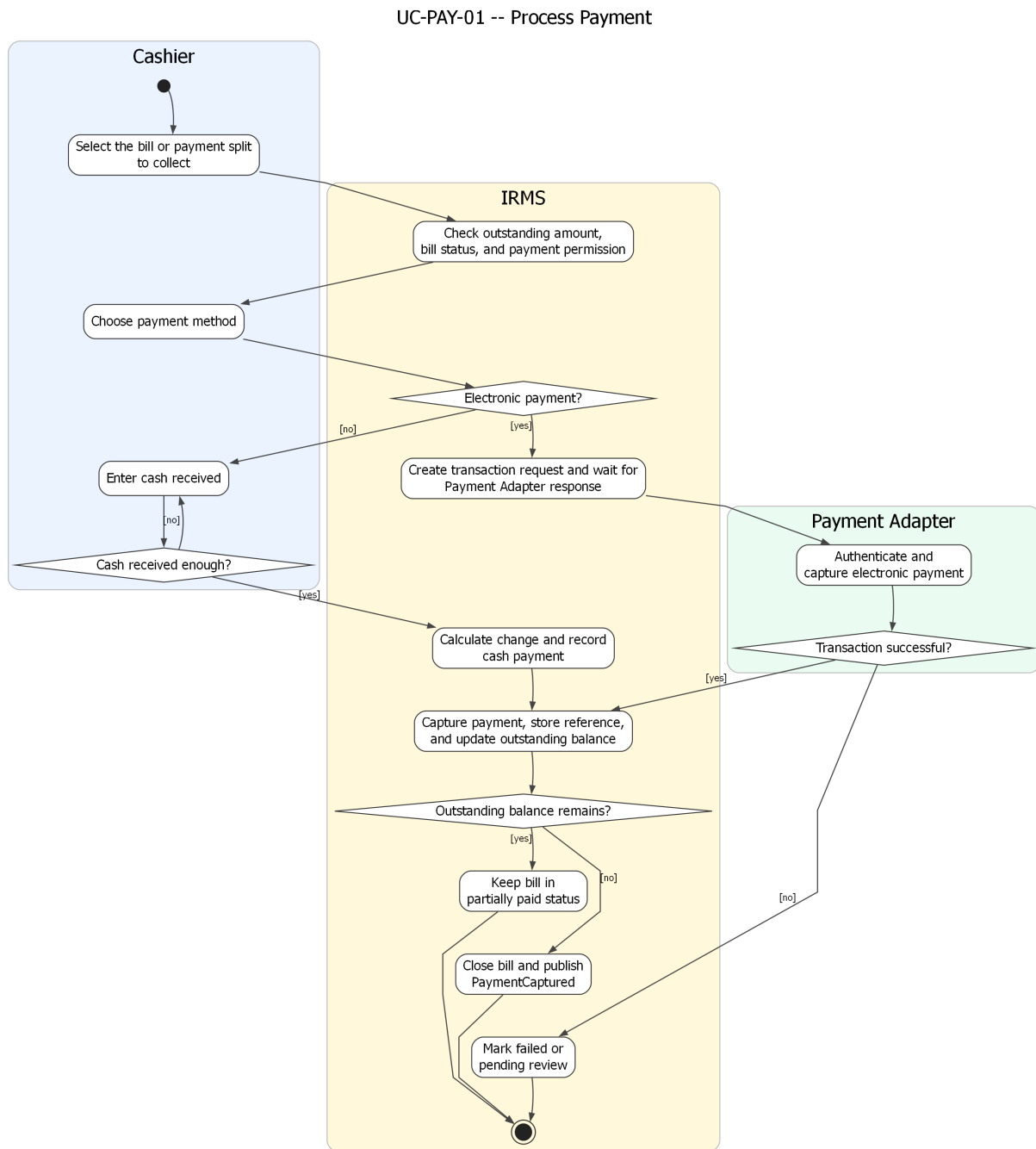
Phân tích sơ đồ UC-BIL-01.

- **Cơ chế Preview:** Bước “xem trước hóa đơn” đóng vai trò chốt chặn cuối cùng để kiểm tra tính nhất quán dữ liệu, phòng ngừa sai sót do các thay đổi tức thời (hủy món hoặc cập nhật giá).
- **Ràng buộc nghiệp vụ:** Hệ thống thiết lập nhánh chặn phát hành khi dữ liệu không hợp lệ. Điều này khẳng định Bill là một thực thể nghiệp vụ (business entity) có các điều kiện phát hành và chuyển đổi trạng thái

chặt chẽ, không đơn thuần là phép cộng tổng dữ liệu.



Hình 52: Sơ đồ hoạt động cho UC-BIL-02 – Tách hóa đơn và thêm tiền boa



Hình 53: Sơ đồ hoạt động cho UC-PAY-01 – Xử lý thanh toán

Phân tích sơ đồ UC-PAY-01.

• Phân luồng thanh toán:

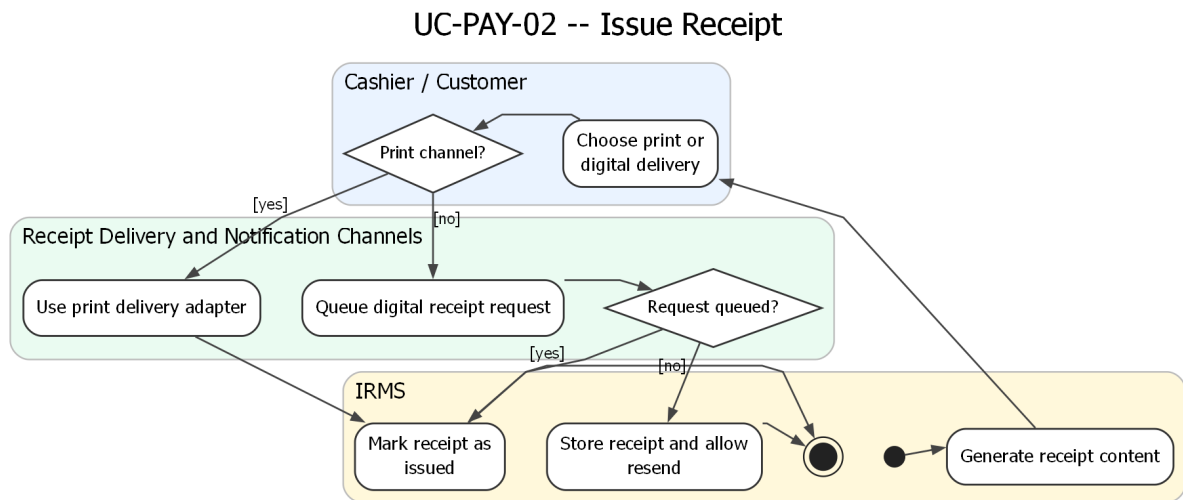
– **PaymentGateway**: Tạo yêu cầu giao dịch → Nhận kết quả → Lưu mã tham chiếu → Xử lý các trạng thái: thành công, thất bại, hoặc chờ rà soát.

– **Tiền mặt**: Tính tiền thối và cập nhật số dư còn lại của hóa đơn.

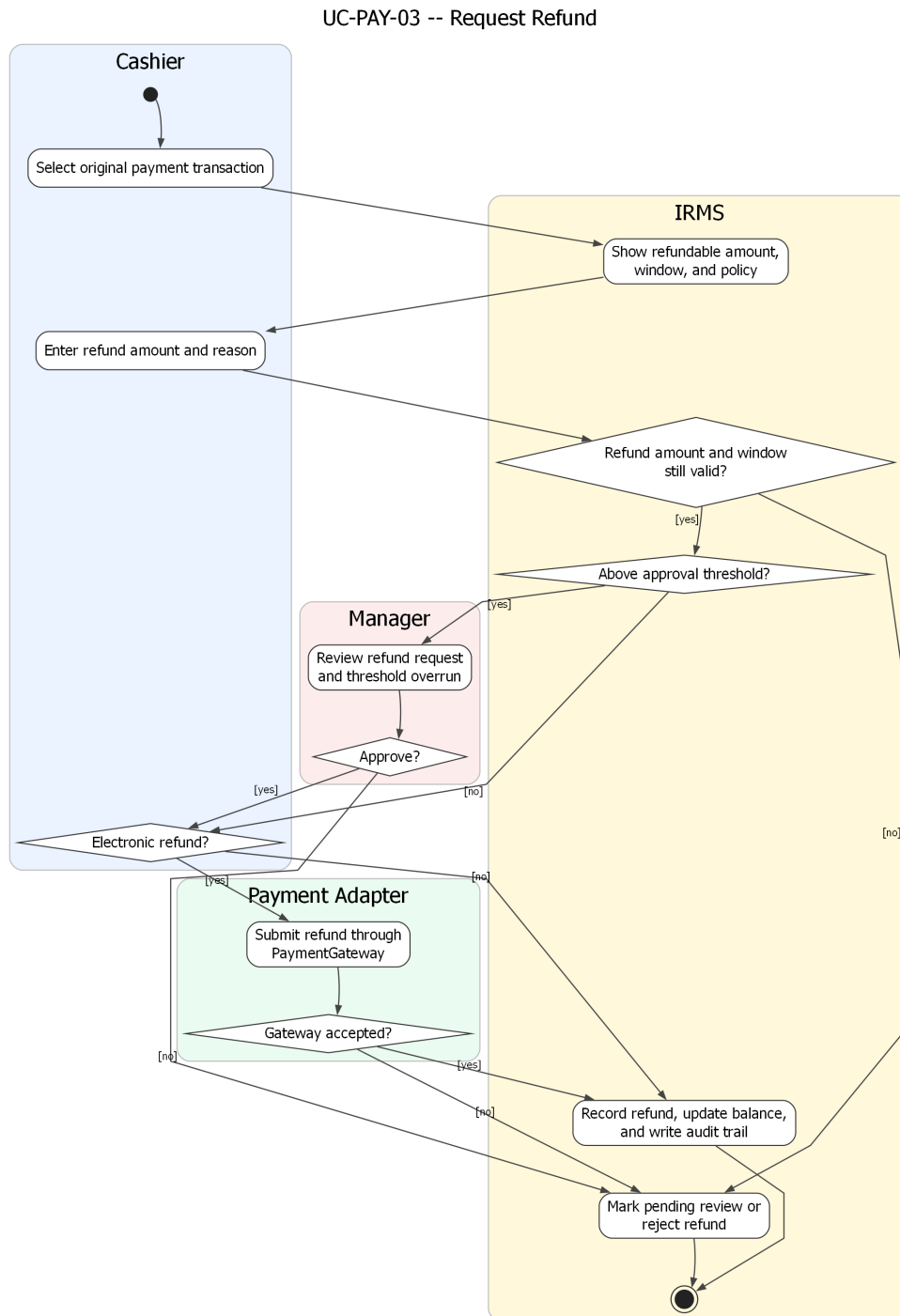
• Logic tắt toán: Sau khi ghi nhận, hệ thống kiểm tra số tiền đã thanh toán:

– **Chưa đủ**: Hóa đơn giữ trạng thái mở (OPEN).

- **Đã đủ:** Đóng hóa đơn (CLOSED) và kích hoạt luồng phát hành biên lai.



Hình 54: Sơ đồ hoạt động cho UC-PAY-02 – Phát hành biên lai



Hình 55: Sơ đồ hoạt động cho UC-PAY-03 – Hoàn tiền

Phân tích sơ đồ UC-PAY-03. Sơ đồ hoàn tiền hiện đã thể hiện đầy đủ các điểm kiểm soát bắt buộc:

- Chọn giao dịch gốc
- Kiểm tra số tiền còn được hoàn
- Nhập lý do hoàn tiền
- So sánh với ngưỡng phê duyệt

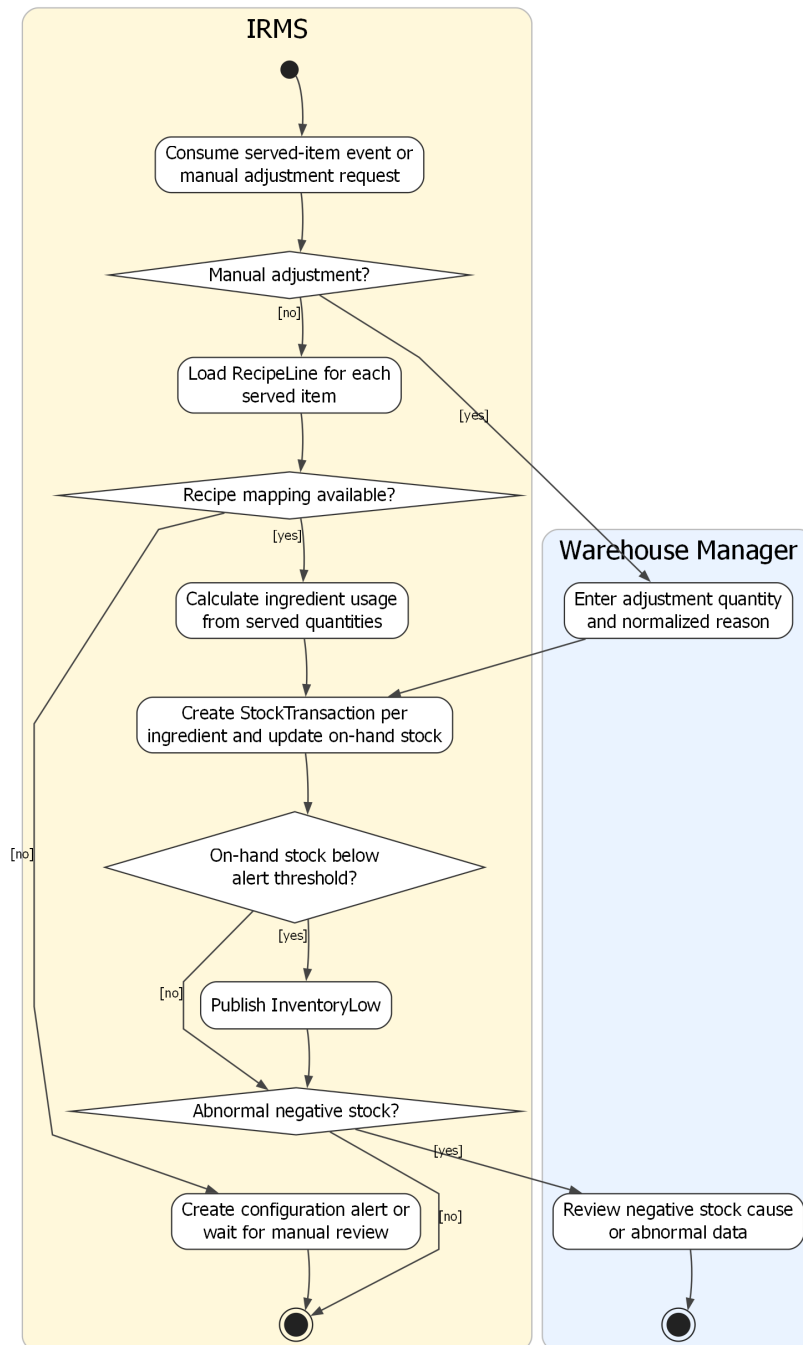
- Xin xác nhận của quản lý nếu cần
- Xử lý hoàn tiền qua PaymentGateway hoặc ghi nhận hoàn tiền thủ công
- Cập nhật sổ kiểm toán

Điều này rất quan trọng vì hoàn tiền là thao tác nhạy cảm nhất trong vùng thanh toán.

Một chi tiết đáng chú ý là sơ đồ đã tách rõ nhánh “chờ rà soát” khi PaymentGateway không chấp nhận hoặc phản hồi không rõ ràng. Nhờ đó báo cáo có thể giải thích trạng thái nghiệp vụ chặt chẽ hơn, thay vì gom mọi trường hợp không thành công vào một nhãn lỗi duy nhất.

4.3.4 Nhóm tồn kho, cảnh báo và phân tích

UC-INV-01 -- Update Stock Consumption After Service



Hình 56: Sơ đồ hoạt động cho UC-INV-01 – Cập nhật tiêu hao tồn kho sau phục vụ

Phân tích sơ đồ UC-INV-01. Sơ đồ cập nhật tiêu hao tồn kho phản ánh đúng bản chất suy diễn của vùng tồn kho. Các bước xử lý bao gồm:

- Nhận sự kiện từ món đã phục vụ hoặc điều chỉnh thủ công
- Tra công thức nguyên liệu
- Tính lượng tiêu hao theo số lượng món

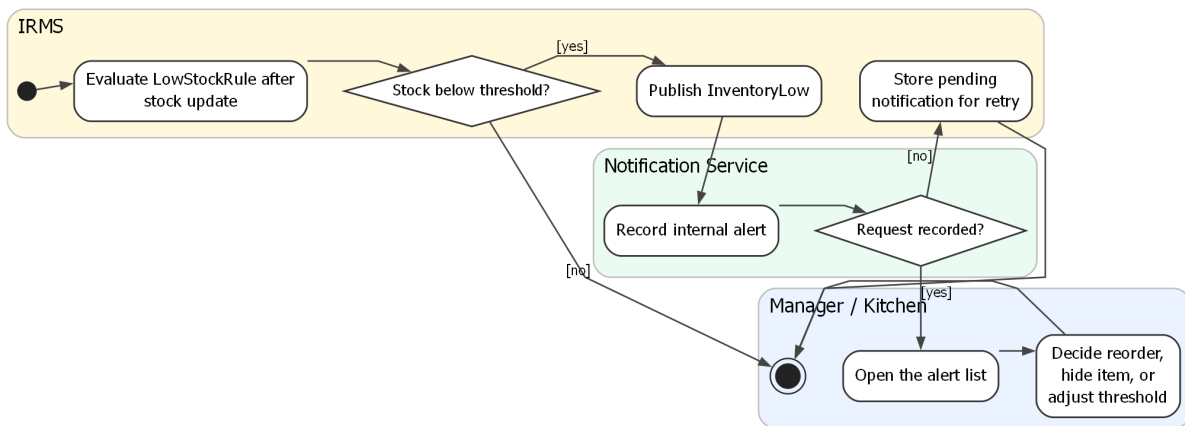
- Sinh StockTransaction theo từng nguyên liệu
- Cập nhật số lượng tồn hiện có
- Đánh giá các điều kiện bất thường

Sơ đồ cũng làm rõ hai nhánh ngoại lệ quan trọng:

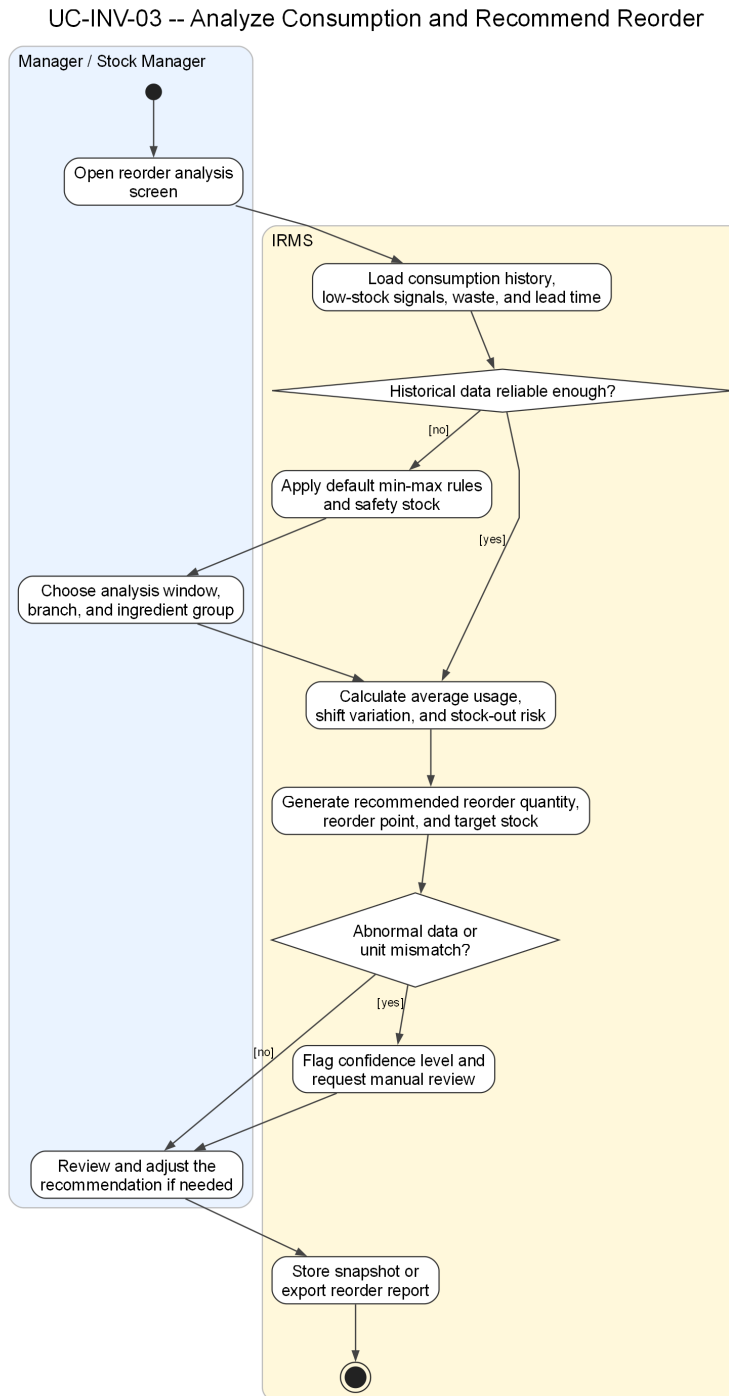
- Thiếu ánh xạ công thức
- Âm kho bất thường

Nhờ đó phân thiết kế hành vi bám sát hơn với ca sử dụng đã đặc tả, đồng thời nối được với sơ đồ cảnh báo sắp hết hàng ngay bên dưới.

UC-INV-02 -- Handle Low-Stock Alert



Hình 57: Sơ đồ hoạt động cho UC-INV-02 – Nhận cảnh báo sắp hết hàng

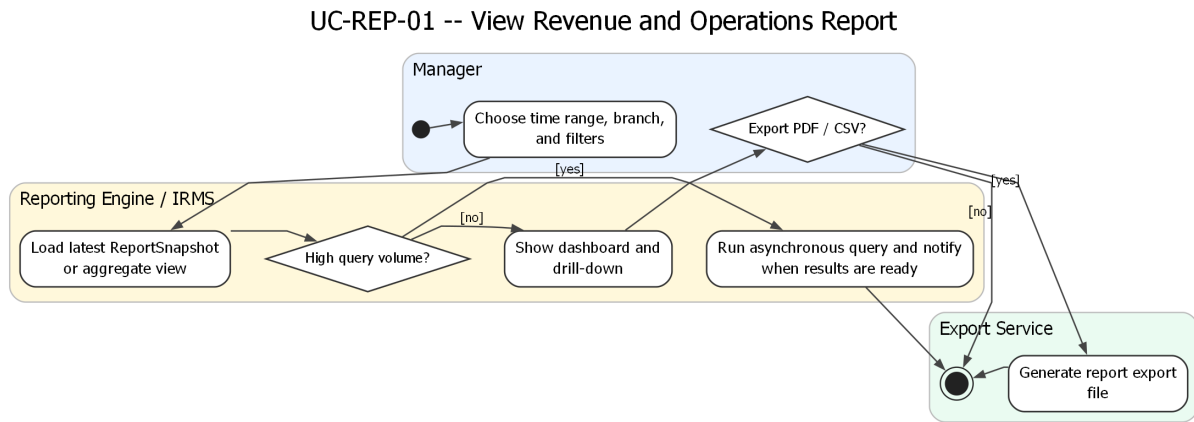


Hình 58: Sơ đồ hoạt động cho UC-INV-03 – Phân tích tiêu hao và đề xuất nhập hàng

Phân tích sơ đồ UC-INV-03. Use case này giúp phần tồn kho bám sát hơn với yêu cầu “historical usage helps predict reorder quantities and manage waste”. Nếu chỉ dừng ở cập nhật tiêu hao và cảnh báo sắp hết hàng, IRMS mới chỉ hỗ trợ phản ứng vận hành ngắn hạn, chưa đủ để ra quyết định chuẩn bị hàng hóa theo chu kỳ ngày, tuần hoặc mùa cao điểm. Do đó, sơ đồ này chuyển trọng tâm của vùng tồn kho từ **theo dõi hiện trạng** sang **hỗ trợ ra quyết định**.

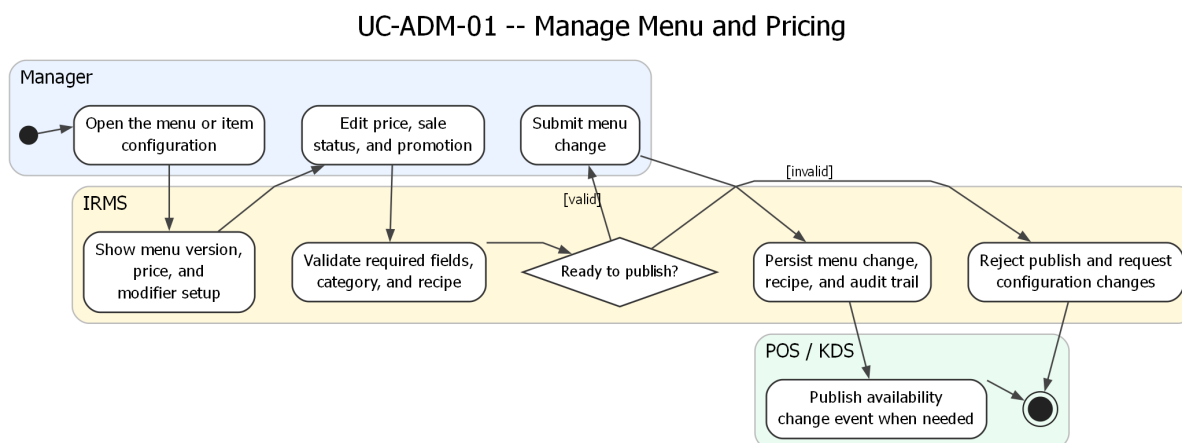
Điểm đáng chú ý là sơ đồ không mô tả dự báo như một thuật toán mơ hồ mà chỉ rõ các dữ liệu đầu vào gồm tiêu hao lịch sử, cảnh báo sắp hết hàng, hao hụt, lead time và mức tồn an toàn. Nhờ đó, đề xuất nhập hàng được

thể hiện là kết quả của mô hình dữ liệu và luồng xử lý đã có sẵn, không phải một chức năng bổ sung rời rạc thiếu nền dữ liệu đủ tin cậy.



Hình 59: Sơ đồ hoạt động cho UC-REP-01 – Xem báo cáo doanh thu và vận hành

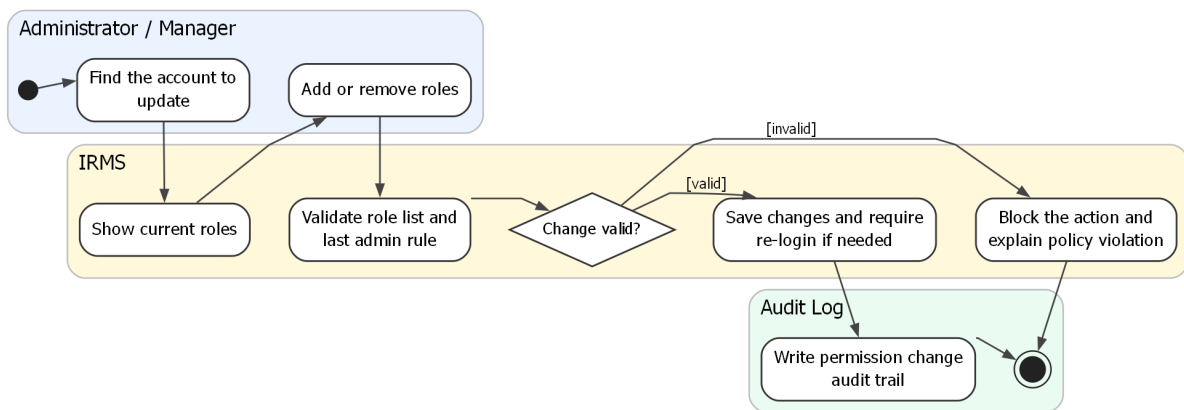
4.3.5 Nhóm quản trị cấu hình và vận hành



Hình 60: Sơ đồ hoạt động cho UC-ADM-01 – Quản lý thực đơn và giá bán

- Quản lý mở màn hình cấu hình thực đơn và chọn món hoặc danh mục cần chỉnh sửa.
- Hệ thống hiển thị thông tin chi tiết của món, bao gồm giá, trạng thái và các chương trình khuyến mãi hiện tại.
- Quản lý thực hiện chỉnh sửa thông tin món và có thể áp dụng khuyến mãi nếu cần.
- Khi lưu thay đổi, quản lý gửi cấu hình món, giá, trạng thái bán, khuyến mãi hoặc công thức nguyên liệu theo form hiện tại.
- Hệ thống kiểm tra dữ liệu cấu hình:
 - Nếu thiếu thông tin bắt buộc, hệ thống yêu cầu bổ sung trước khi tiếp tục.
 - Nếu danh mục không tồn tại hoặc công thức nguyên liệu sai định dạng, hệ thống yêu cầu điều chỉnh trước khi lưu.
- Khi dữ liệu hợp lệ, hệ thống lưu thay đổi thực đơn, công thức liên quan và ghi nhận dấu vết kiểm toán; nếu trạng thái bán thay đổi, hệ thống phát event cập nhật khả dụng.
- Các đơn hàng đang mở vẫn giữ nguyên giá đã ghi nhận trước đó để đảm bảo tính nhất quán.

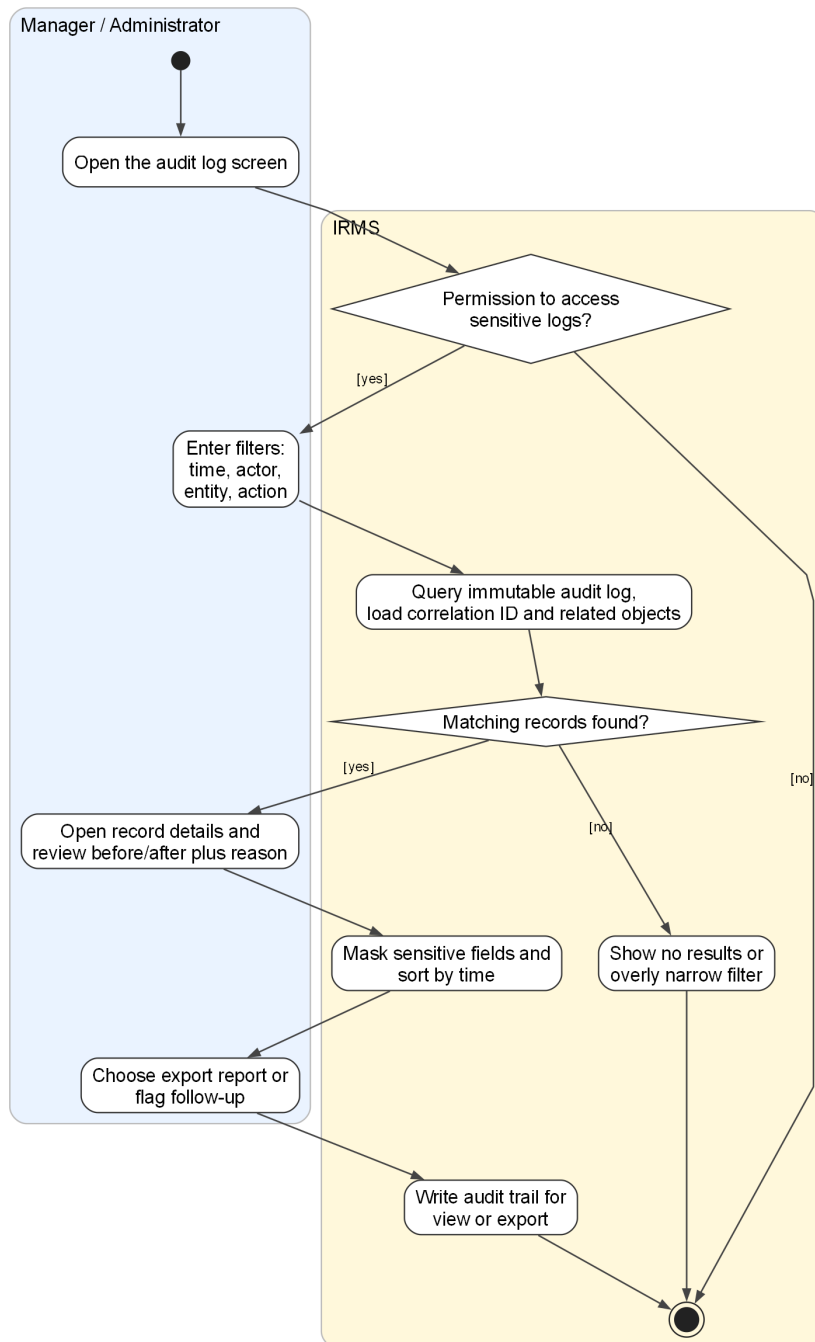
UC-ADM-02 -- Manage Roles and Permissions



Hình 61: Sơ đồ hoạt động cho UC-ADM-02 – Quản lý vai trò và phân quyền

- Quản trị viên mở màn hình quản lý người dùng và tìm kiếm người dùng cần cập nhật quyền.
- Hệ thống hiển thị vai trò và danh sách quyền hiện tại của người dùng.
- Quản trị viên thực hiện gán hoặc điều chỉnh vai trò và quyền theo chính sách RBAC.
- Hệ thống xử lý các trường hợp mở rộng:
 - Nếu không còn vai trò nào được chọn, hệ thống chặn thao tác vì mỗi nhân viên phải giữ ít nhất một vai trò.
 - Nếu vai trò được chọn không tồn tại trong danh sách hiện hành, hệ thống chặn thao tác và yêu cầu chọn lại.
- Hệ thống kiểm tra tính hợp lệ của chính sách phân quyền:
 - Nếu phát hiện xung đột chính sách RBAC, hệ thống yêu cầu quản trị viên điều chỉnh lại quyền.
 - Nếu thao tác dẫn đến việc gỡ bỏ quyền quản trị cuối cùng, hệ thống chặn thao tác để đảm bảo luôn tồn tại ít nhất một tài khoản quản trị.
- Khi dữ liệu hợp lệ, hệ thống thay thế danh sách vai trò của người dùng bằng các vai trò đã chọn.
- Hệ thống yêu cầu người dùng đăng nhập lại để áp dụng quyền mới.
- Cuối cùng, hệ thống ghi nhận toàn bộ thay đổi vào nhật ký kiểm toán nhằm phục vụ truy vết và kiểm soát.

UC-ADM-03 -- Review Audit Log and Trace Sensitive Actions

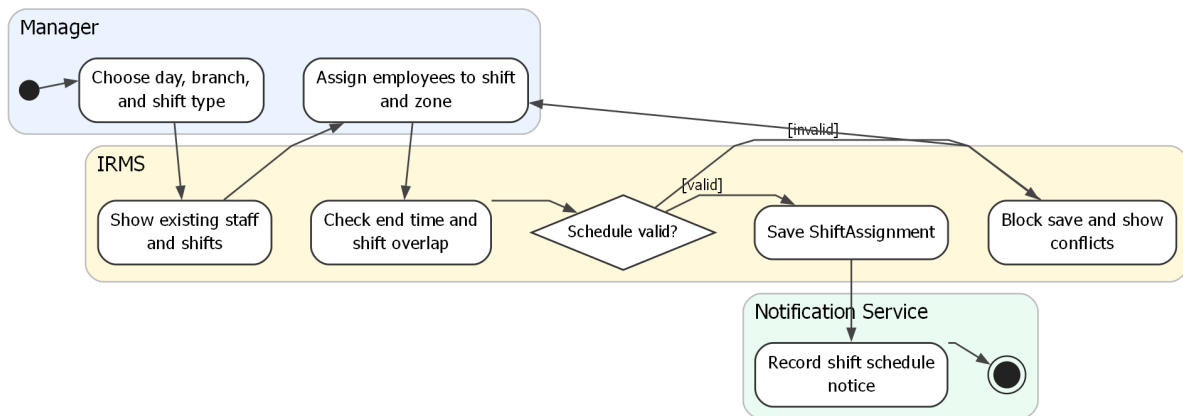


Hình 62: Sơ đồ hoạt động cho UC-ADM-03 – Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

- Người dùng có thẩm quyền mở màn hình nhật ký kiểm toán để thực hiện tra cứu.
- Hệ thống kiểm tra quyền truy cập:
 - Nếu không đủ quyền, hệ thống từ chối truy cập hoặc chỉ hiển thị dữ liệu rút gọn.
 - Nếu hợp lệ, hệ thống cho phép tiếp tục truy vấn.
- Người dùng nhập bộ lọc tìm kiếm (thời gian, người thao tác, loại hành động, mã liên kết).
- Hệ thống truy vấn và hiển thị danh sách nhật ký kiểm toán:

- Nếu không có dữ liệu phù hợp, hệ thống thông báo và yêu cầu điều chỉnh bộ lọc.
- Nếu người dùng không đủ quyền, hệ thống làm mờ hoặc ẩn thông tin nhạy cảm.
- Người dùng chọn một bản ghi để xem chi tiết, bao gồm giá trị trước và sau thay đổi, lý do và ngữ cảnh thực hiện.
- Người dùng có thể xuất báo cáo hoặc đánh dấu bản ghi cần theo dõi.
- Hệ thống ghi nhận hành vi truy cập và thao tác trên nhật ký kiểm toán nhằm phục vụ kiểm soát và truy vết sau này.

UC-OPS-01 -- Manage Staff Shifts

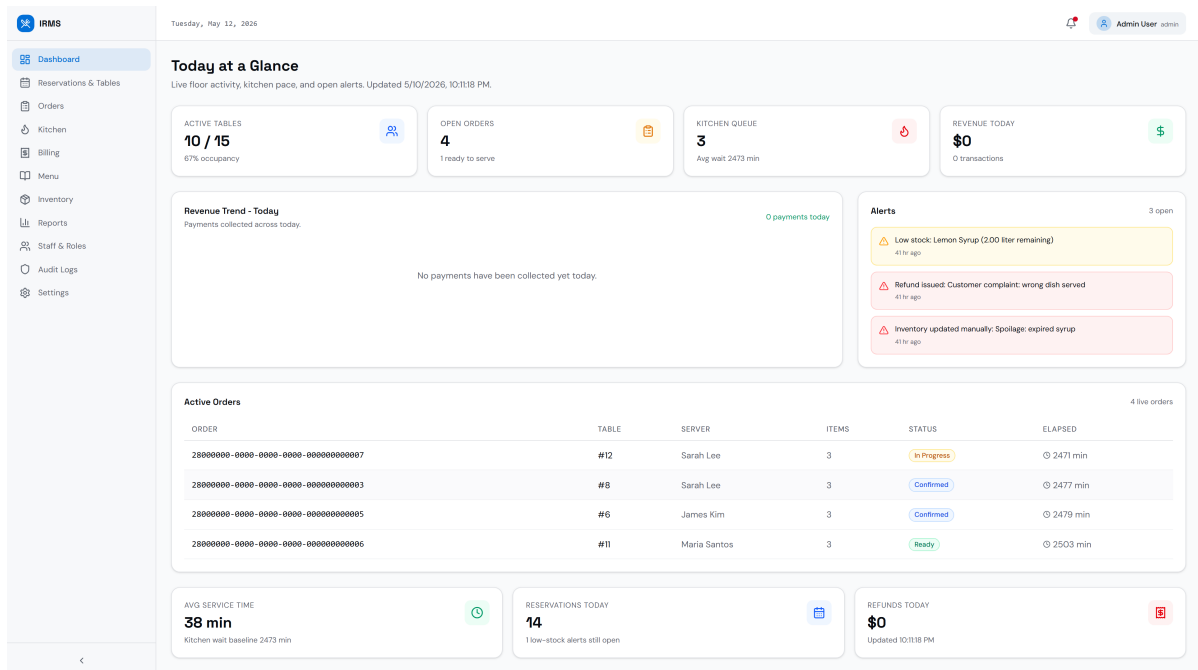


Hình 63: Sơ đồ hoạt động cho UC-OPS-01 – Quản lý ca làm nhân viên

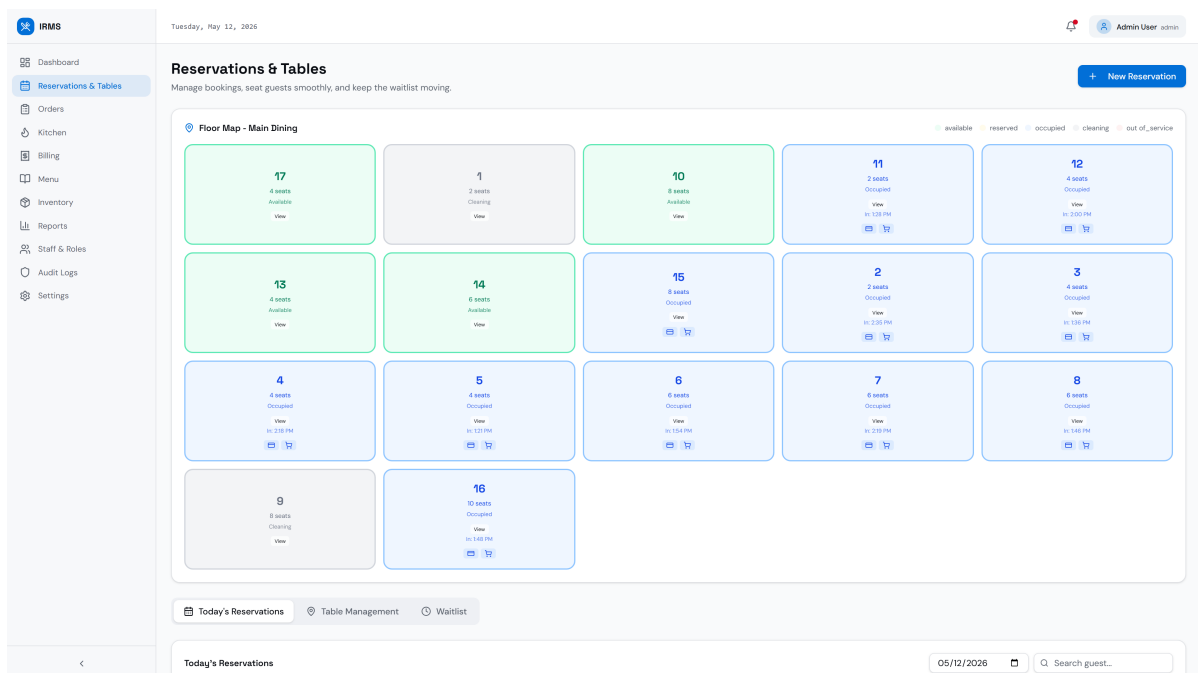
- Quản lý mở màn hình lịch làm việc và chọn ngày, khu vực cùng loại ca cần phân công.
- Hệ thống hiển thị danh sách nhân viên và các ca làm việc đã có.
- Quản lý thực hiện gán nhân viên vào ca và khu vực làm việc.
- Hệ thống hỗ trợ các trường hợp mở rộng:
 - Nếu thời điểm kết thúc không sau thời điểm bắt đầu, hệ thống chặn lưu ca.
 - Nếu nhân viên đã có ca trùng khoảng thời gian, hệ thống chặn lưu ca và yêu cầu chọn lại.
- Hệ thống kiểm tra các ràng buộc lập lịch:
 - Nếu phát hiện trùng ca, hệ thống chặn lưu và yêu cầu điều chỉnh.
 - Nếu dữ liệu hợp lệ, hệ thống lưu ShiftAssignment, ghi nhật ký kiểm toán và đưa thông báo nội bộ vào hàng chờ cho nhân viên.
- Khi dữ liệu hợp lệ, quản lý lưu lịch làm việc.
- Hệ thống cập nhật ShiftAssignment, ghi nhật ký kiểm toán và đưa thông báo nội bộ vào hàng chờ cho nhân viên liên quan.

5 Hiện thực

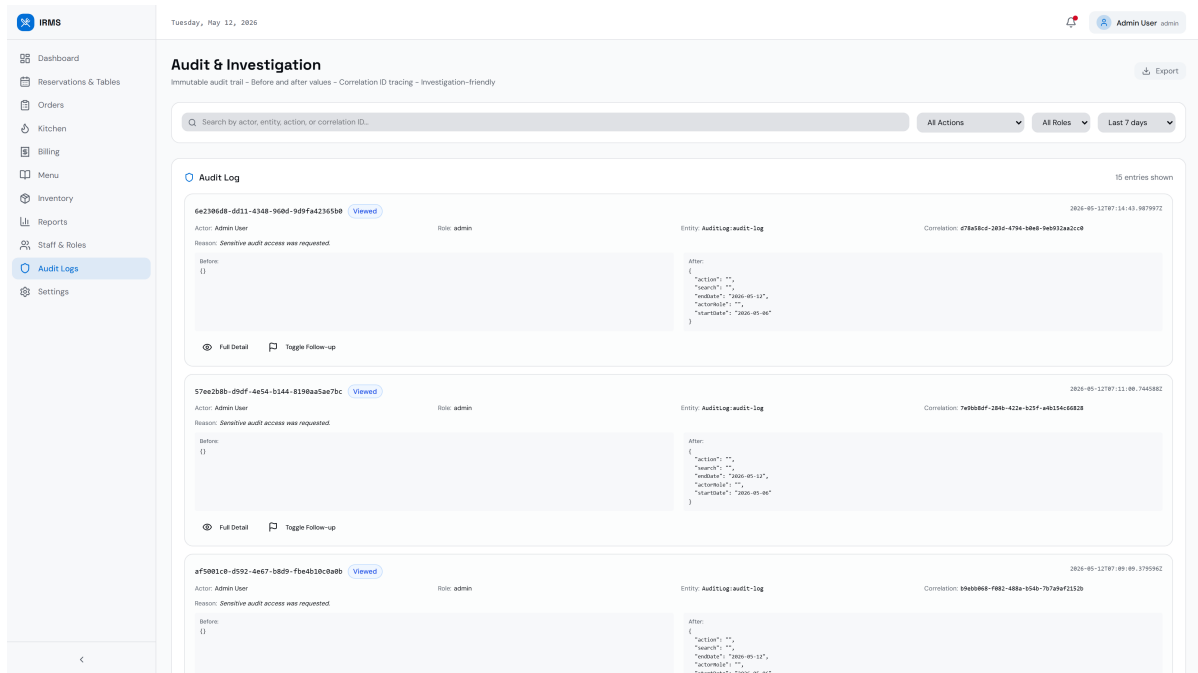
Source code Các giao diện đã được triển khai cho hệ thống IRMS.



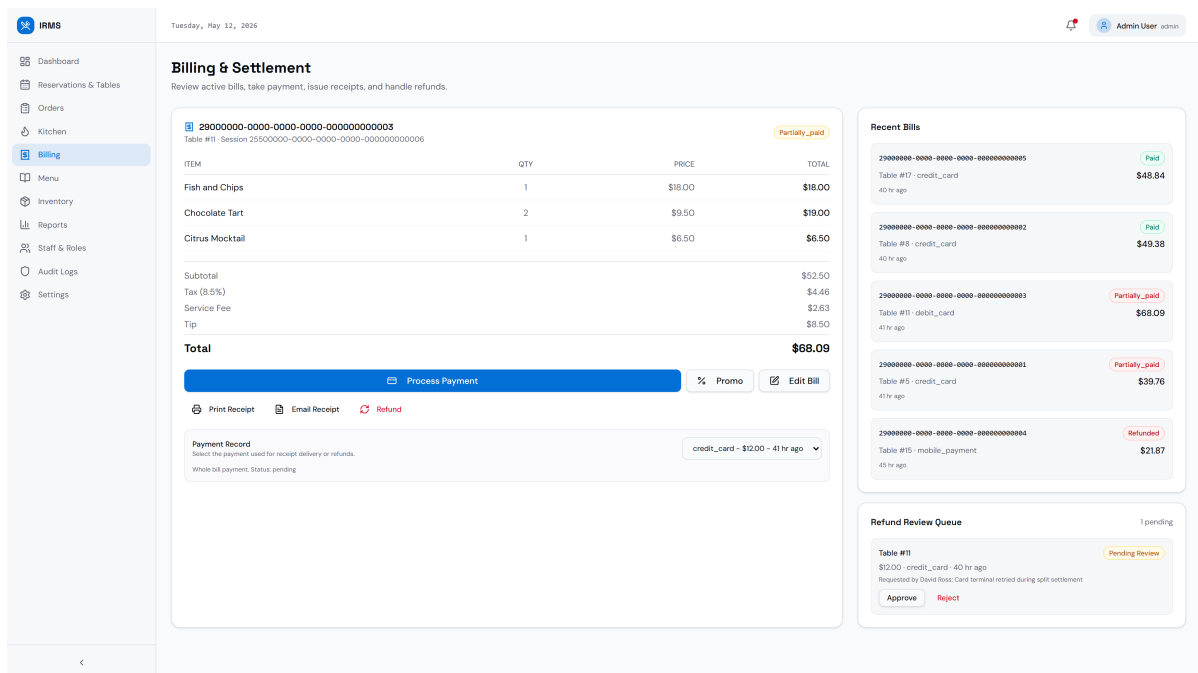
Hình 64: Giao diện dashboard của hệ thống IRMS



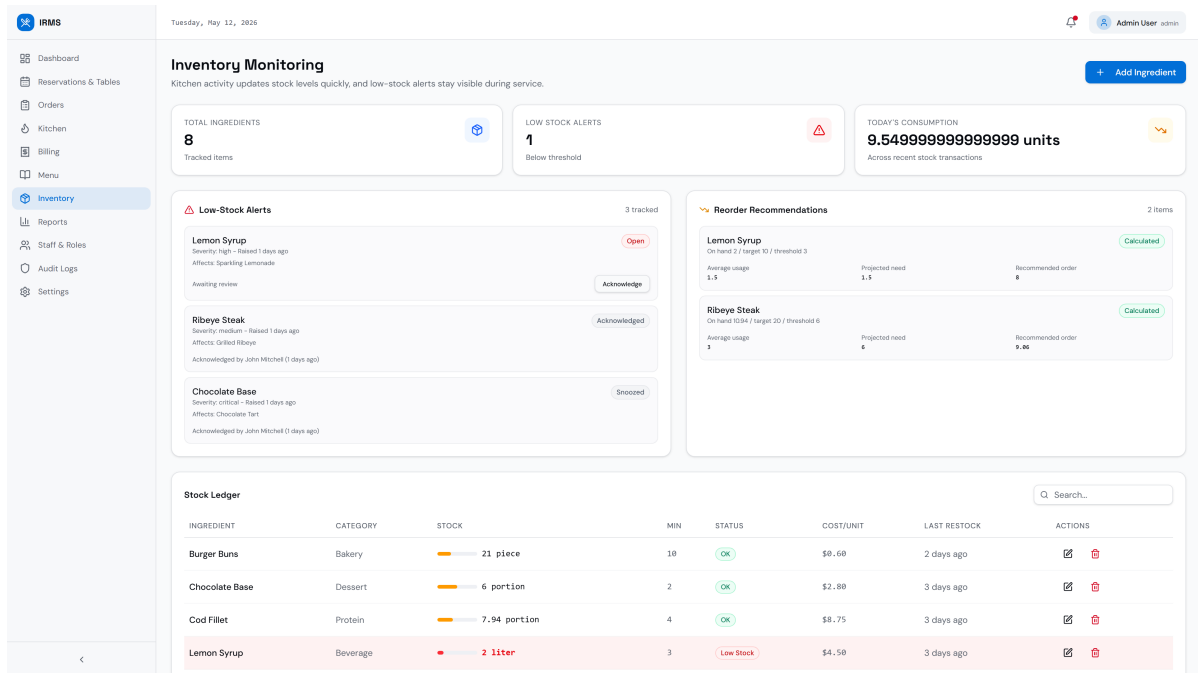
Hình 65: Giao diện đặt bàn của hệ thống IRMS



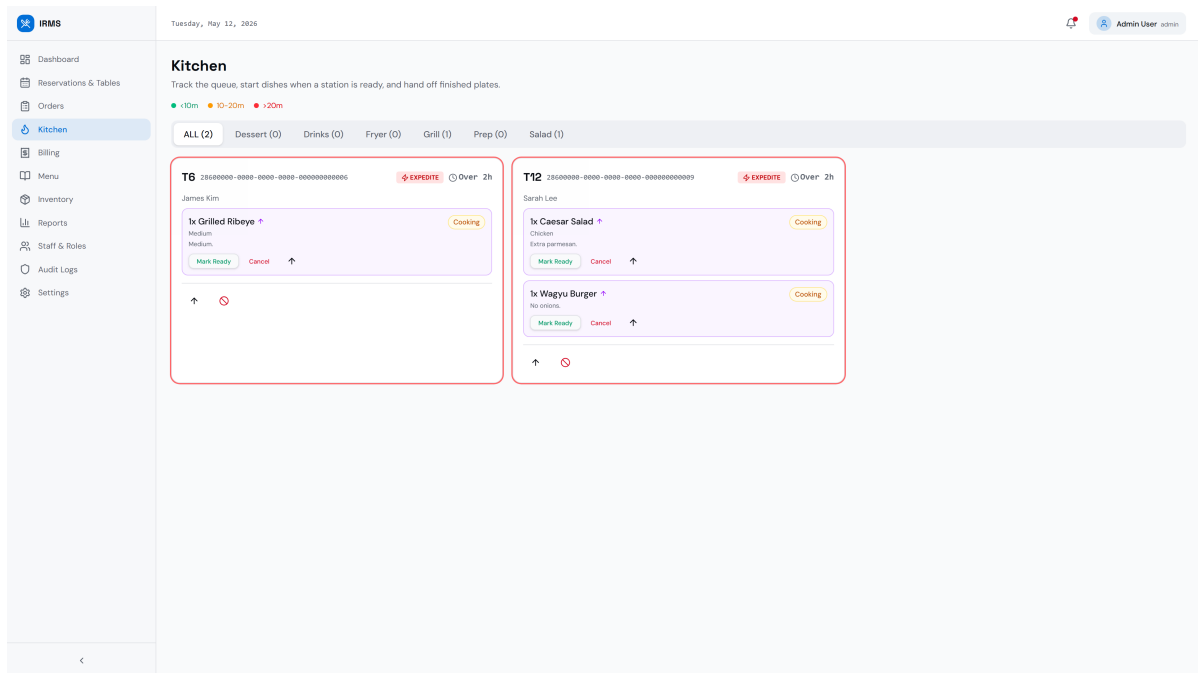
Hình 66: Giao diện kiểm toán của hệ thống IRMS



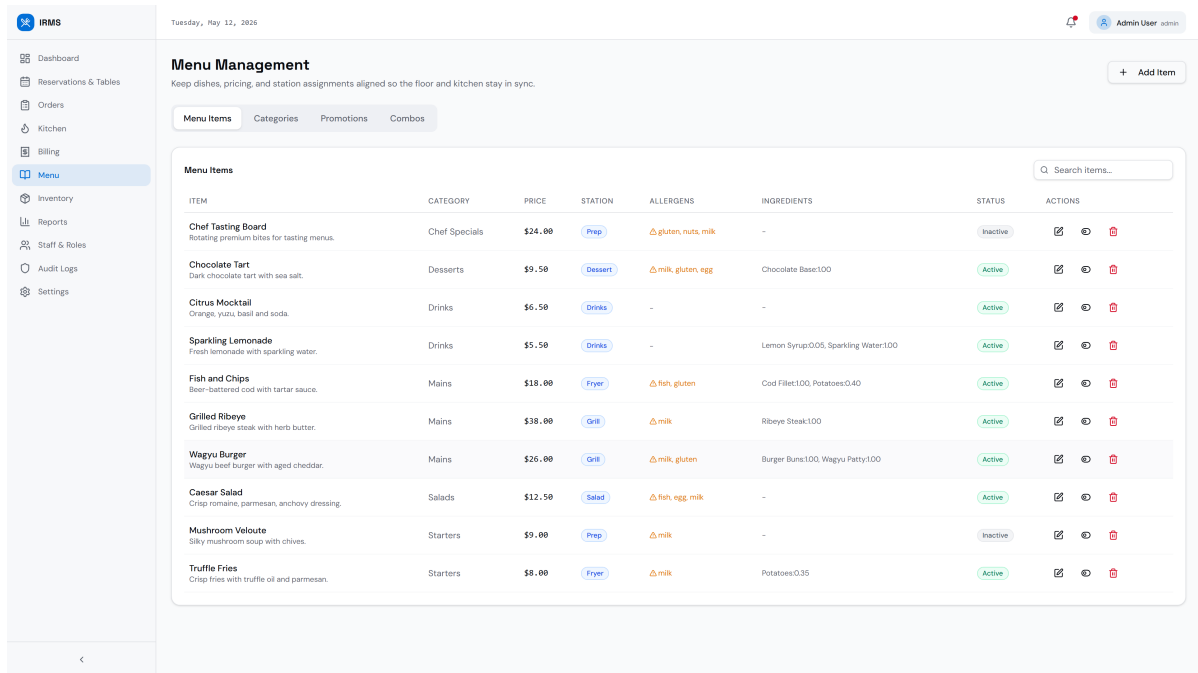
Hình 67: Giao diện thanh toán của hệ thống IRMS



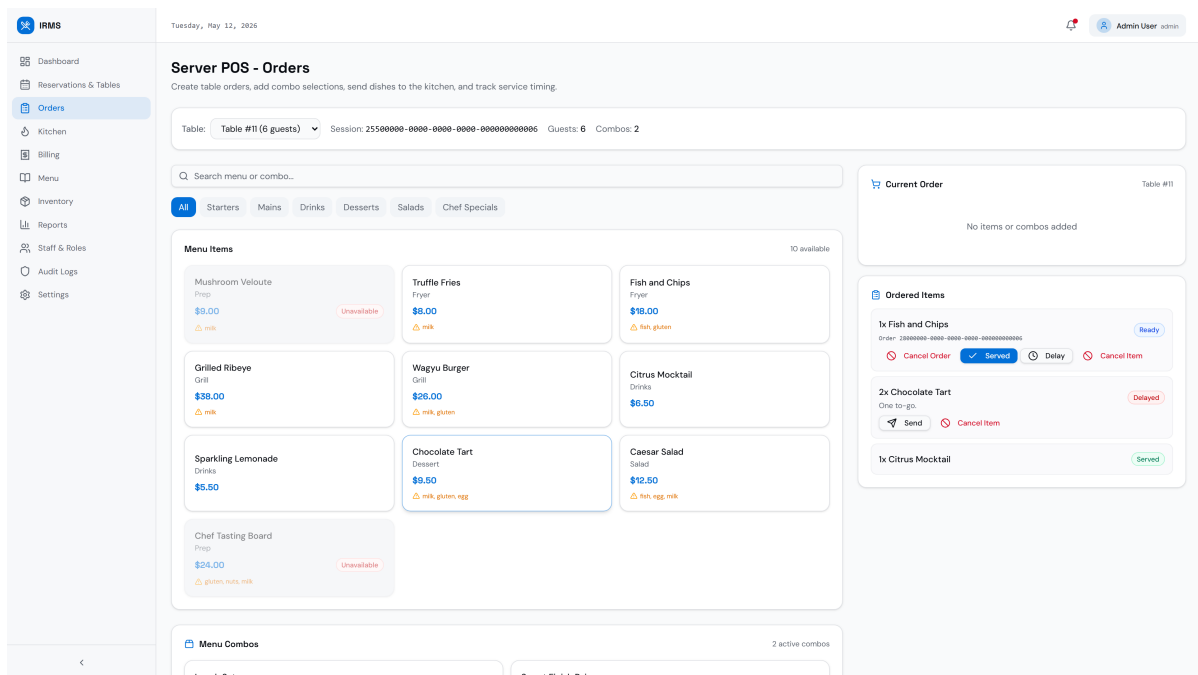
Hình 68: Giao diện quản lý tồn kho của hệ thống IRMS



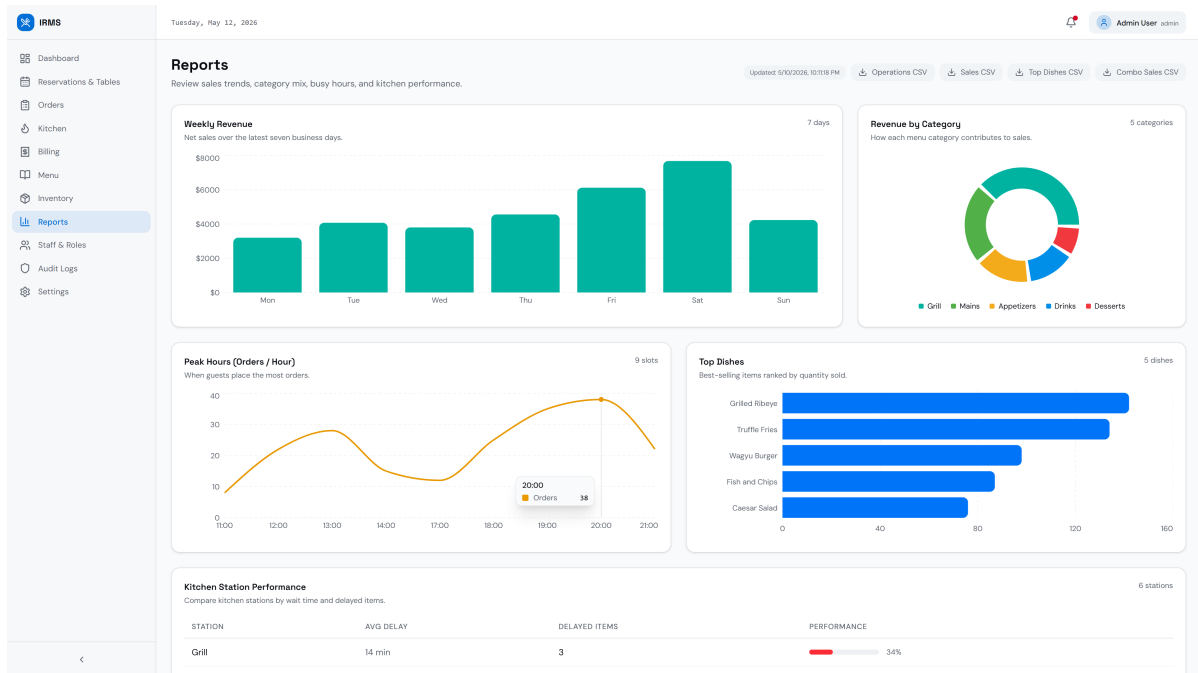
Hình 69: Giao diện quản lý bếp của hệ thống IRMS



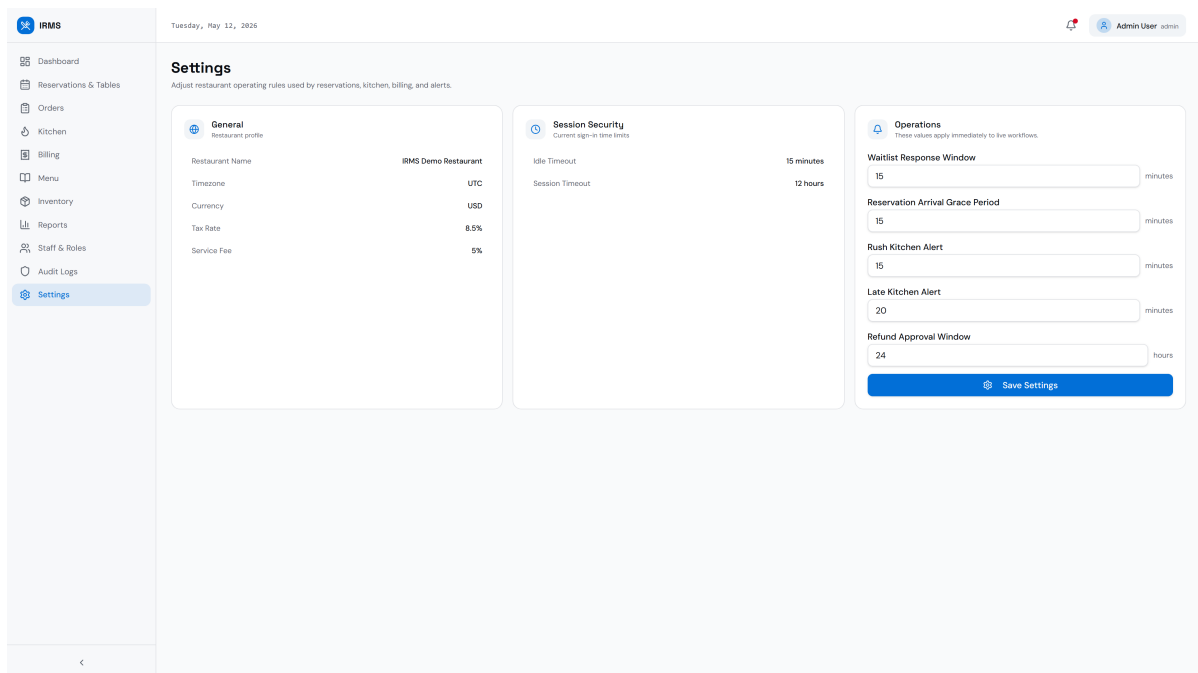
Hình 70: Giao diện quản lý thực đơn của hệ thống IRMS



Hình 71: Giao diện quản lý đơn hàng của hệ thống IRMS



Hình 72: Giao diện quản lý báo cáo của hệ thống IRMS

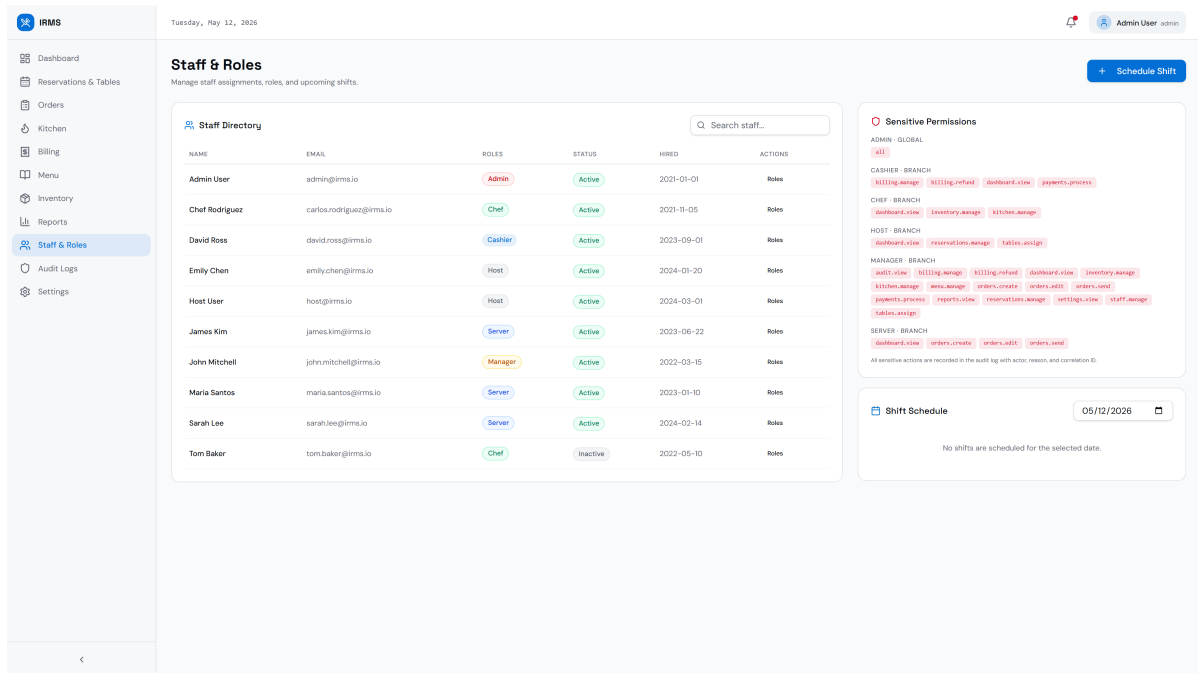


The screenshot displays the 'Settings' section of the IRMS system. It includes a sidebar with navigation options like Dashboard, Reservations & Tables, Orders, Kitchen, Billing, Menu, Inventory, Reports, Staff & Roles, Audit Logs, and Settings (selected). The main content area shows several settings panels:

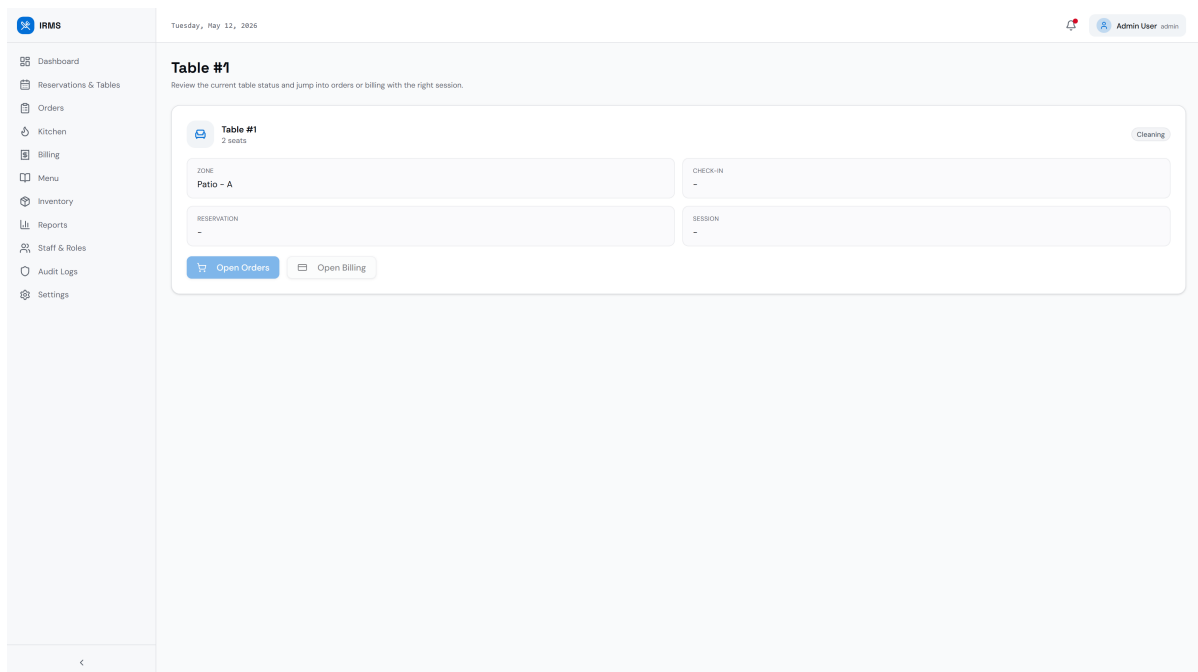
- General:** Restaurant profile settings including Restaurant Name (IRMS Demo Restaurant), Timezone (UTC), Currency (USD), Tax Rate (8.5%), and Service Fee (5%).
- Session Security:** Current sign-in time limits including Idle Timeout (15 minutes) and Session Timeout (12 hours).
- Operations:** Workflow settings including Waitlist Response Window (15 minutes), Reservation Arrival Grace Period (15 minutes), Rush Kitchen Alert (15 minutes), Late Kitchen Alert (20 minutes), and Refund Approval Window (24 hours).

A 'Save Settings' button is located at the bottom right of the Operations panel.

Hình 73: Giao diện quản lý cài đặt của hệ thống IRMS



Hình 74: Giao diện quản lý nhân viên của hệ thống IRMS



Hình 75: Giao diện quản lý bàn của hệ thống IRMS

A Phụ lục

A.1 Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc

Các quyết định kiến trúc của IRMS được đặt trên ba cơ sở: bối cảnh vận hành của nhà hàng, yêu cầu chức năng theo từng miền nghiệp vụ và yêu cầu phi chức năng ở mức toàn dự án. Kiến trúc chủ đạo là **service-based architecture theo miền nghiệp vụ**; **event-driven processing** đóng vai trò hỗ trợ cho các hệ quả nền như thông báo, cập nhật mô hình đọc báo cáo, kiểm toán và cảnh báo tồn kho. Cách lựa chọn này giữ tuyến giao dịch chính

ngắn, đồng thời vẫn cho phép các hoạt động hỗ trợ được xử lý linh hoạt phía sau.

A.1.1 So sánh các kiểu kiến trúc ứng viên

Bảng 40: So sánh các kiểu kiến trúc ứng viên cho IRMS

Kiểu kiến trúc	Điểm phù hợp	Giới hạn trong bối cảnh IRMS
Kiến trúc phân lớp tập trung thuần	Dễ khởi tạo và đơn giản cho các thao tác nội bộ.	Dễ gom các nghiệp vụ khác bản chất vào cùng một tầng kỹ thuật, làm mờ ranh giới giữa đặt bàn, gọi món, bếp, thanh toán, tồn kho và báo cáo.
Service-based theo miền nghiệp vụ	Phù hợp với cách IRMS chia trách nhiệm theo các năng lực vận hành của nhà hàng; mỗi service có ranh giới và hợp đồng giao tiếp rõ hơn.	Cần quản lý chặt hợp đồng giữa các service và tránh phụ thuộc vòng giữa các miền nghiệp vụ.
Microservices đầy đủ	Cô lập mạnh về triển khai và hỗ trợ vòng đời phát hành riêng.	Chi phí vận hành, giám sát, quản trị giao dịch phân tán và quản lý hợp đồng dịch vụ cao hơn nhu cầu của phạm vi hiện tại.
Event-driven thuần	Phù hợp cho phát tán sự kiện, xử lý nền và cập nhật mô hình đọc.	Không phù hợp cho toàn bộ hệ thống vì tạo đơn, lập hóa đơn, thanh toán và hoàn tiền cần phản hồi tức thời và trạng thái lỗi rõ ràng.
Service-based kết hợp event-driven	Giữ ranh giới nghiệp vụ bằng service-based architecture, đồng thời dùng event-driven cho các hệ quả nền như thông báo, báo cáo, kiểm toán và cảnh báo tồn kho.	Cần phân biệt rõ thao tác nào đồng bộ, thao tác nào bất đồng bộ; các luồng nền cần chống xử lý lặp và có khả năng theo dõi trạng thái.

Từ các phương án trên, service-based architecture là cấu trúc chính vì bám sát các miền trách nhiệm đã nêu trong Chương 1. Event-driven processing không thay thế cấu trúc này mà bổ sung một cơ chế phối hợp cho các hệ quả không cần chặn thao tác của nhân viên. Sự kết hợp này tạo ra một kiến trúc vừa rõ ràng ở tuyến giao dịch chính, vừa đủ linh hoạt cho thông báo, báo cáo, kiểm toán và cảnh báo tồn kho.

A.1.2 Hồ sơ quyết định kiến trúc

A.1.3 Tổng quan danh mục ADR

Các quyết định kiến trúc dưới đây được ghi lại để giữ bối cảnh lựa chọn của IRMS, đặc biệt là mối liên hệ giữa yêu cầu phi chức năng ở Mục 1.6 và các góc nhìn kiến trúc ở Chương 3–4. Phân tích chi tiết vẫn nằm trong thân báo cáo; mỗi ADR chốt một quyết định, trạng thái, căn cứ và hệ quả chính. Các mã ADR được nhắc trong các section chính là liên kết trực tiếp tới bản ghi quyết định tương ứng. Người đọc có thể dùng các liên kết này để xem bối cảnh, căn cứ và hệ quả của từng quyết định kiến trúc.

Bảng 41: Danh mục quyết định kiến trúc quan trọng của IRMS

ADR	Trạng thái	Quyết định	Ý nghĩa kiến trúc	Mục liên kết
ADR-0001	Thay thế bởi ADR-0002	Modular architecture cho phân rã ban đầu.	Structure, maintainability	Mục 3.5.
ADR-0002	Được chấp nhận; thay thế ADR-0001	Service-based architecture là kiến trúc chủ đạo; event-driven processing là cơ chế hỗ trợ.	Structure, NFR, dependency	Mục 1.7, Mục 3.1.
ADR-0003	Được chấp nhận	Gateway boundary cho frontend và REST entry point.	Interface, dependency, security boundary	Mục 3.1, Mục 3.6.1.
ADR-0004	Được chấp nhận	REST đồng bộ cho thao tác cần phản hồi trực tiếp.	Interface, responsiveness	Mục 3.6.2.
ADR-0005	Được chấp nhận	Event-driven processing cho hệ quả bất đồng bộ.	NFR, dependency, scalability	Mục 3.6.3.
ADR-0006	Được chấp nhận	Outbox pattern để phát event sau giao dịch nghiệp vụ.	Reliability, construction technique	Mục 3.6.3.
ADR-0007	Được chấp nhận	RabbitMQ làm broker cho luồng event nội bộ.	Dependency, reliability, operation	Mục 3.6.
ADR-0008	Được chấp nhận	Một PostgreSQL vật lý, nhiều nhóm bảng logic.	Data, deployment	Mục 3.7.
ADR-0009	Được chấp nhận	Read model logic cho báo cáo.	Data, performance	Mục 3.1.
ADR-0010	Được chấp nhận	Idempotency/inbox tracking cho consumer nền.	Reliability	Mục 3.6.
ADR-0011	Được chấp nhận	Boundary adapters cho thanh toán, thông báo và biên nhận.	Interface, extensibility	Mục 3.9.
ADR-0012	Được chấp nhận	IAM/RBAC và audit log cho thao tác nhạy cảm.	Security, auditability	Mục 1.6.
ADR-0013	Được chấp nhận	Layering trong từng service.	Structure, maintainability	Mục 3.5.
ADR-0014	Được chấp nhận	Docker Compose cho runtime cục bộ.	Deployment, construction	Mục 3.7.
ADR-0015	Được chấp nhận	Correlation id/logging cho truy vết vận hành.	Observability, auditability	Mục 1.6.

Bảng 42: Liên kết ADR với yêu cầu phi chức năng

ADR	Quyết định	Yêu cầu phi chức năng liên quan	Lý do liên kết
ADR-0001	Modular architecture cho phân rã ban đầu.	Bảo trì và mở rộng chức năng.	Tách hệ thống theo vùng nghiệp vụ giúp tổ chức source code để hiểu ở giai đoạn đầu.
ADR-0002	Service-based architecture là kiến trúc chủ đạo.	Khả năng mở rộng; bảo trì; độ phản hồi.	Ranh giới service giúp cô lập thay đổi và mở rộng các vùng tải cao như gọi món, bếp và thanh toán.
ADR-0003	Gateway boundary cho frontend.	Bảo trì; quan sát vận hành; bảo mật.	Gateway tạo điểm vào thống nhất để kiểm soát route, lỗi, timeout và chính sách ở biên.
ADR-0004	REST đồng bộ cho thao tác cần phản hồi trực tiếp.	Hiệu năng và độ phản hồi.	Các thao tác tại giao diện phục vụ, bếp và thu ngân cần phản hồi rõ trong thời gian ngắn.
ADR-0005	Event-driven processing cho hệ quả bất đồng bộ.	Độ phản hồi; khả năng mở rộng; độ tin cậy.	Báo cáo, thông báo, kiểm toán và cập nhật nền được tách khỏi tuyến giao dịch nóng.
ADR-0006	Outbox pattern cho event sau giao dịch nghiệp vụ.	Độ tin cậy và toàn vẹn giao dịch.	Ghi event cùng giao dịch nghiệp vụ giúp giảm rủi ro mất event khi publish thất bại.
ADR-0007	RabbitMQ cho event nội bộ.	Khả năng mở rộng; quan sát vận hành; độ tin cậy.	Broker tách producer khỏi consumer và hỗ trợ xử lý nền độc lập.
ADR-0008	Một PostgreSQL vật lý, nhiều nhóm bảng logic.	Độ tin cậy; vận hành; bảo trì.	Một database vật lý đơn giản hóa runtime hiện tại, còn module và nhóm bảng giữ ranh giới logic.
ADR-0009	Read model logic cho báo cáo.	Hiệu năng; khả năng mở rộng; quan sát vận hành.	Truy vấn báo cáo không làm nặng dữ liệu giao dịch chính và có thể chấp nhận độ trễ.
ADR-0010	Idempotency/inbox tracking cho consumer nền.	Độ tin cậy và toàn vẹn giao dịch.	Consumer có thể nhận lại event, vì vậy cần tránh xử lý lặp gây tác dụng phụ.
ADR-0011	Boundary adapters cho thanh toán, thông báo và biên nhận.	Bảo trì và mở rộng chức năng.	Provider hoặc kênh gửi có thể thay đổi mà không làm service nghiệp vụ phụ thuộc trực tiếp.
ADR-0012	IAM/RBAC và audit log.	Bảo mật và khả năng kiểm toán.	Hoàn tiền, hủy món, ghi đề giá và thay đổi quyền cần kiểm soát quyền và lưu vết.

ADR	Quyết định	Yêu cầu phi chức năng liên quan	Lý do liên kết
ADR-0013	Layering trong từng service.	Bảo trì và mở rộng chức năng.	Tách controller, use case, domain policy và adapter hạ tầng giúp giảm trộn trách nhiệm.
ADR-0014	Docker Compose cho runtime cục bộ.	Vận hành; khả năng kiểm thử.	Môi trường cục bộ thống nhất giúp kiểm thử tích hợp và giảm sai lệch cấu hình.
ADR-0015	Correlation id/logging.	Quan sát vận hành; kiểm toán.	Request và event cần truy vết xuyên suốt để phân tích lỗi và đối chiếu thao tác.

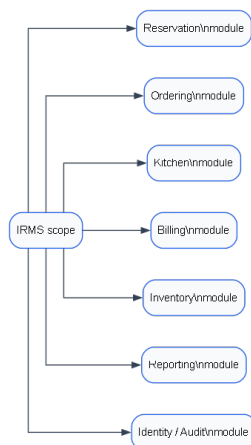
A.1.4 Nhóm quyết định về phong cách kiến trúc và phân rã

ADR-0001: Chọn modular architecture cho phân rã ban đầu. **Trạng thái.** Thay thế bởi [ADR-0002](#).

Bối cảnh. Ở giai đoạn đầu, IRMS cần một cách phân rã dễ hiểu để mô tả các vùng nghiệp vụ như đặt bàn, gọi món, bếp, thanh toán, tồn kho, báo cáo, định danh và kiểm toán. Modular architecture phù hợp cho bước khởi đầu vì giúp tách chức năng theo module và tránh trình bày hệ thống như một khối nguyên khối.

Quyết định. Dùng modular architecture/module-based decomposition làm định hướng phân rã ban đầu. Quyết định này chủ yếu phục vụ cấu trúc và tổ chức hiện thực, không còn là phong cách kiến trúc chủ đạo sau khi [ADR-0002](#) được chấp nhận.

Căn cứ. Quyết định này gắn với yêu cầu bảo trì và mở rộng chức năng ở Mục 1.6; hệ thống cần các vùng trách nhiệm đủ rõ để thay đổi thực đơn, giá, tồn kho hoặc báo cáo mà không kéo theo toàn bộ hệ thống.



Hình 76: Minh họa phân rã ban đầu theo module nghiệp vụ của IRMS

Hệ quả.

- Module View vẫn hợp lệ để mô tả package, trách nhiệm và phụ thuộc tính.
- Cách tiếp cận này chưa đủ để mô tả runtime độc lập, gateway boundary, event/outbox và mở rộng từng vùng chức năng.
- [ADR-0002](#) thay thế quyết định này ở mức architecture style chính, nhưng không phủ nhận giá trị tổ chức module.

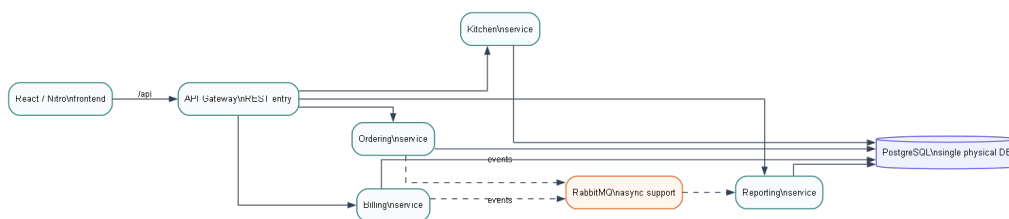
Liên kết với các góc nhìn kiến trúc. [ADR-0001](#) liên kết trực tiếp với Module View ở Mục 3.5; các quyết định runtime được chuyển sang [ADR-0002](#) và nhóm ADR về giao tiếp.

ADR-0002: Chuyển sang service-based architecture làm kiến trúc chủ đạo. **Trạng thái.** Được chấp nhận; thay thế [ADR-0001](#).

Bối cảnh. Các yêu cầu về mở rộng, phản hồi nhanh, cô lập lỗi, xử lý nền và phối hợp nhiều vùng nghiệp vụ đòi hỏi ranh giới runtime rõ hơn modular architecture ban đầu. IRMS cần giữ tuyến giao dịch nóng ngán cho gọi món, bếp và thanh toán, trong khi báo cáo, thông báo, kiểm toán và cảnh báo tồn kho có thể xử lý bất đồng bộ.

Quyết định. Chọn service-based architecture làm kiến trúc chủ đạo. Modular decomposition vẫn được giữ trong Module View để tổ chức source code và trách nhiệm module; event-driven processing chỉ là cơ chế hỗ trợ cho các hệ quả bất đồng bộ.

Căn cứ. Quyết định này xuất phát từ yêu cầu khả năng mở rộng, độ phản hồi và bảo trì ở Mục 1.6, phân lựa chọn kiến trúc ở Mục 1.7 và kiến trúc tổng quan ở Mục 3.1.



Hình 77: Minh họa service-based architecture với event-driven processing là cơ chế hỗ trợ

Hệ quả.

- Cần quản lý hợp đồng API/event giữa các service.
- Gateway boundary, outbox, RabbitMQ và quan sát liên service trở thành quyết định hỗ trợ bắt buộc.
- Việc mở rộng từng vùng chức năng khả thi hơn, nhưng làm tăng nhu cầu kiểm soát cấu hình và lỗi liên service.

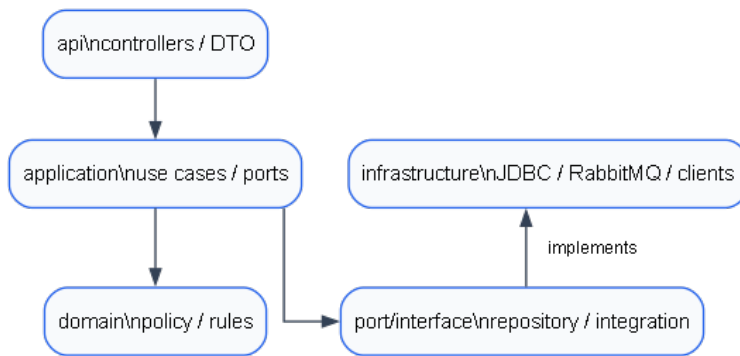
Liên kết với các góc nhìn kiến trúc. [ADR-0002](#) chỉ phối kiến trúc tổng quan, Module View và Component-and-Connector View. Deployment View cụ thể hóa quyết định này bằng các runtime service trong Docker Compose.

ADR-0013: Dùng domain/application layering trong từng service. **Trạng thái.** Được chấp nhận.

Bối cảnh. Khi đã dùng service-based architecture, từng service vẫn cần cấu trúc nội bộ đủ rõ để tránh trộn controller, use case, domain policy, repository và adapter hạ tầng. Các package hiện có như api, application, domain, infrastructure và integration thể hiện cách tổ chức theo lớp trong từng miền.

Quyết định. Trong từng service, mã nguồn được tổ chức theo lớp: api tiếp nhận REST, application điều phối use case và port, domain giữ policy/rule, còn infrastructure/integration hiện thực truy cập dữ liệu hoặc kết nối ngoài.

Căn cứ. Quyết định này liên quan trực tiếp đến yêu cầu bảo trì và mở rộng chức năng ở Mục 1.6; thay đổi rule nghiệp vụ, adapter hoặc controller cần được khu trú trong lớp đúng vai trò.



Hình 78: Minh họa layering trong từng service của IRMS

Hệ quả.

- Quy tắc phụ thuộc một chiều cần được giữ ổn định.
- Domain policy không nên phụ thuộc trực tiếp vào framework hoặc adapter hạ tầng nếu không cần.
- Module View có thể đổi chiều cấu trúc package với trách nhiệm kiến trúc.

Liên kết với các góc nhìn kiến trúc. [ADR-0013](#) cụ thể hóa Module View và hỗ trợ phần SOLID ở Mục 3.9.

A.1.5 Nhóm quyết định về giao tiếp runtime

ADR-0003: Dùng gateway boundary cho frontend và REST entry point. **Trạng thái.** Được chấp nhận.

Bối cảnh. Frontend phục vụ nhiều vai trò vận hành và không nên gọi trực tiếp mọi service nội bộ. Nếu giao diện biết toàn bộ topology backend, thay đổi service URL, port hoặc cấu trúc runtime sẽ lan lên tầng UI.

Quyết định. Frontend đi qua Frontend Node Server và API Gateway; backend dùng GatewayProxyController làm điểm vào REST để định tuyến yêu cầu đến service phù hợp.

Căn cứ. Quyết định này liên quan đến yêu cầu bảo trì, bảo mật và quan sát vận hành ở Mục 1.6. Gateway tạo điểm kiểm soát route, lỗi, timeout và chính sách truy cập ở biên hệ thống.



Hình 79: Minh họa gateway boundary giữa frontend và các REST API nội bộ

Hệ quả.

- Frontend có một entry point ổn định thay vì phụ thuộc trực tiếp vào từng service.
- Gateway trở thành điểm cấu hình quan trọng, cần kiểm soát route, timeout và lỗi định tuyến.
- Khi gateway lỗi, nhiều luồng nghiệp vụ tuyến đầu có thể bị ảnh hưởng nên healthcheck và log ở biên rất quan trọng.

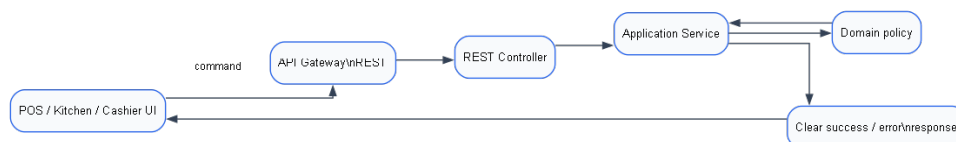
Liên kết với các góc nhìn kiến trúc. [ADR-0003](#) liên kết với kiến trúc tổng quan, C&C View và Deployment View.

ADR-0004: Dùng REST đồng bộ cho thao tác cần phản hồi trực tiếp. **Trạng thái.** Được chấp nhận.

Bối cảnh. Các thao tác như check-in, tạo order, cập nhật bếp, lập hóa đơn, thanh toán và hoàn tiền cần trả trạng thái thành công/thất bại rõ ràng cho nhân viên. Các thao tác này không phù hợp nếu chỉ phát event rồi chờ xử lý nền.

Quyết định. Dùng REST/API đồng bộ cho thao tác mà người dùng cần phản hồi trực tiếp. Event không thay thế các command tuyến đầu; event chỉ xử lý hệ quả sau khi trạng thái chính đã được ghi nhận.

Căn cứ. Quyết định này liên quan trực tiếp đến yêu cầu hiệu năng và độ phản hồi ở Mục 1.6; giao diện phục vụ, bếp và thu ngân cần phản hồi ngắn, nhất quán và dễ hiểu.



Hình 80: Minh họa tuyến REST đồng bộ cho command cần phản hồi trực tiếp

Hệ quả.

- API contract cần ổn định và phản hồi lỗi phải nhất quán.
- Tuyến giao dịch nóng phải ngắn, tránh phụ thuộc vào tác vụ báo cáo hoặc thông báo.
- Các thao tác nhạy cảm vẫn cần kiểm quyền và audit theo [ADR-0012](#).

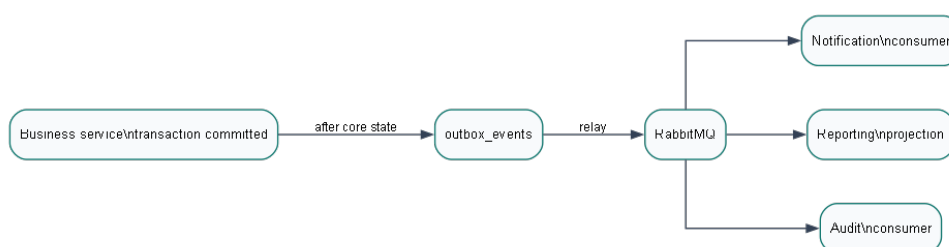
Liên kết với các góc nhìn kiến trúc. [ADR-0004](#) được thể hiện trong C&C View, đặc biệt ở phần kiểu kết nối và tuyến lệnh vận hành.

ADR-0005: Dùng event-driven processing cho hệ quả bất đồng bộ. Trạng thái. Được chấp nhận.

Bối cảnh. Báo cáo, thông báo, kiểm toán, cập nhật read model và cảnh báo tồn kho là các hệ quả quan trọng nhưng không phải lúc nào cũng cần chặn thao tác chính. Nếu đặt toàn bộ vào giao dịch đồng bộ, các luồng gọi món, bếp và thanh toán sẽ dài hơn và dễ bị nghẽn.

Quyết định. Dùng event-driven processing cho các hệ quả bất đồng bộ sau giao dịch chính. Event-driven processing không phải architecture style chính của IRMS, mà là cơ chế phối hợp hỗ trợ service-based architecture.

Căn cứ. Các yêu cầu về độ phản hồi, khả năng mở rộng và độ tin cậy ở Mục 1.6 đòi hỏi hệ quả nền không làm dài thao tác tuyến đầu và có thể được xử lý lại khi lỗi.



Hình 81: Minh họa event-driven processing cho hệ quả sau giao dịch chính

Hệ quả.

- Chấp nhận độ trễ có kiểm soát giữa giao dịch chính và hệ quả nền.
- Consumer cần idempotency, retry và log đủ để vận hành.
- Trạng thái nghiệp vụ tuyến đầu không được phụ thuộc hoàn toàn vào event nền.

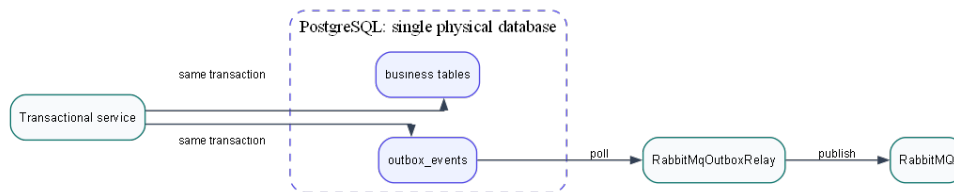
Liên kết với các góc nhìn kiến trúc. [ADR-0005](#) được cụ thể hóa trong Component-and-Connector View và liên kết với [ADR-0006](#), [ADR-0007](#), [ADR-0010](#).

ADR-0006: Dùng outbox pattern để phát event sau giao dịch nghiệp vụ. **Trạng thái.** Được chấp nhận.

Bối cảnh. Khi giao dịch nghiệp vụ đã hoàn tất nhưng bước publish message thất bại, hệ thống có nguy cơ mất event nếu publish trực tiếp. Với các luồng như xác nhận order, thanh toán hoặc cảnh báo tồn kho, mất event có thể làm reporting, notification hoặc audit không đầy đủ.

Quyết định. Event cần phát bất đồng bộ được ghi vào outbox_events trong cùng transaction với thay đổi nghiệp vụ; sau đó RabbitMqOutboxRelay đọc outbox và publish sang RabbitMQ.

Căn cứ. Yêu cầu độ tin cậy và toàn vẹn giao dịch ở Mục 1.6 đòi hỏi sự kiện nghiệp vụ không bị mất khi giao dịch chính đã hoàn tất nhưng bước phát thông điệp gặp lỗi.



Hình 82: Minh họa outbox pattern với một PostgreSQL vật lý duy nhất

Hệ quả.

- Cần relay/worker để chuyển event từ outbox sang broker.
- Cần quan sát backlog và lỗi publish để tránh event bị kẹt.
- outbox_events là nhóm bảng logic trong cùng PostgreSQL, không phải database riêng.

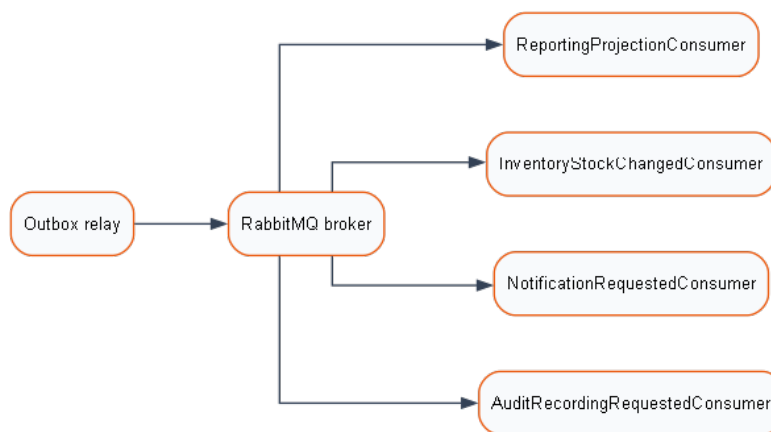
Liên kết với các góc nhìn kiến trúc. [ADR-0006](#) xuất hiện trong C&C View, Deployment View và liên kết với [ADR-0008](#) về dữ liệu.

ADR-0007: Dùng RabbitMQ làm broker cho luồng event nội bộ. **Trạng thái.** Được chấp nhận.

Bối cảnh. Các producer như ordering, billing hoặc inventory không nên phụ thuộc trực tiếp vào từng consumer reporting, notification, audit. Broker giúp tách producer khỏi consumer và cho phép các tác vụ nền phát triển độc lập hơn.

Quyết định. Dùng RabbitMQ làm broker cho event nội bộ được phát từ outbox relay đến các consumer nền.

Căn cứ. Quyết định này liên quan đến khả năng mở rộng, độ tin cậy và quan sát vận hành ở Mục 1.6; broker hỗ trợ hàng đợi, retry và tách nhịp xử lý giữa producer và consumer.



Hình 83: Minh họa RabbitMQ làm broker cho các consumer nền nội bộ

Hệ quả.

- Cần cấu hình broker và theo dõi hàng đợi trong vận hành.
- Consumer lỗi không nhất thiết làm sập tuyến giao dịch chính.
- Event có thể đến muộn hoặc được gửi lại, nên [ADR-0010](#) là điều kiện bổ trợ quan trọng.

Liên kết với các góc nhìn kiến trúc. [ADR-0007](#) nằm trong C&C View và Deployment View, đồng thời gắn với cấu hình RabbitMQ của runtime Docker.

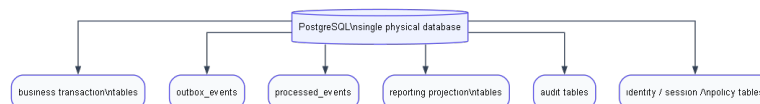
A.1.6 Nhóm quyết định về dữ liệu, nhất quán và báo cáo

ADR-0008: Dùng một PostgreSQL vật lý duy nhất, phân tách theo nhóm bảng logic. Trạng thái. Được chấp nhận.

Bối cảnh. Phạm vi triển khai hiện tại cần đơn giản, có thể khởi động cục bộ và giữ nhất quán cho giao dịch quan trọng. Tuy nhiên hệ thống vẫn cần phân biệt dữ liệu giao dịch, outbox/inbox, audit, identity/session/policy và read model báo cáo.

Quyết định. Runtime hiện tại chỉ có một PostgreSQL vật lý. Các nhóm outbox_events, processed_events, read model, audit và identity/session/policy là nhóm bảng logic trong cùng PostgreSQL.

Căn cứ. Quyết định này liên quan đến độ tin cậy, vận hành và bảo trì ở Mục 1.6; một database vật lý đơn giản hóa triển khai, trong khi phân tách logic vẫn giữ được ranh giới trách nhiệm.



Hình 84: Minh họa một PostgreSQL vật lý duy nhất với nhiều nhóm bảng logic

Hệ quả.

- Không vẽ hoặc mô tả nhiều database vật lý trong runtime hiện tại.
- Cần giữ ranh giới logic qua module, repository, naming và policy truy cập dữ liệu.
- Mô hình này phù hợp phạm vi học phần nhưng chưa phải chiến lược database-per-service.

Liên kết với các góc nhìn kiến trúc. [ADR-0008](#) chi phối Deployment View, Data View và các đoạn mô tả read model/outbox trong kiến trúc tổng quan.

ADR-0009: Dùng read model logic cho báo cáo. **Trạng thái.** Được chấp nhận.

Bối cảnh. Dashboard và báo cáo quản lý cần truy vấn doanh thu, vận hành, tồn kho và hiệu suất mà không làm nặng tuyến giao dịch chính. Báo cáo có thể chấp nhận độ trễ có kiểm soát nếu đổi lại thao tác gọi món, bếp và thanh toán ổn định hơn.

Quyết định. Reporting dùng read model/projection tables trong cùng PostgreSQL để phục vụ dashboard và báo cáo. Projection được cập nhật qua event hoặc tiến trình nền, không được mô tả như một database vật lý riêng.

Căn cứ. Quyết định này liên quan đến hiệu năng, khả năng mở rộng và quan sát vận hành ở Mục 1.6; truy vấn báo cáo không nên tranh chấp trực tiếp với dữ liệu giao dịch nóng.



Hình 85: Minh họa read model báo cáo là nhóm bảng logic trong cùng PostgreSQL

Hệ quả.

- Dữ liệu báo cáo có thể trễ so với giao dịch gốc.
- Cần mô tả rõ read model là projection logic, không phải reporting database riêng.
- Khi event hoặc consumer lỗi, dashboard có thể chậm cập nhật nhưng không nên chặn tuyến giao dịch chính.

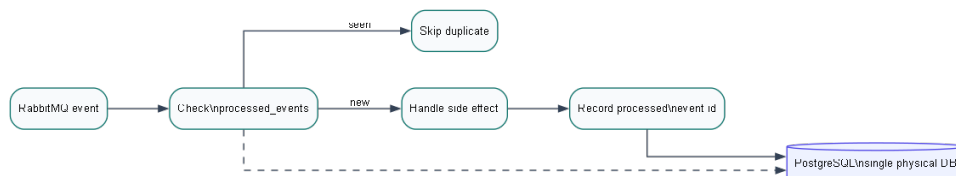
Liên kết với các góc nhìn kiến trúc. ADR-0009 liên kết với kiến trúc tổng quan, C&C View xử lý nền và các activity/reporting use case.

ADR-0010: Dùng idempotency/inbox tracking cho consumer nền. **Trạng thái.** Được chấp nhận.

Bối cảnh. Event delivery có thể retry hoặc trùng. Nếu consumer inventory, reporting, notification hoặc audit xử lý lặp mà không kiểm soát, hệ thống có thể ghi nhận sai projection, gửi thông báo lặp hoặc tạo audit không nhất quán.

Quyết định. Consumer nền ghi nhận event đã xử lý thông qua processed_events hoặc cơ chế tương đương trước/sau khi xử lý theo contract của consumer. Khi nhận event đã xử lý, consumer bỏ qua hoặc xử lý theo nhánh an toàn.

Căn cứ. Quyết định này liên quan trực tiếp đến yêu cầu độ tin cậy và toàn vẹn giao dịch ở Mục 1.6; các luồng nền không được gây tác dụng phụ lặp khi event được gửi lại.



Hình 86: Minh họa kiểm tra processed_events để tránh xử lý event lặp

Hệ quả.

- Consumer cần định danh event ổn định và kiểm tra trạng thái đã xử lý.
- Cần quan sát lỗi consumer và event bị retry nhiều lần.
- processed_events là nhóm bảng logic trong cùng PostgreSQL.

Liên kết với các góc nhìn kiến trúc. [ADR-0010](#) cụ thể hóa phần xử lý nền trong C&C View và hỗ trợ [ADR-0005](#), [ADR-0007](#).

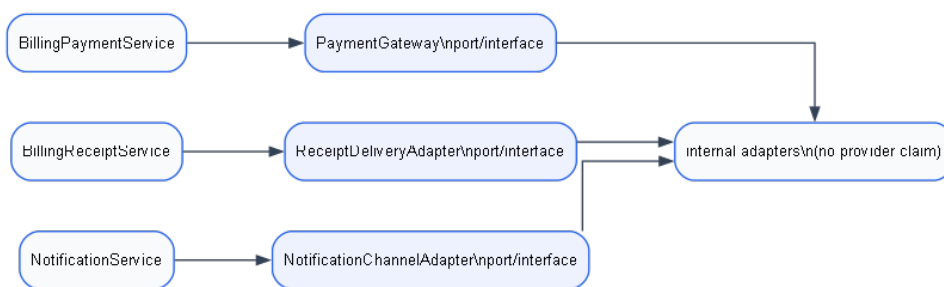
A.1.7 Nhóm quyết định về bảo mật, mở rộng và triển khai

ADR-0011: Dùng boundary adapters cho thanh toán, thông báo và biên nhận. **Trạng thái.** Được chấp nhận.

Bối cảnh. Thanh toán, thông báo và phát hành biên nhận là các điểm dễ thay đổi theo chính sách vận hành hoặc kênh tích hợp. Lỗi nghiệp vụ không nên phụ thuộc trực tiếp vào provider cụ thể, đặc biệt khi phạm vi hiện tại chỉ thể hiện adapter nội bộ.

Quyết định. Các kênh thanh toán, thông báo và biên nhận đi qua port/interface như PaymentGateway, NotificationChannelAdapter và ReceiptDeliveryAdapter.

Căn cứ. Quyết định này liên quan trực tiếp đến yêu cầu bảo trì và mở rộng chức năng ở Mục 1.6; khi thêm phương thức thanh toán hoặc kênh thông báo mới, thay đổi cần khu trú ở adapter và kiểm thử contract.



Hình 87: Minh họa boundary adapters cho thanh toán, thông báo và biên nhận

Hệ quả.

- Service nghiệp vụ phụ thuộc vào contract ổn định thay vì provider cụ thể.
- Cần kiểm thử contract cho từng adapter.
- Không trình bày như đã tích hợp provider thanh toán/SMS/email ngoài nếu phạm vi hiện tại chỉ có adapter nội bộ.

Liên kết với các góc nhìn kiến trúc. [ADR-0011](#) liên kết với Class Diagram, C&C View và phần SOLID ở Mục 3.9.

ADR-0012: Dùng IAM/RBAC và audit log cho thao tác nhạy cảm. **Trạng thái.** Được chấp nhận.

Bối cảnh. Hoàn tiền, hủy món đã gửi bếp, ghi đề giá, thay đổi quyền, điều chỉnh tồn kho và truy cập audit log là các thao tác nhạy cảm. Hệ thống cần kiểm soát ai được thực hiện thao tác nào và lưu vết đủ để đối chiếu sau vận hành.

Quyết định. Dùng IAM/RBAC tại lớp định danh và lớp biên; kết hợp AuthorizationPolicy cho điều kiện nghiệp vụ; ghi AuditLog hoặc audit event cho thao tác nhạy cảm.

Căn cứ. Yêu cầu bảo mật và kiểm toán ở Mục 1.6 quy định các thao tác nhạy cảm phải kiểm soát vai trò, ghi nhận người thực hiện, thời điểm, giá trị và lý do khi cần.



Hình 88: Minh họa kiểm quyền và audit log cho thao tác nhạy cảm

Hệ quả.

- Kiểm quyền không chỉ nằm ở gateway mà còn cần xuất hiện ở quy tắc nghiệp vụ.
- Audit là luồng bắt buộc cho thao tác nhạy cảm.
- Cần giữ user context và correlation id để truy vết đầy đủ.

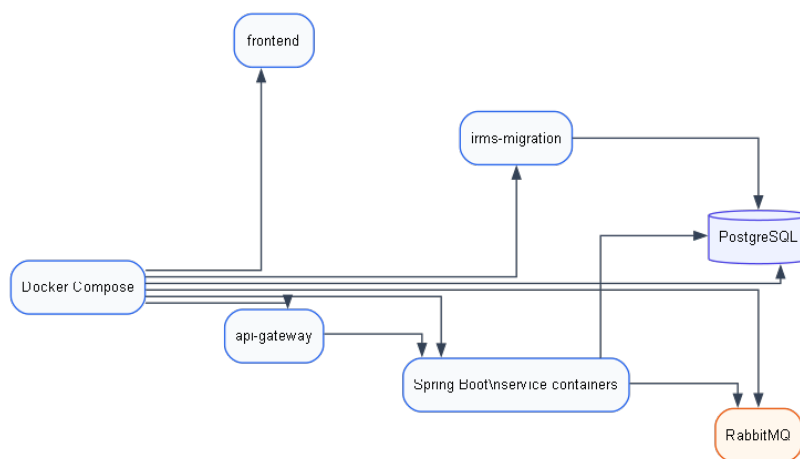
Liên kết với các góc nhìn kiến trúc. [ADR-0012](#) liên kết với yêu cầu phi chức năng, kiến trúc tổng quan, class identity/audit và các activity diagram quản trị.

ADR-0014: Dùng Docker Compose cho runtime cục bộ và trình bày học phần. **Trạng thái.** Được chấp nhận.

Bối cảnh. IRMS cần môi trường có thể khởi động nhất quán để kiểm thử tích hợp và trình bày chức năng. Runtime hiện tại gồm frontend, API gateway, các service Spring Boot, PostgreSQL, RabbitMQ và migration container.

Quyết định. Dùng Docker Compose để khởi động local runtime gồm app services, PostgreSQL, RabbitMQ và migration.

Căn cứ. Quyết định này liên quan đến yêu cầu vận hành và khả năng kiểm thử ở Mục 1.6; Compose giúp giảm sai lệch cấu hình giữa các lần chạy và làm rõ thứ tự khởi động.



Hình 89: Minh họa runtime Docker Compose cục bộ của IRMS

Hệ quả.

- Compose phù hợp kiểm thử tích hợp và demo trong phạm vi học phần.
- Không mô tả load balancer, replica hoặc production cluster như cấu hình đã triển khai nếu chưa có căn cứ.
- Migration cần chạy trước các service nghiệp vụ để schema nhất quán.

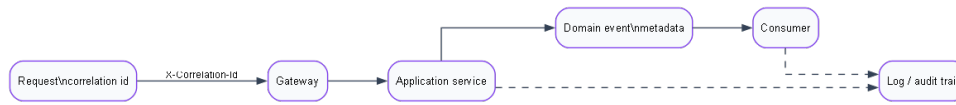
Liên kết với các góc nhìn kiến trúc. [ADR-0014](#) liên kết với Deployment View, Install View và Phụ lục hướng dẫn chạy project.

ADR-0015: Dùng centralized logging/correlation id cho truy vết vận hành. **Trạng thái.** Được chấp nhận.

Bối cảnh. Các yêu cầu ghi dữ liệu, event nền, hoàn tiền, kiểm toán và báo cáo cần truy vết xuyên suốt từ request ban đầu đến service và consumer liên quan. Nếu thiếu correlation id, việc điều tra lỗi hoặc đối chiếu audit sau ca vận hành sẽ khó hơn.

Quyết định. Các request/event quan trọng mang correlation id để truy vết qua gateway, application service, event metadata, consumer và audit/log.

Căn cứ. Quyết định này liên quan đến yêu cầu khả năng quan sát và vận hành ở Mục 1.6; report và thiết kế hiện tại đã mô tả correlation id như một phần của dữ liệu giao dịch, event metadata và audit context.



Hình 90: Minh họa correlation id đi qua request, service, event và audit/log

Hệ quả.

- REST request và event cần truyền correlation id nhất quán.
- Audit/log cần giữ correlation id để đối chiếu sau vận hành.
- Nếu mở rộng observability sau này, correlation id là nền tảng cho tracing đầy đủ hơn.

Liên kết với các góc nhìn kiến trúc. [ADR-0015](#) liên kết với C&C View, yêu cầu phi chức năng về quan sát vận hành và các luồng audit trong thiết kế chi tiết.

Các bản ghi ADR được lưu cùng tài liệu dự án trong thư mục `docs/adr`.

A.2 Liên kết giữa phụ lục và các phần chính

Phụ lục này chỉ đóng vai trò giải thích các quyết định đã được dùng trong thân tài liệu. Các nội dung dưới đây tránh lặp lại toàn bộ phần chính, mà chỉ chỉ ra nơi từng quyết định được thể hiện rõ nhất.

Bảng 43: Liên kết giữa các phần chính và phụ lục biện minh

Phần chính	Điểm cần giải thích	Phụ lục liên quan
Chương 1	Kiến trúc chủ đạo xuất phát từ yêu cầu chức năng và phi chức năng.	A.1 giải thích vì sao service-based là kiến trúc chính và event-driven là cơ chế hỗ trợ.
Mục 3.1	Sơ đồ tổng quan dùng service-based, event-driven, một PostgreSQL và các adapter biên.	A.1 và A.4 liên kết các quyết định chính với kiến trúc tổng quan.
Mục 3.2–3.7	Ba góc nhìn chính gồm Module View, Component-and-Connector View và Allocation/Deployment View.	A.3 giải thích vai trò từng góc nhìn và lý do không xem bản đồ năng lực logic là một view chính độc lập.
Chương 4	Sơ đồ lớp và sơ đồ hoạt động cụ thể hóa các ranh giới service, adapter và luồng xử lý.	A.4 giải thích các thành phần nền như PaymentGateway, NotificationChannelAdapter, ReceiptDeliveryAdapter, phân quyền và kiểm toán.

A.3 Lý do chọn các góc nhìn và loại sơ đồ

Ba góc nhìn chính được chọn để tránh trộn lẫn cấu trúc tĩnh, cấu trúc runtime và cấu trúc triển khai. Bản đồ năng lực logic được giữ như nội dung hỗ trợ đọc nghiệp vụ, không phải một góc nhìn kiến trúc chính độc lập.

Bảng 44: Vai trò của các góc nhìn và loại sơ đồ trong IRMS

Loại sơ đồ/góc nhìn	Vị trí chính	Vai trò
Module View	Mục 3.5	Mô tả các đơn vị hiện thực chính, trách nhiệm module, quan hệ phụ thuộc tĩnh và mapping giữa năng lực nghiệp vụ với cấu trúc backend.
Component-and-Connector View	Mục 3.6	Mô tả thành phần runtime, port, interface và connector giữa frontend, gateway, service nghiệp vụ, PostgreSQL, RabbitMQ và các adapter biên.
Allocation/Deployment View	Mục 3.7	Mô tả cách các thành phần phần mềm được phân bổ lên container, node runtime, cơ sở dữ liệu, broker, volume và cấu hình triển khai.
Bản đồ năng lực logic	Mục 3.4	Hỗ trợ chuyển từ yêu cầu nghiệp vụ sang ranh giới module; không thay thế ba góc nhìn kiến trúc chính.
Class Diagram	Mục 4.1–4.2	Làm rõ cấu trúc lớp, interface, service, policy, strategy và adapter ở mức thiết kế chi tiết.
Activity Diagram	Mục 4.3	Kiểm tra luồng xử lý chi tiết của từng nhóm use case, đặc biệt các điểm đồng bộ, bất đồng bộ, kiểm quyền và ghi nhận kiểm toán.

A.4 Lý do chọn các thành phần và loại hình kiến trúc nền

Các thành phần dưới đây nối yêu cầu vận hành ở Chương 1 với kiến trúc tổng quan và thiết kế chi tiết. Nội dung được trình bày theo vai trò kiến trúc, tránh mô tả như danh sách file triển khai.

Bảng 45: Lý do chọn các thành phần kiến trúc nền của IRMS

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trong kiến trúc
ReactJS cho lớp giao diện	Phù hợp với các màn hình thao tác theo vai trò như lễ tân, phục vụ, bếp, thu ngân và quản lý; hỗ trợ cập nhật trạng thái giao diện và điều hướng nhất quán.	Xuất hiện ở biên người dùng trong kiến trúc tổng quan và các kịch bản nghiệp vụ.
Frontend Node Server và api-gateway	Tạo lớp biên thống nhất: Frontend Node Server phục vụ React và chuyển tiếp /api, còn api-gateway định tuyến yêu cầu đến các service nội bộ.	Gắn với quyết định điểm vào thống nhất và các sơ đồ tổng quan, thành phần và triển khai.
Service-based architecture	Phân rã hệ thống theo các miền nghiệp vụ như đặt bàn, gọi món, bếp, thanh toán, tồn kho, báo cáo và kiểm toán.	Là lựa chọn kiến trúc chính; được phản ánh trong Module View, Component-and-Connector View và sơ đồ tổng quan.
Event-driven processing	Tách các hệ quả không cần phản hồi ngay khỏi tuyến giao dịch nóng, ví dụ thông báo, kiểm toán, cập nhật mô hình đọc và cảnh báo tồn kho.	Là cơ chế hỗ trợ cho service-based architecture; xuất hiện trong C&C View, các bộ xử lý nền và luồng sự kiện.

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trong kiến trúc
RabbitMQ, outbox và bộ xử lý nền	Bảo đảm sự kiện được phát sau khi giao dịch chính đã ghi nhận; hỗ trợ thử lại và xử lý nền mà không làm nghẽn thao tác gọi món, bếp hoặc thanh toán.	Gắn với quyết định kiểu kết nối và các sơ đồ thành phần, kết nối, triển khai.
PostgreSQL vật lý duy nhất	Bảo đảm nhất quán dữ liệu mạnh cho xác nhận đơn, thanh toán, hoàn tiền và kiểm toán.	Các nhóm dữ liệu như outbox, inbox, mô hình đọc báo cáo, định danh và kiểm toán là nhóm bảng logic trong cùng PostgreSQL, không phải database riêng.
Mô hình đọc cho báo cáo	Tách luồng đọc báo cáo khỏi dữ liệu giao dịch nóng, giúp truy vấn dashboard không ảnh hưởng trực tiếp đến gọi món, hóa đơn hoặc thanh toán.	Được cập nhật thông qua sự kiện hoặc tiến trình nền và được mô tả như một nhóm bảng logic trong PostgreSQL.
Docker Compose runtime	Phù hợp với phạm vi triển khai cục bộ gồm frontend, api-gateway, các service Spring Boot, PostgreSQL, RabbitMQ và migration container.	Gắn với Deployment/Allocation View; các hạ tầng sẵn sàng cao được đặt ở định hướng mở rộng thay vì mô hình hiện trạng.

Bảng 46: Lý do chọn các adapter biên và cơ chế kiểm soát của IRMS

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trong kiến trúc
PaymentGateway	Cô lập xử lý thanh toán sau một interface nội bộ, giúp Billing & Settlement Service không phụ thuộc trực tiếp vào một cơ chế thanh toán cụ thể.	Là adapter biên của vùng thanh toán và xuất hiện trong kiến trúc tổng quan, C&C View và sơ đồ lớp.
NotificationChannel-Adapter	Tách lựa chọn kênh thông báo khỏi logic nghiệp vụ chính, cho phép thay đổi kênh gửi mà không sửa lỗi thông báo.	Là adapter biên của vùng thông báo và liên kết với các luồng sự kiện nền.
ReceiptDelivery-Adapter	Tách phát hành biên lai khỏi tuyến thanh toán lỗi; biên lai giấy hoặc điện tử có thể xử lý sau khi trạng thái thanh toán đã rõ ràng.	Liên kết với BillingReceiptService và các luồng phát hành biên lai.
Phân quyền theo vai trò và AuthorizationPolicy	Kiểm soát quyền theo vai trò và theo điều kiện nghiệp vụ, đặc biệt với hoàn tiền, thay đổi giá, điều chỉnh tồn kho và truy cập dữ liệu quản trị.	Gắn với yêu cầu bảo mật ở Chương 1 và xuất hiện trong kiến trúc tổng quan, C&C View và thiết kế chi tiết.

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trong kiến trúc
AuditLog	Ghi nhận thao tác nhạy cảm để hỗ trợ truy vết, đối soát và kiểm toán vận hành.	Dữ liệu kiểm toán là nhóm bảng logic trong cùng PostgreSQL; các activity diagram quản trị cần thể hiện điểm ghi nhận này.

A.5 Bảng thuật ngữ và ký hiệu viết tắt

Bảng này thống nhất cách hiểu các ký hiệu viết tắt xuất hiện trong tài liệu. Tên riêng hoặc thuật ngữ chuẩn được giữ nguyên; phần diễn giải dùng tiếng Việt để tránh nhầm lẫn khi đọc giữa các chương.

Bảng 47: Thuật ngữ và ký hiệu viết tắt dùng trong tài liệu

Ký hiệu	Tên đầy đủ	Cách hiểu trong IRMS
ADR	Architecture Decision Record	Hồ sơ quyết định kiến trúc, dùng để ghi lại lựa chọn quan trọng và hệ quả của lựa chọn đó.
API	Application Programming Interface	Hợp đồng giao tiếp giữa giao diện, gateway và các service.
C&C	Component-and-Connector	Góc nhìn thành phần và kết nối, mô tả thành phần runtime, port, interface và connector.
IAM	Identity and Access Management	Vùng định danh và kiểm soát truy cập của hệ thống.
RBAC	Role-Based Access Control	Cơ chế phân quyền theo vai trò người dùng.
POS	Point of Sale	Điểm bán hàng hoặc giao diện thao tác bán hàng tại nhà hàng.
KDS	Kitchen Display System	Màn hình hiển thị bếp để nhận và cập nhật phiếu chế biến.
REST	Representational State Transfer	Kiểu giao tiếp đồng bộ dùng cho các thao tác cần phản hồi trực tiếp.
AMQP	Advanced Message Queuing Protocol	Giao thức hàng đợi dùng với RabbitMQ.
JDBC	Java Database Connectivity	Cơ chế kết nối từ ứng dụng Java đến PostgreSQL.
SOLID	Năm nguyên tắc thiết kế hướng đối tượng	Nhóm nguyên tắc giúp phân tách trách nhiệm, mở rộng có kiểm soát và đảo ngược phụ thuộc.
UI	User Interface	Giao diện người dùng.
UC	Use Case	Cả sử dụng được đặc tả trong Chương 2.

A.6 Hướng dẫn chạy project

Phần hướng dẫn chạy cục bộ gồm các bước khởi động hệ thống trong phạm vi triển khai hiện tại. Các lệnh dưới đây bám theo cấu trúc `irms_project`, cấu hình `docker-compose.yml`, Maven wrapper của backend và script frontend trong `package.json`.

Môi trường chạy cục bộ

Môi trường chạy cục bộ của IRMS sử dụng Java 17 cho backend Spring Boot, Node.js/npm cho frontend React/TanStack Start, Docker Compose cho runtime container, PostgreSQL cho lưu trữ và RabbitMQ cho xử lý bất đồng bộ. Môi trường chạy cục bộ bằng Docker Compose được ghi nhận trong [ADR-0014](#). Khi chạy không qua Docker Compose, PostgreSQL cần sẵn sàng ở cấu hình local và RabbitMQ chỉ cần thiết khi bật broker-backed runtime thay vì chế độ local mặc định.

Cấu trúc thư mục liên quan

- `irms_project/backend`: mã nguồn backend Spring Boot, Maven wrapper, Dockerfile, cấu hình `application*.properties` và migration Flyway.
- `irms_project/frontend`: mã nguồn giao diện React/TanStack Start, cấu hình Vite/Nitro và Dockerfile frontend.
- `irms_project/docker-compose.yml`: runtime container gồm PostgreSQL, RabbitMQ, migration, gateway, các service nghiệp vụ và frontend.
- `irms_project/scripts`: script hỗ trợ chạy local trên Windows và Linux/macOS.

Khởi động bằng Docker Compose

Cách chạy bằng Docker Compose phù hợp cho kiểm thử tích hợp và trình bày chức năng trong phạm vi học phần. Các thành phần chính có thể được khởi động từ thư mục `irms_project` bằng các lệnh sau:

```
cd irms_project
docker compose config
docker compose build
docker compose up -d
```

Compose khởi động postgres, rabbitmq, irms-migration, api-gateway, các service nghiệp vụ từ cổng 8081 đến 8088 và frontend. Theo cấu hình mặc định, giao diện nằm tại `http://localhost:5173`, gateway tại `http://localhost:8080`, RabbitMQ AMQP tại `localhost:5672` và giao diện quản trị RabbitMQ tại `http://localhost:15672`.

Chạy bằng script local

Nếu chạy không qua Compose đầy đủ, project cung cấp script local để kiểm tra môi trường, biên dịch backend, chạy migration và khởi động các runtime cần thiết. Trên Windows có thể dùng:

```
cd irms_project
.\scripts\run-local.ps1
```

Trên Linux/macOS có thể dùng:

```
cd irms_project
chmod +x scripts/run-local.sh
./scripts/run-local.sh
```

Script local sử dụng `VITE_API_BASE_URL=http://localhost:8080`, ghi log vào thư mục `.logs/local` và dùng PostgreSQL local với database `irms` khi cấu hình mặc định được giữ nguyên.

Chạy backend và frontend riêng

Backend có thể chạy bằng Maven wrapper trong thư mục `irms_project/backend`. Migration được chạy bằng profile `local,migration`; từng runtime service được chọn bằng profile `local,<service-name>`.

```
cd irms_project/backend
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local,migration
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local,api-gateway
```

Các profile service tương ứng gồm `ordering-service`, `kitchen-service`, `billing-service`, `reservation-service`, `inventory-service`, `notification-service`, `reporting-service` và `identity-audit-service`. Trên Linux/macOS dùng `./mvnw` thay cho `.\mvnw.cmd`.

Frontend chạy trong thư mục `irms_project/frontend` bằng `npm`:

```
cd irms_project/frontend
npm install
$env:VITE_API_BASE_URL="http://localhost:8080"
npm run dev
```

Trên Linux/macOS, biến môi trường có thể đặt cùng lệnh: `VITE_API_BASE_URL=http://localhost:8080 npm run dev`. Cổng dev mặc định của Vite là 5173.

Kiểm tra sau khi chạy

Sau khi hệ thống khởi động, có thể kiểm tra giao diện tại `http://localhost:5173` và gateway tại `http://localhost:8080`. Endpoint health chuyên dụng được cấu hình ở `http://localhost:8080/api/health`; các service nội bộ trong Compose cũng dùng endpoint `/api/health` cho healthcheck của container.

Dữ liệu và migration

Migration dùng Flyway trong backend và được bật ở profile/container migration trước khi các service nghiệp vụ chạy. Toàn bộ runtime hiện tại dùng một PostgreSQL vật lý duy nhất; các nhóm bảng giao dịch, `outbox_events`, `processed_events`, kiểm toán, định danh và read model báo cáo là các nhóm logic trong cùng PostgreSQL, không phải nhiều database vật lý riêng.

A.7 Phân công công việc

Phân phân công dưới đây trình bày phạm vi đóng góp chính của từng thành viên. Các nội dung vẫn cần được rà soát chéo để bảo đảm mạch liên kết giữa yêu cầu, kiến trúc, thiết kế chi tiết và phân phụ lục.

Bảng 48: Phân công công việc của Nhóm 8

MSSV	Thành viên	Phạm vi đóng góp chính
2311896	Đỗ Quang Long	Phụ trách tổng hợp bối cảnh nghiệp vụ, hệ thống hóa yêu cầu chức năng và phi chức năng, đồng thời phối hợp rà soát để bảo đảm các yêu cầu được phản ánh nhất quán trong các phần phân tích và thiết kế.
2311982	Lê Hoàng Luân	Phụ trách tổ chức cấu trúc tài liệu LaTeX, chuẩn hóa hình thức trình bày và tích hợp các thành phần nội dung, đồng thời phối hợp với các thành viên khác để bảo đảm bố cục báo cáo và hệ thống sơ đồ thống nhất.
2310990	Lê Thúy Hiền	Phụ trách xây dựng phần đặc tả use case và mô tả các luồng hoạt động nghiệp vụ chính, đồng thời phối hợp đối chiếu với phần yêu cầu và phần kiến trúc để giữ sự liên thông giữa mô tả nghiệp vụ và giải pháp thiết kế.
2213698	Nguyễn Hải Trung	Phụ trách các góc nhìn phân rã dịch vụ, thành phần và kết nối của hệ thống, đồng thời phối hợp với các phần khác để bảo đảm các quyết định kiến trúc bám sát yêu cầu nghiệp vụ và thống nhất với cấu trúc tổng thể của báo cáo.
2211384	Tô Thế Hưng	Phụ trách xây dựng các sơ đồ lớp và rà soát mô hình miền nghiệp vụ, đồng thời phối hợp đối chiếu với phần kiến trúc và phần hành vi để bảo đảm cấu trúc lớp phản ánh đúng các trách nhiệm và điểm mở rộng của hệ thống.
2310737	Trần Nhật Đăng	Phụ trách phần triển khai, phân bổ và phân tư kiến trúc, đồng thời phối hợp rà soát cách diễn đạt và mối liên kết giữa các chương để bảo đảm báo cáo có tính nhất quán và hoàn chỉnh ở mức toàn cục.